

NkCanvas & NKGui

Construire des applications et des jeux
avec le moteur Nkentseu

Cours complet pour débutants

Table des matières

0	Avant-propos	1
0.1	À qui s'adresse ce cours	1
0.1.1	Ce que ce cours couvre, et ce qu'il ne couvre pas	1
0.2	Comment lire les exemples.	2
0.3	Comment le dépôt est organisé	2
0.3.1	Kernel/Foundation — les briques de base	2
0.3.2	Kernel/System — les services système	3
0.3.3	Kernel/Runtime — les modules du moteur	3
0.3.4	Engine et Integrations	3
0.3.5	Applications et Ressources	3
0.4	Les outils de base du moteur.	4
0.4.1	Les types primitifs	4
0.4.2	Chaînes et conteneurs — NKContainers	4
0.4.3	Mathématiques — NKMath	4
0.4.4	Fichiers et chemins — NKFileSystem	5
0.4.5	Mémoire — NKMemory	5
0.4.6	Les habitudes à prendre	5
0.5	Compiler : le système Jenga	6
0.5.1	À quoi ressemble un fichier .jenga	6
0.5.2	Les deux commandes que vous taperez	7
0.5.3	Où sont les journaux	7
0.6	Le point d'entrée : nkmain, pas main	8
0.7	Exercices	9
1	Le décor : la pile complète	10
1.1	Vue d'ensemble	10
1.2	Étage 1 — la fenêtre : NKWindow	10
1.3	Étage 2 — les événements : NKEvent	11
1.4	Étage 3 — le GPU. Attention, il y a deux abstractions	12
1.4.1	NKRHI — l'abstraction GPU bas niveau	12
1.4.2	NKCanvas — sa propre abstraction, pour la 2D	12
1.4.3	Les cinq backends 2D de NKCanvas, et leur état réel	13
1.5	Étage 4 — NKCanvas, le rendu 2D	14
1.6	Étage 5 — NKGui, les widgets.	14
1.6.1	Le point capital : NKGui ne connaît aucun GPU	15
1.6.2	Ce que NKGui produit : NkGuiDrawList	15
1.7	Le pont : marier NKGui et NKCanvas	16
1.7.1	NkGuiCanvasBackend — le pont vers NKCanvas	16
1.7.2	NkGuiRHIBackend — le pont vers NKRHI	17
1.7.3	Pourquoi cette séparation ?	17
1.8	Le chemin complet d'un clic, de bout en bout	18
1.9	Deux mots sur les macros d'export.	18
1.10	Exercices	19
2	NKCanvas : dessiner en deux dimensions	20
2.1	Ce que NKCanvas est, en une phrase.	20
2.2	Les deux niveaux d'API	20
2.2.1	NKIRenderer2D : l'interface	20
2.2.2	NKRenderer2D : la façade	21
2.3	La cible de rendu : NkRenderWindow	21
2.3.1	Le redimensionnement	22

2.5.3	Les types de primitives, et l'absence de QUADS	25
2.6	Ce que NkCanvas n'a PAS	25
2.7	Les couleurs	25
2.8	Les textures.	26
2.8.1	Créer et remplir	26
2.8.2	La texture blanche de 1 pixel	26
2.8.3	Les pixels conservés côté CPU	27
2.9	Les polices et le texte.	27
2.9.1	Comment un texte devient des triangles	27
2.10	Les formes persistantes.	28
2.11	Le batching : le vrai sujet du module	29
2.11.1	L'accumulation	29
2.11.2	La capacité, et l'histoire qui va avec	30
2.11.3	La soumission : Flush()	30
2.11.4	Le vrai appel de dessin	31
2.12	Le découpage (<i>scissor</i>) et sa pile	31
2.12.1	Changer de découpe force un vidage	32
2.12.2	Le retournement d'axe sous OpenGL	32
2.13	La surface réellement utilisée en pratique	33
2.14	Un programme complet	33
2.14.1	L'ossature	34
2.14.2	Fenêtre, cible, état	34
2.14.3	La boucle	35
2.14.4	Le rendu	35
2.14.5	Ce que ce programme raconte	36
2.15	Récapitulatif des pièges du chapitre	36
2.16	Exercices	37
3	NKGui : l'interface immédiate	38
3.1	L'idée : on ne crée rien, on redéclare tout	38
3.1.1	Le mode immédiat	38
3.1.2	Ce que ce choix vous épargne	38
3.1.3	Une note d'honnêteté sur la documentation du module	39
3.2	L'identité d'un widget	39
3.2.1	La pile d'identifiants	40
3.3	Le contexte : NkGuiContext.	40
3.3.1	Cycle de vie	41
3.3.2	Le contexte courant	41
3.3.3	Le thème	42
3.4	L'entrée : NkGuiInput.	42
3.4.1	Ce que l'application pose	43
3.4.2	Ce que NKGui calcule	43
3.4.3	La répétition au maintien	43
3.4.4	Les touches suivies	43
3.5	Le cycle d'une image	44
3.5.1	BeginFrame	44
3.5.2	EndFrame	45
3.5.3	L'invariant d'ordre	45
3.6	Comment un widget conserve son état	46
3.7	Comment un widget reçoit les événements	46
3.7.1	Le problème	46

3.7.2	La solution : glouton, avec une image de retard	47
3.7.3	ButtonBehavior : la sémantique du clic	48
3.7.4	La machine à états d'interaction	49
3.8	Le routeur d'occlusion : gérer vos propres surfaces flottantes	49
3.8.1	Les quatre couches	50
3.8.2	L'API	50
3.8.3	L'usage type	50
3.9	La liste de dessin	52
3.9.1	Choisir la bonne couche : ctx.DL()	52
3.9.2	Les primitives	53
3.9.3	Le découpage	53
3.9.4	Le texte, et le piège du flou	54
3.9.5	La formule de la ligne de base	55
3.10	La mise en page	55
3.10.1	Le modèle : un curseur qui descend	56
3.10.2	Les sept modes de flux	56
3.10.3	Les helpers de curseur	56
3.10.4	Les conteneurs	57
3.10.5	Le DPI	58
3.11	Le catalogue des widgets.	58
3.11.1	Texte et boutons	58
3.11.2	Cases à cocher	59
3.11.3	Valeurs numériques	59
3.11.4	Saisie de texte	60
3.11.5	Graphiques et couleur	60
3.11.6	Images	61
3.11.7	Arbres, sélection, onglets	61
3.11.8	Zones défilables et tables	62
3.11.9	Popups, combos, menus, infobulles	62
3.11.10	Le contexte utilisé comme un widget	63
3.12	Panneaux, fenêtres, docking	63
3.12.1	BeginPanel / EndPanel	63
3.12.2	Begin / EndWindow	64
3.12.3	Le docking	65
3.13	Le hook de style	65
3.14	Les limites dures	66
3.15	Récapitulatif des idiomes.	67
3.16	Exercices	67
4	Construire une application, de zéro à l'exécutable	69
4.1	Vue d'ensemble du squelette.	69
4.2	Étape 1 — la fenêtre	69
4.3	Étape 2 et 3 — contexte GPU et cible de rendu	70
4.4	Étape 4 — le contexte NKGui.	71
4.5	Étape 5 — le pont vers NkCanvas	71
4.6	Étape 6 — la police, et l'upload de son atlas	72
4.6.1	Ce que fait LoadEmbedded	72
4.6.2	Ce que fait UploadFontGray8	73
4.6.3	Les polices de repli	74
4.7	Étape 7 — brancher l'entrée	74

4.7.1	Souris et fermeture	74
4.7.2	La molette	75
4.7.3	Le double-clic	75
4.7.4	Le texte et les touches	76
4.7.5	Le presse-papiers	76
4.8	La boucle principale	76
4.8.1	Le temps	77
4.8.2	Les événements — avant tout le reste	77
4.8.3	Le redimensionnement	77
4.8.4	L'image d'interface	78
4.8.5	Le curseur	79
4.8.6	La présentation	79
4.8.7	Ce que fait Submit, en détail	80
4.9	Le programme complet	81
4.10	Ajouter une image	84
4.11	Le fichier de build	85
4.11.1	Pourquoi chaque dépendance est là	86
4.11.2	Enregistrer la cible et compiler	86
4.12	Diagnostiquer les silences	87
4.13	Pour aller plus loin	87
4.14	Exercices	88
5	La coquille d'application : ce que vous n'écrivez plus	89
5.1	Le plus petit programme complet	89
5.2	Les méthodes que vous remplissez.	90
5.3	Les trois services que la coquille rend	90
5.3.1	La boucle cède la main, et sur le Web c'est vital	91
5.3.2	La zone sûre, mesurée et non devinée	91
5.3.3	Le pointeur : souris et doigt unifiés	92
5.4	Deux coquilles, et il faut choisir.	92
5.5	L'écran d'ouverture, offert	93
6	NKImage : charger, fabriquer, afficher des images	95
6.1	Ce qu'est NKImage, et ce qu'il n'est pas	95
6.1.1	Aucune dépendance externe	95
6.1.2	NKImage ne connaît aucun GPU	96
6.1.3	L'arborescence	96
6.2	Les deux vocabulaires de formats	96
6.3	Deux familles d'API, et pourquoi il faut choisir	97
6.3.1	L'API instance : l'objet vit sur la pile	98
6.3.2	L'API statique : vous possédez le pointeur	98
6.3.3	Copie interdite, déplacement autorisé	99
6.4	Charger une image	99
6.4.1	La signature qui compte	99
6.4.2	Les accesseurs	99
6.4.3	Le stride : la cause numéro un des images « penchées »	100
6.5	Fabriquer une image de toutes pièces	100
6.5.1	Le squelette canonique de l'API statique	100
6.5.2	Créer une image remplie d'une couleur	101
6.5.3	Dessiner dans une image (sur le processeur)	101
6.6	Écrire sur le disque	102

6.6.1	Le dispatch par extension	102
6.6.2	Les deux dernières lignes sont des mensonges	102
6.6.3	Encoder en mémoire	103
6.7	Transformer une image.	104
6.7.1	Redimensionner	104
6.7.2	Recadrer, convertir, recopier	105
6.7.3	Le cas HDR	105
6.8	Le cas particulier du GIF animé.	105
6.9	Libérer correctement : les quatre cas	106
6.10	Afficher une image dans une interface NKGui	107
6.10.1	Le widget	107
6.10.2	L'upload	107
6.10.3	La séquence complète	108
6.10.4	Le chemin plus court : NkTexture de NkCanvas	109
6.11	Décoder ailleurs, uploader ici.	110
6.12	État réel du module	110
6.12.1	Ce qui fonctionne	110
6.12.2	Ce qui ne fonctionne pas	111
6.12.3	Les deux classes utilitaires exportées	111
6.13	Exercices	111
7	NKFont : polices, atlas et métriques	113
7.1	Le module ne connaît ni image, ni GPU, ni fenêtre	113
7.2	Les quatre objets à connaître.	114
7.2.1	NKFontAtlas — le propriétaire de tout	114
7.2.2	NKFont — une face rastérisée à une taille	115
7.2.3	NKFontGlyph — la structure qui fait tout le travail	115
7.2.4	NKFontConfig — les réglages, posés avant l'ajout	116
7.3	Les dix polices embarquées : écrire du texte sans aucun fichier	117
7.4	La séquence canonique en quatre temps.	118
7.4.1	Le mécanisme de repli en action	119
7.5	Dessiner une chaîne de caractères	119
7.5.1	Le patron universel	119
7.5.2	Le pixel-snap, et l'invariant subtil	120
7.5.3	La ligne de base, pas le haut du texte	120
7.5.4	Mesurer avant de dessiner	121
7.6	De l'atlas à l'écran	121
7.6.1	L'atlas est en alpha 8 bits	121
7.6.2	Le piège d'upload	122
7.6.3	Le chemin NKGui, celui que vous emploierez	122
7.6.4	Changer d'échelle : recharger et ré-uploader	123
7.7	Le mode SDF : du texte net à toute taille	124
7.8	Les limites dures, et l'invariant du Build()	124
7.8.1	Des tableaux de taille fixe	124
7.8.2	Build() n'est ni réentrant ni parallélisable	125
7.8.3	Le mergeMode duplique les pointeurs	125
7.8.4	Le cache de tailles rend nullptr la première fois	126
7.9	Ce que le module ne fait pas	126
7.10	L'autre wrapper : render::NkFont	126
7.11	Récapitulatif des trois chemins	127
7.12	Exercices	128

8	NKAudio : faire du bruit	129
8.1	Le module, et sa dépendance surprenante	129
8.2	L'invariant fondateur : un seul format interne	130
8.3	Le patron canonique en cinq temps	130
8.3.1	Temps 1 — initialiser le moteur	131
8.3.2	Temps 2 — décoder le fichier	132
8.3.3	Temps 3 — lancer une voix	132
8.3.4	Temps 4 — piloter pendant la lecture	133
8.3.5	Temps 5 — l'ordre de fermeture, qui n'est pas négociable	134
8.4	Jouer un son au clic d'un bouton	134
8.4.1	Au démarrage — une seule fois	134
8.4.2	Dans la frame — quand le widget renvoie <code>true</code>	135
8.4.3	À la fermeture	135
8.5	Les bus : régler des groupes de sons ensemble	135
8.6	Les fichiers longs : le streaming	137
8.7	Le <i>callback</i> procédural, et le fil audio	138
8.7.1	L'ordre de destruction avec un lecteur de flux	138
8.8	Le micro	139
8.9	Tester sans carte son	140
8.10	État réel du module	141
8.10.1	Ce qui fonctionne	141
8.10.2	Ce qui ne fonctionne pas	141
8.10.3	Le reste de la surface, en survol	142
8.11	Relier le son à l'image	142
8.12	Exercices	143
9	NKMedia : lire et écrire de la vidéo	145
9.1	Le module, et une inversion inhabituelle	145
9.1.1	Ici, les commentaires <i>sous-estiment</i> le module	146
9.2	NkMediaProbe : qu'y a-t-il dans ce fichier ?	147
9.3	Écrire une vidéo	148
9.3.1	L'API	148
9.3.2	Le squelette de référence	149
9.3.3	Deux limites de l'écriture	150
9.3.4	La séquence d'images, à la manière de Blender	150
9.4	Lire une vidéo	151
9.4.1	L'API	151
9.4.2	La boucle de lecture	151
9.4.3	Le <code>seek</code> n'est pas ce que vous croyez	152
9.4.4	Décoder coûte cher	152
9.5	Afficher la vidéo	153
9.5.1	Dans une fenêtre NkCanvas	153
9.5.2	Dans une interface	154
9.6	Enregistrer ce que l'application affiche	155
9.6.1	L'API	155
9.6.2	La capture, de bout en bout	155
9.6.3	Trois pièges du recorder	156
9.6.4	Le sens des pixels	157
9.7	Le démux audio, en deux mots	157
9.8	Vérifier que tout marche sur votre machine	157
9.9	Une leçon de performance qui dépasse la vidéo	158

9.10 Exercices	159
10 NkNetwork : faire parler deux machines.	160
10.1 Où il vit, et ce dont il a besoin	160
10.1.1 Deux avertissements sur la documentation du module	161
10.2 Le périmètre de ce chapitre	161
10.3 Le socle : la plateforme.	162
10.3.1 Les codes de retour	162
10.3.2 Les adresses	163
10.4 NkConnectionManager : la classe du chapitre	163
10.4.1 Trois invariants de fil d'exécution	164
10.4.2 Le bout-en-bout, tel qu'il est testé	164
10.4.3 On ne reçoit pas, on draine	165
10.5 Choisir son canal	166
10.6 NkBitStream : ne pas gaspiller le réseau	167
10.6.1 L'argument : la quantification	167
10.7 NkNetWorld : répliquer des entités	169
10.8 Le socket brut : la découverte sur le réseau local	170
10.9 HTTP, et pourquoi HTTPS échoue	172
10.10 Comment structurer une application en réseau	173
10.11 L'ordre de fermeture	174
10.12 Exercices	175
11 Projet final : NkAtelier.	176
11.1 Ce que nous allons construire	176
11.2 Étape 0 — l'arborescence	177
11.3 Étape 1 — le fichier de build	177
11.3.1 Le contenu	177
11.3.2 Ce qu'il faut comprendre dans ce fichier	178
11.3.3 Enregistrer le projet dans l'espace de travail	178
11.4 Étape 2 — le squelette : fenêtre, contexte, backend, police	179
11.4.1 Les inclusions	179
11.4.2 Fenêtre et cible de rendu	179
11.4.3 Contexte NKGui et backend	180
11.4.4 La police	181
11.5 Étape 3 — les événements et la boucle.	181
11.5.1 Le branchement des entrées	181
11.5.2 La boucle, et l'invariant d'ordre	182
11.6 Étape 4 — l'image (NKImage)	182
11.6.1 Fabriquer l'image	183
11.6.2 L'appeler, et l'afficher	184
11.7 Étape 5 — le son (NKAudio)	184
11.7.1 Synthétiser trois sons	184
11.7.2 Initialiser, jouer, couper	185
11.8 Étape 6 — la vidéo (NKMedia)	186
11.8.1 Écrire la vidéo	186
11.8.2 Relire la vidéo	187
11.8.3 Cadencer et afficher	188
11.9 Étape 7 — la fermeture.	189
11.10 Compiler et lancer	190
11.11 Le catalogue des silences.	191

11.12	Le programme, vu de haut	192
11.13	Pour aller plus loin	192
11.13.1	Enregistrer ce que l'atelier affiche	192
11.13.2	Ajouter le réseau	193
11.13.3	Faire du décodage un travail de fond	193
11.14	Exercices	193

Avant-propos

0.1 À qui s'adresse ce cours

Ce cours s'adresse à une personne qui *sait déjà programmer en C++* et qui ne connaît *rien* au moteur Nkntseu. C'est une distinction importante : nous n'expliquerons pas ce qu'est un pointeur, une référence, une lambda ou un constructeur. En revanche, nous expliquerons *tout* du moteur — y compris les choses qui paraissent évidentes une fois qu'on les a comprises, et particulièrement celles-là, parce que ce sont elles qui font perdre une soirée quand personne ne les a écrites.

Ce que vous devez savoir avant d'ouvrir ce document :

- du C++ moderne, niveau C++17 : auto, lambdas, références, constexpr, noexcept, énumérations fortement typées (enum class);
- la notion de compilation séparée : en-têtes, unités de traduction, édition de liens;
- ce qu'est un pointeur non possédant (*non-owning*) par opposition à un pointeur possédant. Ce cours en parlera souvent : le moteur en est rempli et c'est la première source de plantages au démarrage;
- quelques notions vagues de rendu : un sommet (*vertex*), un triangle, une texture, une matrice de projection. Vous n'avez pas besoin de savoir écrire un *shader* : nous n'en écrirons aucun.

Ce que vous n'avez *pas* besoin de savoir : OpenGL, Vulkan, Direct3D, Metal. Tout le cours se tient au-dessus de ces API ; le moteur les cache, et c'est précisément son intérêt.

0.1.1 Ce que ce cours couvre, et ce qu'il ne couvre pas

Les cinq premiers chapitres — ceux que vous tenez — forment un parcours complet et fermé : à la fin du chapitre 4, vous savez écrire, compiler et lancer une application graphique interactive complète avec le moteur. Le parcours est le suivant :

1. **Chapitre 0** (celui-ci) : le dépôt, les règles du projet, la compilation.
2. **Chapitre 1** : la pile logicielle complète, de la fenêtre du système d'exploitation jusqu'aux boutons.
3. **Chapitre 2** : **NkCanvas**, le moteur de rendu 2D. Comment on dessine des triangles, et ce que le module ne sait *pas* faire.
4. **Chapitre 3** : **NKGui**, le framework d'interface. C'est le cœur du cours et le chapitre le plus long.
5. **Chapitre 4** : construire une application finie, du fichier de build jusqu'à l'exécutable qui s'affiche.

Ne sont *pas* couverts ici : le rendu 3D (NKRenderer, NKRI), l'audio (NKAudio), la vidéo (NKMedia), le réseau (NKNetwork), l'intelligence artificielle (Kernel/AI). Certains de ces modules font l'objet de chapitres ultérieurs de ce même cours.

Un avertissement d'honnêteté

Ce cours dit aussi ce qui *ne marche pas*. Le moteur est un projet vivant : certains backends graphiques sont validés à l'exécution, d'autres non ; certaines fonctions déclarées retournent une valeur neutre. Chaque fois que c'est le cas, il est écrit noir sur blanc, avec le fichier et la ligne. Un cours qui promet un moteur parfait vous ferait perdre plus de temps qu'il ne vous en ferait gagner.

0.2 Comment lire les exemples

Tous les blocs de code de ce cours portent leur **provenance** en titre, sous la forme `chemin/vers/fichier.cpp:ligne`. Ce n'est pas de la coquetterie : cela signifie que vous pouvez ouvrir le fichier et vérifier. Les extraits sont pris tels quels dans le dépôt, parfois abrégés (les coupures sont signalées par des points de suspension ou un commentaire). Quand un exemple est *écrit pour le cours* et n'existe pas tel quel dans le dépôt, c'est dit explicitement.

Trois encadrés reviennent régulièrement :

- **Ce qu'il faut retenir** — la phrase à emporter, si vous ne deviez retenir qu'une chose de la section ;
- **Le piège** — un comportement qui coûte une soirée. Presque tous ceux de ce cours sont issus de bugs réellement rencontrés dans le dépôt et documentés en commentaire dans les sources ;
- **Exercice** — à faire, pas à lire.

Le dépôt de référence est `D:/Projets/2026/Nkentseu/Nkentseu`. Tous les chemins de ce cours sont relatifs à cette racine.

0.3 Comment le dépôt est organisé

Le moteur est découpé en **modules**. Un module est un dossier autonome avec ses sources, son fichier de build (`.jenga`) et, souvent, sa documentation (`README.md`, `ARCHITECTURE.md`, `ROADMAP.md`, `USAGE.md`). Les modules sont rangés par **couche**, et le principe fondamental est énoncé dans `ARCHITECTURE.md` : *chaque couche ne connaît que les couches situées en dessous d'elle*.

0.3.1 Kernel/Foundation — les briques de base

Module	Rôle
NKCore	Types primitifs (<code>int32</code> , <code>float32</code> , <code>usize...</code>), macros, asserts
NKMath	<code>NkVec2/3/4</code> , <code>NkMat4</code> , <code>NkQuat</code> , <code>NkColor</code> , <code>NkRect</code>
NKMemory	Allocateurs, <code>NkUniquePtr</code> — la porte d'entrée de toute allocation
NKContainers	<code>NkVector</code> , <code>NkString</code> , <code>NkHashMap</code>
NKPlatform	Détection de plateforme à la compilation, macros <code>NKENTSEU_PLATFORM_*</code>

Ces cinq modules n'ont *aucune* dépendance externe. Tout le reste du moteur repose dessus.

0.3.2 Kernel/System — les services système

NKFileSystem (chemins, fichiers), NKLogger (journalisation), NKStream (lecture/écriture binaire et texte), NKThreading (fils d'exécution, mutex, atomiques), NKTime (horloges, *delta time*), NKNetwork, NKReflection, NKSerialization.

0.3.3 Kernel/Runtime — les modules du moteur

C'est là que vivent les deux vedettes de ce cours :

- [Kernel/Runtime/NKCanvas](#) — le rendu 2D (chapitre 2);
- [Kernel/Runtime/NKGui](#) — le framework d'interface (chapitre 3).

et leurs voisins immédiats : NKWindow (la fenêtre native), NKEvent (les événements typés), NKFont (rasterisation de polices), NKImage (décodage PNG/JPG/SVG...), NKRHI (abstraction GPU bas niveau), NKRenderer (rendu 3D), NKAudio, NKMedia, NKSL (langage de *shaders*), NKUI (l'ancien module d'interface, remplacé par NKGui).

0.3.4 Engine et Integrations

[Engine/NKEditorKit](#) est une *coquille* d'application d'édition : fenêtre sans décoration, barre de titre maison, docking, palette de commandes, préférences. C'est ce qui permet à l'IDE NKCode de n'écrire que ses panneaux. Ce cours ne s'en sert pas — nous construirons l'application à la main au chapitre 4 — mais il faut savoir qu'il existe, parce que la plupart des exemples réels du dépôt passent par lui.

[Integrations/NKGui](#) contient le pont alternatif entre NKGui et NKRHI (NKGuiRHIBackend), pour les applications qui font de la 3D. Nous y reviendrons au chapitre 1.

0.3.5 Applications et Resources

[Applications/](#) contient toutes les applications et démos, une par dossier. Trois nous serviront de référence tout au long du cours :

- [Applications/NKGuiDemo](#) — la démo de référence de NKGui (environ 1 067 lignes dans un seul `main.cpp`). Elle prouve le pipeline complet et couvre presque toute l'API. C'est le fichier à ouvrir à côté de ce cours;
- [Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp](#) — la démo NkCanvas *seul*, sans interface, à laquelle nous consacrerons la fin du chapitre 2;
- [Applications/NKCode](#) — l'IDE complet, l'application la plus grosse du dépôt. Elle sert d'étude de cas.

[Resources/](#) contient les données partagées (polices, textures, icônes, modèles, shaders). [Documentation/](#) et [Guides/](#) contiennent la documentation écrite. [Build/](#) reçoit tout ce que la compilation produit, et [logs/](#) les journaux d'exécution.

Ce qu'il faut retenir

Le dépôt suit une règle simple : **un dossier = un module = un fichier .jenga**. Pour comprendre ce dont un module dépend, il suffit d'ouvrir son `.jenga` et de lire l'appel à `nkentseudependson`. Nous le ferons souvent : c'est le moyen le plus fiable de savoir qui connaît qui.

0.4 Les outils de base du moteur

Avant d'écrire du code Nkentsu, il faut connaître la poignée d'outils qu'on retrouve dans chaque fichier du dépôt. Ils viennent tous de `Kernel/Foundation` et de `Kernel/System`, et ils reviendront à toutes les pages de ce cours.

0.4.1 Les types primitifs

Le moteur nomme explicitement la taille de ses entiers, dans `NKCore` : `int8`, `int16`, `int32`, `int64`, `uint8`, `uint16`, `uint32`, `uint64`, `float32`, `float64`, et `usize` pour une taille en octets. Ils sont disponibles dans l'espace de noms `nkentsu`. Une largeur de fenêtre est un `uint32`, une coordonnée un `float32`, un compteur de boucle un `int32` : on lit la taille dans le nom, sans avoir à connaître la plateforme.

0.4.2 Chaînes et conteneurs — `NKContainers`

Type	Rôle
<code>NkString</code>	chaîne de caractères, <code>CStr()</code> donne le <code>const char *</code>
<code>NkStringView</code>	vue non possédante sur une chaîne
<code>NkVector<T></code>	tableau dynamique — le conteneur le plus utilisé du moteur
<code>NkHashMap</code>	table associative

Les méthodes suivent la convention de nommage du moteur, en `PascalCase` : `Size()`, `Data()`, `PushBack()`, `PopBack()`, `Resize()`, `Reserve()`, `Clear()`, `Erase()`, `Begin()`, `End()`. L'accès indexé s'écrit comme partout, avec les crochets. Un parcours typique, tel qu'on en croise des centaines dans le dépôt :

`Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:122-125 (extrait)`

```
1 mScratch.Resize(dl.vtx.Size());
2 for (uint32 i = 0; i < dl.vtx.Size(); ++i) {
3     const nkgui::NkGuiVertex &s = dl.vtx[i];
4     renderer::NkVertex2D &d = mScratch[i];
```

Deux réflexes visibles ici et à reprendre : on dimensionne d'abord (`Resize`), on remplit ensuite ; et on prend une *référence* sur l'élément plutôt qu'une copie.

Pour construire une chaîne à partir de valeurs, le moteur fournit `NkPrintf`, qui renvoie une `NkString` :

`Applications/NKCode/src/NKCode/Editor/NkCodeEditor.h:4585-4586 (extrait)`

```
1 const NkString nb = NkPrintf("%d", i + 1);
2 const float32 nw = ctx.font->MeasureWidth(nb.CStr());
```

0.4.3 Mathématiques — `NKMath`

Tout vit dans l'espace de noms `nkentsu::math` : `NkVec2f`, `NkVec2i`, `NkVec2u`, `NkVec3f`, `NkVec4f` pour les vecteurs ; `NkMat4f` pour les matrices ; `NkQuat` pour les quaternions ; `NkRect2f`, `NkRect2i` pour les rectangles ; `NkColor` pour les couleurs ; et les fonctions trigonométriques `NkCos`, `NkSin`, etc. Les vecteurs se construisent en agrégat, ce qui donne un code très compact :

```

1  math::NkVec2f ball{220.f, 200.f};
2  math::NkVec2f vel{260.f, 215.f}; // pixels / seconde
3  const float32 R = 42.f;

```

0.4.4 Fichiers et chemins — NKFileSystem

NkPath manipule les chemins, NkFile lit et écrit. Deux appels que ce cours croisera : NkPath::GetExecutableDirectory(), qui donne le dossier de l'exécutable, et NkFile::ReadAllBytes, qui charge un fichier entier — c'est ainsi que NkFont lit une fonte ([Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkFont.cpp:65-81](#)).

0.4.5 Mémoire — NKMemory

Toute la mémoire du moteur passe par NKMemory, dans l'espace de noms nkentseu::memory. Deux outils suffisent pour ce cours :

- `memory::NkMakeUnique<T>(args...)` crée un objet possédé par un `NkUniquePtr<T>` : il se libère tout seul en fin de portée;
- `memory::NkGetDefaultAllocator()` donne l'allocateur par défaut, dont les méthodes `New<T>(args...)` et `Delete(ptr)` servent quand on gère la durée de vie soi-même.

Il existe aussi des allocateurs spécialisés — linéaire pour les données « par image », *pool* pour des objets identiques, arène pour un niveau de jeu — détaillés dans [Guides/01-NKMemory.md](#).

La règle qui va avec, et qui n'admet pas d'exception : **un objet obtenu d'un allocateur se rend au même allocateur**. Le dépôt en porte la cicatrice :

```

1  // IMPORTANT : on alloue via l'allocateur NKMemory (NkGetDefaultAllocator)
2  // et NON via `new`. Raison : Le NkUniquePtr<NkIRenderer2D> (CreateUnique)
3  // desalloue via NkDefaultDelete -> ce MEME allocateur (-> _aligned_free).
4  // Allouer avec `new` (malloc) puis libérer avec _aligned_free = mismatch
5  // d'allocateur -> heap corruption c0000374 au free du renderer (shutdown).
6  // cf BugReports/NKCanvas/heap-c0000374-renderer-alloc-mismatch.md.

```

Le mélange d'allocateurs

Rendre à un allocateur un bloc qu'il n'a pas donné produit une **corruption de tas**. Sous Windows, cela se manifeste par un plantage `0xc0000374` — et, ce qui rend le bug diabolique, ce plantage survient à *la fermeture de l'application*, très loin de la ligne fautive. Le fichier [Kernel/Runtime/NKCanvas/src/NKCanvas/Factory/NkContextFactory.h](#) le dit en une ligne (:39) : « RÈGLE : ne JAMAIS faire `delete` (objet alloué par NKMemory) ». Un objet créé par une *factory* du moteur se détruit par la méthode `Destroy` correspondante, et par elle seule.

0.4.6 Les habitudes à prendre

Voici la forme que prend tout cela dans le code que vous écrirez au chapitre 4 :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:262-269 (extrait)

```
1 auto target = memory::NkMakeUnique<NkRenderWindow>(window, desc);
2 if (!target || !target->IsValid())
3     return -1;
4
5 auto ctxPtr = memory::NkMakeUnique<NkGuiContext>();
6 if (!ctxPtr)
7     return -1;
8 NkGuiContext &ctx = *ctxPtr;
```

Trois habitudes à prendre immédiatement :

- on crée les objets à durée de vie longue avec `memory::NkMakeUnique<T>` : ils se libèrent seuls, au bon endroit;
- on teste toujours le résultat. `NkMakeUnique` peut retourner un pointeur nul ; les constructeurs du moteur ne lancent pas d'exception — ils posent un état « invalide » qu'on interroge par `IsValid()`;
- quand un objet est possédé par un `NkUniquePtr`, on passe partout ailleurs un *pointeur brut non possédant* obtenu par `.Get()`. Il faut alors garantir la durée de vie à la main. Nous verrons au chapitre 4 que c'est exactement le cas de la police.

Ce qu'il faut retenir

`NkString` et `NkVector` pour les données, `math::Nk*` pour la géométrie, `NkFileSystem` pour les fichiers, `NKMemory` pour la mémoire : ce sont les outils du moteur, et ils suffisent. Les noms de méthodes sont en PascalCase et les tailles d'entiers sont écrites dans le nom du type.

0.5 Compiler : le système Jenga

Le moteur n'utilise ni CMake, ni Meson, ni Visual Studio directement. Il utilise **Jenga**, un système de build maison écrit en Python. Chaque module porte un fichier `<NomDuModule>.jenga` qui est, littéralement, un script Python.

0.5.1 À quoi ressemble un fichier `.jenga`

Applications/NKGuiDemo/NKGuiDemo.jenga:41-58 (extrait)

```
1 with project("NKGuiDemo"):
2     windowedapp()
3     language("C++")
4     cppdialect("C++17")
5     location(".")
6
7     files(["src/**/*.cpp"])
8
9     nkentseudependson(
10         ["NKGui", "NKReflection", "NKCanvas", "NKWindow", "NkEvent", "NKGlad",
11          "NKFont", "NKImage", "NKStream", "NKFileSystem", "NKLogger", "NKMath",
12          "NKTime", "NKContainers", "NKMemory", "NKCore", "NKPlatform", "NKThreading"],
13         extra_includes=["src", "src/NKGuiDemo", "%{NKGlad.location}/include"],
14     )
15
16     objdir("%{wks.location}/Build/Obj/{cfg.buildcfg}-{cfg.system}/{prj.name}")
```

```
17 targetdir("%{wks.location}/Build/Bin/%{cfg.buildcfg}-%{cfg.system}/%{prj.name}")
```

Les quatre appels à connaître :

- `project("Nom")` — ouvre la déclaration d'une cible. Le nom est celui qu'on passera à `--target`;
- `windowedapp()` — le type de cible : application fenêtrée. Une bibliothèque déclarerait `staticlib()`;
- `files([...])` — les sources à compiler. Notez le motif `"src/**/*.cpp"` : *seuls les .cpp*. Un module entièrement en .h avec des fonctions `inline` ne compile rien du tout — c'est le cas du pont NKGui → NkCanvas, et c'est délibéré (chapitre 1);
- `nkentseudependson([...])` — la liste des modules dont on dépend. C'est *la* ligne à lire pour savoir qui connaît qui.

0.5.2 Les deux commandes que vous taperez

Commande — à taper depuis la racine du dépôt

```
1 jenga build --target NKGuiDemo --config Release
2 jenga build --target NKGuiDemo --config Debug
```

L'exécutable produit atterrit à un emplacement déterminé par le `targetdir` ci-dessus :

Chemin de l'exécutable produit (Windows)

```
1 Build\Bin\Release-Windows\NKGuiDemo\NKGuiDemo.exe
2 Build\Bin\Debug-Windows\NKGuiDemo\NKGuiDemo.exe
```

La configuration Debug garde les symboles et les assertions; Release optimise. Un réflexe du dépôt, énoncé dans les règles de travail du projet : quand on livre un lot, on construit **les deux** configurations, parce qu'un bug qui n'existe qu'en Release (ordre d'initialisation, dépassement de tampon masqué) est le pire de tous.

Lancer depuis la racine du dépôt

Beaucoup d'applications du dépôt cherchent leurs ressources par des chemins *relatifs au répertoire courant* : `Resources/Fonts/...`, `Applications/NKCode/data/textures/...`. Lancer l'exécutable depuis son propre dossier (`Build/Bin/...`) donne alors une fenêtre sans polices, sans icônes, parfois entièrement noire — et *aucun* message d'erreur explicite. La parade adoptée par le dépôt est de chercher dans plusieurs dossiers candidats, dont celui de l'exécutable, comme le documente [Applications/NKCode/src/NKCode/Shell/NkAppFonts.h:18-21](#) : « Candidats RELATIFS AU CWD (dev, lancement depuis la racine du repo) PUIS relatifs à l'EXECUTABLE ». Tant que vous êtes en développement : **lancez depuis la racine du dépôt**.

0.5.3 Où sont les journaux

Le moteur journalise via NKLogger. La trace d'exécution est écrite dans `logs/app.log`, *relative-ment au répertoire courant* — donc, si vous lancez depuis la racine, dans `logs/app.log` à la racine du dépôt. La trace précédente est conservée sous `logs/app.prev.log`.

C'est le premier endroit à regarder quand une application se lance et n'affiche rien : le choix du backend graphique, les échecs de création de contexte, les polices introuvables y sont écrits. Dans votre propre code, la journalisation s'écrit ainsi :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:319 et 1055 (extraits)

```
1 logger.Info("Image chargée : {0}x{1}", imgW, imgH);
2 logger.Error("Capture écran ECHEC ({0})", path);
```

(l'objet global `logger` vient de `NKLogger/NkLog.h`; il existe aussi des variantes `Infof/Warnf/Errorf` au format `printf`).

0.6 Le point d'entrée : `nkmain`, pas `main`

Dernière particularité à connaître avant d'écrire la moindre ligne : le moteur fournit lui-même le `main` natif de chaque plateforme (`WinMain` sous Windows, `main` ailleurs, un point d'entrée Java/JNI sur Android). Votre programme, lui, déclare :

Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp:29-33 et 57

```
1 NKENTSEU_DEFINE_APP_DATA([]() {
2     NkAppData d{};
3     d.appName = "NkCanvas Demo";
4     d.appVersion = "1.0.0";
5     return d;
6 })();
7
8 int nkmain(const NkEntryState &state) {
```

Il faut pour cela inclure `NKWindow/NKMain.h`. Le paramètre `state` porte notamment les arguments de la ligne de commande, sous forme d'un `NkVector<NkString>` :

Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp:36-40 (extrait)

```
1 static NkGraphicsApi ParseBackend(const NkVector<NkString> &args) {
2     for (usize i = 1; i < args.Size(); ++i) {
3         const NkString &a = args[i];
4         if (a == "--backend=vulkan" || a == "-bvk")
5             return NkGraphicsApi::NK_GFX_API_VULKAN;
```

Ce qu'il faut retenir

Trois choses à retenir de ce chapitre, et rien d'autre :

1. Les outils du moteur : `NkString`, `NkVector`, `math::Nk*`, `NkFileSystem`, et `memory::NkMakeUnique` pour la mémoire.
2. `jenga build --target <Cible> --config Release` depuis la racine, exécutable dans `Build/Bin/Release-Windows/<Cible>/<Cible>.exe`, lancé **depuis la racine**.
3. Le point d'entrée s'appelle `nkmain(const NkEntryState &)`, et les journaux sont dans `logs/app.log`.

0.7 Exercices

1 — Reconnaître la carte

Sans compiler quoi que ce soit, ouvrez `Kernel/Runtime/NKGui/NKGui.jenga` et `Kernel/Runtime/NKCanvas/NKCanvas.jenga`. Relevez la liste passée à `nkentseudependson` dans chacun. Question : **NKGui dépend-il de NKCanvas**? Notez votre réponse; le chapitre 1 y revient et en tire toute une architecture.

2 — Le premier build

Construisez et lancez la démo de référence :

```
cd D:\Projets\2026\Nkentseu\Nkentseu
jenga build --target NKGuiDemo --config Release
.\Build\Bin\Release-Windows\NKGuiDemo\NKGuiDemo.exe
```

Puis relancez-la depuis *son propre dossier* (`cd Build\Bin\Release-Windows\NKGuiDemo` puis `.\NKGuiDemo.exe`) et comparez : qu'est-ce qui change à l'écran? Ouvrez ensuite `logs/app.log` et retrouvez la ligne qui indique le backend graphique retenu.

3 — Chasse aux allocations

Cherchez dans `Kernel/Runtime/NKCanvas/src/` toutes les occurrences de `NkGetDefaultAllocator`. Pour chacune, identifiez qui libère l'objet et par quel appel. Vous devriez tomber sur le destructeur du pont NKGui, que le chapitre 1 détaille : `Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NKGuiCanvasBackend.h:24-32`.

Le décor : la pile complète

Avant d'écrire une seule ligne de code, il faut savoir *qui fait quoi*. Ce chapitre pose la carte du terrain. Il se lit de bas en haut : de la fenêtre que le système d'exploitation nous donne, jusqu'au bouton sur lequel l'utilisateur clique. À la fin, vous saurez nommer chaque étage, dire de quoi il dépend, et — c'est le point le plus important du chapitre — vous comprendrez pourquoi l'étage du haut ne connaît pas l'étage du bas.

1.1 Vue d'ensemble

Vue d'ensemble — synthèse (voir Documentation, section « Raccordement au reste du moteur »)

```

1  Application (NKGuiDemo, NKCode, ...NK3DModeler)
2      |   cree une NkWindow, pompe les NKEvent -> remplit ctx.input
3      |   declare ses widgets entre BeginFrame / EndFrame
4      v
5  NKGui (NkGuiContext -> NkGuiDrawList : vtx + idx + cmds)
6      |   ne connaît AUCUN GPU
7      v
8  Backend (au choix)
9      | - NkGuiCanvasBackend (header-only, dans NKCanvas/UI/) -> NkIRenderer2D
10     | - NkGuiRHIBackend (Integrations/NKGui/) -> NKRHI
11     v
12  NKCanvas (NkBatchRenderer2D -> SubmitBatches) | NKRHI
13     v
14  GPU

```

Cinq étages, donc, mais qui ne s'empilent pas comme on l'imaginerait. Prenons-les un par un.

1.2 Étage 1 — la fenêtre : NkWindow

NkWindow est l'abstraction de fenêtre native. C'est le seul module de la pile qui parle au système d'exploitation : HWND sous Windows, Window X11 ou surface Wayland sous Linux, NSWindow sous macOS.

Son usage se réduit à trois gestes :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:243-251

```

1  NkWindow window;
2  NkWindowConfig cfg;
3  cfg.title = "NKGui - Demo Phase 2 (Coeur : drawlist + interaction)";
4  cfg.width = 1100;
5  cfg.height = 800;
6  cfg.centered = true;
7  cfg.resizable = true;
8  if (!window.Create(cfg))
9      return -1;

```

puis, dans la boucle, `window.IsOpen()` pour savoir si elle vit encore, et `window.SetCursor(...)` pour changer le pointeur. Le reste — taille courante, position, état maximisé — s'interroge par des accesseurs (`GetSize()` renvoie un `math::NkVec2u`).

Une option mérite d'être signalée dès maintenant parce qu'elle explique l'apparence des applications du dépôt :

Engine/NKEditorKit/src/NKEditorKit/NkEditorShell.cpp:234

```
1      wc.frame = false;           // SANS bordure OS -> barre de titre custom (VSCode)
```

Avec `frame = false`, le système ne dessine plus ni barre de titre ni bordure : l'application les dessine elle-même, comme le fait Visual Studio Code. C'est ce que fait `NKCode`. Notre application du chapitre 4 gardera la décoration native, qui est plus simple.

X11 définit None

Sous Linux, `NKWindow` tire `<X11/Xlib.h>`, qui contient `#define None 0L`. Or plusieurs énumérations de `NKGui` ont un membre nommé `None`. Le préprocesseur le transforme alors en `0L = 0` et le compilateur signale « expected identifier » sur des lignes parfaitement correctes, très loin de la vraie cause. La parade est en tête de l'en-tête parapluie de `NKGui` :

Kernel/Runtime/NKGui/src/NKGui/NKGui.h:18-27 (abrégé)

```
1  // Retire Les macros X11 (None, Bool, Status...) AVANT toute declaration de
2  // NKGui. Sur Linux, NKWindow tire <X11/Xlib.h>, dont le `#define None 0L`
3  // transforme ensuite `None = 0` - membre de plusieurs enumerations de NkGuiTypes
4  // - en `0L = 0`. Le compilateur signale alors « expected identifier » sur des
5  // lignes parfaitement correctes, tres loin de la vraie cause.
6  #include "NKPlatform/NkX11Clean.h"
```

La leçon générale : **incluez toujours `NKGui/NKGui.h` et jamais un en-tête interne de `NKGui`**. L'ordre d'inclusion de l'en-tête parapluie n'est pas arbitraire, c'est l'ordre des dépendances.

1.3 Étape 2 — les événements : `NKEvent`

`NKEvent` transforme les messages du système en **événements typés** : `NkWindowCloseEvent`, `NkMouseMoveEvent`, `NkMouseButtonPressEvent`, `NkKeyPressEvent`, `NkTextInputEvent`...

Le modèle est celui des *callbacks* enregistrés une fois pour toutes, puis d'une file qu'on vide à chaque image :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:326-331 (extrait)

```
1      auto &events = NkEvents();
2      bool running = true;
3      events.AddEventCallback<NkWindowCloseEvent>([&](NkWindowCloseEvent *) { running = false;
4      });
5      events.AddEventCallback<NkMouseMoveEvent>([&](NkMouseMoveEvent *e) {
6          ctx.input.mousePos = {static_cast<float32>(e->GetX()),
7                                static_cast<float32>(e->GetY())};
8      });
```

et, dans la boucle :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:438-440

```
1     while (NkEvent *ev = NkEvents().PollEvent()) {
2         (void)ev;
3     }
```

Ce `while` déroutant mérite un mot : **il ne fait rien du résultat**. Le travail a déjà été accompli par les *callbacks* enregistrés plus haut ; `PollEvent` sert uniquement à *pomper* la file jusqu'à ce qu'elle soit vide, ce qui déclenche au passage l'appel de tous les *callbacks*. On retrouve exactement le même idiome dans la boucle principale de `NKEditorKit` (`Engine/NKEditorKit/src/NKEditorKit/NkEditorShell.cpp:314`).

Une distinction fondamentale, qu'on retrouvera au chapitre 4 :

- `NkTextInputEvent` porte le **texte** tapé, déjà décodé par le système en *codepoint* Unicode — claviers AZERTY et méthodes de saisie asiatiques comprises. C'est lui qui alimente les champs de saisie ;
- `NkKeyPressEvent` / `NkKeyReleaseEvent` portent les **touches** : flèches, Suppr, Entrée, F2, Ctrl+quelque chose. Ce sont elles qui alimentent la navigation.

Confondre les deux est un classique : on branche les flèches sur le texte, et plus rien ne fonctionne. Le commentaire du pont de `NKEditorKit` le dit :

Engine/NKEditorKit/src/NKEditorKit/NkEditorShell.cpp:367-370

```
1 // Mappe une touche OS d'edition vers L'etat ENFONCE NKGui (press/release ->
2 // repetition au maintien geree par NKGui). Sans ce pont, aucune navigation
3 // clavier ni edition de texte dans Les panneaux.
```

1.4 Étage 3 — le GPU. Attention, il y a deux abstractions

C'est ici que le débutant se perd, et c'est normal : le moteur contient **deux** abstractions GPU distinctes, qui ne se superposent pas.

1.4.1 NKRHI — l'abstraction GPU bas niveau

`Kernel/Runtime/NKRHI` est l'abstraction « moderne » du GPU : périphérique (`NkIDevice`), tampons de commandes, passes de rendu, *pipelines*, descripteurs. Son arborescence de backends parle d'elle-même : `DirectX11`, `DirectX12`, `Metal`, `OpenGL`, `Software`, `Vulkan`. C'est sur elle que repose le rendu 3D (`NKRenderer`) et, par ricochet, les applications qui font de la 3D.

1.4.2 NKCanvas — sa propre abstraction, pour la 2D

`NKCanvas` **ne passe pas par NKRHI**. Il possède ses propres contextes graphiques et ses propres renderers 2D. La preuve tient en une ligne de son fichier de build :

Kernel/Runtime/NKCanvas/NKCanvas.jenga:65

```
1     _canvasDeps = [ "NKWindow", "NKFont", "NKImage", "NKStream", "NKTime", "NKGlad",
                    "NKThreading" ]
```

Pas de NKRHI dans la liste. L'arborescence `Kernel/Runtime/NKCanvas/src/NKCanvas/Backend/` contient DirectX/ (DX11 et DX12), Metal/, OpenGL/, Software/ et Vulkan/ : ce sont les implémentations *de NKCanvas*, pas celles de NKRHI.

Il y a donc, dans le dépôt, deux familles de backends portant les mêmes noms d'API graphique et ne se connaissant pas. Cette duplication est assumée et même documentée à l'endroit où elle pose problème :

`Engine/NKEditorKit/src/NKEditorKit/NkIEditorRenderer.h:28-31`

```
1 // Choix d'API graphique NEUTRE (decouple de NKCANVAS ET NKRHI : Leurs enums
2 // NkGraphicsApi se dupliquent dans Le namespace nkentseu et ne peuvent
3 // cohabiter dans un meme TU). Chaque impl mappe vers son propre enum.
4 enum class NkEditorGfxApi : uint8 { Auto = 0, OpenGL, Vulkan, DX11, DX12, Software };
```

Lisez bien : les deux énumérations `NkGraphicsApi` — celle de `NKCanvas` et celle de `NKRHI` — *ne peuvent pas cohabiter dans une même unité de traduction*. D'où le besoin d'un troisième énuméré, neutre, quand une couche doit pouvoir parler aux deux.

Une fenêtre = une pile

C'est la doctrine du dépôt, énoncée mot pour mot :

`Integrations/NKGui/NkEditorRHIRenderer.h:8-13 (extrait)`

```
1 * toute application moteur (NkAnimaEditor, Noguee, ...) qui veut la coquille
2 * NkEditorShell rendue sur NKRHI/NKRenderer (et PAS NKCanvas - regle
3 * « une fenetre = une pile ») injecte cette classe via
4 * NkEditorShellConfig::renderer.
```

Une fenêtre est rendue **soit** par `NKCanvas`, **soit** par `NKRHI` — jamais les deux. Le choix se fait une fois, à la création. Pour ce cours, ce sera toujours `NKCanvas`.

1.4.3 Les cinq backends 2D de NKCanvas, et leur état réel

Les cinq dossiers de backends ne fournissent pas tous un renderer 2D, et ceux qui le fournissent ne sont pas tous validés à l'exécution. Voici l'état, tel qu'il ressort des sources et des feuilles de route :

Backend	Renderer 2D	État à l'exécution
OpenGL	<code>NkOpenGLRenderer2D</code> (926 l.)	le seul validé
Vulkan	<code>NkVulkanRenderer2D</code> (1524 l.)	le code le plus complet; plantages signalés
DX12	<code>NkDX12Renderer2D</code> (1320 l.)	plantages signalés
DX11	<code>NkDX11Renderer2D</code> (764 l.)	écran vide/noir signalé
Software	<code>NkSoftwareRenderer2D</code> (674 l.)	écran vide/noir signalé
Metal	aucun	non implémenté

La ligne « Metal » n'est pas une supposition, elle est écrite dans la fabrique :

`Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2DFactory.cpp:86-89`

```
1 case NkGraphicsApi::NK_GFX_API_METAL:
2     NK_R2D_FACTORY_ERR("Metal 2D renderer not yet implemented");
```

```
3     return nullptr;
```

Quant aux autres verdicts, ils viennent de [Kernel/Runtime/NKCanvas/ROADMAP.md](#) (lignes 185-200 et 601-607), à la date du 30 mai 2026 : seul OpenGL était alors validé à l'exécution sur la démo Pong. **Ces états sont datés et méritent d'être revérifiés** avant de tirer une conclusion définitive : le dépôt bouge. Mais ils expliquent pourquoi, quand une application « ne montre rien », la première chose à essayer est de changer de backend.

Le choix, lui, est explicite. Soit on nomme l'API :

[Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:253-262](#)

```
1     NkContextDesc desc;
2     desc.api = NkGraphicsApi::NK_GFX_API_AUTO;
3     if (desc.api == NkGraphicsApi::NK_GFX_API_AUTO) {
4     #if defined(NKENTSEU_PLATFORM_WINDOWS)
5         desc.api = NkGraphicsApi::NK_GFX_API_DX11;
6     #else
7         desc.api = NkGraphicsApi::NK_GFX_API_OPENGL;
8     #endif
9     }
```

soit on laisse la fabrique essayer une liste, par `NkContextFactory::CreateWithFallback(window, preferenceOrder, count)`. L'ordre conseillé documenté en [.../Factory/NkContextFactory.h:46](#) est {DX12, DX11, Metal, Vulkan, OpenGL, Software}.

1.5 Étage 4 — NkCanvas, le rendu 2D

NkCanvas est décrit dans [Guides/05-NKCanvas.md](#) comme « la couche de rendu 2D conviviale de Nkntseu, l'équivalent de la partie *Graphics* de SFML ». Il fournit :

- un **contexte graphique** (`NkIGraphicsContext`) créé par une fabrique selon l'API choisie;
- une **cible de rendu** (`NkRenderTarget`), dont la spécialisation `NkRenderWindow` est celle qu'on utilise 99 % du temps : elle possède le cycle d'image;
- un **render 2D** (`NkIRenderer2D`, façade `NkRenderer2D`) qui offre les primitives : lignes, rectangles, cercles, triangles, et le point d'entrée bas niveau `DrawVertices`;
- des **ressources** : `NkTexture`, `NkFont`, `NkSprite`, `NkText`, `NkShader`;
- des **formes persistantes** façon SFML : `NkRectangleShape`, `NkCircleShape`, `NkLineShape`, `NkConvexShape`.

Le chapitre 2 lui est entièrement consacré. Retenez pour l'instant une seule chose : NkCanvas dessine des **triangles**. Tout le reste — cercles, lignes épaisses, texte — est fabriqué à partir de triangles par le module lui-même.

1.6 Étage 5 — NKGui, les widgets

NKGui est le framework d'interface : boutons, cases à cocher, sliders, champs de saisie, arbres, tables, menus, fenêtres flottantes, docking. Environ 7 780 lignes réparties sur 13 fichiers source, dont [Kernel/Runtime/NKGui/src/NKGui/Widgets/NKGuiWidgets.cpp](#) qui pèse à lui seul 4 973 lignes.

Son arborescence est petite et lisible — c'est par là qu'il faut commencer :

Kernel/Runtime/NKGui/ — arborescence

```
1 Kernel/Runtime/NKGui/
2   NKGui.jenga · ARCHITECTURE.md · README.md · ROADMAP.md
3   src/NKGui/
4     NKGui.h                <- en-tete PARAPLUIE (la seule chose a inclure)
5     NKGuiExport.h -> NKGuiApi.h <- macros d'export (defaut : statique)
6     Core/
7       NKGuiTypes.h        (333 l.) types de base : NKGuiId, enums, themes de flags
8       NKGuiInput.h        (145 l.) etat d'entree par frame (header-only, tout inline)
9       NKGuiDrawList.h/.cpp (101/342 l.) la LISTE DE COMMANDES de dessin
10      NKGuiFont.h/.cpp    ( 81/158 l.) police + atlas (wrapper NKFont)
11      NKGuiContext.h/.cpp (551/573 l.) LE contexte : etat complet de l'UI
12      Widgets/
13      NKGuiWidgets.h/.cpp (405/4973 l.) TOUS les widgets
```

1.6.1 Le point capital : NKGui ne connaît aucun GPU

Voici la ligne la plus importante de ce chapitre :

Kernel/Runtime/NKGui/NKGui.jenga:22-27

```
1 nkentseudependson(
2   ["NKPlatform", "NKCore", "NKMemory", "NKMath", "NKThreading",
3    "NKLogger", "NKContainers", "NKEvent", "NKFont", "NKImage"],
4   selfexport="NKGui",
5   extra_includes=["src"],
6 )
7 files(["src/**/*.cpp"])
```

Relisez la liste. Il n'y a ni **NKCanvas**, ni **NKRHI**, ni même **NKWindow**. NKGui ne connaît que les mathématiques, les conteneurs, la mémoire, les polices, les images et les événements. Il ne sait pas ce qu'est une texture GPU, ni un *shader*, ni une fenêtre.

Ce qu'il faut retenir

NKGui ne dépend ni de NKCanvas ni de NKRHI. Il ne dessine rien : il *produit une liste de commandes* — des sommets, des indices, des commandes de dessin — et laisse quelqu'un d'autre les envoyer au GPU. Ce quelqu'un s'appelle un **backend**, et le mariage se fait par un **pont**.

1.6.2 Ce que NKGui produit : NKGuiDrawList

La sortie de NKGui est purement géométrique :

Kernel/Runtime/NKGui/src/NKGui/Core/NKGuiDrawList.h:23-46 (abrégé)

```
1 struct NKGuiVertex {
2     NKVec2 pos;
3     NKVec2 uv;    ///< (0,0) = couleur unie (pas de texture)
4     uint32 col;
5 };
6
7 enum class NKGuiDrawCmdType : uint8 {
8     Triangles,          ///< triangles unis
9     TexturedTriangles   ///< triangles textures (texte/atlas/image)
10 };
```



```

11
12 struct NkGuiDrawCmd {
13     NkGuiDrawCmdType type = NkGuiDrawCmdType::Triangles;
14     uint32 idxOffset = 0;
15     uint32 idxCount = 0;
16     uint32 texId = 0;
17     NkRect clipRect = {0.f, 0.f, 1.0e9f, 1.0e9f}; // 1e9 = « pas de clip »
18 };

```

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.h:48-56 (abrégé)

```

1 struct NkGuiDrawList {
2     NkVector<NkGuiVertex> vtx;
3     NkVector<uint32> idx;
4     NkVector<NkGuiDrawCmd> cmds;
5     NkRect clipStack[32] = {};
6     int32 clipDepth = 0;
7     float32 thickScale = 1.f; ///< echelle DPI des epaisseurs (preservee par Reset)
8 };

```

Trois tableaux, donc : les sommets, les indices, et les commandes. Une *commande* décrit une tranche d'indices à dessiner avec une texture donnée et un rectangle de découpe donné. Notez le `texId` : c'est un simple entier, pas un pointeur vers une texture GPU. NKGui manipule des *identifiants*, à charge du backend de savoir à quoi ils correspondent.

Notez aussi la sentinelle `1e9` pour « pas de découpe ». Le backend la reconnaît par un test tolérant (`w < 1e8`), ce qui évite toute comparaison exacte de flottants.

1.7 Le pont : marier NKGui et NkCanvas

Puisque NKGui ne connaît pas NkCanvas, et que NkCanvas ne connaît pas NKGui, il faut un tiers qui connaisse les deux. Le dépôt en fournit deux, selon le public.

1.7.1 NkGuiCanvasBackend — le pont vers NkCanvas

Il vit dans `Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h` et il est **header-only**. Ce détail est délibéré et sa justification est en tête de fichier :

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:2-9

```

1 // NkGuiCanvasBackend.h — rend un nkgui::NkGuiDrawList via NKCanvas (NkIRenderer2D).
2 // Backend RÉUTILISABLE (Lib) : Le cœur NKGui reste render-agnostique ; ce pont
3 // traduit ses draw-lists en appels NkIRenderer2D. Gère l'atlas de police
4 // (gray8 -> RGBA blanc+alpha) et les images RGBA pour les commandes texturées.
5 //
6 // HEADER-ONLY : NKCanvas ne le compile pas (pas de .cpp) ; seuls les consommateurs
7 // (qui dépendent déjà de NKGui ET NKCanvas) l'incluent. Modelé sur NkUICanvasBackend.

```

Comprenez bien l'astuce. Ce fichier inclut `"NKGui/NKGui.h"`, donc il dépend de NKGui. Mais il vit dans NKCanvas, dont le `files([...])` ne prend que les `.cpp` : **personne ne le compile dans NKCanvas**. Il n'est compilé que dans l'application qui l'inclut — application qui, elle, dépend bien des deux modules. Résultat : NKCanvas n'acquiert aucune dépendance vers NKGui, et le pont existe quand même.

Son API tient en quatre méthodes :

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:34, 41, 94, 120

```
1 bool Init(nkentseu::renderer::NkIRenderer2D *renderer);
2 bool UploadFontGray8(uint32 texId, const uint8 *gray, int32 w, int32 h);
3 bool UploadImageRGBA(uint32 texId, const uint8 *rgba, int32 w, int32 h);
4 void Submit(const nkentseu::nkgui::NkGuiDrawList &dl, uint32 fbW, uint32 fbH);
```

C'est tout. Init lui donne le renderer 2D; les deux Upload enregistrent une texture sous un identifiant (un atlas de police, une image); Submit traduit une liste de commandes en appels NkIRenderer2D. Le chapitre 4 montrera la séquence complète.

1.7.2 NkGuiRHIBackend — le pont vers NKRHI

L'alternative vit dans `Integrations/NKGui/NkGuiRHIBackend.h`, compilée dans une bibliothèque statique NkGuiIntegration. Son API est le miroir de la précédente, avec un tampon de commandes en plus :

Integrations/NKGui/NkGuiRHIBackend.h:45-59

```
1 bool Init(NkIDevice* device, NkRenderPassHandle renderPass, NkGraphicsApi api);
2 void Destroy();
3 void Submit(NkICommandBuffer* cmd, const NkGuiDrawList& dl, uint32 fbW, uint32 fbH);
4 bool UploadTextureRGBA8(uint32 texId, const uint8* data, int32 width, int32 height);
5 bool UploadTextureGray8(uint32 texId, const uint8* data, int32 width, int32 height);
6 bool RegisterTexture(uint32 texId, NkTextureHandle texture);
7 bool HasTexture(uint32 texId) const noexcept;
```

La doctrine de répartition est écrite dans le fichier de build de l'intégration :

Integrations/NKGui/NKGuiIntegration.jenga:5-8

```
1 Rend une UI NKGui (nkgui::NkGuiDrawList) via NKRHI/NKRenderer, sans NKCanvas.
2 Sert aux applications 2D/3D (app d'animation, moteur de jeu). NKCanvas reste
3 le backend de l'IDE (NKCode).
```

Un point de conception intéressant de ce second pont : il enregistre aussi des textures *externes* (RegisterTexture). C'est ainsi qu'une application 3D affiche le rendu de sa scène dans un panneau de son interface : elle rend la scène dans une texture hors écran, l'enregistre sous un texId, et NKGui la dessine comme une image ordinaire.

1.7.3 Pourquoi cette séparation ?

On pourrait trouver la construction alambiquée. Elle rapporte trois choses concrètes :

1. **On peut changer de moteur de rendu sans toucher un seul widget.** L'IDE utilise NkCanvas, l'application d'animation utilise NKRHI, et le code des boutons est le même. Mieux : NKEditorKit rend cette substitution explicite, avec une interface NkIEditorRenderer injectable (`Engine/NKEditorKit/src/NKEditorKit/NkIEditorRenderer.h:7-15`).
2. **NKGui est testable sans GPU.** Une liste de commandes est une structure de données : on peut la produire, l'inspecter et la comparer sans jamais ouvrir de fenêtre.
3. **Les dépendances restent en arbre, pas en graphe.** NKGui ne connaît pas NkCanvas; NkCanvas ne connaît pas NKGui; l'application connaît les deux. Aucun cycle.

Il y a un prix, et il faut le connaître : **c'est à l'application de faire le raccordement**. Personne ne le fera à votre place. Si vous oubliez d'appeler `Submit`, rien ne s'affiche et aucun message d'erreur n'apparaît. Si vous oubliez d'uploader l'atlas de police, tous les textes sont invisibles et aucun message d'erreur n'apparaît non plus. Le chapitre 4 revient longuement sur ces silences.

1.8 Le chemin complet d'un clic, de bout en bout

Récapitulons en suivant un seul clic de souris, de l'appui jusqu'au pixel qui change de couleur.

1. L'utilisateur appuie. Le système envoie un message ; `NKWindow` le reçoit, `NKEvent` le transforme en `NkMouseButtonPressEvent`.
2. La boucle appelle `PollEvent()` ; le *callback* enregistré écrit `ctx.input.mouseDown[0] = true`.
3. L'application appelle `ctx.BeginFrame(dt)`. `NKGui` en déduit la *transition* : `mouseClicked[0] = mouseDown[0] && !mousePrev[0]`.
4. L'application appelle `Button(ctx, "OK")`. Le widget calcule son rectangle, demande son identité, teste le survol, voit le clic, et retourne `true` — au *relâchement*, comme nous le verrons au chapitre 3. Au passage, il écrit deux rectangles et six sommets dans la liste de commandes.
5. L'application appelle `ctx.EndFrame()`. `NKGui` fusionne les listes des fenêtres par ordre de profondeur, ferme les popups, remet à zéro la molette et le texte consommés.
6. L'application appelle `target->Clear()`, puis soumet les deux listes au pont : `ctx.dl` d'abord, `ctx.dlOverlay` ensuite, chacune par `backend.Submit(liste, w, h)`. Le pont convertit chaque sommet, résout chaque `texId` en `NkTexture*`, applique chaque rectangle de découpe et appelle `DrawVertices`.
7. `NkCanvas` accumule tout dans ses tampons, regroupe par texture, et à la fermeture de l'image envoie un appel de dessin par groupe.
8. L'application appelle `target->Display()` : l'image est présentée.

L'invariant d'ordre

événements → `ctx.BeginFrame(dt)` → widgets → `ctx.EndFrame()` → `Clear` → `Submit` ×2 → `Display`.

Cet ordre n'est pas négociable et le chapitre 3 expliquera pourquoi `BeginFrame` doit venir *après* le pompage des événements.

1.9 Deux mots sur les macros d'export

En lisant les en-têtes, vous rencontrerez partout des macros du genre `NKENTSEU_NKGUI_API` :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h (forme générale)

```
1 NKENTSEU_NKGUI_API void Text(NkGuiContext &ctx, const char *s) noexcept;
```

Par défaut, le module est construit en **statique** et cette macro est **vide** (`Kernel/Runtime/NKGui/src/NKGui/NkGuiApi.h:32-40`). Quand vous lisez une signature, effacez-la mentalement : il faut lire `void Text(NkGuiContext &ctx, const char *s) noexcept`. Il existe deux variantes : `NKENTSEU_NKGUI_CLASS_EXPORT` (classe entière) et `NKENTSEU_NKGUI_API_INLINE` (fonction inline exportée). Dans la suite de ce cours, nous les omettons pour la lisibilité.

1.10 Exercices

1 — Vérifier la carte soi-même

Ouvrez les trois fichiers de build suivants et relevez, pour chacun, la liste des modules dont il dépend : `Kernel/Runtime/NKGui/NKGui.jenga`, `Kernel/Runtime/NKCanvas/NKCanvas.jenga`, `Engine/NKEditorKit/NKEditorKit.jenga`. Dessinez le graphe. Vérifiez qu'il n'y a aucun cycle, et identifiez le seul module qui connaît à la fois NKGui et NKCanvas.

2 — Trouver le pont

Ouvrez `Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h` et comptez : combien de méthodes publiques ? Combien de lignes au total ? Comparez avec les 4 973 lignes de `NkGuiWidgets.cpp`. Que vous dit ce rapport sur la quantité de travail nécessaire pour porter NKGui vers un nouveau moteur de rendu ?

3 — Changer de backend à chaud

Reprenez la démo `NkCanvasDemo` du dossier `Sandbox`, qui accepte un argument de ligne de commande :

```
NkCanvasDemo.exe --backend=opengl  
NkCanvasDemo.exe --backend=dx11  
NkCanvasDemo.exe --backend=sw
```

Lancez-la avec chacun des backends et notez ce que vous observez. Recoupez avec le tableau d'état de la section 1.4 et avec [logs/app.log](#). Ce petit exercice vous évitera plus tard de chercher un bug dans votre code alors que le problème est dans le choix du backend.

NkCanvas : dessiner en deux dimensions

Nous descendons maintenant d'un étage pour examiner NkCanvas en profondeur. C'est le module qui, en dernier ressort, fait apparaître des pixels. Tout ce que fera NKGui au chapitre suivant finira ici.

Le plan de ce chapitre : d'abord *quelle est la forme de l'API* (deux niveaux, une cible de rendu, un repère); ensuite *ce qu'on peut dessiner* et surtout *ce qu'on ne peut pas*; puis *comment ça arrive au GPU* — c'est-à-dire le *batching*, qui est le vrai sujet du module; enfin un programme complet qui tourne.

2.1 Ce que NkCanvas est, en une phrase

Guides/05-NkCanvas.md:11-19 (extrait)

```
1 **NkCanvas** est la couche de **rendu 2D conviviale** de Nkntseu. C'est
2 l'équivalent de la partie *Graphics* de **SFML** : ... multi-backend ...
```

La comparaison avec SFML est juste et vous fera gagner du temps si vous connaissez cette bibliothèque : on retrouve la cible de rendu qui ouvre et ferme l'image, les formes persistantes qu'on positionne et qu'on dessine, le *drawable*, la vue. Ce qui change : le multi-backend, c'est-à-dire cinq API graphiques possibles derrière une seule et même interface.

Environ 23 400 lignes réparties sur 101 fichiers, organisées ainsi :

Dossier	Contenu
Core/	NkIGraphicsContext, NkContextDesc, NkGraphicsApi
Factory/	NkContextFactory — choisit et crée le contexte GPU
Backend/	OpenGL/, Vulkan/, DirectX/, Metal/, Software/
Renderer/Core/	NkIRenderer2D, NkRenderer2D, NkVertexArray, NkTransform
Renderer/Batch/	NkBatchRenderer2D — <i>le batcher</i> , base commune
Renderer/Resources/	NkTexture, NkFont, NkSprite/NkText, NkShader
Renderer/Shapes/	NkRectangleShape, NkCircleShape, NkLineShape...
Renderer/Targets/	NkRenderTarget, NkRenderWindow, NkRenderTexture
UI/	NkGuiCanvasBackend.h — le pont vu au chapitre 1
Compute/	NkIComputeContext — GPGPU, hors sujet ici

Tous les types vivent dans l'espace de noms nkntseu::renderer.

2.2 Les deux niveaux d'API

2.2.1 NkIRenderer2D : l'interface

`Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkIRenderer2D.h` déclare l'interface de rendu 2D. Il y a une implémentation par API graphique, mais elles héritent toutes d'une classe intermédiaire, `NkBatchRenderer2D`, qui fait l'essentiel du travail (section 2.9).

2.2.2 NkRenderer2D : la façade

`.../Renderer/Core/NkRenderer2D.h` est la classe concrète que manipule le code applicatif. Tout est transféré *inline* vers l'interface. Elle n'ajoute presque rien — sauf une surcharge de commodité — et son intérêt est ailleurs : elle documente la propriété.

`Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2D.h:31-35`

```
1  /// PROPRIETE
2  ///   NkRenderer2D ne POSSEDE pas son backend NkIRenderer2D* — Le NkRenderTarget
3  ///   qui le contient est propriétaire (typiquement via memory::NkUniquePtr).
4  ///   Le NkRenderer2D est juste une projection. La durée de vie suit celle du
5  ///   NkRenderTarget englobant.
```

avec, en clair, dans les membres privés (lignes 314-315) : `NkIRenderer2D *mBackend{nullptr};`
`///< non-owning et NkRenderTarget *mTarget{nullptr}; ///< non-owning.`

La façade meurt avec sa cible

Un `NkRenderer2D` récupéré par `target.GetRenderer2D()` n'est qu'une vue sur le renderer possédé par la cible. Le conserver au-delà de la durée de vie de la `NkRenderWindow` — dans un membre de classe, par exemple — donne un objet qui pointe vers de la mémoire libérée. C'est le même raisonnement, et le même risque, que pour `ctx.font` au chapitre 4.

2.3 La cible de rendu : NkRenderWindow

En pratique, on ne manipule ni le contexte graphique ni le renderer directement : on crée une `NkRenderWindow`, qui fait les deux et possède le cycle d'image.

`Applications/Sandbox/src/DemoNkentseu/NKCanvas/NKCanvasDemo.cpp:69-77`

```
1  // — 2. Cible de rendu NKCanvas (la voie haut-niveau, comme Pong) —————
2  NkContextDesc desc;
3  desc.api = ParseBackend(state.args);
4  NkRenderWindow target(window, desc);
5  if (!target.IsValid()) {
6      logger.Error("[nkcanvas] NkRenderWindow init FAILED");
7      window.Close();
8      return -2;
9  }
```

Une ligne de commentaire, en tête de cette même démo, résume la doctrine d'usage — il faut la retenir :

`Applications/Sandbox/src/DemoNkentseu/NKCanvas/NKCanvasDemo.cpp:6-7`

```
1  // NkRenderWindow possède le cycle de frame -> Clear() ouvre+efface, on dessine,
2  // Display() termine+présente (pas de Begin/End/SetView/SetViewport à la main).
```

Voici ce que ces deux méthodes font réellement :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Targets/NkRenderWindow.cpp:145-171 (abrégé)

```
1 void NkRenderWindow::Clear(const NkColor2D &color) {
2     if (!mRenderer) return;
3     // Pousser la couleur de clear au contexte AVANT Begin() : sur Vulkan/
4     // Software, BeginFrame() ouvre le render pass / clear le back-buffer avec
5     // cette couleur (sinon ils utilisent un gris en dur -> fond incorrect).
6     // No-op sur OpenGL/DX (ils clearent via le renderer->Clear ci-dessous).
7     if (mContext) mContext->SetClearColor(color.r/255.f, ...);
8     // Begin() ouvre la frame si pas déjà ouverte (idempotent par convention
9     // du backend) ; Clear() est appellable en milieu de frame.
10    if (!mFrameOpen) { mRenderer->Begin(); mFrameOpen = true; }
11    mRenderer->Clear(color);
12 }
13
14 void NkRenderWindow::Display() {
15     if (mRenderer && mFrameOpen) { mRenderer->End(); mFrameOpen = false; }
16     if (mContext) mContext->Present();
17 }
```

Autrement dit : `Clear` ouvre l'image (si elle ne l'est pas déjà) et l'efface; `Display` la ferme — ce qui vide les tampons accumulés vers le GPU — puis la présente à l'écran. Vous n'appellerez jamais `Begin()` ni `End()` vous-même.

Ne pas ré-initialiser le renderer

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Targets/NkRenderWindow.cpp:95-98

```
1 // NB : NE PAS appeler mRenderer->Initialize(mContext) ici !
2 // NkRenderer2DFactory::Create l'a déjà fait (cf. NkRenderer2DFactory.cpp:83).
3 // Double-init -> NkOpenGLRenderer2D::Initialize log "Already initialized"
4 // ERR et corrompt l'état (rectangles creux observés en runtime).
```

Symptôme mémorable : des « rectangles creux » — les formes apparaissent en contour, vides. Si vous voyez ça, cherchez une double initialisation ou une matrice dégénérée (section 2.8).

2.3.1 Le redimensionnement

`OnResize(w, h)` recrée la chaîne d'échange (*swapchain*) et recalcule la vue par défaut. Deux subtilités :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Targets/NkRenderWindow.cpp:353-358 (extrait)

```
1 // Si une frame est en cours, on la termine avant la recreation
2 // (sinon submit sur swapchain détruite -> UB sur Vulkan/DX12).
3 if (mFrameOpen && mRenderer) { mRenderer->End(); mFrameOpen = false; }
```

et, du côté du batcher, `OnResize` recalcule la vue *par défaut* mais **préserve une vue personnalisée** posée par `SetView` (`../Renderer/Batch/NkBatchRenderer2D.cpp:169-196`).

Détecter le resize sur la fenêtre, pas sur la swapchain

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:444-448

```
1 // Synchroniser la SWAPCHAIN à la taille de la FENÊTRE. target->GetSize()
2 // renvoie la taille de la swapchain (qui ne change QU'À OnResize) – s'en
3 // servir pour détecter le resize était faux : la swapchain restait figée à
4 // sa taille de création (rendu basse-résolution étiré). On pilote OnResize
5 // depuis la taille fenêtre (inclut la 1re frame → sync initiale).
```

Le code correct compare `target->GetWindow().GetSize()` — la fenêtre — à la dernière taille connue, et appelle `OnResize` si elle a changé. Se servir de `target->GetSize()` pour détecter le changement est une boucle logique : cette valeur ne change qu'après `OnResize`. Le symptôme est très reconnaissable : l'application semble marcher, mais tout est flou et étiré dès qu'on agrandit la fenêtre.

2.4 Le repère : Y vers le bas

L'origine est en haut à gauche et l'axe Y descend. C'est la convention de toutes les interfaces graphiques, et elle est confirmée dans le batcher :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp:183 (commentaire)

```
1 // Vue par défaut = écran plein-cadre (origine haut-gauche, Y-down)
```

La projection orthographique correspondante est construite ici :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2DTypes.cpp:85-87

```
1 const float32 a = 2.f / size.x;
2 const float32 b = -2.f / size.y; // Y-down (positive Y goes down in screen space)
```

Le signe moins sur b est toute l'histoire : c'est lui qui retourne l'axe.

Quelques lignes plus haut, une note sur la convention mémoire des matrices, qui est un piège classique dès qu'on vise plusieurs API :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2DTypes.cpp:81-84

```
1 // Column-major output (compatible with glUniformMatrix4fv with transpose=GL_FALSE,
2 // and with HLSL cbuffer when uploaded as-is since HLSL is row-major – we compensate
3 // by transposing at upload in the DX backends)
```

2.4.1 La caméra 2D

`NkView2D` (`.../Renderer/Core/NkRenderer2DTypes.h:41-47`) porte un `center`, une `size` et une `rotation`. On la pose par `SetView`. Deux conversions utiles existent pour passer du monde à l'écran et réciproquement : `MapPixelToCoords(NkVec2i)` et `MapCoordsToPixel(NkVec2f)` (`.../Renderer/Batch/NkBatchRenderer2D.cpp:538-550`).

Tant que vous dessinez une interface, vous n'y toucherez pas : la vue par défaut est exactement l'écran en pixels, ce qui est ce qu'on veut.

2.5 Les primitives disponibles

Toutes sont déclarées dans `NkIRenderer2D.h` et **toutes sont implémentées** dans `NkBatchRenderer2D`, donc disponibles sur les cinq backends actifs.

Signature (abrégée)	Ligne	Réalisation
<code>Draw(const NkSprite &)</code>	135	quad texturé
<code>Draw(const NkText &)</code>	138	délègue à <code>NkText::Draw</code>
<code>DrawPoint(pos, color, size)</code>	141	quad <code>size×size</code>
<code>DrawLine(a, b, color, thickness)</code>	143	quad orienté
<code>DrawRect(rect, color, outline, outlineColor)</code>	146	rempli + contour
<code>DrawFilledRect(rect, color)</code>	149	un quad
<code>DrawCircle(center, radius, color, segments, ...)</code>	151	polygone
<code>DrawFilledCircle(center, radius, color, segments)</code>	155	éventail de triangles
<code>DrawTriangle(a, b, c, ...)</code>	158	contour
<code>DrawFilledTriangle(a, b, c, color)</code>	161	un triangle
<code>DrawVertices(verts, n, indices, m, texture)</code>	192	le point d'entrée bas niveau

La dernière ligne est celle qui compte : `DrawVertices` est la primitive *universelle*, celle qu'utilise le pont `NKGui`, et à travers laquelle transite absolument toute l'interface d'une application.

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkIRenderer2D.h:192-193

```
1 virtual void DrawVertices(const NkVertex2D *vertices, uint32 vertexCount,
2                           const uint32 *indices, uint32 indexCount,
3                           const NkTexture *texture = nullptr) = 0;
```

2.5.1 Le sommet

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2DTypes.h:73-77

```
1 struct NkVertex2D { float32 x, y; float32 u, v; uint8 r, g, b, a; };
```

Vingt octets : deux flottants de position, deux de coordonnées de texture, quatre octets de couleur. C'est délibérément compact — on en pousse des dizaines de milliers par image.

2.5.2 Les primitives composites, réalisées dans l'en-tête

Trois fonctions sont implémentées *par défaut* directement dans `NkIRenderer2D.h`, donc sans que le moindre backend ait à s'en occuper :

- `DrawRectOutline(rect, color, thickness)` — quatre `DrawFilledRect` (lignes 167-175);
- `DrawCircleOutline(center, radius, color, thickness, segments)` — un anneau en segments de `DrawLine` (lignes 178-189);
- `DrawTexturedRect(rect, texture, color, uv)` — construit un quad `NkVertex2D[4]` et appelle `DrawVertices` (lignes 199-233).

C'est un patron de conception qu'on retrouvera : **on n'ajoute une méthode virtuelle que si elle a besoin du backend**. Tout ce qui se compose à partir des primitives existantes s'écrit une fois, dans l'interface.

2.5.3 Les types de primitives, et l'absence de QUADS

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2DTypes.h:60-70

```
1 // Pas de QUADS — Les quads sont decomposes en 2 triangles par Le batcher
2 // (compatible cross-API : Vulkan/DX/Metal n'ont pas de QUADS natif)
3 enum class NkPrimitiveType : uint8 {
4     NK_POINTS = 0, NK_LINES = 1, NK_LINE_STRIP = 2,
5     NK_TRIANGLES = 3, NK_TRIANGLE_STRIP = 4, NK_TRIANGLE_FAN = 5,
6 };
```

Le commentaire est une bonne illustration de ce que « multi-backend » veut dire en pratique : l'API commune ne peut offrir que l'intersection de ce que les API sous-jacentes savent faire. OpenGL avait des quads; Vulkan, Direct3D et Metal n'en ont pas. Donc l'abstraction n'en a pas non plus, et le batcher découpe.

2.6 Ce que NkCanvas n'a PAS

Cette section est courte mais elle vous fera gagner des heures. Voici ce que vous chercherez en vain :

- **Pas de DrawText.** Il n'existe aucune méthode qui prenne une chaîne et l'affiche. Le texte passe obligatoirement par le *drawable* `NkText` — `Draw(const NkText &)` — ou par le patron SFML `target.Draw(drawable, states)`.
- **Pas de DrawImage.** Même chose : on passe par `NkSprite`, ou par `DrawTexturedRect`, ou par `DrawVertices` avec une texture.
- **Pas de dégradé.** Aucune primitive `DrawGradient*`. Le seul moyen d'obtenir un dégradé est de donner des couleurs différentes aux sommets d'un même triangle et de laisser le GPU interpoler — donc via `DrawVertices` ou `NkVertexArray`.
- **Pas de coins arrondis.** Aucun `DrawRoundedRect`.

Ce qu'il faut retenir

Ces quatre absences ne sont pas des oublis : ce sont des choix de niveau. `NkCanvas` dessine des *formes géométriques*. Le dégradé, les coins arrondis et le texte mis en page sont des affaires de *présentation*, et c'est `NKGui` qui les fabrique — en triangulant lui-même des arcs de coin, en donnant quatre couleurs à un quad, en émettant un quad par glyphe. Nous verrons tout cela au chapitre 3.

2.7 Les couleurs

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkRenderer2DTypes.h:12

```
1 using NkColor2D = math::NkColor;
```

`math::NkColor` (`Kernel/Foundation/NKMath/src/NKMath/NkColor.h:1046`) est une classe **RGBA sur 8 bits**, quatre octets en tout (`uint8 r, g, b, a`). On l'écrit en agrégat : `NkColor2D{18, 20, 28, 255}`.

Il existe un pendant flottant, `NkColorF` (composantes dans $[0, 1]$, 16 octets), qui porte tout l'arsenal colorimétrique : HSL, HSV, LAB, LCH, XYZ, OKLab, OKLch, CMYK, hexadécimal, luminance WCAG, DeltaE et DeltaE2000, modes de fusion *Multiply*, *Screen*, *Overlay*, *SoftLight*, *HardLight*. Utile pour calculer une palette; inutile pour dessiner. Le rendu 2D n'emploie que les quatre `uint8`.

2.8 Les textures

`.../Renderer/Resources/NkTexture.h:36` — une ressource GPU, ni copiable ni déplaçable.

2.8.1 Créer et remplir

Méthode	Rôle
<code>Create(renderer, w, h, fillColor)</code>	texture vide d'une couleur
<code>LoadFromFile(renderer, path)</code>	depuis un fichier image
<code>LoadFromImage(renderer, image, area)</code>	depuis un <code>NkImage</code> déjà décodé
<code>LoadFromMemory(...)</code>	depuis un tampon d'octets
<code>Update(pixels, destX, destY)</code>	mise à jour <i>partielle</i>
<code>SetFilter / SetWrap / GenerateMipmap</code>	paramètres d'échantillonnage
<code>GetTexCoords(const NkRect2i &)</code>	sous-rectangle en pixels → UV dans $[0, 1]^2$

Les filtres et modes de répétition sont deux petites énumérations : `NkTextureFilter{NK_NEAREST, NK_LINEAR}` et `NkTextureWrap{NK_CLAMP, NK_REPEAT, NK_MIRROR_REPEAT}` (`NkTexture.h:22-33`).

SetFilter ne fait rien partout

Sur DX12 et Vulkan, il n'y a pas de *sampler* par texture : le *sampler* est global et immuable, donc `SetFilter` et `SetWrap` sont des *no-op*. C'est écrit dans les « pièges connus » du module (`Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkTexture.h:121`), avec deux autres limites de la même liste : `NkRenderTexture` n'est un vrai tampon hors-écran que sur OpenGL (ailleurs c'est un *stub*), et `NkShader::Compile()` ne fonctionne que sur OpenGL — il retourne *false* ailleurs.

2.8.2 La texture blanche de 1 pixel

Voici une idée simple et instructive :

`Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkTexture.h:121`

```
1 static NkTexture *NkTexture::GetWhiteTexture(NkIRenderer2D &);
```

Cette texture d'un pixel blanc est utilisée par *toutes* les primitives *non texturées* : `DrawLine`, `DrawFilledRect`, `DrawFilledCircle`, `DrawFilledTriangle` (`.../Renderer/Batch/NkBatchRenderer2D.cpp:394, 401, 426, 479`). Pourquoi? Pour que **tout passe par le même *shader* et le même *pipeline***. Il n'y a pas un chemin « uni » et un chemin « texturé » : il y a un seul chemin, texturé, et le uni est simplement du blanc multiplié par la couleur du sommet.

Ce qu'il faut retenir

« Tout est texturé ; le uni, c'est juste du blanc. » Cette astuce supprime un changement d'état GPU par forme, et surtout permet de regrouper dans un même appel de dessin des rectangles pleins et du texte — à condition qu'ils partagent la même texture, ce qui n'est pas le cas ici, mais nous verrons que le principe se généralise.

2.8.3 Les pixels conservés côté CPU

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkTexture.cpp:109-111

```
1 // (=> GetCPUPixels()==null). Le rasterizer Software echantillonne
2 // directement mCPUPixels ; sans cette copie -> textures/texte
3 // INVISIBLES en Software. Fix bug 2026-06-05.
```

Chaque texture garde donc une copie de ses pixels en mémoire centrale (`mCPUPixels`). C'est du gaspillage sur GPU, mais c'est la seule façon de faire fonctionner le rasteriseur logiciel. À connaître si vous vous demandez pourquoi une texture consomme deux fois sa taille.

2.9 Les polices et le texte

`NkFont`, dans `NkCanvas`, est un **enveloppeur GPU** au-dessus du module externe `NKFont`, qui fait la rasterisation (celle-ci a été sortie de `NkCanvas` le 28 mai 2026, cf. [.../Renderer/Resources/NkFont.h:56-63](#)).

- **Chargement** : `NkFont::LoadFromFile(NKIRenderer2D &renderer, const char *path)` ou `LoadFromMemory(...)`. L'implémentation lit les octets et *conserve la copie en mémoire* (`mFontData`, [NkFont.cpp:65-81](#)) : la fonte doit rester disponible pour rasteriser de nouvelles tailles.
- **Atlas** : une *page* par taille de caractère demandée (`struct Page`, [NkFont.h:102-107](#)), créée paresseusement par `GetOrCreatePage` ([NkFont.cpp:86-128](#)). Chaque page construit un atlas puis l'envoie dans une `NkTexture`.
- **Glyphes** : `GetGlyph(codepoint, characterSize, bold)` renvoie un `NkGlyph` = `textureRect` (pixels dans l'atlas), `bounds` (décalage et taille), `advance` (avance horizontale).

Deux limites à connaître, écrites dans le code :

- **Le crénage n'est pas exposé**. `GetKerning` retourne toujours `0.f` ([NkFont.cpp:163-167](#)) : « Le module `NKFont` n'expose pas le kerning par paire (integre dans l'advance des glyphes) ».
- **Le faux-gras n'existe pas**. Le paramètre `bold` est ignoré ([NkFont.cpp:133](#), paramètre nommé `/*bold*/`) : « charger une fonte bold dediee si necessaire ».

2.9.1 Comment un texte devient des triangles

Chaque glyphe devient un quad de quatre `NkVertex2D` ([.../Renderer/Resources/NkSprite.cpp:213-228](#)). Puis `NkText::Draw` applique la transformation — rotation, échelle, origine — côté CPU, sommet par sommet ([NkSprite.cpp:290-316](#)), et soumet le tout d'un coup :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkSprite.cpp:318

```
1 renderer.DrawVertices(tverts.Data(), ..., atlas);
```

Résultat : **un seul appel de dessin par texte**, quelle que soit sa longueur. C'est la logique du *batching*, appliquée localement.

Une fonction reste non implémentée et il vaut mieux le savoir avant de compter dessus :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkSprite.cpp:332-334

```
1 return 0; // TODO: binary search on glyph bounds
```

(NkText::FindCharacterPos — convertir un index de caractère en position à l'écran).

2.10 Les formes persistantes

Pour du contenu qui ne change pas à chaque image, NkCanvas offre des objets persistants façon SFML. Ils héritent tous de NkShape : public NkTransformable, public NkDrawable (.. ./Renderer/Shapes/NkShape.h:40), qui factorise le remplissage (triangulation en éventail), le contour et une texture optionnelle.

Classe	API propre
NkRectangleShape	explicit NkRectangleShape(NkVec2f size), SetSize/GetSize
NkCircleShape	explicit NkCircleShape(float32 r, uint32 segments = 32)
NkLineShape	NkLineShape(NkVec2f a, NkVec2f b, float32 thickness = 1.f)
NkConvexShape	SetPointCount(n), SetPoint(i, p)

Limite documentée : la triangulation en éventail n'est valable que pour des polygones **convexes** (NkShape.h:21-26, NkConvexShape.h:19-23). Un polygone concave dessiné ainsi produira des recouvrements aberrants.

Le tampon temporaire détruit avant l'envoi

C'est le meilleur exemple pédagogique du dépôt et il mérite d'être lu deux fois :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Shapes/NkShape.cpp:62-71

```
1 // On triangule en fan COTE NkShape (1 triangle = (p0, p_i, p_{i+1}))
2 // et on emet directement en NK_TRIANGLES. Pourquoi pas NK_TRIANGLE_FAN
3 // (n vertices, plus economique) ? Car NkRenderWindow::Draw raw fait
4 // une expansion TRIANGLE_FAN -> TRIANGLES dans un buffer scratch
5 // local au switch ; si le backend batch lazyement ce buffer (et ne
6 // recopie pas immediatement), le scratch est detruit avant flush ->
7 // donnees garbage / artefacts (visible : « rectangles creux »).
8 // En emettant TRIANGLES direct, le buffer `v` de NkShape reste vivant
9 // pendant tout l'appel target.Draw, et NkRenderWindow se contente
10 // d'un identity-indices submit deterministe.
```

La leçon dépasse largement ce cas : **dès qu'un système accumule des données pour les envoyer plus tard, tout tampon local devient suspect**. Vous retrouverez exactement ce raisonnement au chapitre 3 (une surcouche différée ne doit jamais détenir de pointeur vers la pile de l'appelant) et au chapitre 4 (la police doit survivre à toute la boucle).

Une matrice dégénérée aplatit tout

Corrigé depuis, mais très instructif :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NkTransformable.h:161-172

```
1  /// a01 = -sy*sin = -sys      <- fix 2026-05-30 : etait -syc (FAUX, degenerait)
2  /// a11 = sy*cos = sys       <- fix 2026-05-30 : etait sys (FAUX, degenerait)
3  ///
4  /// Bug d'origine : avec rotation=0, scale=(1,1) on obtenait sys=0
5  /// et syc=1, et le code mettait m01=-syc=-1, m11=sys=0 -> matrice
6  /// degeneratee transformant tout en ligne Y=ty. Conséquence :
7  /// paddles NkRectangleShape invisibles (vertices alignés en ligne),
8  /// meme symptome sur ball/dashes/circles. Les scores marchent car
9  /// DrawFilledRect ne passe PAS par NkTransformable.
```

Notez le détail final, qui est la clé du diagnostic : *ce qui marchait encore* indiquait précisément le chemin de code fautif. Quand une partie de l’affichage disparaît et pas une autre, cherchez ce qui les distingue.

2.11 Le batching : le vrai sujet du module

Nous arrivons au cœur de NkCanvas. Fichier central : `Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp` (552 lignes), classe de base de *tous* les backends actifs.

L’idée : au lieu d’envoyer chaque forme au GPU au moment où on la dessine — ce qui coûterait un appel de pilote par rectangle — on **accumule** tout dans des tableaux en mémoire centrale, et on n’envoie qu’à la fin, en gros paquets.

2.11.1 L’accumulation

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.h:157-159

```
1  NkVector<NkVertex2D> mVertices;
2  NkVector<uint32>      mIndices;
3  NkVector<NkBatchGroup> mGroups;
```

Un NkBatchGroup est un futur appel de dessin :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.h:23-28 (abrégé)

```
1  struct NkBatchGroup {
2      const NkTexture *texture;
3      NkBlendMode blendMode;
4      uint32 indexStart;
5      uint32 indexCount;
6  };
```

Et voici la règle qui gouverne la performance de toute votre interface :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp:207-229 (principe)

```
1  // EnsureGroup(tex, blend) ferme le groupe courant et en ouvre un nouveau
2  // DES QUE la texture ou le mode de fusion change (et incremente mStats.textureSwap).
```

La règle d'or du 2D

Un appel de dessin par groupe; un nouveau groupe à chaque changement de texture ou de mode de fusion. Donc : moins on change de texture, moins il y a d'appels de dessin. C'est la raison d'être des *atlas* — une seule texture qui contient tous les glyphes, ou toutes les icônes — et c'est pourquoi alterner « une icône, un texte, une icône, un texte » coûte quatre appels là où « deux icônes puis deux textes » n'en coûte que deux.

2.11.2 La capacité, et l'histoire qui va avec

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.h:33-40

```
1 // Maximum vertices / indices before an automatic flush.
2 // 262144 (~5 Mo GPU) : une UI dense (IDE plein écran : arbre + onglets +
3 // icônes) dépasse 65536 sommets ENTRE DEUX CLIPS — La draw-list NkGui
4 // arrive alors en UNE commande plus grosse que l'ancien buffer, et
5 // l'upload écrivait HORS du buffer GPU (crash memcpy). Les 3 backends
6 // (DX11/DX12/GL) créent leurs buffers avec ces constantes.
7 static constexpr uint32 kMaxVertices = 262144;
8 static constexpr uint32 kMaxIndices = kMaxVertices * 6 / 4;
```

Deux cent soixante-deux mille sommets, environ 5 Mo côté GPU. Le commentaire raconte pourquoi : une interface dense produit, *entre deux changements de découpe*, une seule commande de plus de 65 536 sommets. L'ancien tampon faisait exactement cette taille; l'envoi écrivait donc au-delà, et le programme plantait dans un memcpy.

Quand l'accumulation risque de déborder, un vidage est déclenché automatiquement :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp:235-237

```
1 if (mVertices.Size() + 4 > kMaxVertices || mIndices.Size() + 6 > kMaxIndices) {
2     Flush();
3 }
```

2.11.3 La soumission : Flush()

Flush() fait quatre choses (NkBatchRenderer2D.cpp:56-98) : fermer le dernier groupe et compacter les groupes vides; appliquer le découpage courant; vérifier qu'on ne dépasse pas la capacité; envoyer.

Le troisième point est une garde défensive exemplaire :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp:80-88

```
1 // GARDE-FOU : ne JAMAIS soumettre plus que la capacité des buffers GPU
2 // (l'upload écrivait hors buffer -> crash). Troncature (indices en
3 // multiple de 3) : dégradation visuelle plutôt que corruption mémoire.
4 uint32 vSub = mVertices.Size(), iSub = mIndices.Size();
5 if (vSub > kMaxVertices) vSub = kMaxVertices;
6 if (iSub > kMaxIndices) iSub = kMaxIndices - (kMaxIndices % 3);
7 SubmitBatches(mGroups.Data(), validCount, mVertices.Data(), vSub, mIndices.Data(), iSub);
```

Ce qu'il faut retenir

« Dégradation visuelle plutôt que corruption mémoire. » Cette phrase est un principe de conception à part entière : quand on ne peut pas satisfaire une demande, on rend un résultat incomplet mais *sûr*, jamais un résultat qui détruit la mémoire. Notez aussi le % 3 : tronquer des indices de triangles à un multiple de trois, faute de quoi le dernier triangle serait incomplet.

2.11.4 Le vrai appel de dessin

SubmitBatches est la seule méthode virtuelle pure que chaque backend doit fournir. Voici la version OpenGL, la plus lisible :

Kernel/Runtime/NKCanvas/src/NKCanvas/Backend/OpenGL/NkOpenGLRenderer2D.cpp:591-613

```
1 void NkOpenGLRenderer2D::SubmitBatches(const NkBatchGroup *groups, uint32 groupCount,
2                                       const NkVertex2D *verts, uint32 vCount,
3                                       const uint32 *idx, uint32 iCount) {
4     glBindVertexArray((GLuint)mVAO);
5     glBindBuffer(GL_ARRAY_BUFFER, (GLuint)mVBO);
6     glBufferSubData(GL_ARRAY_BUFFER, 0, (GLsizeiptr)(vCount * sizeof(NkVertex2D)), verts);
7     glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, (GLuint)mEBO);
8     glBufferSubData(GL_ELEMENT_ARRAY_BUFFER, 0, (GLsizeiptr)(iCount * sizeof(uint32)), idx);
9     // Issue one draw call per group
10    for (uint32 i = 0; i < groupCount; ++i) {
11        const auto &g = groups[i];
12        ApplyBlendMode(g.blendMode);
13        BindTexture(g.texture);
14        glDrawElements(GL_TRIANGLES, (GLsizei)g.indexCount, GL_UNSIGNED_INT,
15                      (void *) (uintptr_t)(g.indexStart * sizeof(uint32)));
16    }
17 }
```

Tout le lot est envoyé en **un seul** glBufferSubData par vidage, puis **un** glDrawElements par groupe. Voilà, très concrètement, ce que « batching » veut dire.

Begin() n'est pas ré-entrant

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp:27-31

```
1 bool NkBatchRenderer2D::Begin() {
2     if (mInFrame) {
3         logger.Warnf("[NkBatch] Begin() called twice");
4         return false;
5     }
6 }
```

Si ce message apparaît dans `logs/app.log`, c'est que quelque chose ouvre une image sans la fermer. En pratique, cela signifie qu'on a appelé Begin() à la main alors que NkRenderWindow::Clear() le fait déjà.

2.12 Le découpage (scissor) et sa pile

Le découpage restreint le rendu à un rectangle. C'est indispensable pour les panneaux défilables : un élément qui dépasse doit être coupé net, pas dessiné par-dessus le voisin.

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NklRenderer2D.h:105-121

```
1 virtual void SetClip(const NkRect2i &rectPixels) { (void)rectPixels; }
2 virtual void PopClip() {}
3 virtual void ResetClip() {}
4 virtual bool HasClip() const { return false; }
5 virtual NkRect2i GetClip() const { return NkRect2i{}; }
```

Le contrat est écrit juste au-dessus :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Core/NklRenderer2D.h:98-104

```
1 // Pile : SetClip empile Le rect (intersecte avec Le clip courant) ; PopClip
2 // depile. ResetClip vide la pile. Implementation backend : glScissor (GL),
3 // VkRect2D dynamique (Vulkan), RSetScissorRects (DX11/12), clamp CPU (Software).
```

Deux propriétés à retenir : **c'est une pile**, et **l'empilement intersecte**. Un enfant ne peut donc jamais dessiner en dehors de son parent, quoi qu'il demande.

2.12.1 Changer de découpe force un vidage

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Batch/NkBatchRenderer2D.cpp:120-126

```
1 void NkBatchRenderer2D::SetClip(const NkRect2i &rect) {
2     Flush(); // committe la geometrie en cours avec Le clip actuel
3     const NkRect2i clip = mHasClip ? NkIntersectClip(mClipRect, rect) : rect;
4     mClipStack.PushBack(clip);
5     mClipRect = clip;
6     mHasClip = true;
7 }
```

C'est logique — le *scissor* est un état GPU, pas un attribut de sommet : tout ce qui est accumulé avant le changement doit partir avec l'ancien rectangle. Mais c'est aussi une conséquence de performance directe :

Ce qu'il faut retenir

Chaque **SetClip** et chaque **PopClip** coûte un vidage, donc au moins un appel de dessin. Une interface très découpée — un **PushClipRect** par ligne de liste, par exemple — multiplie les appels de dessin. Le bon réflexe est de poser *une* découpe pour toute une zone, puis d'écarter soi-même les éléments hors vue avant de les émettre (nous verrons ce « culling manuel » au chapitre 3).

2.12.2 Le retournement d'axe sous OpenGL

Kernel/Runtime/NKCanvas/src/NKCanvas/Backend/OpenGL/NkOpenGLRenderer2D.cpp:532-546 (extrait)

```
1 // Clip en pixels, origine haut-gauche -> glScissor a l'origine bas-gauche :
2 // on inverse Y avec la hauteur de la surface (= mViewport.height, viewport
3 // plein ecran a top=0).
4 const int32 y = mViewport.height - rect.y - h; // flip Y
5 glEnable(GL_SCISSOR_TEST);
6 glScissor((GLint)x, (GLint)y, (GLsizei)w, (GLsizei)h);
```

OpenGL place son origine en bas à gauche. Le module a choisi la convention « haut-gauche » pour toute son API et compense dans le seul backend concerné. C'est exactement le bon endroit pour faire ce genre de correction : *une fois, au plus près de l'API*, et jamais dans le code applicatif.

L'état OpenGL survit d'une image à l'autre

Kernel/Runtime/NKCanvas/src/NKCanvas/Backend/OpenGL/NkOpenGLRenderer2D.cpp:524-526

```
1 // Chaque frame démarre sans scissor (sinon un Clear serait clipé par le
2 // scissor laissé par la frame précédente, l'état GL étant persistant)
```

Symptôme, si on l'oublie : l'écran ne s'efface que partiellement, et une zone garde l'image de la frame précédente comme une traînée. Les machines à états globales — OpenGL en est l'archétype — demandent qu'on remette explicitement tout à zéro au début de chaque image.

2.13 La surface réellement utilisée en pratique

Nous venons de parcourir une API large : des dizaines de méthodes, des formes, des sprites, des vues, des shaders. Voici maintenant un fait qui remet tout en perspective.

L'IDE NKCode — l'application la plus grosse du dépôt, plus de trente mille lignes d'interface — n'utilise de NkCanvas que **sept méthodes** :

Méthode	Ligne	Appelée par
bool Begin()	59	NkRenderWindow::Clear
void End()	60	NkRenderWindow::Display
void Clear(const NkColor2D &)	73	le fond de l'image
void OnResize(uint32, uint32)	89	redimensionnement
void SetClip(const NkRect2i &)	105	découpage
void PopClip()	109	dépile le découpage
void DrawVertices(...)	192	tout le dessin

plus, côté ressources, NkTexture::Create, NkTexture::Update et NkTexture::SetFilter.

Voilà tout. NKCode ne dessine ni sprite, ni forme, ni texte NkCanvas : il envoie des triangles pré-calculés par NKGui. Une recherche exhaustive des `#include "NKCanvas/..."` dans tout `Applications/NKCode/src/` ne renvoie **aucun** résultat — le module n'apparaît que dans les dépendances d'édition de liens.

Ce qu'il faut retenir

Pour écrire une interface, la surface utile de NkCanvas se réduit à : Clear / Display au niveau de la fenêtre, SetClip / PopClip pour découper, DrawVertices pour dessiner, et NkTexture pour les atlas. Le reste — formes, sprites, vues, shaders — sert aux jeux et aux démos, pas aux interfaces.

2.14 Un programme complet

Assemblons. Le programme ci-dessous est la démo NkCanvas du dépôt, prise telle quelle : une fenêtre, une balle qui rebondit, un rectangle qui pulse, un triangle qui tourne, un éventail

de lignes. Aucun NKGui, aucune police.

2.14.1 L'ossature

Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp:11-33 (extrait)

```
1  #include "NKWindow/NKWindow.h"
2  #include "NKWindow/NKMain.h"
3  #include "NKWindow/Core/NKWindowConfig.h"
4  #include "NKEvent/NKWindowEvent.h"
5  #include "NKTime/NKTime.h"
6  #include "NKLogger/NKLog.h"
7  #include "NKMATH/NKColor.h"
8  #include "NKMATH/NKMath.h" // math::NkCos / NkSin
9
10 #include "NKCanvas/Core/NKContextDesc.h"
11 #include "NKCanvas/Core/NKGraphicsApi.h"
12 #include "NKCanvas/Renderer/Targets/NKRenderWindow.h"
13 #include "NKCanvas/Renderer/Core/NKRenderer2D.h"
14
15 using namespace nkentseu;
16 using namespace nkentseu::renderer;
17
18 NKENTSEU_DEFINE_APP_DATA([]() {
19     NkAppData d{};
20     d.appName = "NkCanvas Demo";
21     d.appVersion = "1.0.0";
22     return d;
23 })());
```

2.14.2 Fenêtre, cible, état

Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp:57-93 (abrégé)

```
1  int nkmain(const NkEntryState &state) {
2      // — 1. Fenetre —————
3      NkWindowConfig cfg;
4      cfg.title = "NkCanvas Demo (NkCanvas seul, style SFML/SDL)";
5      cfg.width = 900;
6      cfg.height = 600;
7      cfg.centered = true;
8      cfg.resizable = true;
9      NkWindow window;
10     if (!window.Create(cfg)) {
11         logger.Error("[nkcanvas] window failed");
12         return -1;
13     }
14
15     // — 2. Cible de rendu NKCanvas —————
16     NkContextDesc desc;
17     desc.api = ParseBackend(state.args);
18     NkRenderWindow target(window, desc);
19     if (!target.IsValid()) { window.Close(); return -2; }
20     logger.Info("[nkcanvas] backend = %s", NkGraphicsApiName(desc.api));
21
22     // — 3. Etat de la scene —————
23     bool running = true;
24     auto &events = NkEvents();
```

```

25     events.AddEventCallback<NkWindowCloseEvent>([&](NkWindowCloseEvent *) { running = false;
    });
26
27     NkClock clock;
28     uint32 lastW = 0, lastH = 0;
29     float32 t = 0.f;
30
31     math::NkVec2f ball{220.f, 200.f};
32     math::NkVec2f vel{260.f, 215.f}; // pixels / seconde
33     const float32 R = 42.f;

```

2.14.3 La boucle

Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp:96-118 (abrégé)

```

1     while (running && window.IsOpen()) {
2         float32 dt = clock.Tick().delta;
3         if (dt > 0.1f)
4             dt = 1.0f / 60.0f;
5         t += dt;
6         while (NkEvent *ev = events.PollEvent()) {
7             (void)ev;
8         }
9         if (!running)
10            break;
11
12         // Resize : la cible suit la fenetre (vue par défaut = ecran) -> pas de clip.
13         const math::NkVec2u sz = target.GetSize();
14         if (sz.x != lastW || sz.y != lastH) {
15             if (lastW != 0 && sz.x > 0 && sz.y > 0)
16                 target.OnResize(sz.x, sz.y);
17             lastW = sz.x;
18             lastH = sz.y;
19         }
20         const float32 W = static_cast<float32>(sz.x), H = static_cast<float32>(sz.y);

```

2.14.4 Le rendu

Applications/Sandbox/src/DemoNkentseu/NkCanvas/NkCanvasDemo.cpp:139-170

```

1     // — 5. Rendu : NkRenderer2D direct (Clear -> formes -> Display) —————
2     target.Clear(NkColor2D{18, 20, 28, 255});
3     NkRenderer2D &r = target.GetRenderer2D();
4
5     // Rectangle "pulsant" au centre (remplissage + contour).
6     const float32 hp = 60.f + 28.f * math::NkSin(t * 2.f);
7     const NkRect2f box{cx - hp, cy - hp, hp * 2.f, hp * 2.f};
8     r.DrawFilledRect(box, NkColor2D{52, 84, 150, 255});
9     r.DrawRectOutline(box, NkColor2D{140, 180, 255, 255}, 2.f);
10
11     // Triangle qui tourne autour du centre.
12     const float32 tr = 95.f;
13     const math::NkVec2f p0{cx + tr * math::NkCos(t), cy + tr * math::NkSin(t)};
14     const math::NkVec2f p1{cx + tr * math::NkCos(t + 2.094f), cy + tr * math::NkSin(t +
15         2.094f)};
16     const math::NkVec2f p2{cx + tr * math::NkCos(t + 4.188f), cy + tr * math::NkSin(t +
17         4.188f)};

```

```

16     r.DrawFilledTriangle(p0, p1, p2, NkColor2D{255, 180, 60, 200});
17
18     // Eventail de Lignes depuis le coin haut-gauche.
19     for (int32 i = 0; i < 7; ++i) {
20         const float32 a = t * 0.6f + static_cast<float32>(i) * 0.22f;
21         r.DrawLine(math::NkVec2f{0.f, 0.f},
22                 math::NkVec2f{110.f + 90.f * math::NkCos(a), 110.f + 90.f *
math::NkSin(a)}},
23                 NkColor2D{80, 200, 160, 180}, 1.5f);
24     }
25
26     // La balle, par-dessus (remplissage + contour clair).
27     r.DrawFilledCircle(ball, R, NkColor2D{255, 110, 80, 255}, 48);
28     r.DrawCircleOutline(ball, R, NkColor2D{255, 220, 200, 255}, 2.f, 48);
29
30     target.Display();
31 }

```

2.14.5 Ce que ce programme raconte

Quelques observations qui valent pour toute application NkCanvas :

- **L'ordre de dessin est l'ordre de recouvrement.** La balle est dessinée en dernier, elle passe donc devant. Il n'y a ni tampon de profondeur, ni tri : ce qui est émis après recouvre.
- **Le temps est explicite.** `clock.Tick().delta` donne le temps écoulé depuis l'image précédente; toute animation se calcule en « unités par seconde » multipliées par `dt`. Notez la borne `if (dt > 0.1f) dt = 1/60`, qui empêche un bond énorme quand la fenêtre a été bloquée (déplacement, mise en veille).
- **Aucune ressource n'est allouée dans la boucle.** La position de la balle est une variable locale à `nkmain`; il n'y a ni `new`, ni conteneur créé par image.
- **On ne touche jamais à Begin/End.** `Clear` et `Display` suffisent.

2.15 Récapitulatif des pièges du chapitre

1. Ne jamais appeler `Initialize` sur un renderer déjà initialisé par la fabrique (*rectangles creux*).
2. Détecter le redimensionnement sur la taille de la *fenêtre*, jamais sur celle de la *swapchain* (rendu basse résolution étiré).
3. Terminer l'image avant de recréer la *swapchain* (comportement indéfini sur Vulkan et DX12).
4. Ne jamais conserver un tampon local vers des données que le batcher lira plus tard (données parasites, formes creuses).
5. Chaque `SetClip` / `PopClip` coûte un vidage : découper largement, et filtrer soi-même le hors-vue.
6. Sous OpenGL, l'état est persistant entre les images : le *scissor* doit être explicitement désactivé au début de chaque image.
7. `SetFilter` / `SetWrap` sont sans effet sur DX12 et Vulkan; `NkShader::Compile` ne fonctionne que sur OpenGL; `NkRenderTexture` n'est réel que sur OpenGL.
8. `NkFont::GetKerning` retourne toujours zéro et le paramètre `bold` est ignoré.

2.16 Exercices

1 — Une horloge

Partez de `NkCanvasDemo`. Remplacez la scène par une horloge analogique : un `DrawCircleOutline` pour le cadran, douze `DrawLine` courts pour les graduations, trois `DrawLine` d'épaisseurs différentes pour les aiguilles. Faites tourner les aiguilles avec `t`. Contrainte : *aucun* allocation dans la boucle.

2 — Compter les appels de dessin

Le batcher tient des statistiques (`mStats`, dont `textureSwap`). Trouvez comment y accéder depuis l'application, puis affichez leur valeur dans le journal une fois par seconde. Dessinez ensuite cent rectangles pleins : combien d'appels de dessin ? Ajoutez maintenant, *entre chaque rectangle*, une forme utilisant une autre texture. Que devient le compte ? Concluez.

3 — Un dégradé sans primitive de dégradé

`NkCanvas` n'a pas de `DrawGradient`. Construisez-en un : remplissez un tableau de quatre `NkVertex2D` formant un rectangle, avec quatre couleurs différentes, un tableau de six indices, et appelez `DrawVertices` avec `texture = nullptr`. Vérifiez que le GPU interpole. Puis expliquez, en vous appuyant sur la section « la texture blanche de 1 pixel », pourquoi passer `nullptr` fonctionne.

4 — Le découpage, et son coût

Dessinez une grille de 20×20 petits carrés. Version A : un seul `SetClip` autour de toute la grille. Version B : un `SetClip` et un `PopClip` autour de *chaque* carré. Comparez les statistiques du batcher et le temps par image. Vous devriez mesurer, à l'écran, la règle énoncée à la section 2.10.

NKGui : l'interface immédiate

Voici le cœur du cours. NKGui est le framework qui vous donnera des boutons, des cases à cocher, des champs de saisie, des arbres, des tables, des menus et des fenêtres ancrables. Mais avant la liste des widgets, il faut comprendre *comment il pense* — parce qu'il ne pense pas comme la plupart des frameworks d'interface, et que tout le reste en découle.

3.1 L'idée : on ne crée rien, on redéclare tout

3.1.1 Le mode immédiat

Dans un framework classique, un bouton est un *objet*. On le crée une fois, on lui donne un texte, une position, on lui branche un gestionnaire de clic, et il vit jusqu'à ce qu'on le détruise. L'interface est un arbre d'objets qu'on modifie.

Dans NKGui, il n'y a **aucun objet bouton**. Il y a un *appel de fonction*, qu'on refait à chaque image :

Forme canonique d'une frame NKGui

```
1 ctx.BeginFrame(dt);
2     Text(ctx, "Bonjour");
3     if (Button(ctx, "Cliquez-moi")) {
4         /* reaction au clic */
5     }
6 ctx.EndFrame();
```

Le bouton n'existe que le temps de l'appel. À l'image suivante, on le redéclare. S'il ne faut plus l'afficher, on ne l'appelle pas — il n'y a rien à détruire, rien à cacher, rien à retirer d'un parent.

Ce qu'il faut retenir

NKGui est une interface immédiate. Les widgets ne sont pas des objets qu'on crée, mais des appels qu'on refait à chaque image. Ce qui survit d'une image à l'autre, ce n'est pas le widget : c'est un *état*, rangé dans le contexte et retrouvé par *identifiant*. Toute la mécanique du chapitre découle de cette phrase.

3.1.2 Ce que ce choix vous épargne

Le gain principal est qu'il n'y a **aucune synchronisation à écrire**. Considérez une case à cocher qui reflète un booléen de votre application. Dans un modèle à objets, il faut : écrire le booléen vers la case quand il change, écrire la case vers le booléen quand l'utilisateur clique, et se souvenir de ne pas boucler. Dans NKGui :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:45

```
1 NKENTSEU_NKGUI_API bool Checkbox(NkGuiContext &ctx, const char *label, bool &value) noexcept;
```

La valeur est passée *par référence*. Le widget la lit pour se dessiner et l'écrit si l'utilisateur clique. Il n'y a plus qu'une seule copie de la vérité, et c'est la vôtre. Ce point est fondamental et mérite d'être formulé explicitement :

Qui possède quoi

Les **valeurs métier** — le booléen d'une case, le flottant d'un *slider*, le tampon de caractères d'un champ de saisie — appartiennent à **votre application**. NKGui ne les stocke pas. NKGui ne stocke que l'**état d'interaction** : ce nœud d'arbre est-il ouvert ? où est le curseur de saisie ? quelle est la largeur des colonnes de cette table ? quel onglet est actif ? Ces informations-là n'ont pas de sens pour votre modèle de données, et c'est pour cela que le framework s'en charge.

3.1.3 Une note d'honnêteté sur la documentation du module

La documentation interne de NKGui annonce deux paradigmes :

Kernel/Runtime/NKGui/ARCHITECTURE.md:5-6

```
1 Deux paradigmes : immédiat (façon ImGui) et retenu (façon Qt/Unity).
```

Le **mode retenu n'existe pas dans le code**. Il est placé en « Phase 6 » ([ARCHITECTURE.md:191](#)) et l'arborescence réelle du module ne contient ni dossier Retained/, ni Layout/, ni Window/, ni Dock/ : tout ce qui est implémenté tient dans Core/ et Widgets/.

Symétriquement, [README.md:14](#) et [ARCHITECTURE.md:185](#) annoncent une « Phase 1 — squelette », alors que [NkGuiWidgets.cpp](#) fait 4 973 lignes et couvre les tables, le docking, le sélecteur de couleur et les menus imbriqués. **Les fichiers d'état sont en retard sur le code. Fiez-vous au code.** Ce cours ne documente que ce qui existe vraiment.

3.2 L'identité d'un widget

Puisqu'un widget n'est pas un objet, comment le framework sait-il, d'une image à l'autre, qu'il s'agit du *même* bouton ? Par son identifiant.

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiTypes.h:26-46 (abrégé)

```
1 using NkGuiId = uint32;  
2 static constexpr NkGuiId NKGUI_ID_NONE = 0;  
3  
4 NKENTSEU_NKGUI_API_INLINE NkGuiId NkGuiHashStr(const char *s, NkGuiId seed = 2166136261u)  
5     noexcept {  
6     NkGuiId h = seed;  
7     while (s && *s) { h ^= static_cast<uint8>(*s++); h *= 16777619u; }  
8     return h ? h : 1u; // jamais 0 (0 = « aucun »)  
9 }  
10 NKENTSEU_NKGUI_API_INLINE NkGuiId NkGuiHashPtr(const void *p, NkGuiId seed = 2166136261u)  
11     noexcept { ... }
```


C'est un **hachage FNV-1a sur 32 bits** du libellé. Deux constantes, une boucle : le décalage initial 2166136261 et le multiplicateur 16777619, qui sont les valeurs canoniques de FNV-1a 32 bits.

Notez le détail final : `return h ? h : 1u`. La valeur zéro est *réservée* — elle signifie « aucun widget » (`NKGUI_ID_NONE`). Si le hachage tombe sur zéro, on renvoie 1. Un identifiant valide n'est donc jamais nul, ce qui permet d'utiliser zéro comme sentinelle sans ambiguïté.

3.2.1 La pile d'identifiants

Deux boutons « Supprimer » dans deux panneaux différents auraient le même libellé, donc le même hachage, donc la même identité — et cliquer sur l'un activerait l'autre. La parade est une *pile de portées* :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:144-164

```
1 void NkGuiContext::PushId(const char *s) noexcept {
2     const NkGuiId seed = idDepth > 0 ? idStack[idDepth - 1] : 2166136261u;
3     if (idDepth < 32) idStack[idDepth++] = NkGuiHashStr(s, seed);
4 }
5 void NkGuiContext::PopId() noexcept { if (idDepth > 0) --idDepth; }
6 NkGuiId NkGuiContext::GetId(const char *s) const noexcept {
7     const NkGuiId seed = idDepth > 0 ? idStack[idDepth - 1] : 2166136261u;
8     return NkGuiHashStr(s, seed);
9 }
```

Le mécanisme est élégant : **la graine de chaque niveau est l'identifiant du niveau précédent**. L'identifiant final dépend donc du *chemin complet*, pas du seul libellé. Il existe une surcharge `PushId(const void *)` qui hache une adresse — pratique pour donner une identité distincte à chaque élément d'une liste.

Idiome — écrit pour ce cours, d'après NkGuiContext.h:448-451

```
1 for (int32 i = 0; i < itemCount; ++i) {
2     ctx.PushId(&items[i]); // ou ctx.PushId(items[i].name.CStr())
3     if (Button(ctx, "Supprimer")) { /* supprime items[i] */ }
4     ctx.PopId();
5 }
```

Un identifiant qui change pendant l'édition

Plusieurs widgets acceptent un libellé *modifiable* — renommage d'un fichier dans un arbre, d'un onglet, d'une cellule. Si l'identité était calculée sur ce libellé, elle changerait à chaque frappe, et le framework croirait à chaque caractère qu'il s'agit d'un widget différent : le curseur de saisie sauterait, le focus se perdrait.

C'est pourquoi toutes les variantes `*Editable` exigent un `idStr` **stable**, distinct du libellé affiché (`NkGuiWidgets.h:198`, :209-210, :221-222). La règle générale : **l'identité ne se calcule jamais sur une donnée qui bouge**.

3.3 Le contexte : NkGuiContext

Tout l'état de l'interface vit dans une seule structure, `Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:131`. C'est un struct volumineux — plus de cent champs. Voici la carte, par famille :

Famille	Champs clés	Lignes
Vue / échelle	viewW, viewH, scale (DPI)	132-134
Thème	theme, syntax	135-136
Entrée	input	137
Dessin	d1, d1Overlay	138-139
Mise en page	layout	140
Polices	font, codeFont (non possédés)	141-142
Popups	popupStack[8], popupRects[8], popupDepth	147-152
Occlusion	occlRects, occlLayers, curInputLayer	175-182
Fenêtres	winDL[32], winMeta[32], winZ[32]	228-245
Docking	dockNodes, dockRoot, dockSpaceId	248-259
Presse-papiers	clipboardGetFn, clipboardSetFn	278-292
Interaction	hotId, hotIdPrev, activeId	317-321
Piles	idStack[32], disabledStack[16]	324-388
Saisie	inputId, inputCaret, inputAnchor	333-338
Stockage	openNodes, tabBarSel, scrollVals...	358-425
Hook de style	styleFn, styleUser	429-431

3.3.1 Cycle de vie

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:437-443

```

1      // — Cycle de vie —————
2      bool Init(int32 width, int32 height) noexcept;
3      void Shutdown() noexcept;
4
5      // — Cycle de frame —————
6      // BeginFrame : APRÈS que l'app ait posé l'input brut (events). Calcule
7      // les transitions, réinitialise hotId + la draw list.
8      void BeginFrame(float32 dt) noexcept;
```

3.3.2 Le contexte courant

Le contexte est **explicite** : toutes les fonctions de widget le prennent en premier paramètre. Il n'y a pas de singleton. Mais un « contexte courant » propre à chaque fil d'exécution existe, pour le code qui n'a pas le contexte sous la main :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:14-25

```

1  namespace {
2      // Contexte courant per-thread (non-singleton, comme NkUIContext).
3      thread_local NkGuiContext *gCurrentContext = nullptr;
4  }
5  void SetCurrentContext(NkGuiContext *ctx) noexcept { gCurrentContext = ctx; }
6  NkGuiContext *GetCurrentContext() noexcept { return gCurrentContext; }
```

Une fois ctx.Init(...) passé, appelez SetCurrentContext(&ctx); avant de rendre la main, appelez SetCurrentContext(nullptr). Rien de plus.

3.3.3 Le thème

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:28-49

```
1 struct NKENTSEU_NKGUI_CLASS_EXPORT NkGuiTheme {
2     NkColor bgPrimary   = {26, 29, 36, 255};
3     NkColor panel       = {38, 42, 52, 255};
4     NkColor header      = {46, 51, 63, 255};
5     NkColor button      = {58, 64, 80, 255};
6     NkColor buttonHover = {78, 92, 120, 255};
7     NkColor buttonActive= {96, 150, 230, 255};
8     NkColor border      = {86, 94, 112, 255};
9     NkColor text        = {230, 232, 240, 255};
10    NkColor textDisabled= {120, 124, 134, 255};
11    NkColor selection    = {64, 110, 200, 235};
12    NkColor accent       = {96, 165, 250, 255};
13    NkColor track        = {30, 34, 43, 255};
14    NkColor tabBar       = {22, 25, 31, 255};
15    NkColor tab          = {40, 45, 56, 255};
16    NkColor tabHover     = {58, 64, 80, 255};
17    NkColor tabActive    = {52, 58, 72, 255};
18    float32 rounding    = 5.f;
19    float32 framePadX   = 10.f;
20    float32 framePadY   = 6.f;
21 };
```

Ce sont des couleurs *nommées par usage*, lues directement (`ctx.theme.button`). Il n’y a pas de rôles sémantiques abstraits. Pour changer l’apparence, on écrase les champs après `Init` — c’est ce que fait le shell de l’éditeur ([Engine/NKEditorKit/src/NKEditorKit/NKEditorShell.cpp:155-179](#), qui y écrit une palette «GitHub Dark»).

Un second thème, `NkGuiSyntax` (`NkGuiContext.h:53-66`), porte les couleurs de coloration syntaxique : `keyword`, `type`, `string`, `comment`, `number`, `preproc`, `heading`, `mdcode`, `function`, `constant`, `oper`. Il sert à l’éditeur de code, et rien ne vous empêche de vous en servir pour autre chose.

Un idiome courant dans le dépôt, pour savoir si l’on est en thème clair ou sombre sans le stocker :

Engine/NKEditorKit/src/NKEditorKit/NKEditorScrollbar.h:29

```
1 const bool light = ((int32)th.bgPrimary.r + th.bgPrimary.g + th.bgPrimary.b) > 384;
```

3.4 L’entrée : `NkGuiInput`

`Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiInput.h` — 145 lignes, entièrement dans l’entête, tout `inline`. C’est la structure que *votre application remplit*, et c’est la frontière entre le monde des événements et le monde des widgets.

3.4.1 Ce que l'application pose

Champ	Sens
mousePos	position du pointeur
mouseDown[3]	0 = gauche, 1 = droit, 2 = milieu
wheel, wheelH	molette verticale et horizontale (cumulées)
ctrlDown, shiftDown, altDown	modificateurs
PushChar(cp)	un caractère saisi (<i>codepoint</i> Unicode)
SetKey(NkGuiKey, bool)	une touche enfoncée ou relâchée
SetDoubleClick(button)	double-clic détecté par le système
wantCopy, wantCut, wantPaste, wantSelectAll	raccourcis d'édition

3.4.2 Ce que NKGui calcule

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiInput.h:103-135 (abrégé)

```
1 void NewFrame() noexcept {
2     for (int32 i = 0; i < 3; ++i) {
3         mouseClicked[i] = mouseDown[i] && !mousePrev[i];
4         mouseReleased[i] = !mouseDown[i] && mousePrev[i];
5         ...
6         // Double-clic : (a) détection interne (2e clic < 0.40 s) OU
7         // (b) injection OS via SetDoubleClick (consommée puis remise à 0).
8         ...
9     }
10    for (int32 i = 0; i < KeyCount; ++i) {
11        keyInit[i] = keyDown[i] && !keyPrev[i];
12        ...
13    }
14 }
```

Les champs calculés — `mouseClicked`, `mouseReleased`, `mouseDoubleClicked`, `mouseDownDur`, `keyInit`, `keyDur` — sont ce que les widgets consultent. Vous ne les écrivez jamais; vous ne posez que l'état *brut*.

3.4.3 La répétition au maintien

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiInput.h:87-94 (signature)

```
1 int32 KeyPressedRepeat(NkGuiKey k, float32 delay = 0.30f, float32 rate = 0.04f) const
    noexcept;
```

Cette fonction renvoie le *nombre* de déclenchements franchis entre deux durées d'appui. C'est ce qui fait qu'une flèche maintenue déplace le curseur en rafale après un court délai, comme dans n'importe quel éditeur de texte. Elle sert aussi aux boutons à rafale.

3.4.4 Les touches suivies

NKGui ne suit pas tout le clavier. Il maintient une liste explicite (`NkGuiTypes.h:75-129`) : Left, Right, Up, Down, Home, End, Backspace, Delete, Enter, Escape, Tab, F2, F5, plusieurs lettres, Space,

F8, F12, etc.

Ce qu'il faut retenir

La saisie de texte passe par `chars[]` — des *codepoints* Unicode poussés par `PushChar` — et **jamais** par les touches. Les touches servent à la navigation et aux commandes. Confondre les deux donne une application où l'on ne peut pas taper d'accents, ou bien où les flèches écrivent des caractères.

3.5 Le cycle d'une image

3.5.1 BeginFrame

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:39-75

```
1 void NkGuiContext::BeginFrame(float32 dt) noexcept {
2     input.dt = dt;
3     input.NewFrame();      // transitions clic/relâche
4     time += dt;           // blink du caret
5     hotIdPrev = hotId;    // le survol résolu de la frame précédente
6     hotId = NKGUI_ID_NONE; // re-calculé par les widgets (greedy)
7     interact = NkGuiInteract::None;
8     lastItemHovered = false;
9     wantCursor = NkGuiCursor::Arrow;
10    idDepth = 0;
11    disabledDepth = 0;
12    inputClickConsumed = false;
13    curPopupLevel = -1;    // le dessin reprend sur la couche principale
14    // Routeur d'occlusion : la liste écrite la frame PRECEDENTE devient la
15    // liste LUE (stable toute la frame, comme hotIdPrev) ; on repart à zero
16    // pour l'écriture de cette frame.
17    occlCount = occlCountNew;
18    for (int32 i = 0; i < occlCountNew; ++i) { occlRects[i] = occlRectsNew[i]; occlLayers[i]
19    = occlLayersNew[i]; }
20    occlCountNew = 0;
21    curInputLayer = 0;
22    winCount = 0;         // pool de fenêtres ré-attribué cette frame
23    curWindow = -1; curWindowId = NKGUI_ID_NONE; curWindowDocked = false;
24    containerDepth = 0;   // pile de conteneurs ré-attribuée
25    overlayDepth = 0;
26    for (uint32 i = 0; i < windowMeta.Size(); ++i) {
27        windowMeta[i].hostRendered = false;
28        windowMeta[i].frameDL = -1;
29        windowMeta[i].dockDL = -2;
30    }
31    dl.Reset();
32    dlOverlay.Reset();
33 }
```

Lisez cette fonction comme une remise à zéro sélective : **tout ce qui est recalculé chaque image est effacé** (les listes de dessin, les piles, le survol courant, le *pool* de fenêtres), et **tout ce qui doit survivre est préservé** (les états persistants indexés par identifiant, la position des fenêtres, l'arbre de docking).

Deux lignes méritent qu'on s'y arrête, parce qu'elles portent le même mécanisme : `hotIdPrev = hotId` et le recopiage des rectangles d'occlusion. Dans les deux cas, **ce qui a été écrit à l'image précédente devient ce qu'on lit à l'image courante**. Nous verrons pourquoi à la section 3.6.

3.5.2 EndFrame

EndFrame (NkGuiContext.cpp:77-142) fait cinq choses :

1. **Fusionne les listes de dessin des fenêtres dans d1, dans l'ordre de profondeur** (tri par insertion, lignes 80-94). C'est ce qui donne un recouvrement correct entre fenêtres.
2. **Détermine la fenêtre survolée** pour l'image suivante (hoveredWindowId, lignes 96-106) : celle qui est le plus en avant sous le curseur.
3. **Ferme la chaîne de popups** : Échap ferme le niveau le plus profond ; un clic hors de tous les popups et hors de leur ancre ferme tout (lignes 113-124).
4. **Libère activeId si le bouton est relâché** — la garde anti-gel, ci-dessous.
5. **Consomme la molette et le texte** (lignes 139-141) : input.wheel et input.wheelH repassent à zéro, puis input.ClearPerFrameText().

L'anti-gel d'activeId

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:125-132

```
1 // ANTI-GEL : aucun glissement légitime ne conserve activeId bouton RELÂCHÉ.
2 // Si Le widget détenant activeId a disparu (hôte redevenu flottant, onglet
3 // caché, fenêtre fermé...e) il ne libère jamais activeId et l'occlusion bloque
4 // TOUTE interaction. Souris haute + activeId encore posé ☐ on libère d'office.
5 if (!input.mouseDown[0] && activeId != NKGUI_ID_NONE) {
6     activeId = NKGUI_ID_NONE;
7     movingWindowId = NKGUI_ID_NONE;
8 }
```

Retenez le symptôme, car il est spectaculaire et déroutant : **plus rien ne répond**. Aucun bouton ne réagit, aucun survol ne s'allume, l'application semble figée alors qu'elle continue de tourner à soixante images par seconde. La cause est presque toujours un activeId resté posé par un widget qui a disparu. Cette garde le libère d'office, mais si vous écrivez votre propre mécanisme de glissement (comme la barre de défilement de NKEditorKit), c'est à vous de remettre ctx.activeId = 0 au relâchement.

3.5.3 L'invariant d'ordre

Ce qu'il faut retenir

événements → ctx.BeginFrame(dt) → widgets → ctx.EndFrame() → backend.Submit(...)

BeginFrame appelle input.NewFrame() en interne. Si vous pompez les événements *après* BeginFrame, les transitions clic/relâche portent sur l'état de l'image *précédente* : vos clics arrivent avec une image de retard, ou pas du tout.

Ne pas transposer l'ordre de l'ancien module

L'ancien module d'interface, NKUI, avait l'ordre *inverse*, et il est documenté ainsi :

Applications/Sandbox/src/DemoNkntseu/NkUICanvas/NkUICanvasDemo.cpp:268-270

```
1 // Ordre input CORRECT : BeginFrame (vide Les deltas clic/release) PUIS Les
2 // events (re-remplissent pos/boutons/clics) -> Les widgets voient Le clic.
```

Ce commentaire ne s'applique pas à NKGui. La démo NKGui pompe les événements *avant* BeginFrame (main.cpp:438 contre :468), précisément parce que c'est BeginFrame qui fait le travail de NewFrame(). Si vous copiez du code de l'ancien module, c'est la première chose à vérifier.

3.6 Comment un widget conserve son état

Réponse : il ne conserve rien lui-même. L'état vit dans le contexte, indexé par identifiant, dans des tableaux parallèles clé/valeur.

État persistant	Stockage	Lignes
Nœuds d'arbre ouverts	openNodes	358
Onglet sélectionné	tabBarKeys + tabBarSel	359-360
Défilement d'une zone	scrollKeys + scrollVals	377-378
Focus/ancre de liste	selKeys + selFocusStore	363-365
Largeurs de colonnes	tblKeys + tblWidths	419-420
Teinte du sélecteur couleur	pickerKeys + pickerHSV	424-425
Fenêtres (pos, taille, z)	windowMeta	236
Arbre de dock	dockNodes	248

Le mécanisme est volontairement simple :

Kernel/Runtime/NKGui/src/NKGui/Core/NKGuiContext.cpp:336-355 (abrégé)

```
1 bool NKGuiContext::IsNodeOpen(NKGuiId id) const noexcept {
2     for (uint32 i = 0; i < openNodes.Size(); ++i)
3         if (openNodes[i] == id) return true;
4     return false;
5 }
6 void NKGuiContext::SetNodeOpen(NKGuiId id, bool open) noexcept { ... /* swap-remove */ }
```

Recherche linéaire, pas de table de hachage. C'est assumé : ces listes restent petites — quelques dizaines de nœuds ouverts au plus.

3.7 Comment un widget reçoit les événements

Nous arrivons au mécanisme le plus subtil de NKGui. Prenez le temps.

3.7.1 Le problème

En mode immédiat, quand on traite le bouton numéro trois, **on ne connaît pas encore les rectangles des widgets qui viendront après**. Or ce sont eux qui seront dessinés par-dessus.

Comment savoir si le pointeur est réellement sur le bouton trois, ou sur une fenêtre qui le recouvre et qu'on n'a pas encore déclarée?

Un framework à objets répondrait facilement : il a l'arbre complet, il fait un test de haut en bas. Le mode immédiat n'a pas cet arbre.

3.7.2 La solution : glouton, avec une image de retard

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:491-528

```
1 bool NkGuiContext::ItemHoverable(const NkRect &r, NkGuiId id) noexcept {
2     // Routeur d'occlusion UNIFIE : un widget n'est jamais survolable si une
3     // surface flottante d'une couche SUPERIEURE (modal, palette, popover
4     // declares via PushOcclusion) recouvre le pointeur – quel que soit
5     // l'ordre de dessin des panneaux.
6     if (!PointReachable(input.mousePosition)) return false;
7     // Désactivé : aucune interaction.
8     if (IsDisabled()) return false;
9     // Un popup ouvert capture le pointeur : un widget ne réagit pas si le
10    // pointeur est au-dessus d'un popup PLUS PROFOND que le niveau courant
11    // (couche principale = -1 ; un item de menu ne capture pas sous son
12    // sous-menu déployé).
13    for (int32 i = curPopupLevel + 1; i < popupDepth; ++i)
14        if (NkGuiRectContains(popupRects[i], input.mousePosition)) return false;
15    // Occlusion par fenêtre : hors popup, un widget ne réagit que si SA fenêtre
16    // est celle survolée au-dessus (curWindowId). Bloque le fond ET les fenêtres
17    // recouvertes. (hoveredWindowId/curWindowId = NONE pour le fond hors fenêtre.)
18    if (curPopupLevel < 0 && hoveredWindowId != NKGUI_ID_NONE && hoveredWindowId !=
        curWindowId)
19        return false;
20    // Hors du CLIP courant (zone défilable, panneau) : pas d'interaction –
21    // un item scrollé hors-vue ne doit pas capturer le pointeur.
22    if (!NkGuiRectContains(DL().CurrentClip(), input.mousePosition)) return false;
23    // Bloqué si un AUTRE widget capture le pointeur.
24    if (activeId != NKGUI_ID_NONE && activeId != id) return false;
25    if (!NkGuiRectContains(r, input.mousePosition)) return false;
26    // Greedy : Le DERNIER widget soumis sous le pointeur écrase hotId →
27    // celui dessiné par-dessus gagne. On ne déclare « survolé » QUE le
28    // front-most de la frame précédente (hotIdPrev) ; le widget masqué
29    // dessous, lui, met à jour hotId mais retourne false → ne capture pas.
30    hotId = id;
31    return hotIdPrev == id;
32 }
```

Chaque `return false` est une leçon, et l'ordre des tests est celui du moins cher au plus cher. Mais l'essentiel est dans les deux dernières lignes.

Glouton (*greedy*) : tout widget qui se trouve sous le pointeur écrase `hotId`. Comme les widgets sont déclarés dans l'ordre de dessin, le dernier qui écrit est celui qui est visuellement au-dessus. À la fin de l'image, `hotId` contient donc bien l'identifiant du widget de premier plan.

Une image de retard : mais on ne peut pas exploiter cette information pendant l'image en cours — elle n'est complète qu'à la fin. Alors on la reporte : `BeginFrame` copie `hotId` dans `hotIdPrev`, et `ItemHoverable` ne déclare « survolé » que le widget dont l'identifiant correspond à `hotIdPrev`. Le survol effectif est donc celui résolu à l'image précédente.

Glouton plus une image de retard

Chaque widget sous le pointeur écrit `hotId` (le dernier gagne, donc celui du dessus); le résultat sert à l'image suivante, via `hotIdPrev`. Un widget masqué met bien à jour `hotId`

mais retourne false : il ne capture rien.

Conséquence pratique et **contre-intuitive** : **ce qui est dessiné plus tard capte le clic**. L'ordre de peinture définit la priorité d'interaction. La démo l'écrit noir sur blanc :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:806-808

```
1 // Soumise APRÈS Les boutons → au-dessus. ItemHoverable résout Le z-ordre :  
2 // au-dessus d'un bouton, c'est La boîte qui capture, pas Le bouton dessous.
```

Le prix à payer est un décalage d'une image sur le survol. À soixante images par seconde, il est invisible. Mais il explique un comportement qu'on remarque parfois : le tout premier clic sur un élément jamais survolé auparavant peut ne pas prendre. Nous verrons à la section 3.12 que les menus contournent ce point par un test géométrique direct.

3.7.3 ButtonBehavior : la sémantique du clic

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:541-543

```
1 bool ButtonBehavior(NkGuiId id, const NkRect &r, NkGuiButtonFlags flags =  
    NkGuiButtonFlags::None,  
2 float32 repeatDelayOverride = -1.f, float32 repeatRateOverride = -1.f,  
3 bool *outHovered = nullptr, bool *outHeld = nullptr) noexcept;
```

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:530-570 (extraits)

```
1 const bool hovered = ItemHoverable(r, id); // respecte Le z-ordre...  
2  
3 if (hovered && input.mouseClicked[0]) {  
4     activeId = id;  
5     if (repeat) pressed = true; // rafale : déclenche dès L'appui  
6 }  
7 const bool held = (activeId == id);  
8 if (held) {  
9     interact = NkGuiInteract::EditWidget;  
10    ...  
11    if (input.mouseReleased[0]) {  
12        // Sans Repeat : clic validé au relâchement DANS Le rect.  
13        if (!repeat && NkGuiRectContains(r, input.mousePos)) pressed = true;  
14        activeId = NKGUI_ID_NONE;  
15    }  
16 } else if (hovered) {  
17     interact = NkGuiInteract::HoverWidget;  
18 }...  
19  
20 lastItemHovered = hovered; // pour IsItemHovered() / SetTooltip
```

Ce qu'il faut retenir

Un clic se valide au **RELÂCHEMENT**, à l'intérieur du rectangle. C'est la convention de tous les systèmes de fenêtrage : on peut appuyer sur un bouton, glisser en dehors, relâcher, et rien ne se produit — l'action est annulée. Avec le drapeau Repeat, en revanche, le déclenchement se fait dès l'appui, puis en rafale.

3.7.4 La machine à états d'interaction

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiTypes.h:52-61

```
1 // — Machine à états d'interaction (Le coeur du fix UX) —————
2 // À chaque frame, UN SEUL mode actif et explicite. Zones de préhension
3 // disjointes et priorisées (resize > move > contenu), chacune avec son
4 // curseur et son affordance. Voir ARCHITECTURE.md §4.
5 enum class NkGuiInteract : uint8 {
6     None = 0,
7     HoverWidget, ///< survol d'un widget
8     EditWidget,  ///< édition active (slider/...input)
9     MoveWindow,  ///< déplacement d'une fenêtre (barre de titre / onglet)
10    ResizeWindow, ///< redimensionnement (bord/coin – cf. edge)
11    DragSplitter, ///< glissement d'un séparateur de dock
12    DragTab,      ///< glissement d'un onglet (réordre / détache)
13    DockTarget    ///< visée d'une cible de dock (boussole)
14 };
```

La règle de conception associée mérite d'être citée, parce qu'elle décrit un défaut d'ergonomie que beaucoup d'interfaces ont :

Kernel/Runtime/NKGui/ARCHITECTURE.md:121-124

```
1 les zones de préhension sont disjointes et priorisées (bord/coin de resize >
2 barre de titre/onglet pour move > contenu), chacune avec son curseur
3 (NkWindow::SetCursor, déjà en place) et son affordance visuelle.→
4 Plus jamais « je voulais redimensionner et ça a docké ».
```

3.8 Le routeur d'occlusion : gérer vos propres surfaces flottantes

NKGui sait gérer ses propres popups. Mais dès que votre application dessine *elle-même* une surface flottante — une boîte de dialogue modale, une palette de commandes, un aperçu au survol — le framework ne peut pas deviner qu'elle est là. Sans précaution, les clics la traversent.

Le problème est décrit précisément dans l'en-tête :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:163-182

```
1 // — ROUTEUR D'OCCCLUSION UNIFIÉ (surfaces flottantes « maison ») ———
2 // PROBLÈME DE FOND résolu ici : Les overlays applicatifs (modals, palettes,
3 // popovers, ...peek) faisaient des hit-tests BRUTS (rect + mouseClicked) qui
4 // ignoraient ce qui est dessiné AU-DESSUS d'eux → traversées de clics,
5 // boutons inertes, consommations manuelles fragiles dupliquées partout.
6 // PRINCIPE : chaque surface flottante déclare son rect + sa couche CHAQUE
7 // frame (PushOcclusion) pendant son dessin ; tous les hit-tests passent par
8 // InputHits/ClickIn qui refusent un point recouvert par une couche PLUS
9 // HAUTE que celle en cours de dessin (curInputLayer). Comme hotIdPrev, la
10 // liste lue est celle de la FRAME PRÉCÉDENTE (stable pour toute la frame,
11 // indépendante de l'ordre de dessin des panneaux). Couches conseillées :
12 // 0 fond/panneaux • 50 menus/palettes/popovers • 100 modals • 200 debug.
```

3.8.1 Les quatre couches

Couche	Contenu
0	fond, panneaux, contenu ordinaire
50	menus, palettes, <i>popovers</i>
100	boîtes de dialogue modales
200	superpositions de débogage

Ce ne sont que des entiers : rien n'interdit d'en intercaler d'autres. Ce qui compte est la relation d'ordre.

3.8.2 L'API

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:185-223 (abrégé)

```
1 void PushOcclusion(const NkRect &r, int32 layer) noexcept; // déclarer une surface
2 bool PointReachable(const NkVec2 &p) const noexcept; // point atteignable ?
3 bool InputHits(const NkRect &r) const noexcept; // hit-test UNIFIE
4 bool ClickIn(const NkRect &r) const noexcept; // clic gauche unifie
5
6 // RAI : fixe la couche d'input pendant le dessin d'une surface flottante.
7 struct NkInputLayerScope {
8     NkGuiContext *c; int32 saved;
9     NkInputLayerScope(NkGuiContext &ctx, int32 layer) : c(&ctx), saved(ctx.curInputLayer)
10     {
11         ctx.curInputLayer = layer;
12     }
13     ~NkInputLayerScope() { c->curInputLayer = saved; }
```

Et la directive, écrite dans le fichier même (:202-203) :

Ce qu'il faut retenir

« Hit-test UNIFIÉ : souris dans *r* ET non recouverte par une couche supérieure. À utiliser PARTOUT à la place de `NkGuiRectContains(mousePos)`. »

Donc : `ctx.InputHits(r)` et `ctx.ClickIn(r)`, jamais un test géométrique brut, sauf pour du chrome de très bas niveau.

3.8.3 L'usage type

Applications/NKCode/src/NKCode/Shell/NkAppCommands.h:137-164 (condensé, idiome des modales)

```
1  const NkRect box = { (W-pw)*0.5f, (Vh-ph)*0.5f, pw, ph };
2
3  ctx.PushOcclusion(box, 100);                                // routeur d'occlusion, couche
   100
4  NkGuiContext::NkInputLayerScope _layer(ctx, 100);          // RAI : mes hit-tests passent
5
6  // MODALITE GLOBALE : popupDepth > 0 est LE signal verifie par les points
7  // d'interaction des panneaux – sans lui les clics TRAVERSENT vers l'editeur.
8  if (ctx.popupDepth == 0) ctx.popupDepth = 1;
9  ctx.popupRects[0] = box;
10 ctx.popupAnchor = box;
```

Le blocage protège ce qui est dessous, jamais ce qui est au-dessus

Cette phrase vient d'un rapport de défaut du dépôt ([Kernel/Runtime/NKGui/ROADMAP.md](#), « Défaut 1 ») et c'est la meilleure formulation du principe :

« ma propre correction du point 3 : en armant le blocage, j'ai neutralisé la liste elle-même, peinte après. D'où la règle qui manquait, à inscrire dans le socle : **le blocage protège ce qui est dessous, jamais ce qui est au-dessus.** »

et la directive qui suit : « Dans un socle correct, l'ordre de peinture définit la priorité d'interaction, sans rien à armer. Ce qui est peint plus tard capte le clic en premier ; le reste en découle. »

Quatre bugs de la même famille sont recensés dans ce document : un menu d'en-tête dont les clics étaient inopérants et qui ne se refermait pas, un panneau qui laissait passer ses clics, une liste déroulante dans laquelle « je ne peux choisir aucun format ». Tous avaient la même cause : un test de survol brut qui ignorait ce qui était peint au-dessus.

Les tests négatifs

Applications/NKCode/src/NKCode/Shell/Panels.h:3461-3465

```
1  // « Clic en dehors du champ -> valider » : test NEGATIF, donc on
2  // exige d'abord que le clic ATTEIGNE notre couche
3  // (PointReachable). Sinon un clic destine a une surface
4  // flottante posee au-dessus (modale, menu) validait le
5  // renommage au passage.
6  else if (mRenameArmed && ctx.input.mouseClicked[0] && !ctx.input.mouseDoubleClicked[0]
   &&
7      ctx.PointReachable(ctx.input.mousePos) &&
8      !NkGuiRectContains(mRenameRect, ctx.input.mousePos)) {
```

Un test *positif* (« le clic est-il dans mon rectangle ? ») échoue naturellement quand quelque chose recouvre. Un test *négatif* (« le clic est-il *en dehors* de mon rectangle ? ») réussit au contraire dans ce cas — et déclenche à tort. Chaque fois que vous écrivez « clic en dehors □ fermer ou valider », commencez par exiger `ctx.PointReachable(mousePos)`.

Ne jamais écraser `ctx.input.mousePos`

C'est probablement le piège le plus coûteux du dépôt, et le commentaire qui le documente est le plus important sur le sujet de l'entrée :

Applications/NKCode/src/NKCode/Shell/NkExplorer.h:89-98 (extrait)

```
1 // /\ NE PAS toucher mousePos : il n'est mis à jour QUE sur un événement
2 // de DÉPLACEMENT (jamais re-sondé) -> L'écraser Le GÈLE pour toute
3 // L'app à la frame suivante (clic sans bouger = position figée). On
4 // neutralise donc SEULEMENT les clics/molette/frappes.
```

La neutralisation correcte est *sélective* :

Applications/NKCode/src/NKCode/Shell/NkExplorer.h:99-107

```
1 if (mPeekOpen || mDelMenu.open) {
2     for (int32 b = 0; b < 3; ++b) {
3         ctx.input.mouseClicked[b] = false;
4         ctx.input.mouseDown[b] = false;
5         ctx.input.mouseDoubleClicked[b] = false;
6     }
7     ctx.input.wheel = ctx.input.wheelH = 0.f;
8     ctx.input.charCount = 0;
9 }
```

Un shell peut se permettre de téléporter la souris hors de l'écran, parce qu'il restaure l'intégralité de input quelques lignes plus loin. Un panneau, non : ce qu'il écrase reste écrasé.

3.9 La liste de dessin

3.9.1 Choisir la bonne couche : ctx.DL()

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:299-303

```
1 NkGuiDrawList &DL() noexcept {
2     if (curPopupLevel >= 0 || overlayDepth > 0)
3         return dlOverlay; // popup / couche overlay forcée
4     return curWindow >= 0 ? winDL[curWindow] : dl;
5 }
```

Trois destinations possibles : la liste de superposition (si l'on dessine un popup), la liste propre à la fenêtre courante, ou la liste principale.

Ce qu'il faut retenir

On écrit toujours dans `ctx.DL()`, jamais dans `ctx.dl` en dur. Un panneau qui écrirait directement dans `ctx.dl` se dessinerait *sous* tout le reste et *sans* découpe — parce qu'il aurait court-circuité à la fois le *pool* de fenêtres et la pile de clip.

Chaque fenêtre a sa propre liste, ce qui rend le tri par profondeur possible :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:225-232

```
1 // — Fenêtres flottantes (Begin/End) —————
2 // Chaque fenêtre dessine dans SA draw-List (pool `winDL`), fusionnées dans
3 // `dl` triées par z-order à EndFrame → recouvrement correct + passage devant.
4 static constexpr int32 WinMax = 32;
```

```
5 NkGuiDrawList winDL[WinMax];
```

3.9.2 Les primitives

Toutes sont implémentées dans `NkGuiDrawList.cpp`.

Primitive	Décl.	Remarque
<code>AddRectFilled(r, col, rounding)</code>	.h :66	coins arrondis inclus
<code>AddRect(r, col, thickness)</code>	.h :67	contour
<code>AddRectFilledMultiColor(r, tl, tr, br, bl)</code>	.h :69	dégradé bilinéaire
<code>AddImage(texId, r, uv0, uv1, tint)</code>	.h :73	quad texturé
<code>AddLine(a, b, col, thickness)</code>	.h :75	
<code>AddTriangleFilled(a, b, c, col)</code>	.h :76	
<code>AddTriangleMultiColor(a, b, c, ca, cb, cc)</code>	.h :78	dégradé barycentrique
<code>AddCircleFilled(center, r, col, segs)</code>	.h :80	segs = 0 → auto (12...128)
<code>AddText(face, texId, baseline, s, col, maxWidth, skew)</code>	.h :87	
<code>AddTextRange(face, texId, baseline, begin, end, col)</code>	.h :91	sous-chaîne

Il n'y a pas de `AddPolyline`, `AddBezier`, `AddArc`, ni `AddCircle` (contour non plein), contrairement à ce qu'annonce [ARCHITECTURE.md:101-102](#).

Rappelez-vous du chapitre 2 : `NkCanvas` n'a ni dégradé ni coins arrondis. Voici donc où ils sont fabriqués. Le rectangle arrondi est quatre arcs de coin, quatre segments chacun, triangulés en éventail depuis le centre du rectangle (`NkGuiDrawList.cpp:114-141`). Le dégradé est simplement quatre couleurs de sommets différentes.

Une bordure n'est pas quatre lignes

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:205-208

```
1 // Bordure = 4 rectangles pleins tracés STRICTEMENT à l'intérieur du rect.
2 // (AddLine centrerait l'épaisseur sur l'arête → la moitié extérieure sort
3 // du rect et est rognée par le clip - scissor exclusif à droite/bas → bord
4 // droit invisible/aminci.) Ici chaque bord tient dans [r.x, r.x+r.w] etc.
```

Symptôme : les bordures droite et basse de vos panneaux sont plus fines que les autres, ou disparaissent complètement. Cause : une épaisseur centrée sur l'arête, dont la moitié extérieure tombe hors de la zone de découpe.

3.9.3 Le découpage

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:34-55 (abrégé)

```
1 NkRect NkGuiDrawList::CurrentClip() const noexcept {
2     return clipDepth > 0 ? clipStack[clipDepth - 1] : NkRect{0.f, 0.f, 1.0e9f, 1.0e9f};
3 }
4 void NkGuiDrawList::PushClipRect(const NkRect &r, bool intersect) noexcept { ... }
5 void NkGuiDrawList::PopClipRect() noexcept { if (clipDepth > 0) --clipDepth; }
```

Pile de 32 niveaux, intersection avec le parent par défaut. Et la règle de découpage des commandes :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:57-75 (extrait)

```
1  if (need) {
2      NkGuiDrawCmd c;
3      c.texId = texId;
4      c.clipRect = clip;
5      c.idxOffset = static_cast<uint32>(idx.Size());
6      c.idxCount = 0;
7      c.type = texId ? NkGuiDrawCmdType::TexturedTriangles : NkGuiDrawCmdType::Triangles;
8      cmds.PushBack(c);
9  }
```

Une nouvelle commande est ouverte **dès que la texture ou le rectangle de découpe change**. Puisque, côté NkCanvas, un changement de découpe force un vidage (chapitre 2), on voit la chaîne de causalité complète : *beaucoup de PushClipRect* → *beaucoup de commandes* → *beaucoup d'appels de dessin*.

Le double filtrage

Le découpage garantit la *correction* visuelle ; il ne dispense pas d'éviter le travail inutile. L'idiome systématique du dépôt combine les deux :

Applications/NKCode/src/NKCode/Shell/NkAppCommands.h:344-354 (abrégé)

```
1  u.dl->PushClipRect(body, true);
2  float32 y = body.y - d->helpScroll;
3  for (int32 i = 0; i < N; ++i) {
4      if (y + rowH >= body.y && y <= body.y + body.h && SC[i].k[0]) { // culling manuel
5          u.Text(body.x, y + u.s(4), SC[i].k, nkcode::NkCol::primary);
6          u.Text(body.x + keyW, y + u.s(4), SC[i].a, nkcode::NkCol::foreground);
7      }
8      y += SC[i].k[0] ? rowH : u.s(10.f);
9  }
10 u.dl->PopClipRect();
```

3.9.4 Le texte, et le piège du flou

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:226-235

```
1  // — ALIGNEMENT D'UN GLYPHE SUR LA GRILLE DE PIXELS —————
2  // Le curseur de texte accumule des avances FRACTIONNAIRES. Sans arrondi,
3  // seul le PREMIER glyphe d'une chaîne tombe sur un pixel entier ; tous les
4  // suivants dérivent et échantillonnent l'atlas ENTRE deux texels, ce qui
5  // rend le texte uniformément flou. Le défaut est d'autant plus marqué que le
6  // corps est petit : l'erreur vaut un demi-texel CONSTANT, soit 4 % de la
7  // hauteur d'un caractère à 13 px et 3,3 % à 15 px.
```

La solution tient en une fonction d'arrondi, mais sa *subtilité* est dans ce qu'elle n'arrondit pas :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:258-266

```
1 // On arrondit la POSITION du quad, jamais L'AVANCE : `x` continue
2 // d'accumuler la valeur exacte, donc la largeur totale de la chaîne
3 // est inchangée et MeasureWidth reste d'accord avec le rendu. ...[]
4 // La LARGEUR est reportée telle quelle : arrondir les deux bords
5 // séparément étirerait le glyphe d'un pixel et le rééchantillonnerait,
6 // ce qu'on cherche précisément à éviter.
```

Trois décisions, trois raisons : on arrondit la position (sinon flou), on n'arrondit pas l'avance (sinon la mesure ne correspond plus au rendu), on ne touche pas à la largeur (sinon on rééchantillonne, donc on refloute). C'est un excellent exemple de ce qu'est une correction *bien* faite.

3.9.5 La formule de la ligne de base

AddText prend une **ligne de base**, pas un coin haut-gauche. Les deux conversions canoniques sont :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.cpp:111-118

```
1 float32 TextAt(NkGuiContext &ctx, const NkVec2 &topLeft, const char *s, const NkColor &col)
2     noexcept {
3     if (!ctx.font || !ctx.font->Valid() || !s) return 0.f;
4     // topLeft = coin haut-gauche → ligne de base = y + ascender.
5     const float32 baseY = topLeft.y + ctx.font->Ascent();
6     ctx.DL().AddText(ctx.font->Face(), ctx.font->TexId(), {topLeft.x, baseY}, s, col);
7     return ctx.font->MeasureWidth(s);
8 }
```

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.cpp:125-133

```
1 static void DrawCenteredLabel(NkGuiContext &ctx, const NkRect &r, const char *label, const
2     NkColor &col) {
3     const float32 tw = ctx.font->MeasureWidth(label);
4     const float32 tx = r.x + (r.w - tw) * 0.5f;
5     const float32 baseY = r.y + (r.h - ctx.font->LineHeight()) * 0.5f + ctx.font->Ascent();
6     ctx.DL().AddText(ctx.font->Face(), ctx.font->TexId(), {tx, baseY}, label, col, r.w - 6.f);
7 }
```

Deux formules à mémoriser

- Aligné en haut : `baseline.y = y + font->Ascent();`
- Centré dans un rectangle : `baseline.y = r.y + (r.h - font->LineHeight()) * 0.5f + font->Ascent();`

Et la garde qui doit précéder tout appel à AddText ou MeasureWidth : `if (!ctx.font || !ctx.font->Valid()) return;`

Attention : `NkGuiFont::LineHeight()` renvoie `face->lineAdvance`, et **non** `ascent + descent`.

3.10 La mise en page

3.10.1 Le modèle : un curseur qui descend

NkGuiLayout (NkGuiContext.h:71-95) est un curseur, comme celui d'une machine à écrire : region (la zone disponible), cursor (où l'on est), lineStartX, curLineH, prevItem, maxX/maxY, plus les espacements (padding = 10, itemSpacingX = 8, itemSpacingY = 6).

Le mode le plus simple, le mode vertical, tient en douze lignes :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.cpp:303-317

```
1  if (w <= 0.f) w = ContentWidth();
2  const NkRect rect = {layout.cursor.x, layout.cursor.y, w, h};
3  layout.prevItem = rect;
4  if (rect.x + rect.w > layout.maxX) layout.maxX = rect.x + rect.w; // Largeur contenu
5  if (h > layout.curLineH) layout.curLineH = h;
6  layout.cursor.x = layout.lineStartX;
7  layout.cursor.y += layout.curLineH + layout.itemSpacingY;
8  if (layout.cursor.y > layout.maxY) layout.maxY = layout.cursor.y;
9  layout.curLineH = 0.f;
10 return rect;
```

3.10.2 Les sept modes de flux

Le champ layout.flow sélectionne le comportement de NextItemRect (NkGuiContext.cpp:194-318) :

flow	Mode	Comportement
0	Vertical (défaut)	un élément par ligne, le curseur descend; $w \leq 0$ = remplir la largeur
1	HBox	le curseur avance en X, pas de retour à la ligne; $w \leq 0 \rightarrow 120$ px
2	Grid	colonnes régulières, retour à la ligne tous les gridCols
3	Stack	enfants superposés, ancrés dans une boîte (9 ancrés)
4	Flow	comme HBox mais passe à la ligne quand ça déborde (étiquettes, barres d'outils)
5	Row (flex)	rangée horizontale, largeurs pré-calculées, hauteur étirée
6	Column (flex)	colonne verticale, hauteurs pré-calculées, largeur étirée

On ne règle pas flow à la main : on ouvre un conteneur.

3.10.3 Les helpers de curseur

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:453-466

```
1 void BeginLayout(const NkRect &region) noexcept; // ouvre une region de contenu
2 NkRect NextItemRect(float32 w, float32 h) noexcept; // w<=0 = remplir la largeur
3 void SameLine(float32 spacingX = -1.f) noexcept; // item suivant a droite du precedent
4 void Spacing(float32 px = -1.f) noexcept; // saut vertical
5 float32 ContentWidth() const noexcept; // Largeur restante au curseur
```

```

6 float32 AvailHeight() const noexcept;           // hauteur restante sous le curseur
7 float32 ItemHeight() const noexcept;           // hauteur standard d'un widget
8 float32 S(float32 px) const noexcept;           // px Logiques -> px ecran (DPI)
9 void Indent(float32 w) noexcept;                // decale le debut de ligne (arbres)

```

ItemHeight() vaut simplement lineHeight + 2 * theme.framePadY, avec un repli de 16 px si aucune police n'est chargée (NkGuiContext.cpp:189-192).

AvailHeight() vaut un milliard dans une zone défilable

Applications/NKCode/src/NKCode/Shell/Panels.h:676-685 (extrait)

```

1 // AvailHeight()/ContentWidth() = taille du CONTENU (scrollable, ~1e9), PAS
2 // la taille visible -> on borne par le rect de CLIP (zone visible du dock)
3 // sinon viewH gigantesque (pas de scrollbar, barre H hors ecran).
4 const NkRect clip = ctx.DL().CurrentClip();
5 NkRect r = {ctx.layout.cursor.x, ctx.layout.cursor.y, ctx.ContentWidth(),
6             ctx.AvailHeight()};
7 if (r.x + r.w > clip.x + clip.w) r.w = clip.x + clip.w - r.x;
8 if (r.y + r.h > clip.y + clip.h) r.h = clip.y + clip.h - r.y;

```

Règle : CurrentClip() donne le VISIBLE; ContentWidth() et AvailHeight() donnent le CONTENU — qui, dans une zone défilable, est volontairement gigantesque. Il faut toujours intersecter les deux. Symptôme si on l'oublie : plus de barre de défilement, et une barre horizontale placée à un million de pixels du bord.

3.10.4 Les conteneurs

Ils sauvegardent le layout parent, le remplacent, puis le restaurent en avançant le parent du bloc consommé (NkGuiWidgets.cpp:1322-1365). Ils sont **composables** — un HBox dans une cellule de grille, par exemple.

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:146-189 (signatures)

```

1 void BeginVBox(NkGuiContext &ctx, float32 gap = -1.f) noexcept; void EndVBox(NkGuiContext &) noexcept;
2 void BeginHBox(NkGuiContext &ctx, float32 gap = -1.f) noexcept; void EndHBox(NkGuiContext &) noexcept;
3 void BeginGrid(NkGuiContext &ctx, int32 columns, float32 gap = -1.f) noexcept; void EndGrid(NkGuiContext &) noexcept;
4 void BeginGroup(NkGuiContext &ctx) noexcept; void EndGroup(NkGuiContext &) noexcept;
5 void BeginFlow(NkGuiContext &ctx, float32 gap = -1.f) noexcept; void EndFlow(NkGuiContext &) noexcept;
6 void BeginRow(NkGuiContext &ctx, float32 height, const float32 *sizes, int32 count, float32 gap = -1.f) noexcept;
7 void EndRow(NkGuiContext &ctx) noexcept;
8 void BeginColumn(NkGuiContext &ctx, float32 width, const float32 *sizes, int32 count, float32 totalHeight = -1.f, float32 gap = -1.f) noexcept;
9 void EndColumn(NkGuiContext &ctx) noexcept;
10 void BeginStack(NkGuiContext &ctx, float32 width, float32 height) noexcept;
11 void StackAnchor(NkGuiContext &ctx, int32 anchor) noexcept; // 0=HG 1=HC 2=HD 3=CG 4=C 5=CD 6=BG 7=BC 8=BD
12 void EndStack(NkGuiContext &ctx) noexcept;
13 void Spacer(NkGuiContext &ctx, float32 w, float32 h) noexcept;
14 void SpringRight(NkGuiContext &ctx, float32 width) noexcept;
15 bool Splitter(NkGuiContext &ctx, const char *idStr, const NkRect &handle, bool vertical,

```

```

17         float32 *value, float32 minV, float32 maxV) noexcept;
18 void SetUiScale(NkGuiContext &ctx, float32 s) noexcept;
19 float32 Scaled(NkGuiContext &ctx, float32 px) noexcept;
20 void PushOverlay(NkGuiContext &ctx) noexcept; void PopOverlay(NkGuiContext &) noexcept;

```

Le flex à poids mérite une note :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:159-160

```

1 `sizes[i] > 0` = px fixes ; `sizes[i] < 0` = poids (-1 = poids 1) qui se
2 partagent l'espace RESTANT. Single-pass exact (total connu).

```

3.10.5 Le DPI

SetUiScale(ctx, s) multiplie les marges, l'arrondi et les espacements. Mais attention : l'application doit recharger la police à tailleBase * scale, faute de quoi la mise en page grossit et le texte reste petit. La démo le fait ainsi :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:461-466

```

1     if (pendingScale > 0.f) { // F9 : appliquer la nouvelle echelle DPI
2         SetUiScale(ctx, pendingScale);
3         if (fontPtr->LoadEmbedded(NkEmbeddedFontId::DroidSans, 18.f * pendingScale))
4             backend.UploadFontGray8(fontPtr->TexId(), fontPtr->pixels, fontPtr->atlasW,
5             fontPtr->atlasH);
6         pendingScale = 0.f;
7     }

```

Notez où ce code est placé : **avant** BeginFrame. Recharger un atlas au milieu d'une image laisserait des commandes de dessin pointant vers l'ancienne texture.

3.11 Le catalogue des widgets

Voici l'inventaire de ce qui existe réellement. Toutes les signatures viennent de `Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h`; la macro `NKENTSEU_NKGUI_API` est omise. Toutes ont une définition dans le `.cpp` — cela a été vérifié fonction par fonction.

3.11.1 Texte et boutons

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:16-44, 140

```

1 void PanelBackground(NkGuiContext &ctx, const NkRect &r) noexcept; //
   .cpp:106
2 float32 TextAt(NkGuiContext &ctx, const NkVec2 &topLeft, const char *s) noexcept; //
   .cpp:120
3 float32 TextAt(NkGuiContext &ctx, const NkVec2 &topLeft, const char *s, const NkColor &col)
   noexcept;
4 void Text(NkGuiContext &ctx, const char *s) noexcept; //
   .cpp:166
5 void TextWrapped(NkGuiContext &ctx, const char *text, float32 wrapWidth = -1.f) noexcept;
6 bool Button(NkGuiContext &ctx, const char *label, const NkRect &r) noexcept; //
   rect explicite
7 bool Button(NkGuiContext &ctx, const char *label) noexcept; //
   rect auto

```

```

8  bool ButtonEx(NkGuiContext &ctx, const char *label, const NkRect &r, NkGuiButtonFlags
    flags,
9      float32 repeatDelay = -1.f, float32 repeatRate = -1.f) noexcept;
10 bool RepeatButton(NkGuiContext &ctx, const char *label, const NkRect &r,
11     float32 repeatDelay = -1.f, float32 repeatRate = -1.f) noexcept;
12 void Separator(NkGuiContext &ctx) noexcept;

```

Le widget le plus simple du framework, en entier — il résume tout ce que nous avons vu :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.cpp:166-172

```

1  void Text(NkGuiContext &ctx, const char *s) noexcept {
2      const float32 lh = (ctx.font && ctx.font->Valid()) ? ctx.font->LineHeight() : 16.f;
3      const float32 w  = (ctx.font && ctx.font->Valid()) ? ctx.font->MeasureWidth(s) : 0.f;
4      const NkRect r = ctx.NextItemRect(w, lh);
5      TextAt(ctx, {r.x, r.y}, s, ctx.IsDisabled() ? ctx.theme.textDisabled : ctx.theme.text);
6  }

```

Mesurer, demander un rectangle au layout, dessiner. Quatre lignes, et la garde sur la police est présente deux fois avec son repli chiffré.

3.11.2 Cases à cocher

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:45-64

```

1  bool Checkbox(NkGuiContext &ctx, const char *label, bool &value) noexcept;
2  bool CheckboxTristate(NkGuiContext &ctx, const char *label, NkGuiCheck &state) noexcept;
3  bool CheckBox3(NkGuiContext &ctx, const char *idStr, NkGuiCheck &state) noexcept; //
    compacte, sans libelle
4  void NkGuiTreeCascade(NkGuiCheck *states, const int32 *parent, int32 count, int32 node,
5      NkGuiCheck value) noexcept;
6  void NkGuiTreeRecomputeMixed(NkGuiCheck *states, const int32 *parent, int32 count) noexcept;

```

Les deux dernières fonctions illustrent un choix de conception intéressant :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:55-58

```

1  NKGui ne stocke PAS la hiérarchie (façon ImGui) : l'app possède `states[]`
2  (1 NkGuiCheck par ænue) + `parent[]` (index du parent, -1 = racine). Ces
3  utilitaires agissent sur ces tableaux quand l'app le décide (ex. Alt+clic).

```

3.11.3 Valeurs numériques

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:65, 112-122

```

1  bool SliderFloat(NkGuiContext &ctx, const char *label, float32 &value, float32 vmin, float32
    vmax) noexcept;
2  bool DragFloat (NkGuiContext &ctx, const char *label, float32 &v, float32 speed = 0.1f,
3      float32 vmin = -1.0e30f, float32 vmax = 1.0e30f,
4      NkGuiDragDir dir = NkGuiDragDir::Horizontal) noexcept;
5  bool DragInt   (NkGuiContext &ctx, const char *label, int32 &v, float32 speed = 0.25f,
6      int32 vmin = -2147483640, int32 vmax = 2147483640,
7      NkGuiDragDir dir = NkGuiDragDir::Horizontal) noexcept;
8  bool InputFloat (NkGuiContext &ctx, const char *label, float32 &v, float32 step = 1.f, ...)
    noexcept;

```

```
9 bool InputInt (NkGuiContext &ctx, const char *label, int32 &v, int32 step = 1, ...) noexcept;
```

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:106-111

```
1 DragFloat/DragInt : GLISSER sur le champ pour changer la valeur (vitesse
2 `speed`/pixel) + DOUBLE-CLIC pour saisir au clavier. InputFloat/InputInt : idem
3 + boutons -/+ (pas `step`). ...[] Pendant le survol/glisser, le widget pose
4 `ctx.wantCursor` ⇨( ou ⇩) que l'app peut appliquer.
```

Ce dernier point est à retenir : le widget *demande* un curseur, il ne le pose pas. C'est à votre boucle d'appliquer `ctx.wantCursor` en appelant `window.SetCursor(...)`. Nous le ferons au chapitre 4.

3.11.4 Saisie de texte

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:126-138

```
1 bool InputText(NkGuiContext &ctx, const char *label, char *buf, int32 bufSize) noexcept;
2 bool InputTextEx(NkGuiContext &ctx, const char *label, char *buf, int32 bufSize,
3                 NkGuiInputFlags flags, int32 maxChars = -1) noexcept;
4 bool InputTextMultiline(NkGuiContext &ctx, const char *idStr, char *buf, int32 bufSize,
5                         const NkRect &rect, NkGuiInputFlags flags = NkGuiInputFlags::None,
6                         int32 maxChars = -1, bool wrap = false) noexcept;
```

Drapeaux disponibles (`NkGuiTypes.h:212-220`) : Password, CharsDecimal, CharsHex, Uppercase, NoBlank, ReadOnly.

Deux distinctions à ne pas rater

- `maxChars` compte en **codepoints**, `bufSize` borne les **octets**. Un caractère accentué occupe deux octets.
- `InputText` renvoie *true* à la **pression d'Entrée**; `InputTextMultiline` renvoie *true* si **le texte a changé**. Ce n'est pas la même sémantique, et confondre les deux donne soit une validation qui n'arrive jamais, soit une action déclenchée à chaque frappe.

3.11.5 Graphiques et couleur

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:71-93

```
1 void ProgressBar(NkGuiContext &ctx, float32 fraction, const char *overlay = nullptr) noexcept;
2 void PlotLines(NkGuiContext &ctx, const char *label, const float32 *values, int32 count,
3               float32 minV = 0.f, float32 maxV = 0.f, float32 height = 0.f) noexcept;
4 void PlotHistogram(NkGuiContext &ctx, const char *label, const float32 *values, int32 count,
5                   float32 minV = 0.f, float32 maxV = 0.f, float32 height = 0.f) noexcept;
6
7 bool ColorButton(NkGuiContext &ctx, const char *idStr, const float32 *col, float32 w = 0.f,
8                 float32 h = 0.f, NkGuiColorFlags flags = NkGuiColorFlags::None) noexcept;
9 bool ColorPicker4(NkGuiContext &ctx, const char *label, float32 *col,
10                  NkGuiColorFlags flags = NkGuiColorFlags::None) noexcept;
11 bool ColorEdit4(NkGuiContext &ctx, const char *label, float32 *col,
12                 NkGuiColorFlags flags = NkGuiColorFlags::None) noexcept;
```

NkGuiColorFlags ([NkGuiTypes.h:151-159](#)) : NoAlpha, Wheel (roue et triangle SV), NoInputs, Disc, Hexagon, Honeycomb. Détail de conception noté dans l'en-tête (:87-88) : « La TEINTE est préservée même au noir/blanc (HSV mémorisé par id) » — d'où les tableaux pickerKeys/pickerHSV du contexte.

3.11.6 Images

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:99-105

```
1 void Image(NkGuiContext &ctx, uint32 texId, float32 w, float32 h,
2           NkColor tint = NkColor{255,255,255,255},
3           NkVec2 uv0 = NkVec2{0.f,0.f}, NkVec2 uv1 = NkVec2{1.f,1.f}) noexcept;
4 bool ImageButton(NkGuiContext &ctx, const char *idStr, uint32 texId, float32 w, float32 h,
5                 NkColor tint = NkColor{255,255,255,255},
6                 NkVec2 uv0 = NkVec2{0.f,0.f}, NkVec2 uv1 = NkVec2{1.f,1.f}) noexcept;
```

Le texId est l'identifiant côté *backend* : au préalable, votre application doit avoir enregistré l'image sous cet identifiant par backend.UploadImageRGBA(texId, ...) (chapitre 4). La teinte multiplie l'échantillon : blanc donne l'image telle quelle, une couleur donne une icône monochrome recolorée.

3.11.7 Arbres, sélection, onglets

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:191-239

```
1 bool CollapsingHeader(NkGuiContext &ctx, const char *label) noexcept;
2 bool TreeNode(NkGuiContext &ctx, const char *label) noexcept;
3 void TreePop(NkGuiContext &ctx) noexcept;
4 bool TreeNodeEditable(NkGuiContext &ctx, const char *idStr, char *label, int32 bufSize,
5                      bool allowRename = true) noexcept;
6 bool Selectable(NkGuiContext &ctx, const char *label, bool selected) noexcept;
7 bool SelectableEditable(NkGuiContext &ctx, const char *idStr, char *label, int32 bufSize,
8                        bool selected, bool allowRename = true) noexcept;
9 bool SelectItem(NkGuiContext &ctx, const char *label) noexcept;
10 bool SelectItemEditable(NkGuiContext &ctx, const char *idStr, char *label, int32 bufSize,
11                       bool allowRename = true) noexcept;
12 int32 TabBar(NkGuiContext &ctx, const char *id, const char *const *labels, int32 count)
13             noexcept;
14 int32 TabBarEx(NkGuiContext &ctx, const char *id, const char *const *labels, int32 count,
15               const bool *enabled) noexcept;
16 int32 TabBarEditable(NkGuiContext &ctx, const char *id, char *const *labels, int32 count,
17                    int32 labelBufSize, const bool *enabled = nullptr,
18                    const bool *allowRename = nullptr) noexcept;
```

Les variantes *Editable sont une signature de NKGui : le renommage en place — par double-clic ou Maj+Entrée — est intégré au socle, pas laissé à l'application. Elles prennent toutes un idStr stable, pour la raison vue à la section 3.2.

3.11.8 Zones défilables et tables

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:315-364

```
1 bool BeginChild(NkGuiContext &ctx, const char *idStr, const NkRect &rect, bool border = true,
2               bool horizontal = false) noexcept;
3 void EndChild(NkGuiContext &ctx) noexcept;
4 bool BeginListBox(NkGuiContext &ctx, const char *idStr, const NkRect &rect) noexcept;
5 void EndListBox(NkGuiContext &ctx) noexcept;
6
7 bool BeginTable(NkGuiContext &ctx, const char *idStr, int32 columns,
8               NkGuiTableFlags flags = NkGuiTableFlags::Borders) noexcept;
9 void TableSetupColumn(NkGuiContext &ctx, const char *label, float32 width = 0.f) noexcept;
10 void TableHeadersRow(NkGuiContext &ctx) noexcept;
11 void TableNextRow(NkGuiContext &ctx, float32 minHeight = 0.f) noexcept;
12 bool TableNextColumn(NkGuiContext &ctx) noexcept;
13 bool TableSetColumnIndex(NkGuiContext &ctx, int32 n) noexcept;
14 void EndTable(NkGuiContext &ctx) noexcept;
15 bool TableGetSortColumn(NkGuiContext &ctx, int32 *outCol, bool *outAscending) noexcept;
16 bool TableCellText(NkGuiContext &ctx, const char *idStr, char *buf, int32 bufSize,
17                  bool editable = false) noexcept;
```

L’idiome complet est donné en commentaire dans l’en-tête :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:323-334

```
1 if (BeginTable(ctx, "t", 3, Borders | RowBg | Resizable)) {
2     TableSetupColumn(ctx, "Nom", 0); // 0 = colonne étirable
3     TableSetupColumn(ctx, "Type", 90); // largeur fixe px
4     TableSetupColumn(ctx, "Taille", 70);
5     TableHeadersRow(ctx);
6     for ...() { TableNextRow(ctx);
7                 TableNextColumn(ctx); Text(ctx, nom);
8                 TableNextColumn(ctx); Text(ctx, type);
9                 TableNextColumn(ctx); Text(ctx, taille); }
10    EndTable(ctx);
11 }
```

NkGuiTableFlags (NkGuiTypes.h:132-140): Borders, RowBg, Resizable, Header, ResizableOuter, Sortable. Limites : NkGuiTableMaxCols = 16, et une seule table active à la fois — « pas d’imbrication v1 » (NkGuiContext.h:397).

3.11.9 Popups, combos, menus, infobulles

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:373-402

```
1 bool BeginCombo(NkGuiContext &ctx, const char *label, const char *preview, int32 itemCount,
2               float32 heightOverride = 0.f) noexcept;
3 void EndCombo(NkGuiContext &ctx) noexcept;
4 void EndPopup(NkGuiContext &ctx) noexcept;
5
6 bool BeginMenuBar(NkGuiContext &ctx, const NkRect &rect) noexcept;
7 void EndMenuBar(NkGuiContext &ctx) noexcept;
8 bool BeginMenu(NkGuiContext &ctx, const char *label) noexcept;
9 void EndMenu(NkGuiContext &ctx) noexcept;
10 bool MenuItem(NkGuiContext &ctx, const char *label, const char *shortcut = nullptr,
11              bool enabled = true) noexcept;
12 bool BeginPopupMenu(NkGuiContext &ctx, const char *idStr) noexcept;
```



```

13 void EndPopupMenu(NkGuiContext &ctx) noexcept;
14
15 void SetTooltip(NkGuiContext &ctx, const char *text) noexcept;

```

Usage de l'infobulle, documenté en :400-401:if (ctx.IsItemHovered()) SetTooltip(ctx, "...");. Pour un menu contextuel, l'application appelle ctx.OpenPopupAt(ctx.GetId(idStr), mousePos) au clic droit, puis dessine entre BeginPopupMenu et EndPopupMenu.

Voici, en passant, comment un menu contourne le décalage d'une image :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.cpp:4806-4812

```

1 // Ouverture au PRESS via test geometrique direct : ne depend PAS du
2 // survol resolu (hotIdPrev) de la frame precedente. Sinon un clic sur
3 // un titre jamais survole avant n'ouvrirait pas le menu au 1er coup.
4 // On N'utilise PAS `clicked` (relachement) ici : sinon press ouvrirait
5 // et release refermerait aussitot (double bascule).

```

3.11.10 Le contexte utilisé comme un widget

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:448-543 (sélection)

```

1 void PushId(const char *s); void PushId(const void *p); void PopId();
2 NkGuiId GetId(const char *s) const;
3 void BeginDisabled(bool disabled = true); void EndDisabled(); bool IsDisabled() const;
4 void BeginSelectList(const char *id, bool *mask, int32 count, NkGuiSelectFlags flags);
5 void ApplySelectClick(int32 idx); void EndSelectList();
6 void OpenPopup(NkGuiId); void OpenPopupAt(NkGuiId, NkVec2); void ClosePopup();
7 bool IsPopupOpen(NkGuiId) const;
8 bool IsHovered(const NkRect &r) const; bool IsItemHovered() const;
9 bool ItemHoverable(const NkRect &r, NkGuiId id);
10 NkString GetClipboard() const; void SetClipboard(const char *t);

```

3.12 Panneaux, fenêtres, docking

3.12.1 BeginPanel / EndPanel

Le conteneur le plus simple. Son code entier tient sur une page et vaut d'être lu en entier — il utilise presque tout ce que nous avons vu :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.cpp:2047-2078

```

1 bool BeginPanel(NkGuiContext &ctx, const char *title, const NkRect &r) noexcept {
2     ctx.DL().AddRectFilled(r, ctx.theme.panel, ctx.theme.rounding); // fond
3     float32 top = r.y;
4     if (title && *title) {
5         const float32 th = ctx.ItemHeight();
6         ctx.DL().AddRectFilled({r.x, r.y, r.w, th}, ctx.theme.header, ctx.theme.rounding);
7         ctx.DL().AddRectFilled({r.x, r.y + th - 1.f, r.w, 1.f}, ctx.theme.border); //
        séparateur
8         if (ctx.font && ctx.font->Valid()) {
9             ctx.DL().AddText(ctx.font->Face(), ctx.font->TexId(),
10                             {r.x + 10.f, r.y + (th - ctx.font->LineHeight()) * 0.5f +
11                             ctx.font->Ascent()},
12                             title, ctx.theme.text);
12     }

```



```

13     top = r.y + th;
14 }
15 // Bordure EN DERNIER → entoure tout le panneau (en-tête inclus), n'est
16 // plus recouverte par le fond de la barre de titre.
17 ctx.DL().AddRect(r, ctx.theme.border, 1.f);
18 // Le contenu (sous l'en-tête) est une ZONE DÉFILABLE : si les widgets
19 // dépassent la hauteur visible, une scrollbar apparaît ...automatiquement
20 const NkRect content = {r.x, top, r.w, r.y + r.h - top};
21 const NkGuiId id = ctx.GetId((title && *title) ? title : "##panel");
22 return BeginScrollFrame(ctx, id, content, false);
23 }
24
25 void EndPanel(NkGuiContext &ctx) noexcept { EndScrollFrame(ctx); }

```

Trois choses à en retirer : la formule de la ligne de base est exactement celle de la section 3.7; la bordure est dessinée *en dernier* pour ne pas être recouverte; et le contenu est automatiquement une zone défilable.

3.12.2 Begin / EndWindow

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:253-263

```

1 // Appeler End UNIQUEMENT si Begin retourne true (convention NKGui, comme
2 // BeginPanel).
3 // if (Begin(ctx, "Inspecteur", &open)) { .....widgets EndWindow(ctx); }
4 void SetNextWindowPos(NkGuiContext &ctx, float32 x, float32 y) noexcept;
5 void SetNextWindowSize(NkGuiContext &ctx, float32 w, float32 h) noexcept;
6 bool Begin(NkGuiContext &ctx, const char *title, bool *open = nullptr,
7            NkGuiWindowFlags flags = NkGuiWindowFlags::None) noexcept;
8 void EndWindow(NkGuiContext &ctx) noexcept;

```

La convention Begin/End de NKGui

On appelle **End*** uniquement si **Begin*** a retourné **true**. C'est vrai pour **Begin**, **BeginPanel**, **BeginCombo**, **BeginTable**, **BeginChild**, **BeginMenu**, **BeginPopupMenu**. Les conteneurs de layout (**BeginVBox**, **BeginHBox**...) ne retournent rien : ceux-là s'apparentent toujours.

NkGuiWindowFlags (**NkGuiTypes.h:245-252**) : **NoResize**, **NoMove**, **NoCollapse**, **NoTitleBar**, **NoClose**.

À la création, une fenêtre reçoit un décalage en cascade pour ne pas se superposer aux précédentes :

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.cpp:2286-2288

```

1 const float32 off = static_cast<float32>(ctx.windowMeta.Size() % 8) * 28.f;
2 nm.rect = {90.f + off, 80.f + off, 320.f, 210.f};

```

SetNextWindowPos et **SetNextWindowSize** ne s'appliquent qu'à la création — c'est la sémantique « la première fois seulement » — et sont consommés aussitôt (**ctx.hasNextPos** = **ctx.hasNextSize** = **false**; **cpp:2293**). Vous ne pouvez donc pas vous en servir pour forcer une position à chaque image; c'est délibéré, sans quoi l'utilisateur ne pourrait jamais déplacer la fenêtre.

Interactions avant dessin

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.cpp:2396-2397

```
1 // — PHASE 1 : INTERACTIONS — mettent à jour `wr` AVANT Le dessin (sinon le
2 //   contenu, dessiné à la nouvelle position, « court après » Le chrome). —
```

Si vous écrivez vous-même une surface déplaçable, traitez le glissement *avant* de dessiner, sinon le contenu accuse une image de retard sur le cadre et l'ensemble semble élastique.

3.12.3 Le docking

Kernel/Runtime/NKGui/src/NKGui/Widgets/NkGuiWidgets.h:271-306

```
1 void DockSpace(NkGuiContext &ctx, const char *idStr, const NkRect &rect) noexcept;
2 void DockBuilderDock(NkGuiContext &ctx, const char *windowTitle, int32 zone) noexcept;
3 void DockBuilderDockTab(NkGuiContext &ctx, const char *windowTitle, const char *targetTitle)
   noexcept;
4 bool DockFocusWindow(NkGuiContext &ctx, const char *windowTitle) noexcept;
5 bool DockIsWindowDocked(NkGuiContext &ctx, const char *windowTitle) noexcept;
6 int32 DockWindowNode(NkGuiContext &ctx, const char *windowTitle) noexcept;
7 void DockDetachWindow(NkGuiContext &ctx, const char *windowTitle) noexcept;
8 void DockWindowHideSingleTab(NkGuiContext &ctx, const char *windowTitle, bool hide) noexcept;
9 void DockSpaceOverViewport(NkGuiContext &ctx, float32 topMargin = 0.f) noexcept;
10 void DockWindowIntoWindow(NkGuiContext &ctx, const char *hostTitle, const char *winTitle)
   noexcept;
11 int32 DockTabAddRequest(NkGuiContext &ctx) noexcept;
12 void DockAddTab(NkGuiContext &ctx, const char *windowTitle, int32 node) noexcept;
```

Les zones de DockBuilderDock (NkGuiWidgets.h:273) : «0 = onglet, 1 = gauche, 2 = droite, 3 = haut, 4 = bas (sur la 1re feuille)».

DockSpace avant les Begin

« À appeler AVANT les Begin des fenêtres dockables » (NkGuiWidgets.h:266-267). L'espace de docking calcule les rectangles que les fenêtres ancrées viendront occuper; si les fenêtres sont déclarées d'abord, elles utilisent les rectangles de l'image précédente.

L'arbre de dock est un arbre binaire de nœuds (NkGuiDockNode, NkGuiTypes.h:288-299) : kind (0 = vide, 1 = division, 2 = feuille), vertical, ratio, child0/child1/parent, windows[8], activeTab, rect. Huit onglets au maximum par feuille.

3.13 Le hook de style

Dernier mécanisme, et le plus élégant : re-styler entièrement l'interface sans toucher à la logique des widgets.

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiTypes.h:301-325

```
1 // — Hook de style : override de DESSIN par widget —————
2 // L'app enregistre `ctx.styleFn` ; pour chaque élément visuel, NKGui l'appelle
3 // AVANT Le rendu par défaut. Si Le callback retourne true, il a dessiné lui-même
4 // (Le défaut est sauté) → re-skin TOTAL sans toucher la logique du widget.
5 enum class NkGuiStyleKind : uint8 {
6     Button = 0, FrameBg, CheckMark, Header, Selectable, DockTarget, Count
```

```

7  };
8
9  struct NkGuiStyleItem {
10      NkGuiStyleKind kind = NkGuiStyleKind::Button;
11      NkRect rect = {0.f, 0.f, 0.f, 0.f};
12      NkGuiId id = NKGUI_ID_NONE;
13      const char *label = nullptr;
14      bool hovered = false;
15      bool active = false;    ///< pressé / en cours d'édition / cible visée
16      bool selected = false;
17      bool disabled = false;
18      int32 value = 0;        ///< donnée par type (DockTarget :
                             0=onglet, 1=G, 2=D, 3=H, 4=B, 5=bord)
19  };

```

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:429-431

```

1  using NkGuiStyleFn = bool (*)(NkGuiContext &, const NkGuiStyleItem &, void *);
2  NkGuiStyleFn styleFn = nullptr;
3  void *styleUser = nullptr;

```

Le contrat est simple : votre fonction reçoit la description de l'élément à dessiner; si elle retourne true, elle a dessiné et NKGui saute son rendu par défaut; si elle retourne false, le rendu par défaut a lieu. Vous pouvez donc ne surcharger que les boutons et laisser le reste inchangé. La démo en donne un exemple complet, DemoStyle ([Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:193-238](#), branché ligne 271).

Ce qu'il faut retenir

Pour changer les *couleurs*, on écrase `ctx.theme`. Pour changer la *forme* — un bouton en biseau, une case à cocher dessinée comme un interrupteur — on branche `ctx.styleFn`. On ne modifie jamais `NkGuiWidgets.cpp`.

3.14 Les limites dures

Toutes ces constantes sont des tailles de tableaux fixes, décidées à la compilation. Les dépasser ne plante pas : le framework ignore silencieusement ce qui excède.

Limite	Valeur	Fichier :ligne
Popups imbriqués	8	NkGuiContext.h:147
Surfaces d'occlusion	16	NkGuiContext.h:175
Fenêtres	32	NkGuiContext.h:228
Zones défilables imbriquées	8	NkGuiContext.h:379
Conteneurs de layout	32	NkGuiContext.h:385
Pile d'identifiants	32	NkGuiContext.h:324
Pile « désactivé »	16	NkGuiContext.h:328
Colonnes de table	16	NkGuiContext.h:120
Onglets par feuille de dock	8	NkGuiTypes.h:295
Caractères saisis par image	32	NkGuiInput.h:53
Pile de découpe (draw-list)	32	NkGuiDrawList.h:52

3.15 Récapitulatif des idiomes

1. Récupérer la liste de dessin par `ctx.DL()`, jamais `ctx.dl` en dur.
2. Convertir un coin haut-gauche en ligne de base par `y + font->Ascent()`; centrer par `r.y + (r.h - font->LineHeight()) * 0.5f + font->Ascent()`.
3. Toujours vérifier `if (!ctx.font || !ctx.font->Valid())` avant `AddText` ou `MeasureWidth`, avec un repli chiffré.
4. `PushClipRect(zone, true) ...PopClipRect()` — toujours appariés, toujours avec intersection.
5. Ajouter un filtrage manuel du hors-vue en plus de la découpe.
6. Molette : `if (Hit(zone) && ctx.input.wheel != 0.f) { scroll -= wheel * pas; ctx.input.wheel = 0.f; }` puis bornage.
7. Glissement : poser `ctx.activeId = monId` à l'appui, agir tant que `ctx.activeId == monId && mouseDown[0]`, remettre `ctx.activeId = 0` au relâchement.
8. Préférer `ctx.InputHits(r)` et `ctx.ClickIn(r)` à un test géométrique brut.
9. Surface flottante : `ctx.PushOcclusion(rect, couche) + NkInputLayerScope`.
10. Toute taille en dur passe par `ctx.S(px)`.
11. Identifiant stable, distinct du libellé affiché ; `PushId/PopId` autour des éléments d'une liste.
12. Lire les couleurs dans `ctx.theme` et `ctx.syntax`, jamais de littéral RGBA dans le contenu.
13. Ne jamais recharger un atlas de police pendant une image : le différer avant `BeginFrame`.
14. Différer toute mutation d'état lourde (ouverture de fichier, reconstruction d'arbre, accès disque) hors du rendu : lever un drapeau, agir à l'image suivante.

3.16 Exercices

1 — Le compteur

Écrivez, dans la boucle de la démo, un petit panneau contenant : un `Text` affichant une valeur entière, un bouton « + » et un bouton « - ». La valeur est une variable locale de votre boucle. Vérifiez que le compteur ne s'incrémente qu'au *relâchement* du bouton, et qu'appuyer puis glisser en dehors avant de relâcher n'incrémente pas. Remplacez ensuite `Button` par `RepeatButton` et observez la différence.

2 — Le piège des identifiants

Affichez une liste de cinq éléments, chacun suivi d'un bouton « Supprimer ». Version A : sans `PushId`. Version B : avec `PushId(&items[i])` autour de chaque ligne. Observez ce qui se passe en version A quand vous survolez et cliquez. Expliquez le comportement à partir de `GetId` et de `ItemHoverable`.

3 — Comprendre le z-ordre

Dessinez deux boutons qui se chevauchent partiellement, le second déclaré après le premier. Cliquez dans la zone de recouvrement : lequel réagit ? Inversez l'ordre de déclaration et recommencez. Ajoutez ensuite un `logger.Info` qui affiche `ctx.hotId` et `ctx.hotIdPrev` à chaque image, et observez le décalage d'une image en déplaçant lentement la souris

d'un bouton à l'autre.

4 — Une modale correcte

Écrivez une boîte de dialogue modale maison : un rectangle centré dessiné dans `ctx.d10verlay` (via `PushOverlay/PopOverlay`), avec un fond assombri sur toute la fenêtre et deux boutons « Valider » et « Annuler ». Appliquez les gestes de la section 3.6 : `PushOcclusion(box, 100)`, `NkInputLayerScope`, et la fermeture au clic en dehors — en n'oubliant pas d'exiger `PointReachable` pour ce test négatif. Vérifiez qu'un clic sur la modale n'atteint pas les widgets qui sont dessous.

Construire une application, de zéro à l'exécutable

Nous avons vu la pile (chapitre 1), le rendu (chapitre 2) et les widgets (chapitre 3). Il reste à tout assembler. Ce chapitre construit une application complète, dans l'ordre, en expliquant à chaque étape *pourquoi* elle est là et *ce qui casse* si on l'oublie.

À la fin, vous aurez un programme qui compile, se lance, affiche une interface et répond à la souris et au clavier.

4.1 Vue d'ensemble du squelette

Avant le détail, voici la carte. Une application NKGui sur NkCanvas se construit en sept étapes, puis boucle.

Les sept étapes de l'initialisation

```

1 1. FENETRE      NkWindow + NkWindowConfig
2 2. CONTEXTE GPU NkContextDesc (choix de l'API graphique)
3 3. CIBLE DE RENDU NkRenderWindow (possede le cycle de frame)
4 4. CONTEXTE UI   NkGuiContext::Init + SetCurrentContext
5 5. BACKEND (PONT) NkGuiCanvasBackend::Init(target->GetRenderer())
6 6. POLICE        NkGuiFont::LoadEmbedded PUIS backend.UploadFontGray8
7 7. ENTREE        callbacks NKEvent -> ctx.input
8
9 BOUCLE :
10 evenements -> [resize] -> ctx.BeginFrame(dt) -> widgets -> ctx.EndFrame()
11 -> target->Clear() -> backend.Submit(ctx.dl) -> backend.Submit(ctx.dlOverlay)
12 -> target->Display()
```

L'ordre n'est pas indifférent. Le pont a besoin du renderer, donc de la cible. L'upload de l'atlas a besoin du pont *et* de la police chargée. Les *callbacks* ont besoin du contexte, puisqu'ils écrivent dedans.

4.2 Étape 1 — la fenêtre

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:243-251

```

1 NkWindow window;
2 NkWindowConfig cfg;
3 cfg.title = "NKGui - Demo Phase 2 (Coeur : drawlist + interaction)";
4 cfg.width = 1100;
5 cfg.height = 800;
6 cfg.centered = true;
7 cfg.resizable = true;
8 if (!window.Create(cfg))
9     return -1;
```

Rien de subtil, mais notez le test de retour : Create renvoie `false` si la fenêtre n'a pas pu être créée, et il n'y a pas d'exception à attraper. Chaque étape de cette initialisation suit le même modèle — retour booléen ou méthode `IsValid()` — et chacune doit être testée.

Les en-têtes nécessaires :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:7-11 (extrait)

```
1 #include "NKWindow/NKWindow.h"
2 #include "NKWindow/NKMain.h"
3 #include "NKEvent/NkWindowEvent.h"
4 #include "NKEvent/NkMouseEvent.h"
5 #include "NKEvent/NkKeyboardEvent.h"
```

4.3 Étape 2 et 3 — contexte GPU et cible de rendu

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:253-265

```
1     NkContextDesc desc;
2     desc.api = NkGraphicsApi::NK_GFX_API_AUTO;
3     if (desc.api == NkGraphicsApi::NK_GFX_API_AUTO) {
4 #if defined(NKENTSEU_PLATFORM_WINDOWS)
5         desc.api = NkGraphicsApi::NK_GFX_API_DX11;
6 #else
7         desc.api = NkGraphicsApi::NK_GFX_API_OPENGL;
8 #endif
9     }
10    auto target = memory::NkMakeUnique<NkRenderWindow>(window, desc);
11    if (!target || !target->IsValid())
12        return -1;
```

Trois observations :

- `NK_GFX_API_AUTO` est résolu ici, à la main, par une directive de préprocesseur. On aurait pu utiliser `NkContextFactory::CreateWithFallback` et laisser le moteur essayer une liste ordonnée; la démo préfère être explicite.
- La cible est créée par `memory::NkMakeUnique` et vivra jusqu'à la fin de `nkmain`. Elle possède le contexte graphique *et* le renderer 2D.
- Le double test `!target || !target->IsValid()` couvre les deux modes d'échec : l'allocation, et l'initialisation GPU.

Les en-têtes correspondants :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:12-16 (extrait)

```
1 #include "NKCanvas/Core/NkContextDesc.h"
2 #include "NKCanvas/Core/NkGraphicsApi.h"
3 #include "NKCanvas/Renderer/Targets/NkRenderWindow.h"
4 #include "NKTime/NkClock.h"
5 #include "NKMemory/NkUniquePtr.h"
```

Ce qu'il faut retenir

Si votre fenêtre reste noire alors que le programme tourne, changez d'API graphique avant de chercher dans votre code. Rappel du chapitre 1 : seul OpenGL était validé à l'exécution au 30 mai 2026 selon [Kernel/Runtime/NKCanvas/ROADMAP.md](#), et DX11 comme le rasteriseur logiciel affichaient un écran vide. Le journal ([logs/app.log](#)) vous dit quel backend a réellement été retenu.

4.4 Étape 4 — le contexte NKGui

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:266-272

```
1 auto ctxPtr = memory::NkMakeUnique<NKGuiContext>();
2 if (!ctxPtr)
3     return -1;
4 NKGuiContext &ctx = *ctxPtr;
5 ctx.Init(static_cast<int32>(cfg.width), static_cast<int32>(cfg.height));
6 ctx.styleFn = &DemoStyle; // hook de style (override de dessin par widget)
7 SetCurrentContext(&ctx);
```

Le contexte est volumineux — plus de cent champs et trente-deux listes de dessin de fenêtres — ce qui explique qu'on l'alloue plutôt que de le poser sur la pile. On prend ensuite une référence, ce qui rend tout le code des widgets lisible.

Init(width, height) pose la taille de vue initiale et prépare les structures. SetCurrentContext installe le contexte courant du fil d'exécution; la ligne ctx.styleFn est facultative.

C'est ici, juste après Init, que vous personnaliserez le thème :

Personnalisation du thème — écrit pour ce cours, d'après NkEditorShell.cpp:155-179

```
1 ctx.Init(1100, 800);
2 ctx.theme.bgPrimary = {13, 17, 23, 255};
3 ctx.theme.panel = {22, 27, 34, 255};
4 ctx.theme.accent = {88, 166, 255, 255};
5 ctx.theme.rounding = 6.f;
6 SetCurrentContext(&ctx);
```

4.5 Étape 5 — le pont vers NKCanvas

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:274-276

```
1 renderer::NKGuiCanvasBackend backend;
2 if (!backend.Init(target->GetRenderer()))
3     return -1;
```

Deux lignes, et c'est tout le mariage entre les deux modules. L'en-tête à inclure est celui qui vit dans NKCanvas :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:25

```
1 #include "NKCanvas/UI/NKGuiCanvasBackend.h" // backend NKGui->NKCanvas (lib réutilisable)
```


Rappelez-vous du chapitre 1 : ce fichier est *header-only*. Il n'est compilé nulle part ailleurs que dans votre propre unité de traduction. C'est pour cela que votre .jenga doit dépendre à la fois de NKGui et de NKCanvas.

backend est déclaré sur la pile de nkmain. Son destructeur libère les textures qu'il a créées :

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:24-32

```
1 ~NkGuiCanvasBackend() {
2     auto &alloc = nkentseu::memory::NkGetDefaultAllocator();
3     for (uint32 i = 0; i < mFonts.Size(); ++i) if (mFonts[i].tex)
4         alloc.Delete(mFonts[i].tex);
5     for (uint32 i = 0; i < mImages.Size(); ++i) if (mImages[i].tex)
6         alloc.Delete(mImages[i].tex);
7 }
```

L'ordre de destruction

Le pont détruit des textures GPU, donc il doit mourir *avant* le renderer qui les a créées. Comme backend est déclaré *après* target dans nkmain, il est détruit *avant* — les objets locaux se détruisent dans l'ordre inverse de leur déclaration. Cet ordre est correct par construction; inverser les deux déclarations produirait une libération de textures sur un renderer déjà mort.

4.6 Étape 6 — la police, et l'upload de son atlas

C'est l'étape où l'on se trompe le plus souvent, parce qu'elle est en deux temps et que rien ne signale l'oubli du second.

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:281-290

```
1 auto fontPtr = memory::NkMakeUnique<NkGuiFont>();
2 if (!fontPtr->LoadEmbedded(NkEmbeddedFontId::DroidSans, 18.f)) {
3     fontPtr->LoadEmbedded(NkEmbeddedFontId::ProggyClean, 16.f);
4 }
5 ctx.font = fontPtr.Get();
6 if (fontPtr->Valid()) {
7     backend.UploadFontGray8(fontPtr->TexId(), fontPtr->pixels, fontPtr->atlasW,
8     fontPtr->atlasH);
9 }
```

4.6.1 Ce que fait LoadEmbedded

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiFont.h:19-40 (abrégé)

```
1 struct NKENTSEU_NKGUI_CLASS_EXPORT NkGuiFont {
2     NkFontAtlas atlas;          ///< possède la texture + les glyphes
3     NkFont *face = nullptr;     ///< face produite (détenue par l'atlas)
4     uint32 texId = 0x4E4B4654u; ///< 'NKFT' - id stable pour le backend
5     uint8 *pixels = nullptr;    ///< atlas alpha8 (détenu par l'atlas)
6     int32 atlasW = 0;
7     int32 atlasH = 0;
8     bool dirty = false;         ///< à (ré)uploader côté backend
9
10    NkGuiFont() = default;
11    NkGuiFont(const NkGuiFont &) = delete;           // NON COPIABLE
```

```

12     NkGuiFont &operator=(const NkGuiFont &) = delete;
13
14     bool LoadEmbedded(NkEmbeddedFontId id, float32 sizePx, bool extFallback = true)
noexcept;
15     bool LoadFromFile(const char *path, float32 sizePx, bool extFallback = true) noexcept;

```

L'appel rasterise les glyphes dans un atlas et laisse pixels pointer sur un bitmap en niveaux de gris (un octet par pixel, l'alpha). **Cet atlas est en mémoire centrale ; le GPU ne le connaît pas encore.**

Les polices embarquées disponibles (`Kernel/Runtime/NKFont/src/NKFont/Embedded/NkFontEmbedded.h:49-67`) : ProgyClean, ProgyTiny (bitmap monospace), DroidSans, Karla, Roboto (sans-serif), Cousine, SourceCodePro (monospace vectorielles), DroidSerif, DejaVuSansMono.

4.6.2 Ce que fait UploadFontGray8

Le pont convertit l'atlas en RGBA — blanc partout, l'alpha portant la forme du glyphe — et crée la texture GPU :

`Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:47-55`

```

1  const usize n = (usize)w * (usize)h;
2  mExpand.Resize(n * 4u);
3  uint8 *d = mExpand.Data();
4  for (usize i = 0; i < n; ++i) {
5      d[i*4+0] = 255u; d[i*4+1] = 255u; d[i*4+2] = 255u; d[i*4+3] = gray[i];
6  }

```

C'est là que le `texId` prend son sens : le pont range la texture dans une table indexée par cet identifiant, et `Submit` la retrouvera quand une commande de dessin portera le même. La constante par défaut est `0x4E4B4654` — les quatre lettres «NKFT». Elle n'est **jamais nulle**, car zéro est réservé à la texture blanche interne côté RHI.

Oublier l'upload : du texte parfaitement invisible

Si vous chargez la police et posez `ctx.font` mais oubliez `UploadFontGray8`, il ne se passe *rien* de visible et *aucun* message n'apparaît. Les widgets se dessinent (leurs rectangles sont là), le layout est correct — mais chaque commande de texte référence un `texId` que le pont ne connaît pas, et elle est silencieusement ignorée.

Symptôme : des boutons vides, des cases à cocher sans étiquette, un panneau au titre absent. Diagnostic : vérifiez que `fontPtr->Valid()` est vrai, et que l'upload est bien appelé après le chargement.

La police ne doit jamais mourir avant la boucle

`Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiContext.h:141-142 (sens)`

```

1  NkGuiFont *font = nullptr;    ///< pointeur BRUT, NON POSSEDE
2  NkGuiFont *codeFont = nullptr; ///< idem

```

`ctx.font = fontPtr.Get()` donne au contexte un pointeur *brut*. Le contexte ne possède rien. Si l'objet `NkGuiFont` est détruit — parce qu'il était local à une fonction d'initialisation, par exemple — le contexte pointe sur de la mémoire libérée et la première mesure de texte plante.

La règle : l'objet police doit vivre au moins aussi longtemps que la boucle. La démo le tient dans un `NkUniquePtr` déclaré dans `nkmain`, à côté de tout le reste.

Recharger un atlas à une autre taille

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:65-66

```
1 // (Re)créer la texture si elle n'existe pas OU si la taille change (rechargement
2 // de police à une autre taille = zoom / DPI). Sinon Update() déborderait l'ancienne
   taille.
```

Le pont gère ce cas pour vous. Mais **vous** devez veiller à ne jamais recharger une police au milieu d'une image : les commandes déjà émises référenceraient un atlas remplacé, et leurs coordonnées de texture ne correspondraient plus. Faites-le avant `BeginFrame`, comme la démo le fait pour la touche F9.

4.6.3 Les polices de repli

Pour les alphabets que la police principale ne couvre pas :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiFont.h:71-78

```
1 // Polices de REPLI EXTERNES (fichiers .ttf charges au runtime) : tout glyphe
2 // absent des polices principales (Inter/DejaVu) y est cherché. Rôles :
3 //   broad : large couverture (Latin étendu, grec, cyrillique, hébreu, arabe, symboles)
4 //   cjk   : ideogrammes (chinois/japonais/coreen) - volumineux, opt-in
5 //   emoji : emoji monochrome
6 // A poser par L'APPLICATION (ex. NKCode, depuis son dossier data/fonts) AVANT
7 // de charger les polices. Un chemin nullptr/vidé = rôle désactivé.
8 void NkSetFallbackFontPaths(const char *broad, const char *cjk, const char *emoji) noexcept;
```

Avant de charger la police, donc. C'est un invariant d'ordre : l'appel place des chemins dans des variables globales que la construction de l'atlas consultera.

Pour une police monospace destinée au code, on désactive au contraire les replis (`extFallback = false`) : « atlas minuscule => reconstruction RAPIDE au zoom » (`NkGuiFont.h:34-35`).

4.7 Étape 7 — brancher l'entrée

Les *callbacks* écrivent directement dans `ctx.input`. Voici l'ensemble minimal, puis les compléments.

4.7.1 Souris et fermeture

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:326-343 (extrait)

```
1 auto &events = NkEvents();
2 bool running = true;
3 events.AddEventCallback<NkWindowCloseEvent>([&](NkWindowCloseEvent *) { running = false;
   });
4
5 events.AddEventCallback<NkMouseMoveEvent>([&](NkMouseMoveEvent *e) {
```

```

6      ctx.input.mousePos = {static_cast<float32>(e->GetX()),
static_cast<float32>(e->GetY())};
7  });
8  events.AddEventCallback<NkMouseButtonPressEvent>([&](NkMouseButtonPressEvent *e) {
9      if (e->GetButton() == NkMouseButton::NK_MB_LEFT) ctx.input.mouseDown[0] = true;
10     if (e->GetButton() == NkMouseButton::NK_MB_RIGHT) ctx.input.mouseDown[1] = true;
11     ctx.input.ctrlDown = e->GetModifiers().ctrl;
12     ctx.input.shiftDown = e->GetModifiers().shift;
13     ctx.input.altDown = e->GetModifiers().alt;
14 });
15 events.AddEventCallback<NkMouseButtonReleaseEvent>([&](NkMouseButtonReleaseEvent *e) {
16     if (e->GetButton() == NkMouseButton::NK_MB_LEFT) ctx.input.mouseDown[0] = false;
17     if (e->GetButton() == NkMouseButton::NK_MB_RIGHT) ctx.input.mouseDown[1] = false;
18 });

```

Indices des boutons : 0 = gauche, 1 = droit, 2 = milieu.

4.7.2 La molette

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:344-352 (extrait)

```

1      events.AddEventCallback<NkMouseWheelVerticalEvent>([&](NkMouseWheelVerticalEvent *e) {
2          ctx.input.wheel += static_cast<float32>(e->GetDeltaY()); // >0 = haut ; consommé en
EndFrame
3          const auto m = e->GetModifiers(); // modificateurs au moment
de la molette
4          ctx.input.ctrlDown = m.ctrl; ctx.input.shiftDown = m.shift; ctx.input.altDown = m.alt;
5      });
6      events.AddEventCallback<NkMouseWheelHorizontalEvent>([&](NkMouseWheelHorizontalEvent *e) {
7          ctx.input.wheelH += static_cast<float32>(e->GetDeltaX());
8      });

```

Notez le += et non le = : plusieurs crans peuvent arriver dans la même image, ils s'accumulent. EndFrame remet le compteur à zéro. Notez aussi la relecture des modificateurs : sans elle, le Ctrl+molette (zoom) ne fonctionnerait pas de façon fiable.

4.7.3 Le double-clic

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:353-359 (extrait)

```

1      // Double-clic détecté par L'OS : NKeyEvent L'émet comme événement DÉDIÉ (le 2e
2      // clic n'arrive PAS comme un mouseDown normal) → on l'injecte explicitement.
3      events.AddEventCallback<NkMouseDoubleClickEvent>([&](NkMouseDoubleClickEvent *e) {
4          ctx.input.mousePos = {static_cast<float32>(e->GetX()),
static_cast<float32>(e->GetY())};
5          if (e->GetButton() == NkMouseButton::NK_MB_LEFT) ctx.input.SetDoubleClick(0);
6      });

```

Sans ce *callback*, le second clic d'un double-clic n'existe tout simplement pas pour NKGui : le système l'a converti en événement dédié. Les renommages en place et la saisie au clavier dans les DragFloat, qui reposent sur le double-clic, ne fonctionneraient pas.

4.7.4 Le texte et les touches

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:360-380 (extrait)

```
1 // Saisie texte + touches d'édition (pour InputText). On pose l'état ENFONCÉ
2 // (press/release) pour permettre la répétition au maintien.
3 events.AddEventCallback<NkTextInputEvent>([&](NkTextInputEvent *e) {
4     ctx.input.PushChar(e->GetCodepoint()); });
5
6 auto setKey = [&](NkKey k, bool down) {
7     switch (k) {
8         case NkKey::NK_LEFT:   ctx.input.SetKey(NkGuiKey::Left, down);   break;
9         case NkKey::NK_RIGHT:  ctx.input.SetKey(NkGuiKey::Right, down);  break;
10        case NkKey::NK_UP:     ctx.input.SetKey(NkGuiKey::Up, down);      break;
11        case NkKey::NK_DOWN:   ctx.input.SetKey(NkGuiKey::Down, down);    break;
12        case NkKey::NK_HOME:   ctx.input.SetKey(NkGuiKey::Home, down);    break;
13        case NkKey::NK_END:    ctx.input.SetKey(NkGuiKey::End, down);     break;
14        case NkKey::NK_BACK:   ctx.input.SetKey(NkGuiKey::Backspace, down); break;
15        case NkKey::NK_DELETE: ctx.input.SetKey(NkGuiKey::Delete, down);  break;
16        case NkKey::NK_ENTER:  ctx.input.SetKey(NkGuiKey::Enter, down);   break;
17        case NkKey::NK_ESCAPE: ctx.input.SetKey(NkGuiKey::Escape, down);  break;
18        default: break;
19    }
20 };
21 events.AddEventCallback<NkKeyPressEvent>([&](NkKeyPressEvent *e) { setKey(e->GetKey(),
22     true); ... });
23 events.AddEventCallback<NkKeyReleaseEvent>([&](NkKeyReleaseEvent *e){ setKey(e->GetKey(),
24     false); ... });
```

Deux points essentiels, déjà annoncés au chapitre 1 :

- on pose l'état **enfoncé/relâché**, pas un « appui » ponctuel. C'est ce qui permet à NKGui de calculer les durées et donc la répétition au maintien;
- le **texte** passe par PushChar et les **touches** par SetKey. Ce sont deux canaux séparés.

Il reste à brancher les raccourcis d'édition, sur le modèle du shell de l'éditeur :

Engine/NKEditorKit/src/NKEditorKit/NkEditorShell.cpp:318-326 (extrait)

```
1 if (e->GetModifiers().ctrl) { // raccourcis copier/couper/coller/tout-selectionner
2     if (k == NkKey::NK_C) mUI.input.wantCopy = true;
3     else if (k == NkKey::NK_X) mUI.input.wantCut = true;
4     else if (k == NkKey::NK_V) mUI.input.wantPaste = true;
5     else if (k == NkKey::NK_A) mUI.input.wantSelectAll = true;
```

4.7.5 Le presse-papiers

NKGui ne connaît pas la fenêtre, donc il ne peut pas accéder au presse-papiers du système. On lui donne deux pointeurs de fonction (ctx.clipboardGetFn, ctx.clipboardSetFn, plus un pointeur utilisateur ctx.clipboardUser); c'est ce que fait le shell en [NkEditorShell.cpp:185-190](#). Sans cela, copier-coller dans un champ de saisie ne fait rien.

4.8 La boucle principale

4.8.1 Le temps

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:431-436

```
1 while (running && window.IsOpen()) {
2     float32 dt = clock.Tick().delta;
3     if (dt <= 0.f)
4         dt = 1.f / 60.f;
5     if (dt > 0.1f)
6         dt = 0.1f;
```

Les deux bornes ont chacune leur raison : $dt \leq 0$ arrive à la toute première image (l'horloge n'a pas encore de référence); $dt > 0.1$ arrive après un blocage — déplacement de fenêtre, mise en veille, point d'arrêt du débogueur. Sans la borne haute, tout ce qui dépend du temps ferait un bond : le curseur de saisie clignoterait plusieurs fois d'un coup, les répétitions au maintien se déclencheraient en rafale.

4.8.2 Les événements — avant tout le reste

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:438-442

```
1 while (NkEvent *ev = NkEvents().PollEvent()) {
2     (void)ev;
3 }
4 if (!running)
5     break;
```

Le `if (!running) break;` évite de dessiner une image entière sur une fenêtre qu'on vient de fermer.

4.8.3 Le redimensionnement

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:444-458

```
1 // Synchroniser la SWAPCHAIN à la taille de la FENÊTRE. target->GetSize()
2 // renvoie la taille de la swapchain (qui ne change QU'À OnResize) - s'en
3 // servir pour détecter le resize était faux : la swapchain restait figée à
4 // sa taille de création (rendu basse-résolution étiré). On pilote OnResize
5 // depuis la taille fenêtre (inclut la 1re frame → sync initiale).
6 const math::NkVec2u wsz = target->GetWindow().GetSize();
7 if (wsz.x > 0 && wsz.y > 0 && (wsz.x != lastW || wsz.y != lastH)) {
8     target->OnResize(wsz.x, wsz.y);
9     lastW = wsz.x;
10    lastH = wsz.y;
11 }
12 const math::NkVec2u sz = target->GetSize(); // = wsz après OnResize
13 if (sz.x > 0 && sz.y > 0) {
14     ctx.viewW = static_cast<int32>(sz.x);
15     ctx.viewH = static_cast<int32>(sz.y);
16 }
```

`lastW` et `lastH` sont initialisés à zéro, ce qui garantit que la première image déclenche une synchronisation. Et les tests > 0 évitent de recréer une chaîne d'échange de taille nulle quand la fenêtre est réduite.

Fenêtre minimisée

Une fenêtre en cours de réduction glisse un rectangle intermédiaire — environ 160×28 sous Windows — entre le test « minimisée ? » et la lecture de taille. Le shell de l'éditeur documente une garde qui refuse toute taille sous 32 pixels :

Integrations/NKGui/NkEditorRHIRenderer.h:123-130 (extrait)

```
1 // GARDE ANTI-MINIMISATION : une fenetre en cours de reduction
2 // glisse son rect placeholder (~160x28 sous Windows) entre le
3 // test « minimisee ? » de la boucle principale et la lecture
4 // de taille -- la course est reelle (defaut 4.3, reproduite).
5 // Sous 32 px, les cibles divisees du rendu (bloom /32) tombent
6 // a zero et CreateTexture2D echoue en E_INVALIDARG -> mort a
7 // la restauration. On refuse net : la vraie taille arrivera
8 // avec le retour de la fenetre.
```

4.8.4 L'image d'interface

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:468-481 (extrait)

```
1     ctx.BeginFrame(dt);
2     NkGuiDrawList &d1 = ctx.dl;
3     const float32 W = static_cast<float32>(ctx.viewW);
4     const float32 H = static_cast<float32>(ctx.viewH);
5
6     // Fond + barre d'en-tête + TITRE (texte NKFont).
7     d1.AddRectFilled({0.f, 0.f, W, H}, ctx.theme.bgPrimary);
8     d1.AddRectFilled({0.f, 0.f, W, 40.f}, ctx.theme.header);
9     d1.AddRectFilled({0.f, 38.f, W, 2.f}, ctx.theme.accent);
10    TextAt(ctx, {16.f, 11.f}, "NKGui - Phase 3 : layout + widgets (NKFont)");
```

Le fond est dessiné en premier — tout le reste passera par-dessus. Ici la démo écrit dans `ctx.d1` directement, ce qui est légitime parce qu'aucune fenêtre n'est ouverte à ce moment : nous sommes bien sur la couche principale. Dès qu'on est à l'intérieur d'un `Begin`, il faut passer par `ctx.DL()` (chapitre 3).

Puis les widgets, avec une région de layout ouverte :

Squelette de widgets — écrit pour ce cours

```
1     ctx.BeginLayout({20.f, 60.f, 400.f, H - 80.f});
2     Text(ctx, "Bonjour NKGui");
3     if (Button(ctx, "Cliquez-moi")) {
4         logger.Info("clac !");
5     }
6     static bool visible = true;
7     Checkbox(ctx, "Afficher le panneau", visible);
8     static float32 zoom = 1.f;
9     SliderFloat(ctx, "Zoom", zoom, 0.5f, 4.f);
```

4.8.5 Le curseur

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:1020-1041

```
1      // Curseur souhaité par NKGui → curseur OS (OPTIONNEL : L'app choisit d'appliquer).
2      // Ex. DragFloat pose ↕ / ⇅ pendant Le survol/glisser. SetCursor est persistant.
3      {
4          NkWindow::NkCursorType c = NkWindow::NkCursorType::Arrow;
5          switch (ctx.wantCursor) {
6              case NkGuiCursor::Text:
7                  c = NkWindow::NkCursorType::TextInput;
8                  break;
9              case NkGuiCursor::Hand:
10                 c = NkWindow::NkCursorType::Hand;
11                 break;
12              case NkGuiCursor::ResizeEW:
13                 c = NkWindow::NkCursorType::ResizeWE;
14                 break;
15              case NkGuiCursor::ResizeNS:
16                 c = NkWindow::NkCursorType::ResizeNS;
17                 break;
18              default:
19                 break;
20          }
21          window.SetCursor(c);
22      }
```

Rappel du chapitre 3 : NKGui *demande* un curseur en posant `ctx.wantCursor`, il ne le pose pas — il ne connaît pas la fenêtre. Ce bloc est le traducteur, et il est facultatif : sans lui, tout fonctionne, mais le pointeur ne change jamais de forme au-dessus d'un champ de saisie ou d'un séparateur. Notez qu'il est placé *avant* `EndFrame`, parce que `wantCursor` est réinitialisé par `BeginFrame` et rempli par les widgets.

4.8.6 La présentation

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:1043-1048

```
1      ctx.EndFrame();
2
3      target->Clear();
4      backend.Submit(dl, sz.x, sz.y); // couche principale
5      backend.Submit(ctx.dlOverlay, sz.x, sz.y); // couche popups/overlay (au-dessus)
6      target->Display(); // Capture d'ecran (F12) : APRES Display() – La frame presentee.
      Self-capture
```

DEUX Submit, jamais un seul

C'est le piège le plus coûteux de tout ce cours, parce qu'il est absolument silencieux. NKGui produit **deux** listes de dessin :

- `ctx.dl` — le contenu ordinaire, y compris les fenêtres fusionnées par ordre de profondeur;
- `ctx.dlOverlay` — les popups, les menus déroulants, les sous-menus, les combos, les infobulles, et tout ce que vous dessinez entre `PushOverlay` et `PopOverlay`.

Si vous n'appellez `Submit` que sur la première, **tout fonctionne sauf les surfaces flottantes**. Le menu s'ouvre — l'état est correct, les clics sont capturés, la logique tourne — mais rien

ne s'affiche. Vous cliquez dans le vide sur des éléments invisibles.
Et il n'y a *aucun* message d'erreur : le framework a fait son travail, le pont aussi; c'est simplement qu'on ne lui a jamais donné la seconde liste.

L'ordre compte aussi : ctx.dl d'abord, ctx.dlOverlay ensuite. Inversé, les popups passeraient *sous* le contenu.

Le shell de l'éditeur fait exactement la même chose, à travers son interface de renderer :

Engine/NKEditorKit/src/NKEditorKit/NkEditorShell.cpp:725-732

```
1 mUI.EndFrame();
2
3 mWindow.SetCursor(MapCursor(mUI.wantCursor));
4
5 mRenderer->BeginFrame();
6 mRenderer->SubmitDrawList(mUI.dl, sz.x, sz.y);
7 mRenderer->SubmitDrawList(mUI.dlOverlay, sz.x, sz.y);
8 mRenderer->EndFrame();
```

4.8.7 Ce que fait Submit, en détail

Pour comprendre ce qui peut mal tourner, voici la traduction complète :

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:120-195 (condensé)

```
1 void Submit(const NkGuiDrawList &dl, uint32 fbW, uint32 fbH) {
2     if (!mRenderer || dl.vtx.Size() == 0 || dl.idx.Size() == 0) return;
3
4     // 1) conversion des sommets NkGui -> NkVertex2D (couleur dépaquetée)
5     mScratch.Resize(dl.vtx.Size());
6     for (uint32 i = 0; i < dl.vtx.Size(); ++i) {
7         const nkgui::NkGuiVertex &s = dl.vtx[i];
8         renderer::NkVertex2D &d = mScratch[i];
9         d.x = s.pos.x; d.y = s.pos.y; d.u = s.uv.x; d.v = s.uv.y;
10        d.r = (uint8)( s.col & 0xFF);
11        d.g = (uint8)((s.col >> 8) & 0xFF);
12        d.b = (uint8)((s.col >> 16) & 0xFF);
13        d.a = (uint8)((s.col >> 24) & 0xFF);
14    }
15
16    // 2) une passe par commande
17    for (uint32 ci = 0; ci < dl.cmds.Size(); ++ci) {
18        const nkgui::NkGuiDrawCmd &dc = dl.cmds[ci];
19        if (dc.idxCount == 0u) continue;
20
21        // 2a) clip : Le rect « infini » (1e9) signifie « pas de clip »
22        const bool hasClip = (dc.clipRect.w < 1.0e8f && dc.clipRect.h < 1.0e8f);
23        if (hasClip) {
24            ... clamp a [0, fbW] x [0, fbH] ...
25            if (x1 <= x0 || y1 <= y0) continue; // clip vide -> on saute
26            mRenderer->SetClip(math::NkRect2i...{});
27        }
28
29        // 2b) résolution de la texture : d'abord les atlas de police, sinon les images
30        renderer::NkTexture *tex = nullptr;
31        if (dc.type == nkgui::NkGuiDrawCmdType::TexturedTriangles) { ... }
32
33        // 2c) sous-ensemble de sommets + indices REBASÉS
34        uint32 lo = 0xFFFFFFFFu, hi = 0u;
```

```

35     for (uint32 k = 0; k < dc.idxCount; ++k) { const uint32 v = dl.idx[dc.idxOffset+k];
36                                             if (v<lo) lo=v; if (v>hi) hi=v; }
37     mIdxTmp.Resize(dc.idxCount);
38     for (uint32 k = 0; k < dc.idxCount; ++k) mIdxTmp[k] = dl.idx[dc.idxOffset+k] - lo;
39     mRenderer->DrawVertices(mScratch.Data() + lo, hi - lo + 1u, mIdxTmp.Data(),
    dc.idxCount, tex);
40
41     if (hasClip) mRenderer->PopClip();
42 }
43 }

```

L'étape (2c) mérite l'attention, parce qu'elle corrige un défaut qui coûtait un plantage :

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:176-178

```

1 // Ne soumet que le SOUS-ENSEMBLE de vertices reference par cette
2 // commande (indices rebases). Indispensable : passer tout le buffer
3 // depasse kMaxVertices (65536) des qu'un draw list est gros -> crash.

```

Le pont calcule l'indice minimal et maximal utilisés par la commande, n'envoie que cette tranche de sommets, et *décale tous les indices* d'autant. Sans cela, chaque commande enverrait tout le tampon.

(Le commentaire mentionne 65 536; la constante actuelle est 262 144 — voir chapitre 2, section 2.10. Le commentaire garde la trace du dimensionnement d'origine, et c'est la même famille de bug qui a motivé l'agrandissement.)

4.9 Le programme complet

Rassemblons tout dans un fichier minimal mais fonctionnel. Ce squelette est reconstitué fidèlement depuis `Applications/NKGuiDemo/src/NKGuiDemo/main.cpp` (lignes 240 à 1067), réduit au strict nécessaire.

Squelette complet — reconstitué depuis `Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:240-1067`

```

1 #include "NKWindow/NKWindow.h"
2 #include "NKWindow/NKMain.h"
3 #include "NKEvent/NkWindowEvent.h"
4 #include "NKEvent/NkMouseEvent.h"
5 #include "NKEvent/NkKeyboardEvent.h"
6 #include "NKCanvas/Core/NkContextDesc.h"
7 #include "NKCanvas/Core/NkGraphicsApi.h"
8 #include "NKCanvas/Renderer/Targets/NkRenderWindow.h"
9 #include "NKTime/NkClock.h"
10 #include "NKMemory/NkUniquePtr.h"
11 #include "NKLogger/NkLog.h"
12 #include "NKGui/NKGui.h"
13 #include "NKCanvas/UI/NkGuiCanvasBackend.h"
14
15 using namespace nkentseu;
16 using namespace nkentseu::nkgui;
17 using namespace nkentseu::renderer;
18
19 NKENTSEU_DEFINE_APP_DATA([]() {
20     NkAppData d{};
21     d.appName = "MonApp";
22     d.appVersion = "0.1.0";
23     return d;

```

```

24  }());
25
26  int nkmain(const NkEntryState &state) {
27      (void)state;
28
29      // — 1) FENETRE OS (NkWindow) —————
30      NkWindow window;
31      NkWindowConfig cfg;
32      cfg.title = "Mon app NKGui";
33      cfg.width = 1100; cfg.height = 800;
34      cfg.centered = true; cfg.resizable = true;
35      if (!window.Create(cfg)) return -1;
36
37      // — 2) CONTEXTE GRAPHIQUE + CIBLE DE RENDU (NkCanvas) —————
38      NkContextDesc desc;
39      desc.api = NkGraphicsApi::NK_GFX_API_AUTO;
40      if (desc.api == NkGraphicsApi::NK_GFX_API_AUTO) {
41  #if defined(NKENTSEU_PLATFORM_WINDOWS)
42      desc.api = NkGraphicsApi::NK_GFX_API_DX11;
43  #else
44      desc.api = NkGraphicsApi::NK_GFX_API_OPENGL;
45  #endif
46      }
47      auto target = memory::NkMakeUnique<NkRenderWindow>(window, desc);
48      if (!target || !target->IsValid()) return -1;
49
50      // — 3) CONTEXTE NKGui —————
51      auto ctxPtr = memory::NkMakeUnique<NkGuiContext>();
52      if (!ctxPtr) return -1;
53      NkGuiContext &ctx = *ctxPtr;
54      ctx.Init(static_cast<int32>(cfg.width), static_cast<int32>(cfg.height));
55      SetCurrentContext(&ctx);
56
57      // — 4) BACKEND (NKGui -> NKCanvas) —————
58      renderer::NkGuiCanvasBackend backend;
59      if (!backend.Init(target->GetRenderer())) return -1;
60
61      // — 5) POLICE : charger PUIS uploader L'atlas —————
62      auto fontPtr = memory::NkMakeUnique<NkGuiFont>(); // DOIT survivre a la boucle
63      if (!fontPtr->LoadEmbedded(NkEmbeddedFontId::DroidSans, 18.f)) {
64          fontPtr->LoadEmbedded(NkEmbeddedFontId::ProggyClean, 16.f);
65      }
66      ctx.font = fontPtr.Get(); // pointeur brut NON possede
67      if (fontPtr->Valid()) {
68          backend.UploadFontGray8(fontPtr->TexId(), fontPtr->pixels, fontPtr->atlasW,
fontPtr->atlasH);
69      }
70
71      // — 6) GLUE D'ENTREE (callbacks NkEvent -> ctx.input) —————
72      auto &events = NkEvents();
73      bool running = true;
74      events.AddEventCallback<NkWindowCloseEvent>([&](NkWindowCloseEvent *) { running = false;
});
75      events.AddEventCallback<NkMouseMoveEvent>([&](NkMouseMoveEvent *e) {
76          ctx.input.mousePos = {static_cast<float32>(e->GetX()),
static_cast<float32>(e->GetY())};
77      });
78      events.AddEventCallback<NkMouseButtonPressEvent>([&](NkMouseButtonPressEvent *e) {
79          if (e->GetButton() == NkMouseButton::NK_MB_LEFT) ctx.input.mouseDown[0] = true;
80      });
81      events.AddEventCallback<NkMouseButtonReleaseEvent>([&](NkMouseButtonReleaseEvent *e) {
82          if (e->GetButton() == NkMouseButton::NK_MB_LEFT) ctx.input.mouseDown[0] = false;

```

```

83     });
84     events.AddEventCallback<NkMouseWheelVerticalEvent>([&](NkMouseWheelVerticalEvent *e) {
85         ctx.input.wheel += static_cast<float32>(e->GetDeltaY());
86     });
87     events.AddEventCallback<NkTextInputEvent>([&](NkTextInputEvent *e) {
88         ctx.input.PushChar(e->GetCodepoint());
89     });
90
91     NkClock clock;
92     uint32 lastW = 0, lastH = 0;
93     bool showPanel = true;
94     float32 zoom = 1.f;
95     char nom[64] = "sans titre";
96
97     // — 7) BOUCLE PRINCIPALE —————
98     while (running && window.IsOpen()) {
99         float32 dt = clock.Tick().delta;
100         if (dt <= 0.f) dt = 1.f / 60.f;
101         if (dt > 0.1f) dt = 0.1f; // borne anti-saut
102
103         // a) EVENEMENTS — AVANT BeginFrame (BeginFrame appelle input.NewFrame())
104         while (NkEvent *ev = NkEvents().PollEvent()) { (void)ev; }
105         if (!running) break;
106
107         // b) Synchroniser La SWAPCHAIN sur La taille de La FENETRE
108         const math::NkVec2u wsz = target->GetWindow().GetSize();
109         if (wsz.x > 0 && wsz.y > 0 && (wsz.x != lastW || wsz.y != lastH)) {
110             target->OnResize(wsz.x, wsz.y);
111             lastW = wsz.x; lastH = wsz.y;
112         }
113         const math::NkVec2u sz = target->GetSize();
114         if (sz.x > 0 && sz.y > 0) { ctx.viewW = (int32)sz.x; ctx.viewH = (int32)sz.y; }
115
116         // c) DEBUT DE FRAME UI
117         ctx.BeginFrame(dt);
118         const float32 W = (float32)ctx.viewW;
119         const float32 H = (float32)ctx.viewH;
120         ctx.dl.AddRectFilled({0.f, 0.f, W, H}, ctx.theme.bgPrimary);
121
122         // d) LES WIDGETS
123         if (BeginPanel(ctx, "Reglages", {20.f, 20.f, 340.f, H - 40.f})) {
124             Text(ctx, "Bonjour NKGui");
125             Separator(ctx);
126             Checkbox(ctx, "Afficher l'aperçu", showPanel);
127             SliderFloat(ctx, "Zoom", zoom, 0.5f, 4.f);
128             InputText(ctx, "Nom", nom, sizeof(nom));
129             if (Button(ctx, "Reinitialiser")) {
130                 zoom = 1.f;
131                 showPanel = true;
132             }
133             EndPanel(ctx);
134         }
135
136         // e) FIN DE FRAME UI
137         ctx.EndFrame();
138
139         // f) PRESENTATION : Clear ouvre la frame, Display la ferme et presente
140         target->Clear();
141         backend.Submit(ctx.dl,          sz.x, sz.y); // couche principale
142         backend.Submit(ctx.dlOverlay, sz.x, sz.y); // popups/overlay AU-DESSUS
143         target->Display();
144     }

```

```

145
146     SetCurrentContext(nullptr);
147     ctx.Shutdown();
148     return 0;
149 }

```

Ce qu'il faut retenir

Deux cents lignes, dont la moitié est du branchement d'entrée. Tout le reste de votre application tiendra entre `BeginFrame` et `EndFrame`.

4.10 Ajouter une image

Les icônes et les images suivent exactement le chemin de la police : on charge des pixels, on les envoie sous un identifiant, on dessine.

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:292-321 (abrégé)

```

1     uint32 imgTexId = 0u;
2     int32 imgW = 0, imgH = 0;
3     {
4         static const char *kCandidates[] = {
5             "Applications/Mou/assets/brand/rihen-logo.png",
6             "Resources/Icons/ContentBrowser/FileIcon.png",
7             "../Applications/Mou/assets/brand/rihen-logo.png",
8         };
9         NkImage img;
10        bool ok = false;
11        for (const char *p : kCandidates)
12            if (img.Load(p, 4)) { ok = true; break; }
13        if (ok && img.Width() > 0 && img.Height() > 0 && img.Pixels()) {
14            imgW = img.Width();
15            imgH = img.Height();
16            static NkVector<uint8> rgba;
17            rgba.Resize(static_cast<usize>(imgW) * imgH * 4u);
18            for (int32 y = 0; y < imgH; ++y) {
19                const uint8 *src = img.RowPtr(y);
20                uint8 *dst = rgba.Data() + static_cast<usize>(y) * imgW * 4u;
21                for (int32 x = 0; x < imgW * 4; ++x)
22                    dst[x] = src[x];
23            }
24            imgTexId = 0x494D4731u; // 'IMG1'
25            backend.UploadImageRGBA(imgTexId, rgba.Data(), imgW, imgH);
26            logger.Info("Image chargée : {0}x{1}", imgW, imgH);
27        } else {
28            logger.Info("Image demo introuvable (cwd) -> section image vide");
29        }
30    }

```

Puis, dans la boucle, `Image(ctx, imgTexId, 128.f, 128.f)` ou `ctx.DL().AddImage(imgTexId, rect, {0,0}, {1,1}, tint)`.

Trois choses à noter :

- **La liste de chemins candidats**, avec des chemins relatifs à la racine du dépôt *et* des chemins remontants. C'est la parade au piège du répertoire courant vu au chapitre 0. Le dépôt formalise cette approche : « Candidats RELATIFS AU CWD (dev, lancement depuis la racine du repo) PUIS relatifs à l'EXECUTABLE » ([Applications/NKCode/src/NKCode/Shell/NkAppFonts.h:18-21](#)).

- **L'échec est journalisé, pas fatal.** L'application se lance quand même, avec une section image vide.
- **Le choix du texId.** Ici 0x494D4731, soit « IMG1 ». Il doit être distinct de celui de toute police (0x4E4B4654 pour la première), car le pont résout **les atlas de police d'abord, les images ensuite** (`NkGuiCanvasBackend.h:161-174`). Le shell de l'éditeur va plus loin et réserve des plages entières :

Engine/NKEditorKit/src/NKEditorKit/NkEditorShell.h:392

```
1  uint32 mNextTexId = 0x4E4B0100u; // ids de textures app (Logos/icones) – distincts des
    polices
```

Ce qu'il faut retenir

`texId == 0` est **réservé** : c'est la texture blanche interne côté RHI (`Integrations/NKGui/NkGuiRHIBackend.h:40`), et un `AddImage` avec `texId == 0` est purement ignoré (`NkGuiDrawList.cpp:160-161`). Si votre image ne s'affiche pas, la première chose à vérifier est que son identifiant n'est pas resté à zéro parce que le chargement a échoué.

4.11 Le fichier de build

Il reste à déclarer la cible pour Jenga. Créez `Applications/MonApp/MonApp.jenga` :

Applications/MonApp/MonApp.jenga — écrit pour ce cours, calqué sur Applications/NKGuiDemo/NKGuiDemo.jenga:41-75

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """MonApp – application de demonstration NKGui sur NKCanvas."""
4
5  from Jenga import *
6  from jengaconfig import *
7
8  with project("MonApp"):
9      windowedapp()
10     language("C++")
11     cppdialect("C++17")
12     location(".")
13
14     files(["src/**/*.cpp"])
15
16     nkentseudependson(
17         ["NKGui", "NKCanvas", "NKWindow", "NKEvent", "NKGlad", "NKFont", "NKImage",
18          "NKStream", "NKFileSystem", "NKLogger", "NKMath", "NKTime",
19          "NKContainers", "NKMemory", "NKCore", "NKPlatform", "NKThreading"],
20         extra_includes=["src", "src/MonApp", "%{NKGlad.location}/include"],
21     )
22
23     objdir("%{wks.location}/Build/Obj/%{cfg.buildcfg}-%{cfg.system}%{prj.name}")
24     targetdir("%{wks.location}/Build/Bin/%{cfg.buildcfg}-%{cfg.system}%{prj.name}")
25
26     apppublisher("Rihen Universe")
27     appversion("0.1.0")
28     licensefile("../LICENSE")
29
30     with filter("system:Windows"):
31         windowedapp()
32         usetoolchain(TC_WINDOWS)
```

```

33     defines(["WIN32_LEAN_AND_MEAN", "_UNICODE", "UNICODE"])
34     links(["user32", "gdi32", "opengl32", "dwmapi", "shell32",
35           "uuid", "ole32", "dxguid",
36           "d3d11", "d3d12", "dxgi", "d3dcompiler"])

```

4.11.1 Pourquoi chaque dépendance est là

Module	Pourquoi
NKGui	les widgets et le contexte
NKCanvas	le rendu, la cible, et l'en-tête du pont
NKWindow	la fenêtre et le point d'entrée nkmain
NKEvent	les événements typés
NKFont	la rasterisation des glyphes
NKImage	le décodage des images
NKGLad	le chargeur de fonctions OpenGL
NKTime	NkClock et le <i>delta time</i>
NKLogger	l'objet logger
NKMath, NKContainers, NKMemory, NKCore, NKPlatform	les fondations
NKFileSystem, NKStream, NKThreading	tirés par les modules ci-dessus

Oublier NKCanvas ou NKGui

Le pont `NkGuiCanvasBackend.h` est *header-only*. Il inclut `"NKGui/NKGui.h"` et appelle des méthodes de `NkIRenderer2D`. Si votre `.jenga` ne déclare qu'un seul des deux modules, l'erreur ne se produit pas à la compilation de `NKCanvas` — qui ne compile pas ce fichier — mais *chez vous*, sous forme d'en-têtes introuvables ou de symboles non résolus.

4.11.2 Enregistrer la cible et compiler

Selon l'organisation du dépôt, la nouvelle cible doit être connue du fichier racine `Nkentseu.jenga` — vérifiez comment les autres applications y sont listées avant de lancer la compilation.

Compilation et lancement — depuis la racine du dépôt

```

1 cd D:\Projets\2026\Nkentseu\Nkentseu
2 jenga build --target MonApp --config Release
3 .\Build\Bin\Release-Windows\MonApp\MonApp.exe

```

Et la configuration de mise au point, qui garde symboles et assertions :

Compilation en Debug

```

1 jenga build --target MonApp --config Debug
2 .\Build\Bin\Debug-Windows\MonApp\MonApp.exe

```

Lancer depuis la racine

Toujours depuis `D:/Projets/2026/Nkentseu/Nkentseu`, pas depuis le dossier de l'exécutable. Les chemins de ressources — polices de repli, images, icônes — sont écrits relativement à la racine du dépôt pendant le développement. Et le journal, `logs/app.log`, est écrit relativement au répertoire courant : lancer d'ailleurs le disperse.

4.12 Diagnostiquer les silences

Cette section est un aide-mémoire. Elle liste ce qui, dans une application NKGui, échoue *sans message d'erreur*.

Symptôme	Cause la plus probable
Fenêtre noire, l'application tourne	Backend graphique non fonctionnel; essayer OpenGL, lire <code>logs/app.log</code>
Widgets visibles, aucun texte	<code>UploadFontGray8</code> manquant, ou <code>fontPtr->Valid()</code> faux
Menus et combos invisibles mais cliquables	Le second Submit sur <code>ctx.dlOverlay</code> manque
Popups sous le contenu	Les deux Submit sont dans le mauvais ordre
Les clics arrivent avec une image de retard, ou jamais	Les événements sont pompés <i>après</i> <code>BeginFrame</code>
Plus rien ne réagit, l'application tourne	<code>activeId</code> figé : un glissement maison n'a pas remis <code>ctx.activeId = 0</code>
La souris semble gelée après une modale	Un panneau a écrasé <code>ctx.input.mousePos</code>
Rendu flou et étiré après agrandissement	Le redimensionnement est détecté sur la swapchain au lieu de la fenêtre
Pas de barre de défilement, barre horizontale hors écran	<code>AvailHeight()</code> utilisé sans intersection avec <code>CurrentClip()</code>
Image absente	<code>texId</code> resté à zéro (chargement échoué) ou en collision avec une plage de police
Bords droit et bas des panneaux amincis	Une bordure dessinée à l' <code>AddLine</code> au lieu de quatre rectangles internes
Texte flou	Position de glyphe non alignée sur la grille de pixels
Deux boutons réagissent ensemble	Même libellé, même portée : il manque <code>PushId/PopId</code>

4.13 Pour aller plus loin

Ce que ce chapitre n'a pas fait, et que vous savez maintenant aborder seul :

- **Des fenêtres flottantes et du docking.** Ajoutez `DockSpaceOverViewport(ctx)` avant vos `Begin`, puis déclarez chaque panneau entre `Begin` et `EndWindow`. La démo le fait en `main.cpp:888-1017`.

- **Le HiDPI.** `SetUiScale(ctx, 1.5f)` plus un rechargement de la police à `18.f * 1.5f`, avant `BeginFrame` ([main.cpp:461-466](#)).
- **La capture d'écran**, après `Display()` : `target->Capture("capture.png")` ([main.cpp:1049-1061](#)).
- **La coquille d'éditeur.** Si votre application ressemble à un IDE — barre de titre maison, palette de commandes, préférences, panneaux ancrables — n'écrivez pas tout cela : [Engine/NKEditorKit](#) le fournit, et vous n'avez plus qu'à écrire des panneaux dérivant de `NkEditorPanel`.

4.14 Exercices

1 — Faire tourner le squelette

Créez `Applications/MonApp/` avec son `.jenga` et son `src/MonApp/main.cpp`, recopiez le squelette de la section 4.9, compilez, lancez. Vérifiez que le panneau s'affiche avec son titre, que la case à cocher répond et que le *slider* se déplace. Puis, **volontairement**, supprimez la ligne `backend.Submit(ctx.dlOverlay, ...)`, ajoutez un `BeginCombo` au panneau, et constatez par vous-même ce que « échoue sans message » veut dire.

2 — Un vrai clavier

Complétez le branchement de l'entrée : double-clic, molette horizontale, touches d'édition (`setKey`), raccourcis `wantCopy/wantCut/wantPaste/wantSelectAll`, et le presse-papiers via `ctx.clipboardGetFn / ctx.clipboardSetFn`. Vérifiez ensuite, dans un `InputText`, que les flèches déplacent le curseur, que `Maj+flèche` sélectionne, que `Ctrl+C` et `Ctrl+V` fonctionnent, et qu'un accent s'écrit correctement.

3 — Une image et son identifiant

Chargez une image PNG avec `NkImage`, envoyez-la par `backend.UploadImageRGBA` sous un identifiant de votre choix, affichez-la avec le widget `Image`. Puis attribuez-lui *délibérément* le même identifiant que la police (`0x4E4B4654`) et observez ce qui s'affiche. Expliquez le résultat à partir de l'ordre de résolution du pont : atlas de police d'abord, images ensuite.

4 — Une application complète

Écrivez un petit gestionnaire de tâches : une table à trois colonnes (fait, titre, priorité), un champ de saisie et un bouton pour ajouter une ligne, une case à cocher par ligne, et un bouton « Supprimer les tâches terminées ». Les données vivent dans un `NkVector` de votre propre structure. Contraintes : un `PushId` par ligne, un identifiant stable pour la table, et **aucune** modification du tableau pendant le parcours — levez un drapeau et agissez après `EndTable`, conformément au dernier idiome du chapitre 3.

La coquille d'application : ce que vous n'écrivez plus

Le chapitre précédent vous a fait écrire une application *à la main* : créer la fenêtre, ouvrir le contexte graphique, tenir la boucle, écouter les événements, présenter l'image. C'était la bonne façon de comprendre ce qui se passe — et c'est la seule façon d'apprendre ce qu'une coquille vous cache.

Ce chapitre montre la coquille. Depuis `NkCanvasApp`, cette plomberie devient quelques méthodes à remplir. Surtout, la coquille apporte trois services qu'une application écrite à la main *oublie presque toujours* — et l'oubli ne se voit qu'une fois le programme entre les mains de quelqu'un d'autre.

Ce qu'il faut retenir

Une coquille ne vous fait pas gagner des lignes : elle vous fait gagner les lignes que *vous n'auriez pas écrites*.

Mesure du dépôt, le 2 septembre 2026 : **quatorze** applications créent elles-mêmes leur fenêtre de rendu. Sur ces quatorze, **trois** gèrent le cycle de vie mobile. Les onze autres — dont `Sandbox`, `NkRef`, `NKGuiDemo`, `NkVideoPlayer` et `NkAudioPlayer` — se ferment quand le téléphone se verrouille, et personne ne l'a remarqué parce que personne ne les lance sur un téléphone.

Ce n'est pas de la négligence : c'est que *rien ne rappelle d'écrire ce qu'on ne voit pas manquer*. C'est précisément ce qu'une coquille existe pour donner.

5.1 Le plus petit programme complet

Applications/NkDames/src/main.cpp

```
1 #include "Dames/NkDamesJeu.h"
2 #include "NKWindow/Core/NkEntry.h"
3 #include "NKWindow/NKMain.h"
4
5 NKENTSEU_DEFINE_APP_DATA([]() {
6     nkentseu::NkAppData d{};
7     d.appName = "NkDames";
8     d.appVersion = "1.0.0";
9     return d;
10 })();
11
12 int nkmain(const nkentseu::NkEntryState &state) {
13     return nkentseu::renderer::NkCanvasApp::Run<nkentseu::jeux::dames::NkDamesJeu>(state);
14 }
```

Trois choses seulement, et elles se lisent dans l'ordre.

`NKENTSEU_DEFINE_APP_DATA` déclare le nom et la version de l'application. `nkmain` est le point d'entrée portable : c'est `NKWindow/NKMain.h` qui fournit derrière lui le `winMain` de Windows et

l'android_main d'Android. NkCanvasApp::Run<T> construit votre classe et lui rend la main.

Le piège

La macro attend un NkAppData, **pas une chaîne** — son nom laisse croire le contraire. Écrivez NKENTSEU_DEFINE_APP_DATA("MonJeu") et le compilateur répond no viable overloaded '=', sans jamais nommer la cause. Le lambda ci-dessus est le patron employé par toutes les applications du dépôt.

5.2 Les méthodes que vous remplissez

Kernel/Runtime/NKCanvas/src/NKCanvas/App/NkCanvasApp.h (extraits)

```
1 virtual NkOptional<int> OnCommandLine(const NkVector<NkString> &args);
2 virtual bool OnInit();
3 virtual void OnUpdate(float32 deltaTime);
4 virtual void OnRender(NkRenderWindow &target);
5 virtual bool OnEvent(const NkEvent &event); // true = consomme
6 virtual bool OnPointer(const NkPointer &pointer);
7 virtual void OnLayout(const NkLayoutInfo &layout);
8 virtual bool OnCloseRequested(); // false = refuse la fermeture
9 virtual void OnPause();
10 virtual void OnResume();
11 virtual void OnShutdown();
```

Aucune n'est obligatoire : elles ont toutes un corps par défaut. On remplit ce dont on a besoin.

Méthode	Quand elle est appelée
OnCommandLine	avant toute fenêtre — peut refuser de démarrer
OnInit	la fenêtre et le GPU existent
OnUpdate	une fois par image, avec le pas de temps
OnRender	une fois par image, avec la cible de rendu
OnPointer	souris <i>ou</i> doigt, unifiés
OnLayout	taille, densité et zone sûre ont changé
OnPause, OnResume	l'application part en arrière-plan, revient

Le piège

OnCommandLine rend un NkOptional<int>. Rendre une valeur *arrête l'application avec ce code*, sans ouvrir de fenêtre — c'est ce qui permet à une option de banc de répondre en mode automatique. Rendre un NkOptional vide laisse le programme démarrer.

5.3 Les trois services que la coquille rend

5.3.1 La boucle cède la main, et sur le Web c'est vital

Kernel/Runtime/NKCanvas/src/NKCanvas/App/NkCanvasApp.cpp (CadencerFrame)

```
1  if (mConfig.imagesParSeconde <= 0) {
2      NkChrono::YieldThread();
3      return;
4  }
5  const float64 budgetMs = 1000.0 / static_cast<float64>(mConfig.imagesParSeconde);
6  const float64 ecoleMs = chrono.Elapsed().milliseconds;
7  if (ecoleMs < budgetMs) {
8      NkChrono::Sleep(budgetMs - ecoleMs);
9  } else {
10     NkChrono::YieldThread();
11 }
```

Sous Emscripten, `NkChrono::Sleep` appelle `emscripten_sleep(ms)` et `YieldThread` appelle `emscripten_sleep(0)`. Les deux rendent la main au navigateur.

Le piège

Les deux branches cèdent, et c'est tout l'intérêt. Une version « ne dormir que si on est en avance » gèle l'onglet dès la première image lente — c'est-à-dire tout le temps sur le Web, où une image dépasse presque toujours son budget. Le `else` n'est pas une optimisation : c'est lui qui rend la main.

Le mécanisme existait avant la coquille, et il était juste. Ce qui manquait, c'était *l'appel*.

5.3.2 La zone sûre, mesurée et non devinée

Kernel/Runtime/NKCanvas/src/NKCanvas/App/NkCanvasApp.h (NkLayoutInfo)

```
1  struct NkLayoutInfo {
2      uint32 width = 0;
3      uint32 height = 0;
4      float32 density = 1.f; // facteur d'echelle de l'ecran (DPI)
5      NkSafeAreaInsets safeArea;
6  };
```

`safeArea` porte quatre marges — `top`, `bottom`, `left`, `right` — en pixels. Elles décrivent la partie de l'écran qui n'est masquée ni par l'encoche, ni par les coins arrondis, ni par l'indicateur de geste du bas.

Ce qu'il faut retenir

La zone sûre est une **ancree**, pas une marge : elle change à la rotation, à l'ouverture du clavier, en écran partagé, et d'un appareil à l'autre. On la demande à la plateforme à l'exécution ; une marge écrite en dur est fautive sur le modèle suivant.

Et c'est un choix *par élément* : les fonds, les images de couverture et les listes qui défilent vont jusqu'au bord ; le texte et les boutons restent dans la zone sûre. Un fond qui s'arrête à la zone sûre laisse des bandes disgracieuses ; un bouton qui déborde sous l'indicateur de geste est **inatteignable**.

5.3.3 Le pointeur : souris et doigt unifiés

Kernel/Runtime/NKEvent/src/NKEvent/NkPointerEvent.h

```
1 enum class NkPointerPhase : uint8 {
2     NK_POINTER_NONE = 0,    // l'événement lu ne concerne pas le pointeur
3     NK_POINTER_DOWN,        // clic gauche enfonce, ou doigt pose
4     NK_POINTER_MOVE,        // déplacement
5     NK_POINTER_UP,          // clic relache, ou doigt leve
6     NK_POINTER_CANCEL       // le système a repris la main (appel entrant, geste OS)
7 };
8
9 struct NkPointer {
10     float32 x = 0.f;
11     float32 y = 0.f;
12     NkPointerPhase phase = NkPointerPhase::NK_POINTER_NONE;
13     uint32 id = 0;           // 0 = souris, ou premier doigt
14     bool fromTouch = false;  // true = doigt
15
16     bool IsValid() const noexcept; // phase != NK_POINTER_NONE
17 };
```

Un clic de souris et un appui du doigt arrivent dans la même méthode, avec la même forme. `fromTouch` dit lequel c'était, pour les rares cas où la distinction compte — une infobulle au survol n'a pas de sens au doigt.

Le piège

x et y sont en pixels *client* : le repère de la zone de dessin, origine en haut à gauche, hors bordure de fenêtre. C'est le même repère qu'une cible de rendu. N'y mêlez jamais des coordonnées écran — le dépôt a payé une journée entière de recherche sur une sélection morte, pour avoir comparé des pixels de fenêtre à des pixels de viseur.

Le piège

Agissez au relâchement, pas à l'appui. `NK_POINTER_UP` permet à l'utilisateur de glisser hors d'un bouton pour annuler son geste — comportement attendu partout, et gratuit ici. Un bouton qui déclenche à l'appui ne pardonne pas l'erreur de visée, et sur un téléphone on vise mal.

Et n'oubliez pas `NK_POINTER_CANCEL` : un appel entrant ou un geste système interrompt la saisie. Le traiter comme un relâchement laisse un objet « collé » au doigt qui n'est plus là.

5.4 Deux coquilles, et il faut choisir

`NkCanvasGuiApp` dérive de `NkCanvasApp` et y ajoute `NKGui` : le contexte, les polices, le backend de dessin. Mais elle *prend la place* du rendu.

Kernel/Runtime/NKCanvas/src/NKCanvas/App/NkCanvasGuiApp.h

```
1 void OnUpdate(float32 deltaTime) final {
2     mLastDelta = deltaTime;
3     OnTick(deltaTime);
4 }
5
6 void OnRender(NkRenderWindow &target) final {
```

```

7   const math::NkVec2u size = target.GetSize();
8   mGuiContext.BeginFrame(mLastDelta);
9   OnDraw(mGuiContext.dl);
10  mGuiContext.EndFrame();
11  mGuiBackend.Submit(mGuiContext.dl, size.x, size.y);
12  mGuiBackend.Submit(mGuiContext.dlOverlay, size.x, size.y);
13 }

```

Les deux méthodes sont `final`. Une classe qui dérive de `NkCanvasGuiApp` ne peut donc **pas** redéfinir `OnRender` : elle remplit `OnTick` et `OnDraw` à la place, et elle dessine avec une `NkGuiDrawList`, pas avec un `NkRenderer2D`.

Vous dérivez de	Vous dessinez avec	Vous remplissez
rien (à la main)	<code>NkRenderer2D</code>	tout
<code>NkCanvasApp</code>	<code>NkRenderer2D</code>	<code>OnUpdate</code> , <code>OnRender</code>
<code>NkCanvasGuiApp</code>	<code>NkGuiDrawList</code>	<code>OnTick</code> , <code>OnDraw</code>

Ce qu'il faut retenir

Comment choisir. Vous dessinez des formes, des sprites, un monde de jeu : `NkCanvasApp` et `NkRenderer2D`. Vous dessinez une interface — boutons, listes, panneaux : `NkCanvasGuiApp` et la liste d'affichage. Vous voulez comprendre ce qui se passe dessous, ou vous avez un besoin qu'aucune des deux ne couvre : à la main, comme au chapitre précédent.

Ce ne sont pas trois niveaux de qualité : ce sont trois outils pour trois travaux.

5.5 L'écran d'ouverture, offert

Applications/NkLudo/src/Ludo/NkLudoJeu.cpp

```

1  bool NkLudoJeu::OnGuiInit() {
2      mSplash.PoserJeu("Ludo");
3      Rejouer();
4      return true;
5  }
6
7  void NkLudoJeu::OnTick(float32 deltaTime) {
8      if (mSplash.Avancer(deltaTime)) {
9          return; // L'ouverture tient l'ecran
10     }
11     /* ... Le jeu ... */
12 }

```

`NkCanvasSplash` peint deux volets — la marque Rihen, puis le moteur et le nom du jeu — avec le logo officiel embarqué, rasterisé depuis un SVG de 514 octets. `PoserJeu` est le *seul* réglage : les couleurs, les durées et la mise en page sont les mêmes partout, délibérément.

Le piège

`Avancer` rend `true` tant que l'ouverture tient l'écran, et il faut **s'arrêter là**. Sans ce retour, le jeu tournerait *derrière* l'écran de marque : une IA jouerait ses premiers coups pendant les quatre secondes d'ouverture, et le joueur trouverait la partie déjà entamée en arrivant.

Et l'ouverture doit **manger l'appui** qui la saute. Sans cela, le même appui passe le splash et active le bouton qui se trouve dessous — l'utilisateur lance une partie qu'il n'a pas choisie.

Exercice

Reprenez l'application du chapitre précédent, écrite à la main, et portez-la sur NkCanvasApp. Comptez les lignes avant et après.

Puis répondez à deux questions, en les *mesurant* plutôt qu'en les devinant : votre version manuelle gérait-elle `OnPause` ? Et cédait-elle la main à chaque image ? Compilez-la pour le Web et ouvrez-la dans un navigateur — vous aurez la réponse en une seconde.

NKImage : charger, fabriquer, afficher des images

Jusqu'ici, tout ce que nous avons affiché était fabriqué par le moteur : rectangles, cercles, atlas de police. Ce chapitre ouvre la porte du monde extérieur. NKImage est le module qui transforme un fichier .png posé sur un disque en un tableau d'octets que le GPU sait consommer — et, dans l'autre sens, un tableau d'octets en fichier.

C'est le premier des cinq modules d'intégration, et ce n'est pas un hasard : c'est le plus simple, le plus stable, et surtout *les autres en dépendent*. NKMedia s'en sert pour son codec MJPEG (qui n'est rien d'autre que le codec JPEG de NKImage appliqué image par image), et NKAudio dépend de NKMedia. L'ordre d'apprentissage est donc imposé par les dépendances :

Ordre imposé par les dépendances (voir les .jenga de chaque module)

```

1 NKImage  └─> NKMedia ──> NKAudio
2              (NKMedia depend de NKImage : codec MJPEG = codec JPEG de NKImage)
3              (NKAudio depend de NKMedia : decodeur Opus)
4              └─> NKFont   (independant : ne depend ni de NKImage ni de NkCanvas)
5
6 NKNetwork (totalement independant – peut venir n'importe ou)
```

6.1 Ce qu'est NKImage, et ce qu'il n'est pas

6.1.1 Aucune dépendance externe

La première chose à regarder, comme toujours, c'est le fichier de build :

Kernel/Runtime/NKImage/NKImage.jenga:25-29

```

1 nkentseudependson(
2     ["NKPlatform", "NKCore", "NKMemory", "NKMath", "NKContainers", "NKLogger",
3     "NKThreading", "NKFileSystem", "NKStream"],
4     selfexport="NKImage",
5     extra_includes=["src"],
6 )
```

Que des modules de base. Pas de libpng, pas de libjpeg, pas de stb_image. Le fichier de build le dit en toutes lettres (NKImage.jenga:6) : « Bibliothèque self-contained sans dépendance externe ». Chaque codec — PNG, JPEG, BMP, TGA, HDR, EXR, PPM, QOI, GIF, ICO, WebP, SVG — est écrit dans le dépôt, y compris le deflate du PNG.

Cela a une conséquence pratique immédiate : **il n'y a rien à installer**. Si le moteur compile, NKImage décode.

6.1.2 NKImage ne connaît aucun GPU

Relisez la liste des dépendances : pas de NKCanvas, pas de NKRI, pas de NKWindow. NKImage produit et consomme des *octets en mémoire vive*. Il ne sait pas ce qu'est une texture.

Ce qu'il faut retenir

NKImage produit des octets CPU ; le renderer produit un identifiant GPU. Le point de couture entre les deux est toujours le même : un pointeur `const uint8*`, une largeur, une hauteur. Tout le reste de ce chapitre découle de cette phrase.

6.1.3 L'arborescence

Kernel/Runtime/NKImage/ — arborescence

```
1 NKImage/
2   NKImage.jenga
3   src/NKImage/
4     NKImage.h                <- en-tete PARAPLUIE (tout inclure d'un coup)
5     Core/
6       NKImage.h              (1218 l.)    <- LE conteneur + NKImageStream + NKDeflate
7       NKImage.cpp            (3034 l.)
8       NKImageExport.h        ( 58 l.)    <- macros NKENTSEU_IMAGE_API / NKIMG_INLINE
9     Codecs/
10      PNG/  JPEG/  BMP/  TGA/  HDR/  EXR/
11      PPM/  QOI/  GIF/  ICO/  WEBP/  SVG/
```

Une seule classe compte pour ce chapitre : `NkImage`, dans `Kernel/Runtime/NKImage/src/NKImage/Core/NkImage.h`. Les codecs sont appelables directement, mais on n'en a presque jamais besoin : `NkImage` choisit le bon selon le contenu du fichier.

L'exemple en tête de `NKImage.h` est périmé

Le commentaire de documentation en tête de l'en-tête parapluie (`Kernel/Runtime/NKImage/src/NKImage/NKImage.h:12-33`) contient un exemple `@code` qui utilise `NkSVGRenderer::RenderFromFile`, `NkSVGDOM` et `NkSVGDOMBuilder`. **Ces trois classes n'existent plus.** Le pipeline SVG a été remplacé par un codec unique, et le même fichier le dit vingt lignes plus bas :

Kernel/Runtime/NKImage/src/NKImage/NKImage.h:53-54

```
1 // SVG : NkSVGDOM/Renderer ont ete remplaces par NkSVGCodec (codec unique).
```

Ne recopiez jamais cet exemple : il ne compile pas. C'est le premier cas d'une règle que nous retrouverons dans chacun des cinq chapitres qui suivent : **quand un commentaire d'en-tête contredit le `.cpp`, c'est le `.cpp` qui a raison.**

6.2 Les deux vocabulaires de formats

Avant toute chose, il faut distinguer deux choses que le débutant confond systématiquement : le format des *pixels en mémoire* et le format du *fichier sur le disque*.

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:67-75

```
1 enum class NkImagePixelFormat : uint8 {
2     NK_UNKNOWN = 0, NK_GRAY8 = 1, NK_GRAY_A16 = 2,
3     NK_RGB24 = 3, NK_RGBA32 = 4, NK_RGBA128F = 5, NK_RGB96F = 6,
4 };
```

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:127-142

```
1 enum class NkImageFormat : uint8 { // format de FICHIER
2     NK_UNKNOWN=0, NK_PNG, NK_JPEG, NK_BMP, NK_TGA, NK_HDR,
3     NK_PPM, NK_PGM, NK_PBM, NK_QOI, NK_GIF, NK_ICO, NK_SVG, NK_EXR,
4 };
```

Le premier décrit comment un pixel est rangé en RAM : un octet de gris (NK_GRAY8), trois octets (NK_RGB24), quatre (NK_RGBA32), ou quatre flottants 32 bits (NK_RGBA128F, pour le HDR). Le second décrit l'enveloppe du fichier. Un .png peut contenir du NK_GRAY8 comme du NK_RGBA32; un .hdr contient forcément du flottant.

Deux fonctions constexpr font le pont :

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:78, 98

```
1 constexpr int32 ChannelsOf(NkImagePixelFormat f) noexcept;
2 constexpr int32 BytesPerPixelOf(NkImagePixelFormat f) noexcept;
```

Enfin, le troisième énuméré du module gouverne le rééchantillonnage :

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:157-162

```
1 enum class NkResizeFilter : uint8 {
2     NK_NEAREST, NK_BILINEAR, NK_BICUBIC, NK_LANCZOS3,
3 };
```

Retenez ces quatre noms, nous y reviendrons : deux d'entre eux mentent.

6.3 Deux familles d'API, et pourquoi il faut choisir

C'est le point structurant du module, et la source de la moitié des plantages des débutants. NkImage propose deux façons complètement différentes de faire la même chose. Le commentaire de tête du fichier les nomme lui-même :

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:7-36 (abrégé)

```
1 1. API STATIQUE -> retourne `NkImage*` (ownership explicite, heap-alloue)
2 2. API INSTANCE -> retourne `bool` (opere sur *this, aucune allocation visible)
```

6.3.1 L'API instance : l'objet vit sur la pile

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:232, 242, 262, 282

```
1 bool Create(uint32 width, uint32 height, math::NkColor color,
2             int32 desiredChannels = 4) noexcept;
3 bool LoadFromFile(const char* path) override;
4 bool Load(const char* path, int32 desiredChannels = 0) noexcept;
5 bool LoadFromMemory(const void* data, usize size, int32 desiredChannels) noexcept;
```

Toutes ces méthodes renvoient un bool et travaillent sur *this. On déclare une NkImage sur la pile, on l'utilise, et le destructeur libère les pixels. Il n'y a rien à écrire pour libérer.

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:400-407 (abrégé)

```
1     NkImage img;
2     bool ok = false;
3     for (const char* p : kCandidates)
4         if (img.Load(p, 4)) {
5             ok = true;
6             break;
7         }
8     if (ok && img.Width() > 0 && img.Height() > 0 && img.Pixels()) {
```

6.3.2 L'API statique : vous possédez le pointeur

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:308, 317, 792, 804, 815

```
1 static NkImage* Create(uint32 w, uint32 h, int32 desiredChannels = 0, uint32 color = 0)
   noexcept;
2 static NkImage* Create(uint32 w, uint32 h, NkImagePixelFormat fmt, uint32 color = 0) noexcept;
3 static NkImage* Alloc(int32 w, int32 h, NkImagePixelFormat fmt) noexcept;
4 static NkImage* Wrap(uint8* pixels, int32 w, int32 h, NkImagePixelFormat fmt, int32 stride=0)
   noexcept;
5 static NkImage* ConvertToTexture(const NkImage& hdrImage, float exposure=1.0f, float
   gamma=2.2f) noexcept;
```

Chacune renvoie un NkImage* alloué sur le tas. Vous devez appeler ->Free() dessus. Et ce n'est pas tout : plusieurs méthodes membres renvoient elles aussi une image neuve, avec la même obligation :

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:396, 404, 475, 485, 523

```
1 NkImage* Convert(NkImagePixelFormat newFmt) const noexcept;
2 NkImage* Resize(int32 nw, int32 nh, NkResizeFilter f = NkResizeFilter::NK_BILINEAR) const
   noexcept;
3 NkImage* Crop(int32 x, int32 y, int32 w, int32 h) const noexcept;
4 NkImage* Copy() const noexcept;
5 NkImage* CopyAs(NkImagePixelFormat fmt) const noexcept;
```

C'est le piège le plus courant du module : on charge sur la pile — bien —, puis on appelle Resize — et on oublie que le résultat est un pointeur qu'il faut libérer.

La règle en une phrase

Si la fonction renvoie `boo1`, il n'y a rien à libérer. Si elle renvoie `NkImage*`, vous devez appeler `->Free()`. Aucune exception dans tout le module.

6.3.3 Copie interdite, déplacement autorisé

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:202-215

```
1 NkImage(const NkImage&) = delete; // copie INTERDITE
2 NkImage& operator=(const NkImage&) = delete;
3 NkImage(NkImage&& other) noexcept;
4 NkImage& operator=(NkImage&& other) noexcept;
```

La raison est écrite juste au-dessus ([Core/NkImage.h:178-180](#)) : « La copie par valeur est désactivée (= `delete`) pour éviter les doubles-*free* accidentels. Utiliser `Copy()` (*deep clone*) ou le *move constructor*. » Vous ne pouvez donc pas ranger une `NkImage` dans un `NkVector` par copie; utilisez le déplacement, ou stockez des pointeurs.

6.4 Charger une image

6.4.1 La signature qui compte

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:262

```
1 bool Load(const char *path, int32 desiredChannels = 0) noexcept;
```

Le second paramètre est le seul qui demande réflexion. `0` signifie « garde le nombre de canaux du fichier » : un PNG en niveaux de gris donnera une image `NK_GRAY8`, un PNG avec transparence donnera du `NK_RGBA32`. C'est économe, mais cela veut dire que *vous ne savez pas* ce que vous obtenez avant d'interroger `Format()`.

Passer `4` force la conversion en `NK_RGBA32`. C'est ce que fait la démo, et c'est ce que vous ferez presque toujours dès qu'il s'agit d'envoyer l'image au GPU : les backends veulent du `RGBA`, et faire la conversion à la lecture évite d'écrire une deuxième passe.

6.4.2 Les accesseurs

Une fois l'image chargée, tout se lit par des accesseurs `inline` ([Core/NkImage.h:527-585](#)) :

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:527-585 (liste)

```
1 Pixels() // uint8* : Le premier octet de la première ligne
2 Width() Height()
3 Channels() // 1, 2, 3 ou 4
4 BytesPP() // octets par pixel
5 Stride() // octets par LIGNE <- Lire la section suivante
6 Format() // NkImagePixelFormat courant
7 SourceFormat() // NkImageFormat du fichier d'origine
8 IsValid() IsHDR() TotalBytes()
9 RowPtr(y) // uint8* sur la ligne y
```

6.4.3 Le stride : la cause numéro un des images « penchées »

Kernel/Runtime/NKImage/src/NKImage/Core/NkImage.h:182-184

```
1 STRIDE :
2 Le stride (bytes par ligne) est aligne sur 4 octets : stride = (w*bpp+3)&~3.
3 Utiliser RowPtr(y) pour acceder a la ligne y de facon portable.
```

Traduisons. Pour une image RGB24 de 5 pixels de large, une ligne fait $5 \times 3 = 15$ octets — mais le module en réserve 16, pour que chaque ligne commence à une adresse multiple de 4. Il y a donc **un octet de bourrage invisible à la fin de chaque ligne**.

Conséquence : `Pixels()` n'est pas un tableau contigu de `width * height * bpp` octets. Un `memcpy` global qui suppose le contraire produit une image décalée d'un pixel de plus à chaque ligne — l'effet « penché » caractéristique. La parade est de recopier ligne par ligne :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:410-419

```
1 static NkVector<uint8> rgba;
2 rgba.Resize(static_cast<usize>(imgW) * imgH * 4u);
3 for (int32 y = 0; y < imgH; ++y) {
4     const uint8 *src = img.RowPtr(y);
5     uint8 *dst = rgba.Data() + static_cast<usize>(y) * imgW * 4u;
6     for (int32 x = 0; x < imgW * 4; ++x)
7         dst[x] = src[x];
8 }
```

Quand le stride ne se voit pas

En RGBA32, `bpp = 4`, donc `w*4` est *toujours* multiple de 4 et `stride == w*4`. Un `memcpy` global marchera. Puis un jour vous chargerez un JPEG en trois canaux, et tout se penchera — dans un code que vous n'avez pas touché. **Utilisez `RowPtr(y)` dès la première ligne que vous écrivez**, même quand ce n'est pas nécessaire ce jour-là.

6.5 Fabriquer une image de toutes pièces

6.5.1 Le squelette canonique de l'API statique

L'application `NKImageCodecTest` est le meilleur exemple court du dépôt. Elle alloue une image, écrit les pixels à la main, et la sauvegarde dans neuf formats :

Applications/NKImageCodecTest/src/main.cpp:15-27 (abrégé)

```
1 NkImage *img = NkImage::Alloc(W, H, NkImagePixelFormat::NK_RGB24);
2 if (!img) {
3     printf("[ERREUR] Alloc\n");
4     return 1;
5 }
6 nk_uint8 *db = img->Pixels();
7 const int st = img->Stride();
8 for (int y = 0; y < H; ++y) {
9     for (int x = 0; x < W; ++x) {
10         /* ... remplissage motif ... */
11         nk_uint8 *p = db + (nk_size)y * st + x * 3;
12         p[0] = r; p[1] = g; p[2] = b;
13     }
```

```
14 }
```

Remarquez l'arithmétique : `db + y * st + x * 3`. C'est `Stride()` qui sert de pas vertical, jamais `Width() * 3`. Puis :

Applications/NKImageCodecTest/src/main.cpp:70-78 (abrégé)

```
1     for (const Fmt &f : formats) {
2         const bool ok = img->Save(f.file, 100);
3         printf("  %-12s : %s\n", f.file, ok ? "ecrit" : "ECHEC SAVE");
4     }
5     img->Free();
```

Alloc → Pixels() + Stride() → Save → Free. Voilà le squelette complet.

6.5.2 Créer une image remplie d'une couleur

Kernel/Runtime/NKImage/src/NKImage/Core/NkImage.h:232

```
1 bool Create(uint32 width, uint32 height, math::NkColor color,
2             int32 desiredChannels = 4) noexcept;
```

Cette signature a une histoire, et le dépôt l'a documentée sur les lieux du crime :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkTexture.cpp:72-75

```
1 // NkImage::Create(w, h, NkColor color, int32 channels=4) : La COULEUR
2 // d'abord, Les canaux ensuite. (Avant : args inverses -> channels=0
3 // pour un fill transparent -> Create echouait -> texture jamais creee.)
```

Couleur d'abord, canaux ensuite

Il existe aussi une *Create statique* dont le troisième paramètre est le nombre de canaux et le quatrième la couleur — l'ordre inverse ([Core/NkImage.h:308](#)). Les deux compilent. L'une des deux ne fait pas ce que vous croyez. Vérifiez la famille d'API que vous employez : `bool` ou `NkImage*`.

6.5.3 Dessiner dans une image (sur le processeur)

`NkImage` embarque un petit rasteriseur logiciel, entièrement `inline` dans l'en-tête ([Core/NkImage.h:587-765](#)) — vous pouvez donc lire son code sans quitter la déclaration :

Kernel/Runtime/NKImage/src/NKImage/Core/NkImage.h:587-765 (liste)

```
1 SetPixel GetPixel BlendPixel Fill
2 DrawHLine DrawVLine DrawLine // Bresenham
3 DrawRect FillRect
4 DrawCircle FillCircle
5 DrawEllipse FillEllipse
```

Toutes prennent une `math::NkColor`. Un exemple complet, écrit pour ce cours :

Exemple écrit pour le cours (API : Core/NkImage.h:232, 587-765, 349)

```
1 #include "NkImage/Core/NkImage.h"
2 #include "NkMath/NkColor.h"
3 using namespace nkentseu;
4
5 NkImage canvas;
6 canvas.Create(640, 480, math::NkColor(20, 20, 28, 255), 4);
7 canvas.FillRect(40, 40, 200, 120, math::NkColor(255, 90, 0, 255));
8 canvas.DrawCircle(320, 240, 100, math::NkColor(0, 245, 255, 255));
9 canvas.DrawLine(0, 0, 639, 479, math::NkColor(255, 255, 255, 255));
10 canvas.SavePNG("dessin.png");
```

Deux limites, énoncées par le module lui-même :

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:587-590

```
1  Formats LDR 8-bit (RGBA/RGB/RG/R). No-op si image invalide, hors
2  bornes, ou HDR. Couleur = math::NkColor (= NkColor2D). SetPixel ECRASE
3  (pas de blending) ; BlendPixel fait un src-over alpha.
```

«No-op si hors bornes» : dessiner à côté de l'image ne plante pas et ne prévient pas. Et sur une image HDR, ces méthodes ne font strictement rien — silencieusement.

6.6 Écrire sur le disque

6.6.1 Le dispatch par extension

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:348-360

```
1 bool Save(const char* path, int32 quality = 90) const noexcept; // dispatch par extension
2 bool SavePNG (const char* path) const noexcept;
3 bool SaveJPEG(const char* path, int32 quality = 90) const noexcept;
4 bool SaveBMP (const char* path) const noexcept;
5 bool SaveTGA (const char* path) const noexcept;
6 bool SavePPM (const char* path) const noexcept;
7 bool SaveHDR (const char* path) const noexcept;
8 bool SaveEXR (const char* path) const noexcept; // EXR scanline FLOAT, compression NONE
9 bool SaveQOI (const char* path) const noexcept;
10 bool SaveGIF (const char* path) const noexcept; // palette 256 (median-cut) + LZW
11 bool SaveWebP(const char* path, bool lossless = true, int32 quality = 90) const noexcept;
12 bool SaveSVG (const char* path) const noexcept;
```

Save("photo.png") appelle SavePNG; Save("photo.jpg", 80) appelle SaveJPEG avec une qualité de 80. Le dispatch se lit dans [Core/NkImage.cpp:2010-2030](#).

6.6.2 Les deux dernières lignes sont des mensonges

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.cpp:2119-2134

```
1 /**
2  * SaveWebP - non implémenté.
3  * Nécessiterait Libwebp ou une implémentation custom VP8/VP8L.
4  */
5 bool NkImage::SaveWebP(const char *, bool, int32) const noexcept {
```

```

6     return false;
7 }
8
9 /**
10  * SaveSVG — non implémenté.
11  * L'encodage raster → SVG (vectorisation) n'est pas dans le scope.
12  */
13 bool NkImage::SaveSVG(const char *) const noexcept {
14     return false;
15 }

```

SaveWebP et SaveSVG renvoient toujours false

Ces deux méthodes sont déclarées, exportées, documentées — et vides. Elles ne plantent pas : elles renvoient false sans rien écrire. Si vous ne testez pas la valeur de retour, votre programme fera comme si tout allait bien.

Corollaire moins évident : Save("sortie.webp") échoue aussi, parce que l'extension webp n'est pas dans le dispatch de Save() ([Core/NkImage.cpp:2010-2030](#)). Le fichier de test du dépôt le prouve malgré lui : NkImageCodecTest liste ic_out.webp dans ses formats attendus, et cette ligne affiche ECHEC SAVE à chaque exécution.

Nuance importante : le codec WebP, lui, existe et fonctionne (NkWebPCodec::Encode, [Codecs/WEBP/NkWebPCodec.h:32](#), chemin VP8L sans perte). C'est uniquement la méthode de commodité NkImage::SaveWebP qui est vide. Si vous avez vraiment besoin de WebP en écriture, appelez le codec directement.

6.6.3 Encoder en mémoire

Parfois on ne veut pas de fichier : on veut les octets, pour les envoyer sur le réseau ou les ranger dans une base.

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:369-373

```

1 bool EncodePNG (uint8*& out,  size& size) const noexcept;
2 bool EncodeJPEG(uint8*& out,  size& size, int32 quality = 90) const noexcept;
3 bool EncodeBMP (uint8*& out,  size& size) const noexcept;
4 bool EncodeTGA (uint8*& out,  size& size) const noexcept;
5 bool EncodeQOI (uint8*& out,  size& size) const noexcept;

```

Le paramètre out est une *référence de pointeur* : la fonction alloue le tampon et vous le rend. La question devient donc : qui le libère, et avec quoi ?

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:31-36 (abrégé)

```

1  REGLE DE MEMOIRE :
2  Les buffers alloués via l'API statique DOIVENT être libérés avec img->Free().
3  Les buffers encodés (EncodePNG, EncodeJPEG, ...) DOIVENT être libérés avec
4  nkentseu::memory::NkFree(ptr).
5  [...] l'allocateur custom NKMemory n'est pas compatible avec le heap CRT
6  standard (crash c0000374 sur Windows en cas de mélange).

```

c0000374 est le code d'un tas Windows corrompu. Le plantage n'arrive d'ailleurs pas au moment de la libération fautive, mais bien plus tard, dans une allocation parfaitement innocente — d'où la difficulté à le diagnostiquer.

Exemple écrit pour le cours (API : Core/NkImage.h:369, NKMemory)

```
1 uint8 *buf = nullptr;
2 usize n = 0;
3 if (src.EncodePNG(buf, n)) {
4     /* ... envoyer buf/n sur le reseau, dans une base, ... */
5     memory::NkFree(buf);          // et JAMAIS free() ni delete[]
6 }
```

6.7 Transformer une image

6.7.1 Redimensionner

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:404

```
1 NkImage* Resize(int32 nw, int32 nh, NkResizeFilter f = NkResizeFilter::NK_BILINEAR) const
    noexcept;
```

Rappel : cela renvoie un NkImage*. L'usage complet :

Exemple écrit pour le cours (API : Core/NkImage.h:262, 404, 349, 775)

```
1 NkImage src;
2 if (!src.Load("photo.jpg", 4))          // 4 = force RGBA32
3     return 1;
4
5 NkImage *thumb = src.Resize(256, 256, NkResizeFilter::NK_BILINEAR);
6 if (!thumb) return 1;
7
8 thumb->Save("photo_thumb.png");          // format deduit de l'extension
9 thumb->Free();                          // OBLIGATOIRE (tas)
10 // ~NkImage() libere src : rien a ecrire.
```

NK_BICUBIC et NK_LANCZOS3 ne font pas ce qu'ils disent

Les quatre filtres compilent. Deux seulement existent.

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.cpp:2901-2903

```
1 // — Bilineaire (défaut pour NK_BILINEAR, NK_BICUBIC, NK_LANCZOS3) —
2 //
3 // Note : NK_BICUBIC et NK_LANCZOS3 ne sont pas encore implémentés ici.
```

Et la documentation de la méthode le répète (Core/NkImage.cpp:3009) : «NK_BICUBIC, NK_LANCZOS3 : *fallback* bilinéaire (non encore implémentés).»

Demander du Lanczos ne plante pas, ne journalise rien, et vous rend du bilinéaire. C'est la pire catégorie de bug : celui qui produit un résultat plausible. Si la qualité de vos vignettes compte, sachez que vous avez du bilinéaire, quoi que dise votre code.

6.7.2 Recadrer, convertir, recopier

Kernel/Runtime/NkImage/src/NkImage/Core/NkImage.h:378-388, 415-464, 505-514

```
1 void FlipVertical() noexcept;
2 void FlipHorizontal() noexcept;
3 void PremultiplyAlpha() noexcept; // RGBA32 seulement
4 void Blit(const NkImage& src, int32 dstX, int32 dstY) noexcept;
5 bool BlitRegion(const NkImage& src, const math::NkIntRect& srcRegion,
6                const math::NkIntRect& dstRegion) noexcept;
7 bool BlitRegion(const NkImage& src, const math::NkIntRect& srcRegion,
8                const math::NkIntRect& dstRegion, NkResizeFilter filter) noexcept;
9 bool Copy(const NkImage& src, int32 dstX, int32 dstY,
10          const math::NkIntRect& area, bool clip = true) noexcept;
11 bool CopyTo(NkImage& dst) const noexcept;
```

Notez le jeu de noms : `Copy(...)` avec arguments est une méthode *instance* qui renvoie `bool` et copie *dans* `*this`; `Copy()` sans argument renvoie un `NkImage*` neuf. Deux méthodes homonymes, deux régimes de mémoire opposés. C'est exactement pourquoi la règle « regardez le type de retour » n'est pas une coquetterie.

6.7.3 Le cas HDR

Une image `.hdr` ou `.exr` contient des flottants dont les valeurs dépassent 1,0 — c'est tout l'intérêt. La convertir naïvement en 8 bits détruit cette information :

Applications/NkImageDemo/src/Demo/ViewerApp.cpp:369-372

```
1 NkImage::Load(.hdr, 4) ferait un Convert lineaire-clamp qui
2 crame toutes les valeurs > 1.0. On passe par [le codec HDR]
3 pour préserver le float, puis ConvertToTexture(exposure,gamma).
```

La bonne séquence est donc : décoder en flottant (le codec HDR le fait), puis appliquer un *tone mapping* explicite avec `ConvertToTexture(image, exposure, gamma)` ([Core/NkImage.h:815](#)), qui rend une image 8 bits affichable.

6.8 Le cas particulier du GIF animé

Le codec GIF a une API à part, parce qu'un GIF peut contenir plusieurs images :

Kernel/Runtime/NkImage/src/NkImage/Codecs/GIF/NkGIFCodec.h:36-53

```
1 struct NkGIFFrame {
2     NkImage *image = nullptr; ///< Frame RGBA32, taille canvas global
3     uint32 delayMs = 0;       ///< Duree d'affichage avant frame suivante
4     uint16 left = 0;          ///< Position originale (info, deja composee)
5     uint16 top = 0;
6     uint8 disposal = 0;       ///< 0=undef,1=keep,2=clear,3=restore (info)
7 };
8 struct NkGIFAnimation {
9     uint32 width = 0; uint32 height = 0; uint32 frameCount = 0;
10    NkGIFFrame *frames = nullptr; ///< Array de frameCount entries
11    uint16 loopCount = 0;          ///< 0 = infini (NETSCAPE2.0)
12 };
```

Le mot important est « déjà composée » : vous n'avez pas à gérer vous-même les modes de *disposal* du format GIF, chaque `frames[i].image` est une image RGBA32 complète, à la taille du canevas global. Les champs `left`, `top` et `disposal` ne sont là que pour information.

Exemple écrit pour le cours (API : `Codecs/GIF/NkGIFCodec.h:64, 67`)

```
1 #include "NKImage/Codecs/GIF/NkGIFCodec.h"
2
3 // data/bytes = contenu brut du .gif lu en memoire
4 NkGIFAnimation *anim = NkGIFCodec::DecodeAnimation(data, bytes);
5 if (anim) {
6     for (uint32 i = 0; i < anim->frameCount; ++i) {
7         NkImage *f = anim->frames[i].image;    // RGBA32, deja composee
8         uint32 d = anim->frames[i].delayMs;    // duree d'affichage
9         /* ... uploader f ... */
10    }
11    NkGIFCodec::FreeAnimation(anim);    // Libere le struct ET toutes Les NkImage*
12 }
```

Un seul appel à `FreeAnimation` libère tout. N'appellez pas `Free()` sur les images individuelles : elles appartiennent à l'animation.

6.9 Libérer correctement : les quatre cas

Récapitulons, parce que c'est ici que les débutants perdent leurs soirées.

Origine de l'image	Libération	Exemple
<code>NkImage img;</code> sur la pile	rien (destructeur)	<code>img.Load(...)</code>
Fabrique statique / <code>Resize</code>	<code>img->Free()</code>	<code>NkImage::Alloc(...)</code>
Tampon <code>Encode*</code>	<code>memory::NkFree(buf)</code>	<code>EncodePNG(buf,n)</code>
<code>NkGIFCodec::DecodeAnimation</code>	<code>FreeAnimation(anim)</code>	animation entière

Et une cinquième méthode, qui n'appartient à aucune de ces lignes :

Kernel/Runtime/NKImage/src/NKImage/Core/NkImage.h:769-783

```
1 Libere les pixels (si owning) ET libere le struct NkImage lui-meme via nkFree.
2 A utiliser UNIQUEMENT sur les images creees via les fabriques statiques
3 (Alloc, Wrap, Create statique, Copy, CopyAs, Convert, Resize, Crop, ...).
4 Ne jamais appeler Free() sur une image allouee sur la stack.
5
6 [NKIResource] Unload() : libere les pixels (si owning) et remet *this dans
7 l'etat « image vide » SANS liberer le struct (contrairement a Free()).
```

`Unload()` sert quand vous voulez réutiliser une `NkImage` de pile pour charger autre chose sans attendre la fin de la portée. `Free()` sur une image de pile, en revanche, tente de libérer une adresse qui n'a jamais été allouée : plantage garanti.

Wrap() ne possède rien

Kernel/Runtime/NKImage/src/NKImage/Core/NKImage.h:795-800

- 1 Cree une vue non-owning sur un buffer pixel externe.
- 2 Le buffer n'est PAS libere par le destructeur ni par Free().
- 3 L'appelant reste responsable de la duree de vie du buffer.

Wrap est très pratique : il permet d'appliquer les méthodes de NKImage (dessin, Save, Encode) à des pixels que vous possédez déjà, sans copie. Mais la NKImage obtenue est une *vue* : si le tampon d'origine meurt avant elle, toute lecture est une lecture après libération.

6.10 Afficher une image dans une interface NKGui

Nous y voilà. C'est la partie que vous attendez, et elle tient en trois gestes : **charger, uploader, déclarer un widget**.

6.10.1 Le widget

Kernel/Runtime/NKGui/src/NKGui/Widgets/NKGuiWidgets.h:96-105

```
1 // — Image / Icône (quad texturé) —————
2 // Dessine la texture `texId` (id backend) à la taille w×h. `tint` multiplie
3 // (blanc = telle quelle) ; `uv0/uv1` = sous-région (atlas/sprite). Auto-Layout.
4 void Image(NKGuiContext &ctx, uint32 texId, float32 w, float32 h,
5            NKColor tint = NKColor{255, 255, 255, 255}, NKVec2 uv0 = NKVec2{0.f, 0.f},
6            NKVec2 uv1 = NKVec2{1.f, 1.f}) noexcept;
7 // Bouton-image (icône cliquable) : fond + image centrée + survol. Retourne true au
8 // clic.
9 bool ImageButton(NKGuiContext &ctx, const char *idStr, uint32 texId, float32 w,
10 float32 h,
11                 NKColor tint = NKColor{255, 255, 255, 255}, NKVec2 uv0 = NKVec2{0.f,
12 0.f},
13                 NKVec2 uv1 = NKVec2{1.f, 1.f}) noexcept;
```

Le premier paramètre après le contexte est un `uint32 texId`. Pas un pointeur, pas une `NKImage` : un **simple entier**. NKGui ne sait rien de votre image ; il écrit ce nombre dans sa liste de commandes et laisse le backend le résoudre. C'est la conséquence directe de l'architecture décrite au chapitre 1.

6.10.2 L'upload

Le pont vers NKCanvas expose la méthode qui associe un `texId` à des pixels :

Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NKGuiCanvasBackend.h:94

```
1 bool UploadImageRGBA(uint32 texId, const uint8 *rgba, int32 w, int32 h);
```

Le nom dit l'exigence : **RGBA**, et implicitement *contigu* (pas de stride). D'où la recopie ligne par ligne que nous avons vue plus haut.

6.10.3 La séquence complète

Voici le bloc réel de la démo, dans son intégralité. C'est l'exemple à recopier :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:290-322

```
1 // — VRAIE image chargée depuis le disque (NKImage) -> texture backend —
2 uint32 imgTexId = 0u;
3 int32 imgW = 0, imgH = 0;
4 {
5     static const char *kCandidates[] = {
6         "Applications/Mou/assets/brand/rihen-logo.png",
7         "Applications/Mou/assets/brand/noge-logo.png",
8         "Resources/Icons/ContentBrowser/FileIcon.png",
9         "../Applications/Mou/assets/brand/rihen-logo.png",
10    };
11    NKImage img;
12    bool ok = false;
13    for (const char *p : kCandidates)
14        if (img.Load(p, 4)) {
15            ok = true;
16            break;
17        }
18    if (ok && img.Width() > 0 && img.Height() > 0 && img.Pixels()) {
19        imgW = img.Width();
20        imgH = img.Height();
21        static NkVector<uint8> rgba;
22        rgba.Resize(static_cast<usize>(imgW) * imgH * 4u);
23        for (int32 y = 0; y < imgH; ++y) {
24            const uint8 *src = img.RowPtr(y);
25            uint8 *dst = rgba.Data() + static_cast<usize>(y) * imgW * 4;
26            for (int32 x = 0; x < imgW * 4; ++x)
27                dst[x] = src[x];
28        }
29        imgTexId = 0x494D4731u; // 'IMG1'
30        backend.UploadImageRGBA(imgTexId, rgba.Data(), imgW, imgH);
31        logger.Info("Image chargée : {0}x{1}", imgW, imgH);
32    } else {
33        logger.Info("Image demo introuvable (cwd) -> section image vide");
34    }
35 }
```

Six observations, dans l'ordre où elles comptent :

1. **Une liste de chemins candidats.** L'application ne sait pas depuis quel répertoire on la lance; elle essaie plusieurs chemins et prend le premier qui marche. Ce n'est pas de la paresse, c'est la réponse honnête à un vrai problème.
2. **NKImage img;** sur la pile, donc aucun Free() à écrire. C'est la voie recommandée.
3. **img.Load(p, 4)** — on force quatre canaux, parce que UploadImageRGBA attend du RGBA.
4. **La recopie ligne par ligne** avec RowPtr(y), pour la raison expliquée plus haut.
5. **imgTexId = 0x494D4731u** — soit 'IMG1' en ASCII. L'identifiant est choisi *par vous*, arbitrairement; il suffit qu'il soit stable et distinct des autres (l'atlas de police, par exemple, utilise 'NKFT'). Un entier lisible en hexadécimal aide énormément au débogage.
6. **Le else journalise.** Sans lui, une image manquante donne simplement... rien. Aucun message. C'est le silence dont parlait le chapitre 1.

Puis, dans la frame, le widget :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:626-637

```
1      Text(ctx, "Image réelle (chargee via NKImage) :");
2      if (imgTexId != 0u && imgW > 0 && imgH > 0) {
3          // Affiche l'image a une largeur fixe en preservant le ratio.
4          const float32 dispW = 150.f;
5          const float32 dispH = dispW * static_cast<float32>(imgH) /
static_cast<float32>(imgW);
6          Image(ctx, imgTexId, dispW, dispH);
7          ctx.SameLine();
8          static int32 ibClicks = 0;
9          if (ImageButton(ctx, "ib", imgTexId, 44.f, 44.f))
10             ++ibClicks;
11     } else {
12         Text(ctx, "(aucune image trouvee)");
13     }
```

Le calcul de `dispH` mérite d'être noté : Image ne préserve *pas* le rapport d'aspect, elle dessine exactement le rectangle que vous demandez. Si vous lui donnez une taille carrée pour une image rectangulaire, elle l'écrase sans se plaindre.

Les trois gestes

1. `NKImage img; img.Load(chemin, 4);` — charge et force le RGBA;
 2. `backend.UploadImageRGBA(monId, pixelsContigus, w, h);` — une fois, à l'initialisation;
 3. `Image(ctx, monId, w, h);` — chaque frame, dans la déclaration d'interface.
- La `NKImage` peut mourir après l'étape 2 : le backend a copié les pixels côté GPU. C'est l'entier `monId` qui doit survivre.

6.10.4 Le chemin plus court : `NKTexture` de `NKCanvas`

Si vous n'êtes pas dans une interface `NKGui` mais dans une scène `NKCanvas`, il existe un raccourci qui fait tout :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NKTexture.h:53-58

```
1 bool Create(NKIRenderer2D &renderer, uint32 width, uint32 height, const NKColor2D &fillColor);
2 bool LoadFromFile(NKIRenderer2D &renderer, const char *path);
3 bool LoadFromImage(NKIRenderer2D &renderer, const NKImage &image, const NKRect2i &area =
    NKRect2i{});
4 bool LoadFromMemory(NKIRenderer2D &renderer, const void *data, uint sizeBytes);
```

Son implémentation montre qu'il n'y a aucune magie — c'est exactement ce que vous auriez écrit :

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NKTexture.cpp:78-86

```
1 bool NKTexture::LoadFromFile(NKIRenderer2D &renderer, const char *path) {
2     NKImage img;
3     if (!img.Load(path))
4         return false;
5     return LoadFromImage(renderer, img);
6 }
```

Créer les textures *après* Initialize()

Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkTexture.cpp:17-19

```
1 Active backend dispatch table. Initialement vide (callbacks null) :  
2 les operations sur NkTexture sont des no-ops tant qu'aucun renderer  
3 n'a appele NkTextureSetBackend() a la fin de son Initialize().
```

Autrement dit : une NkTexture créée avant que le renderer soit initialisé ne fait **rien**, et ne dit rien. C'est un invariant d'ordre pur. Créez vos textures après `renderer.Initialize()`, toujours.

6.11 Décoder ailleurs, uploader ici

Décoder un JPEG de huit mégapixels prend des dizaines de millisecondes : bien plus qu'une image à 60 Hz. La tentation est de le faire sur un fil de travail.

Le module ne l'interdit pas, mais il ne le promet pas non plus : le `.jenga` liste `NKThreading` en dépendance, et aucun verrou n'est exposé dans l'API publique. Aucun commentaire du module ne garantit la *thread-safety*. La règle prudente, qui est celle de la visionneuse du dépôt ([Applications/NkImageDemo/src/Demo/ViewerApp.cpp:463](#), « Upload sur le main thread (libere l'image apres) »), tient en une ligne :

Ce qu'il faut retenir

Décodage dans un fil de travail, upload sur le fil principal. Le décodage produit des octets — c'est du calcul pur, isolé. L'upload touche au contexte graphique, qui appartient au fil principal.

6.12 État réel du module

Terminons par l'inventaire honnête, parce que c'est ce qui vous fera gagner du temps.

6.12.1 Ce qui fonctionne

Format	Lecture	Écriture
PNG	oui	oui
JPEG	oui	oui
BMP	oui	oui
TGA	oui	oui
PPM / PGM / PBM	oui	oui
QOI	oui	oui
HDR	oui	oui
EXR	oui	oui (scanline float, compression NONE)
GIF	oui (statique <i>et</i> animé)	oui (palette 256 + LZW)
ICO	oui	aucun encodeur
WebP	oui	par le codec seulement, pas par Save
SVG	oui (rastérisation)	non

S'y ajoutent : le redimensionnement en `NK_NEAREST` et `NK_BILINEAR`, et toutes les primitives de dessin CPU.

6.12.2 Ce qui ne fonctionne pas

- `NkImage::SaveWebP` et `NkImage::SaveSVG` renvoient `false` sans rien faire ([Core/NkImage.cpp:2119-2134](#));
- `NK_BICUBIC` et `NK_LANCZOS3` retombent silencieusement sur du bilinéaire ([Core/NkImage.cpp:2901-2903](#));
- l'exemple en tête de [src/NkImage/NkImage.h](#) cite des classes supprimées.

NkImageDemo ne compile plus

Vous trouverez dans le dépôt une application [Applications/NkImageDemo](#), déclarée dans [Nkentseu.jenga:1447](#). Ne la prenez pas pour modèle d'appel :

[Applications/NkImageDemo/src/Demo/ViewerApp.cpp:450](#)

```
1      NkImage *img = NkImage::Load(path.CStr(), 4);
```

Il n'existe **aucune** `static NkImage* Load(...)`. Le seul `Load` du module est la méthode d'instance `bool Load(const char*, int32)` ([Core/NkImage.h:262](#)). Cette démo n'apparaît d'ailleurs pas dans [Build/Bin/Debug-Windows/](#) : elle ne compile plus contre l'API actuelle. Sa *logique* en revanche reste excellente à lire — parcours de dossier, lecture de GIF animé, *tone mapping* HDR. Lisez-la comme un plan, pas comme une référence d'API.

6.12.3 Les deux classes utilitaires exportées

Pour mémoire, parce que vous les croiserez en lisant les codecs :

- `NkImageStream` ([Core/NkImage.h:933-1043](#)) — un lecteur/écrivain binaire qui sait lire en *big endian* comme en *little endian* : `ReadU16BE`, `ReadU32LE`, `ReadBytes`, `Skip`, `Seek`, `Tell`, `HasError`, et les `Write*` symétriques. C'est l'outil avec lequel tous les codecs du module sont écrits;
- `NkDeflate` ([Core/NkImage.h:1085-1217](#)) — la compression *deflate/inflate* du PNG, exposée publiquement : `Decompress`, `DecompressRaw`, `Compress`. Utile bien au-delà des images.

6.13 Exercices

1 — Le round-trip, et le format qui échoue

Écrivez un petit programme console qui :

1. alloue une image RGB24 de 256×256 via `NkImage::Alloc`;
2. la remplit d'un dégradé à la main, en utilisant `Stride()`;
3. la sauvegarde en `.png`, `.bmp`, `.tga`, `.qoi` et `.webp`, en **testant chaque valeur de retour** et en affichant le résultat;
4. recharge le `.png` et compare octet par octet avec l'original.

Une seule des cinq sauvegardes doit échouer. Laquelle, et pourquoi? Inspirez-vous de [Applications/NkImageCodecTest/src/main.cpp](#), qui fait exactement cela en 79 lignes.

2 — Rendre le stride visible

Chargez le même PNG deux fois : une fois avec `Load(path, 3)`, une fois avec `Load(path, 4)`. Pour chacune, affichez `Width()`, `Channels()`, `BytesPP()` et `Stride()`. Choisissez une largeur qui n'est pas multiple de 4 (par exemple 101 pixels) et vérifiez que `Stride() != Width() * BytesPP()` dans le cas à trois canaux.

Puis, volontairement, écrivez la conversion en RGBA avec un `memcpy` global au lieu de `RowPtr`, affichez le résultat, et regardez la diagonale apparaître. Une erreur qu'on a vue une fois ne se refait pas.

3 — Une vignette dans une interface

Reprenez l'application `NKGui` du chapitre 4 et ajoutez-lui un panneau qui :

- charge une image du dépôt (essayez plusieurs chemins candidats);
- en fabrique une vignette de 128 pixels de large avec `Resize`, **en préservant le rapport d'aspect**;
- upload la vignette sous un `texId` de votre choix;
- l'affiche avec `Image`, et affiche à côté ses dimensions d'origine avec `Text`.

Vérifiez ensuite avec un débogueur — ou un simple compteur — que vous avez bien appelé `Free()` exactement une fois sur le résultat de `Resize`, et zéro fois sur l'image de pile.

4 — Mesurer le mensonge

Chargez une photographie assez grande, puis produisez deux vignettes de la même taille : une avec `NK_BILINEAR`, une avec `NK_LANCZOS3`. Sauvegardez les deux en PNG, puis comparez-les octet par octet.

Elles doivent être **rigoureusement identiques**. Vous venez de démontrer vous-même, sans lire le `.cpp`, que le filtre Lanczos n'existe pas. C'est exactement la démarche à appliquer chaque fois qu'une API vous promet quelque chose que vous n'avez pas vu marcher.

NKFont : polices, atlas et métriques

Afficher du texte est l'opération la plus banale d'une interface et la plus mal comprise. On croit qu'il suffit d'« écrire une chaîne » ; en réalité, il faut convertir des octets UTF-8 en *codepoints*, trouver le dessin de chaque caractère, le rastériser une fois pour toutes dans une grande image, puis émettre deux triangles texturés par lettre en faisant avancer un curseur.

NKFont fait tout cela, et **rien d'autre**. C'est un module remarquablement honnête sur son périmètre : il produit un bitmap en mémoire vive, et vous rend pour chaque caractère un rectangle à l'écran et un rectangle dans le bitmap. Ce qu'on en fait ensuite ne le regarde pas.

7.1 Le module ne connaît ni image, ni GPU, ni fenêtre

Kernel/Runtime/NKFont/NKFont.jenga:35-39

```
1 nkentseudependson(
2   ["NKPlatform", "NKCore", "NKMemory", "NKMath", "NKContainers", "NKThreading",
   "NKLogger"],
3   selfexport="NKFont",
4   extra_includes=["src"],
5 )
```

Ni NKImage, ni NKCanvas, ni NKWindow. NKFont est même plus autonome que NKImage : il ne dépend pas du système de fichiers pour fonctionner, puisqu'il embarque ses propres polices — nous y venons.

Kernel/Runtime/NKFont/ — arborescence

```
1 NKFont/
2   NKFont.jenga
3   src/NKFont/
4     NKFont.h           ( 366 l.) <- EN-TETE PUBLIC PRINCIPAL (NkFontAtlas + NkFont)
5     NkFontAtlas.cpp    (~ 700 l.) <- packing + rasterisation
6     NkFontMesh.cpp     (~ 400 l.) <- extrusion 3D des glyphes
7     Core/
8       NKFontTypes.h    <- alias nkft_*, NkFontCodepoint, NkGlyphId
9       NKFontParser.h/.cpp <- parser TTF/OTF/CFF/WOFF + rasteriseur + SDF
10      NkFontRasterizer.cpp
11      NkFontDetect.h/.cpp <- NkFontProfile / NkFontDetector
12      NkFontSizeCache.h/.cpp <- cache multi-tailles
13      Embedded/
14      NkFontEmbedded.h/.cpp <- 10 polices compressées DANS le binaire
```

La description du .jenga promet six fonctionnalités inexistantes

Kernel/Runtime/NKFont/NKFont.jenga:7-23 annonce fièrement le support de BDF, de Type 1 (PFB/PFA), de WOFF2 avec Brotli, d'une machine virtuelle de *hinting* TrueType, de GSUB pour les ligatures, et de l'algorithme bidirectionnel UAX#9.

Une recherche exhaustive sur `src/` donne **zéro occurrence** de BDF, Type1, PFB, Brotli,

Bidi, GSUB et Hinting. La seule mention de WOFF2 est un avertissement disant qu'il n'est pas supporté. GPOS apparaît une fois, pour lire un décalage de table qui n'est ensuite jamais utilisé ([Core/NkFontParser.cpp:993](#)).

La liste honnête est celle que le parser affiche lui-même :

Kernel/Runtime/NKFont/src/NKFont/Core/NkFontParser.h:5-14

```
1 // Formats supportés :
2 // OK TTF SimpleGlyph + CompositeGlyph
3 // OK TTC (TrueType Collection) via faceIndex
4 // OK OTF avec table glyf (contours TrueType)
5 // OK OTF/CFF (Type 2 charstrings – interpréteur intégré)
6 // OK WOFF (décompresseur zlib intégré)
7 // /\ WOFF2 (Brotli – non supporté, dépendance externe requise)
8 // OK cmap format 4 (BMP) et format 12 (full Unicode)
9 // OK kern table format 0
10 // OK SDF generation (Signed Distance Field)
```

C'est déjà beaucoup — un interpréteur CFF Type 2 écrit à la main n'est pas rien. Mais c'est ça, et pas ce que promet le fichier de build.

7.2 Les quatre objets à connaître

7.2.1 NkFontAtlas — le propriétaire de tout

L'atlas est la seule chose que vous créez vous-même. Il possède la texture, les configurations, et les polices.

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:102-133 (champs publics)

```
1 void *texID = nullptr;
2 nkft_int32 texWidth = 0, texHeight = 0;
3 nkft_uint8 *texPixels = nullptr; // ALPHA8, 1 octet/pixel
4 bool texReady = false;
5 nkft_int32 texDesiredWidth = 0;
6 nkft_int32 texGlyphPadding = 2; ///< Padding recommandé >= 2 pour éviter le
    bleeding.
7 bool sdfMode = false;
8 nkft_int32 sdfSpread = 6; ///< Rayon SDF en pixels (4-8 typique).
9 NkFont *fonts[NK_FONT_ATLAS_MAX_FONTS] = {};
10 NkFontConfig configs[NK_FONT_ATLAS_MAX_FONTS];
11 nkft_uint32 fontCount = 0u;
```

Ce sont des champs publics, sans accesseurs : on les lit et on les écrit directement. C'est délibéré et cohérent avec le reste du moteur.

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:134-157

```
1 NkFont* AddFontFromFile(const char* path, nkft_float32 sizePixels, const NkFontConfig* cfg =
    nullptr);
2 NkFont* AddFontFromMemory(const nkft_uint8* data, nkft_size dataSize,
3     nkft_float32 sizePixels, const NkFontConfig* cfg = nullptr);
4 NkFont* AddFontFromMemoryOwned(nkft_uint8* data, nkft_size dataSize,
5     nkft_float32 sizePixels, const NkFontConfig* cfg = nullptr);
6
7 static const nkft_uint32* GetGlyphRangesDefault();
```

```

8 static const nkft_uint32* GetGlyphRangesLatinExtA();
9 static const nkft_uint32* GetGlyphRangesCyrillic();
10 static const nkft_uint32* GetGlyphRangesGreek();
11 static const nkft_uint32* GetGlyphRangesChineseFull();
12
13 bool Build(); // <- rasterise TOUT
14
15 void GetTexDataAsAlpha8 (nkft_uint8** op, nkft_int32* ow, nkft_int32* oh, nkft_int32* ob =
    nullptr);
16 void GetTexDataAsRGBA32 (nkft_uint8** op, nkft_int32* ow, nkft_int32* oh, nkft_int32* ob =
    nullptr);
17
18 void ClearTexData();
19 void Clear();
20 bool IsBuilt() const;

```

Comme NkImage, l'atlas n'est pas copiable : `NkFontAtlas(const NkFontAtlas&) = delete;` (`NkFont.h:129`).

7.2.2 NkFont — une face rastérisée à une taille

Attention au nom : dans ce module, NkFont n'est pas « une police ». C'est **une police à une taille donnée**, déjà rastérisée. Charger Inter en 18 et en 32 pixels donne *deux* NkFont.

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:211-219

```

1 NkFontAtlas *containerAtlas = nullptr; // L'atlas POSSEDE cette NkFont
2 NkFontConfig config;
3 nkft_float32 fontSize = 0, ascent = 0, descent = 0, lineAdvance = 0, scale = 1;
4 NkFontGlyph glyphs[NK_FONT_MAX_GLYPHS];
5 nkft_uint32 glyphCount = 0u;
6 const NkFontGlyph *fallbackGlyph = nullptr;
7 nkft_float32 fallbackAdvanceX = 0;

```

Le commentaire d'en-tête est catégorique sur la propriété :

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:88-90

```

1 NkFont (le struct produit) n'est pas auto-portant : il est detenu par
2 son `containerAtlas` qui parse le TTF, fournit les glyphes et la
3 texture. NkFont seul ne « se charge » pas.

```

Ce qu'il faut retenir

Les NkFont* que vous recevez appartiennent à l'atlas. **Ne les détruisez jamais vous-même**, et ne les gardez pas au-delà de la vie de l'atlas. Détruire l'atlas détruit toutes ses faces.

7.2.3 NkFontGlyph — la structure qui fait tout le travail

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:32-38

```

1 struct NkFontGlyph {
2     NkFontCodepoint codepoint = 0;
3     nkft_float32 advanceX = 0.f;
4     nkft_float32 x0 = 0, y0 = 0, x1 = 0, y1 = 0; ///< Position quad (pixels écran,
    relative au curseur).

```

```

5         nkft_float32 u0 = 0, v0 = 0, u1 = 0, v1 = 0; ///< UV dans l'atlas [0..1].
6         bool visible = true;
7     };

```

Lisez-la attentivement : c'est la structure la plus importante du chapitre.

- x_0 , y_0 , x_1 , y_1 sont les coins du rectangle à l'écran, **relatifs au curseur** — pas absolus. Vous ajoutez la position du curseur, et c'est tout;
- u_0 , v_0 , u_1 , v_1 sont les coordonnées dans l'atlas, déjà normalisées entre 0 et 1;
- `advanceX` est de combien avancer le curseur après ce caractère;
- `visible` vaut `false` pour l'espace et les caractères qui n'ont pas de dessin : on saute l'émission du quad mais on avance quand même le curseur.

Il n'y a **rien d'autre à calculer**. Pas de métriques à recomposer, pas de conversion d'unités de police en pixels : `Build()` l'a déjà fait.

7.2.4 NkFontConfig — les réglages, posés avant l'ajout

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:44-62

```

1     struct NkFontConfig {
2         const nkft_uint8 *fontData = nullptr;
3         nkft_size fontDataSize = 0u;
4         bool fontDataOwned = false;
5         nkft_int32 fontIndex = 0;
6         nkft_float32 sizePixels = 13.f;
7         nkft_int32 oversampleH = 2;
8         nkft_int32 oversampleV = 1;
9         bool pixelSnapH = false;
10        math::NkVec2f glyphOffset = {0.f, 0.f};
11        nkft_float32 glyphMinAdvanceX = 0.f;
12        nkft_float32 glyphMaxAdvanceX = 1e9f;
13        nkft_float32 glyphExtraSpacing = 0.f;
14        const nkft_uint32 *glyphRanges = nullptr;
15        NkFontCodepoint glyphFallback = 0x003Fu;
16        bool mergeMode = false;
17        nkft_float32 rasterizerMultiply = 1.f;
18        char name[40] = {};
19    };

```

Dix-sept champs, dont deux comptent vraiment au début.

`glyphRanges` est un tableau de *paires* {début, fin} terminé par un zéro. Si vous le laissez à `nullptr`, vous obtenez la plage par défaut. Si vous voulez le latin étendu plus les caractères de dessin de boîtes, vous écrivez :

Exemple écrit pour le cours (API : NkFont.h:56, 134)

```

1     static const uint32 kRanges[] = { 0x0020, 0x00FF, 0x2500, 0x257F, 0 }; // Latin-1 +
    box-drawing
2     NkFontConfig cfg;
3     cfg.glyphRanges = kRanges;
4     NkFont *f = atlas.AddFontFromFile("Resources/Fonts/Inter-Regular.ttf", 20.f, &cfg);

```

`mergeMode` est le mécanisme de *repli* : en ajoutant une seconde police avec `mergeMode = true`, ses glyphes viennent combler ceux qui manquent dans la première, dans la même `NkFont`.

C'est ainsi qu'une police latine peut afficher des idéogrammes ou des émojis. Nous verrons le code réel plus loin.

7.3 Les dix polices embarquées : écrire du texte sans aucun fichier

C'est la meilleure porte d'entrée du module, et sans doute la fonctionnalité la plus sous-estimée du moteur.

Kernel/Runtime/NKFont/src/NKFont/Embedded/NkFontEmbedded.h:49-74

```
1 enum class NkEmbeddedFontId : nkft_uint32 {
2     ProggyClean = 0, ProggyTiny = 1, DroidSans = 2, Karla = 3, Roboto = 4,
3     Cousine = 5, SourceCodePro = 6, DroidSerif = 7, DejaVuSansMono = 8, Inter = 9,
4     Count = 10, Default = ProggyClean,
5 };
```

Ces dix polices sont *réellement* présentes : le fichier `Embedded/NkFontEmbedded.cpp` pèse 2,1 Mo de données compressées, et le registre (`NkFontEmbedded.cpp:20265-20370`) fait pointer chaque entrée sur un tableau non vide. Aucune n'est un espace réservé.

Kernel/Runtime/NKFont/src/NKFont/Embedded/NkFontEmbedded.h:120-158

```
1 static bool IsAvailable(NkEmbeddedFontId id);
2 static const NkEmbeddedFontData* GetData(NkEmbeddedFontId id);
3 static NkFont* AddToAtlas(NkFontAtlas& atlas, NkEmbeddedFontId id,
4     nkft_float32 sizePx = 0.f, const NkFontConfig* cfg = nullptr);
5 static NkFont* AddDefaultFont(NkFontAtlas& atlas, const NkFontConfig* cfg = nullptr);
6 static const char* GetName(NkEmbeddedFontId id);
7 static const NkEmbeddedFontData* GetAll(nkft_int32* outCount);
8 static nkft_uint8* DecompressData(const NkEmbeddedFontData& data, nkft_uint32* outSize =
9     nullptr);
9 static void FreeDecompressedData(nkft_uint8* ptr);
```

Les licences sont déclarées dans le registre : ProggyClean et ProggyTiny sont en MIT ; DroidSans, Roboto, Cousine et DroidSerif en Apache 2.0 ; Karla, SourceCodePro et Inter en OFL 1.1 ; DejaVuSansMono en Bitstream Vera / domaine public. Ce n'est pas un détail décoratif : cela veut dire que vous pouvez distribuer votre exécutable sans vous poser de question.

Ce qu'il faut retenir

Votre première application affichant du texte n'a besoin d'aucun fichier de police, d'aucun dossier de ressources, d'aucun chemin relatif. Une ligne suffit : `NkFontEmbedded::AddToAtlas(atlas, NkEmbeddedFontId::Inter, 18.f)`. Vous n'aurez jamais l'erreur « police introuvable ».

Le dépôt s'en sert d'ailleurs avec un repli en cascade, parce que rien n'oblige une police donnée à être compilée dans un binaire particulier :

Applications/Pong/src/Pong/Render/FontAtlas.cpp:52-58

```
1     NkEmbeddedFontId fontId = NkEmbeddedFontId::ProggyClean;
2     if (NkFontEmbedded::IsAvailable(NkEmbeddedFontId::Karla)) {
3         fontId = NkEmbeddedFontId::Karla;
4     } else if (NkFontEmbedded::IsAvailable(NkEmbeddedFontId::DroidSans)) {
5         fontId = NkEmbeddedFontId::DroidSans;
6     } else if (NkFontEmbedded::IsAvailable(NkEmbeddedFontId::Roboto)) {
```

```

7         fontId = NkEmbeddedFontId::Roboto;
8     }

```

7.4 La séquence canonique en quatre temps

Tout usage de NKFont suit le même ordre, sans exception :

1. `atlas.Clear()` — si l'atlas a déjà servi;
2. `Add*()` — une ou plusieurs fois, une par police et par taille;
3. `atlas.Build()` — **c'est ici que les pixels apparaissent**;
4. `atlas.GetTexDataAsAlpha8(...)` — on récupère le bitmap.

Le pont officiel entre NKFont et NKGui l'applique littéralement :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiFont.cpp:118-133

```

1     bool NkGuiFont::LoadEmbedded(NkEmbeddedFontId id, float32 sizePx, bool extFallback)
2     noexcept {
3         atlas.Clear();
4         face = nullptr;
5         pixels = nullptr; // rechargeable
6         NkFontConfig cfg;
7         cfg.glyphRanges = NkGlyphRanges();
8         face = NkFontEmbedded::AddToAtlas(atlas, id, sizePx, &cfg);
9         if (!face)
10            return false;
11        NkMergeFallback(atlas, sizePx, extFallback); // repli pour les glyphes manquants
12        if (!atlas.Build())
13            return false;
14        int32 bpp = 0;
15        atlas.GetTexDataAsAlpha8(&pixels, &atlasW, &atlasH, &bpp);
16        dirty = (pixels != nullptr && atlasW > 0 && atlasH > 0);
17        return dirty;
18    }

```

L'ordre n'est pas négociable, et l'échec est silencieux

`Build()` refuse de travailler sur un atlas vide et le journalise :

Kernel/Runtime/NKFont/src/NKFont/NkFontAtlas.cpp:330-333

```

1     if (fontCount == 0) {
2         logger.Info("[NkFontAtlas] Build():aucune fonte\n");
3         return false;
4     }

```

Mais `GetTexDataAsAlpha8`, appelé avant `Build()`, ne journalise *rien* : il pose simplement `nullptr` (`NkFontAtlas.cpp:640-642` : `if (!texReady || !texPixels) { *op = nullptr; ... }`). Vous uploadez alors un pointeur nul, le backend ne fait rien, et vous cherchez pendant une heure pourquoi tous vos textes sont invisibles.

Testez toujours le pointeur récupéré, comme le fait `NkGuiFont` avec son champ `dirty`.

7.4.1 Le mécanisme de repli en action

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiFont.cpp:98-113

```
1      static void NkMergeFallback(NkFontAtlas &atlas, float32 sizePx, bool ext) noexcept {
2          if (!ext)
3              return;
4          auto add = [&](const char *path, const uint32 *ranges) {
5              if (!NkFileExists(path))
6                  return;
7              NkFontConfig fb;
8              fb.glyphRanges = ranges;
9              fb.mergeMode = true;
10             atlas.AddFontFromFile(path, sizePx > 0.f ? sizePx : 16.f, &fb);
11         };
12         add(gFbBroad, NkBroadRanges());
13         add(gFbCjk, NkCjkRanges());
14         add(gFbEmoji, NkEmojiRanges());
15     }
```

Trois polices de repli : une couverture large, une pour le chinois-japonais-coréen, une pour les émojis. Chacune n'est ajoutée que si le fichier existe — d'où le `NkFileExists`. Les chemins, eux, sont posés par l'application :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiFont.h:76-78

```
1  A poser par l'APPLICATION (ex. NKCode, depuis son dossier data/fonts) AVANT
2  de charger les polices. Un chemin nullptr/vide = role desactive.
3  void NkSetFallbackFontPaths(const char* broad, const char* cjk, const char* emoji) noexcept;
```

Avant de charger — pas après. Une fois l'atlas construit, il est trop tard.

7.5 Dessiner une chaîne de caractères

7.5.1 Le patron universel

Voici la boucle que vous écrirez, sous une forme ou une autre, chaque fois que vous rendrez du texte vous-même. C'est celle de NKGui :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:245-266

```
1      while (p < end) {
2          const NkFontCodepoint cp = NkFontDecodeUTF8(&p, end);
3          if (cp == 0u)
4              break;
5          const NkFontGlyph *g = face->FindGlyph(cp);
6          if (!g)
7              continue;
8          if (g->visible) {
9              const float32 x0 = NkGuiPixelSnap(x + g->x0), y0 = NkGuiPixelSnap(y +
10             g->y0);
11             const float32 x1 = x0 + (g->x1 - g->x0), y1 = y0 + (g->y1 - g->y0);
12             if (x1 > xEnd)
13                 break; // troncature simple
14             const uint32 i0 = Vtx({x0 + sTop, y0}, {g->u0, g->v0}, c);
15             const uint32 i1 = Vtx({x1 + sTop, y0}, {g->u1, g->v0}, c);
16             const uint32 i2 = Vtx({x1 + sBot, y1}, {g->u1, g->v1}, c);
17             const uint32 i3 = Vtx({x0 + sBot, y1}, {g->u0, g->v1}, c);
```



```

17         Tri(i0, i1, i2, texId);
18         Tri(i0, i2, i3, texId);
19     }
20     x += g->advanceX;
21 }

```

Décortiquons.

1. `NkFontDecodeUTF8(&p, end)` avance le pointeur `p` et rend le *codepoint*. C'est une fonction libre du module (`NkFont.h:313`), doublée d'une méthode statique `NkFont::DecodeUTF8` (`NkFont.h:235`);
2. `FindGlyph(cp)` cherche le glyphe **avec repli** : si le caractère n'existe pas dans la police, on obtient le glyphe de remplacement plutôt que `nullptr`. La variante `FindGlyphNoFallback` existe si vous voulez savoir la vérité;
3. si visible, on émet quatre sommets et deux triangles. Les positions sont `x + g->x0` et `y + g->y0` : le curseur plus le décalage du glyphe;
4. **en dehors du `if`**, on avance : `x += g->advanceX`. L'espace n'a pas de dessin mais fait avancer le curseur.

7.5.2 Le pixel-snap, et l'invariant subtil

Regardez à nouveau : les positions passent par `NkGuiPixelSnap`. Le commentaire explique pourquoi :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:225-231

```

1  Le curseur de texte accumule des avances FRACTIONNAIRES. Sans arrondi,
2  seul le PREMIER glyphe d'une chaîne tombe sur un pixel entier ; tous les
3  suivants dérivent et échantillonnent l'atlas ENTRE deux texels, ce qui
4  rend le texte uniformément flou.

```

Mais l'arrondi est appliqué avec une précaution qu'on ne devine pas :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiDrawList.cpp:257-266

```

1  On arrondit la POSITION du quad, jamais l'AVANCE : `x` continue
2  d'accumuler la valeur exacte, donc la largeur totale de la chaîne
3  est inchangée et MeasureWidth reste d'accord avec le rendu.

```

Arrondir l'affichage, pas le calcul

Si vous arrondissiez `advanceX`, l'erreur s'accumulerait caractère après caractère et la largeur mesurée ne correspondrait plus à la largeur dessinée — vos alignements, vos centrages et vos troncatures deviendraient tous faux. C'est une leçon générale : **on arrondit à l'affichage, jamais dans l'accumulateur**.

7.5.3 La ligne de base, pas le haut du texte

`y` dans la boucle ci-dessus n'est pas le haut du texte : c'est la **ligne de base** — la ligne sur laquelle « posent » les lettres, et sous laquelle descendent les jambages du `p` et du `g`. Les `g->y0` sont donc négatifs pour la plupart des caractères.

Les métriques nécessaires sont dans la `NkFont`, calculées par `Build()` :

- ascent — hauteur au-dessus de la ligne de base;
- descent — profondeur en dessous;
- lineAdvance — de combien descendre pour la ligne suivante;
- fontSize, scale — la taille demandée et le facteur appliqué.

Donc, pour dessiner à partir d'un coin haut-gauche :

Exemple écrit pour le cours (API : NkFont.h:213)

```
1 float x = posX;
2 float y = posY + font->ascent;    // y = LIGNE DE BASE, pas le haut !
3 // ... boucle de glyphs ...
4 y += font->lineAdvance;           // ligne suivante
```

La constante magique de Pong

Le jeu Pong approxime la montée par un facteur empirique :

Applications/Pong/src/Pong/Render/FontAtlas.cpp:206-208

```
1      const float ascent = fontPx * 0.82f;
```

Cela fonctionne pour *cette* police, à *ces* tailles. Changez de police et tout se décale verticalement. La valeur exacte existe pourtant : c'est font->ascent, calculée par Build() (NkFontAtlas.cpp:377). Utilisez-la.

7.5.4 Mesurer avant de dessiner

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:229-235

```
1      nkft_float32 GetCharAdvance(NkFontCodepoint c) const {
2          const NkFontGlyph *g = FindGlyph(c);
3          return g ? g->advanceX : fallbackAdvanceX;
4      }
5
6      nkft_float32 CalcTextSizeX(const char *text, const char *textEnd = nullptr) const;
7      static NkFontCodepoint DecodeUTF8(const char **text, const char *textEnd);
```

CalcTextSizeX parcourt la chaîne et somme les avances. C'est ce qu'il faut pour centrer un libellé, dimensionner un bouton, ou décider d'une troncature. Notez qu'il n'existe pas de CalcTextSizeY : la hauteur d'une ligne, c'est lineAdvance, point.

7.6 De l'atlas à l'écran

7.6.1 L'atlas est en alpha 8 bits

GetTexDataAsAlpha8 rend un octet par pixel : la *couverture* du glyphe, de 0 (rien) à 255 (plein). Ce n'est pas une image en niveaux de gris à afficher telle quelle — c'est un masque.

Or les rasteriseurs 2D du moteur échantillonnent des textures RGBA. Il faut donc convertir, et le dépôt explique exactement pourquoi la recette est celle-là :

Applications/Pong/src/Pong/Render/FontAtlas.cpp:92-104

```

1      // L'atlas NKFont est alpha8 (1 canal = couverture du glyphe). Le
2      // shader 2D de NKCanvas echantillonne la texture en RGBA puis la
3      // multiplie par la couleur du vertex. On convertit donc l'atlas en
4      // RGBA8 "blanc + alpha=couverture" : texture(1,1,1,cov) * color =
5      // (color.rgb, color.a*cov) => exactement le rendu de texte voulu.
6      const usize pixCount = static_cast<usize>(w) * static_cast<usize>(h);
7      NkVector<uint8> rgba;
8      rgba.Resize(pixCount * 4u);
9      for (usize i = 0; i < pixCount; ++i) {
10         rgba[i * 4 + 0] = 255;
11         rgba[i * 4 + 1] = 255;
12         rgba[i * 4 + 2] = 255;
13         rgba[i * 4 + 3] = pixels[i]; // couverture du glyphe
14     }

```

Blanc partout, alpha = couverture. Multiplié par la couleur du sommet, cela donne exactement la couleur voulue avec la bonne transparence. Vous n'avez pas à écrire cette boucle si vous ne le voulez pas : `GetTexDataAsRGBA32` (`NkFontAtlas.cpp:662-680`) fait rigoureusement la même chose, en allouant un second tampon à la demande.

7.6.2 Le piège d'upload

Applications/Pong/src/Pong/Render/FontAtlas.cpp:107-109

```

1      // NB : surtout PAS Create()+Update() : Update exige gTextureBackend.Update
2      // qui n'est pas cable sur tous les backends -> echec silencieux.

```

Ce commentaire vaut avertissement général : dans `NkCanvas`, préférez `LoadFromImage` en un seul appel plutôt que la paire `Create` + `Update`, tant que tous les backends ne câblent pas `Update`.

7.6.3 Le chemin NKGui, celui que vous emploierez

Dans une interface `NKGui`, tout ce qui précède est encapsulé dans un petit *wrapper* :

Kernel/Runtime/NKGui/src/NKGui/Core/NkGuiFont.h:19-69 (abrégé)

```

1      struct NkGuiFont {
2          NkFontAtlas atlas;           ///< possède la texture + les glyphes
3          NkFont *face = nullptr;      ///< face produite (détenue par l'atlas)
4          uint32 texId = 0x4E4B4654u; ///< 'NKFT' - id stable pour le backend
5          uint8 *pixels = nullptr;    ///< atlas alpha8 (détenu par l'atlas)
6          int32 atlasW = 0;
7          int32 atlasH = 0;
8          bool dirty = false;          ///< à (ré)uploader côté backend
9          bool LoadEmbedded(NkEmbeddedFontId id, float32 sizePx, bool extFallback =
10 true) noexcept;
11          bool LoadFromFile(const char *path, float32 sizePx, bool extFallback = true)
12          noexcept;
13          /* Face() TexId() Valid() Ascent() Descent() LineHeight() MeasureWidth() */
14     };

```

et son branchement tient en six lignes :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:281-288

```
1 auto fontPtr = memory::NkMakeUnique<NkGuiFont>();
2 if (!fontPtr->LoadEmbedded(NkEmbeddedFontId::DroidSans, 18.f)) {
3     fontPtr->LoadEmbedded(NkEmbeddedFontId::ProggyClean, 16.f);
4 }
5 ctx.font = fontPtr.Get();
6 if (fontPtr->Valid()) {
7     backend.UploadFontGray8(fontPtr->TexId(), fontPtr->pixels, fontPtr->atlasW,
8     fontPtr->atlasH);
9 }
```

Notez UploadFontGray8 et non UploadImageRGBA : le backend sait qu'un atlas de police est en un seul canal et fait la conversion lui-même ([Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NKGuiCanvasBackend.h:41](#)). Après quoi ctx.font pointe la face, et tous les widgets de NKGui dessinent leur texte tout seuls.

Trois lignes, trois responsabilités

1. fontPtr->LoadEmbedded(id, taille) — NKFont rasterise;
2. backend.UploadFontGray8(...) — le backend crée la texture GPU;
3. ctx.font = fontPtr.Get() — NKGui sait quelle face utiliser.

Oubliez la deuxième : tous les textes sont invisibles, sans message d'erreur. Oubliez la troisième : les widgets n'affichent aucun libellé, sans message d'erreur non plus.

7.6.4 Changer d'échelle : recharger et ré-uploader

Sur un écran à 150 %, il ne suffit pas d'agrandir les quads : l'atlas doit être rasterisé plus gros, sinon le texte est flou. La démo le fait à la touche F9 :

Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:461-466

```
1 if (pendingScale > 0.f) { // F9 : appliquer la nouvelle echelle DPI
2     SetUiScale(ctx, pendingScale);
3     if (fontPtr->LoadEmbedded(NkEmbeddedFontId::DroidSans, 18.f * pendingScale))
4         backend.UploadFontGray8(fontPtr->TexId(), fontPtr->pixels, fontPtr->atlasW,
5         fontPtr->atlasH);
6     pendingScale = 0.f;
7 }
```

Le ré-upload est obligatoire, et le module le dit :

Kernel/Runtime/NKFont/src/NKFont/Core/NkFontSizeCache.h:62-63

```
1 @note Le rebuild de l'atlas invalide toutes les textures GPU.
2 L'application doit être notifiée pour re-uploader la texture.
```

C'est la raison d'être du drapeau dirty de NkGuiFont et des méthodes NeedsGpuUpload() / ClearGpuUploadFlag() du cache de tailles.

7.7 Le mode SDF : du texte net à toute taille

Un atlas classique est rastérisé à une taille précise; l'agrandir le rend flou. Le mode *Signed Distance Field* stocke, pour chaque texel, la distance au bord du glyphe plutôt que sa couverture. Un *shader* reconstitue alors un bord net à n'importe quelle échelle.

Exemple écrit pour le cours (API : `NkFont.h:112-113, 149`)

```
1 NkFontAtlas atlas;
2 atlas.sdfMode = true; // AVANT Build()
3 atlas.sdfSpread = 6; // 4-8 pixels
4 NkFontEmbedded::AddToAtlas(atlas, NkEmbeddedFontId::Inter, 48.f);
5 atlas.Build();
6 // Utiliser le shader fourni : nkentseu::kNkFontFragSDF (NkFont.h:343)
7 // avec uSmoothing ~ 0.1 / fontSize.
```

Ce mode est réellement branché : `NkFontAtlas.cpp:496-535` alloue un tampon temporaire et appelle `nkfont::NkMakeSDFFromBitmap(...)`.

Et le module fournit les deux *shaders* correspondants, en constantes compilées :

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:324-336

```
1 static constexpr const char *kNkFontFragNormal = R"GLSL(
2 #version 460 core
3 in vec2 vUV;
4 in vec4 vColor;
5 out vec4 fragColor;
6 layout(binding=0) uniform sampler2D uAtlas;
7 void main() {
8     float alpha = texture(uAtlas, vUV).a;
9     if (alpha < 0.01) discard;
10    fragColor = vec4(vColor.rgb, vColor.a * alpha);
11 }
12 )GLSL";
```

`kNkFontFragSDF (NkFont.h:343)` est son pendant pour le mode SDF, avec un uniforme `uSmoothing`. Vous n'en aurez pas besoin dans une interface `NKGui` — le backend a déjà les siens — mais ils sont là si vous écrivez votre propre rendu.

7.8 Les limites dures, et l'invariant du `Build()`

Voici la partie que le lecteur pressé saute et regrette.

7.8.1 Des tableaux de taille fixe

Kernel/Runtime/NKFont/src/NKFont/NkFont.h:99-100, 169-170

```
1 static constexpr nkft_uint32 NK_FONT_ATLAS_MAX_FONTS = 16;
2 static constexpr nkft_uint32 NK_FONT_ATLAS_MAX_CUSTOM_RECTS = 64;
3 static constexpr nkft_uint32 NK_FONT_MAX_GLYPHS = 4096u;
4 static constexpr nkft_uint32 NK_FONT_INDEX_SIZE = 256u;
```

`NkFont::glyphs` est un tableau *membre* de 4096 entrées, pas un conteneur dynamique. Conséquences concrètes :

- une face ne peut pas dépasser 4096 glyphes. La plage `GetGlyphRangesChineseFull()` en dépasse largement : elle ne rentrera pas;
- un atlas ne peut pas contenir plus de 16 entrées — et une entrée, c'est *une police à une taille*. Cinq tailles de deux polices, plus trois replis, et vous êtes à treize;
- chaque `NkFont` est un objet volumineux. Ne les copiez pas; manipulez des `NkFont*`, ce que l'API impose de toute façon.

7.8.2 `Build()` n'est ni réentrant ni parallélisable

Kernel/Runtime/NKFont/src/NKFont/NkFontAtlas.cpp:336-337

```
1 static nkfont::NkFontFaceInfo faceInfos[NK_FONT_ATLAS_MAX_FONTS];
2 static nkft_float32 scales[NK_FONT_ATLAS_MAX_FONTS];
```

Ces tableaux sont `static` — partagés par toutes les instances. Et pire, chaque face en garde l'adresse :

Kernel/Runtime/NKFont/src/NKFont/NkFontAtlas.cpp:381-382

```
1 // Stockage du faceInfo pour l'extraction des contours
2 font->m_FaceInfo = &faceInfos[i];
```

Les tableaux de *packing* sont eux aussi statiques (`NkFontAtlas.cpp:381-386`).

Un seul atlas, un seul `Build()`, un seul fil

Trois conséquences, toutes silencieuses :

1. deux `Build()` **en parallèle** sur deux fils écrivent dans les mêmes tableaux : corruption, sans message;
2. un `Build()` sur un atlas B **invalide** les `m_FaceInfo` des faces de l'atlas A. Les fonctions qui s'en servent — `GetGlyphOutlinePoints`, l'extrusion 3D — deviennent fausses. L'atlas A continue pourtant d'afficher du texte normalement : seule l'extraction de contours ment;
3. `Build()` n'est pas réentrant : ne l'appellez pas depuis un *callback* déclenché par lui-même.

La règle à suivre : un seul `NkFontAtlas` par application, sur le fil principal. C'est ce que fait `NKGui`, et c'est ce que vous ferez.

7.8.3 Le `mergeMode` duplique les pointeurs

Kernel/Runtime/NKFont/src/NKFont/NkFontAtlas.cpp:185-189

```
1 Les polices FUSIONNÉES (mergeMode) partagent le MÊME NkFont* que leur police de
2 base (cf. AddFontFromMemoryOwned) : fonts[] contient donc des pointeurs DUPLIQUÉS.
3 Il faut ne détruire chaque pointeur unique QU'UNE fois – sinon double-free / tas
4 corrompu au rechargement (crash zoom).
```

Vous n'aurez pas à gérer cela vous-même — l'atlas s'en charge — mais retenez le symptôme : « crash au zoom » signifie « rechargement d'atlas avec fusion ». Si vous écrivez votre propre `wrap-`

per et que vous parcourez `atlas.fonts[]` pour faire quelque chose sur chaque face, **vous visiterez plusieurs fois le même pointeur**.

7.8.4 Le cache de tailles rend `nullptr` la première fois

Si vous utilisez `NkFontSizeCache` ([Core/NkFontSizeCache.h:85](#)), lisez ce contrat avant :

Kernel/Runtime/NKFont/src/NKFont/Core/NkFontSizeCache.h:114-117

```
1 Si la taille n'est pas dans le cache :
2   1. Ajoute la fonte à l'atlas.
3   2. Marque l'atlas comme nécessitant un rebuild.
4   3. Retourne nullptr jusqu'au prochain appel après BuildAtlas().
```

Un `GetFont(24.f)` qui renvoie `nullptr` n'est donc pas forcément une erreur : c'est peut-être « revenez à la frame suivante ». Le cache plafonne à `MAX_CACHED_SIZES = 16` tailles ([NkFontSizeCache.h:87](#)).

7.9 Ce que le module ne fait pas

Pour être complet, et pour vous éviter de chercher :

Fonctionnalité	État réel
WOFF2 / Brotli	absent (Core/NkFontParser.h:11)
GSUB, ligatures	absent
Bidi (UAX#9)	absent
<i>Hinting</i> TrueType	absent
BDF, Type 1 (PFB/PFA)	absent
GPOS	offset lu, jamais exploité
Kerning par paire	existe au parser, non remonté
Faux-gras	non — charger une fonte grasse
NkFontSdf.h	annoncé « futur », n'existe pas

Deux précisions.

Le **kerning** est un vrai cas curieux : le parser sait lire la table kern format 0 et expose `NkGetGlyphKernAdvance` ([Core/NkFontParser.h:171](#)), mais l'information ne remonte pas jusqu'à `NkFontGlyph`. Le wrapper `NkCanvas` l'écrit franchement : « Le module NKFont n'expose pas le kerning par paire → 0 » ([Kernel/Runtime/NKCanvas/src/NKCanvas/Renderer/Resources/NkFont.h:87](#)). Un « AV » restera donc plus espacé que dans un traitement de texte.

Le **gras** n'est pas synthétisé : « bold ignoré (le module ne fait pas de faux-gras ; charger une fonte bold dédiée si nécessaire) » ([../Resources/NkFont.h:84-85](#)). Si vous voulez du gras, ajoutez une seconde entrée dans l'atlas — en gardant à l'esprit la limite de seize.

7.10 L'autre wrapper : `renderer::NkFont`

Il existe une seconde façade, côté `NkCanvas`, d'inspiration SFML :

```

1      class NkFont {
2      public:
3          bool LoadFromFile(NkIRenderer2D &renderer, const char *path);
4          bool LoadFromMemory(NkIRenderer2D &renderer, const void *data, usize
sizeBytes);
5          const NkGlyph &GetGlyph(uint32 codepoint, uint32 characterSize, bool bold =
false) const;
6          float32 GetKerning(uint32 first, uint32 second, uint32 characterSize) const;
7          float32 GetLineHeight(uint32 characterSize) const;
8          const NkTexture *GetAtlasTexture(uint32 characterSize) const;
9          bool IsValid() const;
10         void Destroy();
11     private:
12         struct Page {                                // une page = une TAILLE de caractère
13             uint32 characterSize = 0;
14             ::nkentseu::NkFontAtlas *atlas = nullptr;
15             ::nkentseu::NkFont *moduleFont = nullptr;
16             NkTexture texture;
17         };
18     };

```

Attention à l'homonymie : `renderer::NkFont` n'est pas `nkentseu::NkFont`. Le premier est un gestionnaire de pages qui délègue au second :

```

1  Depuis le refactoring 2026-05-28, NKCanvas ne réimplémente plus la
2  rasterisation (ex-FreeType supprimé). renderer::NkFont délègue au module
3  NkFont (nkentseu::NkFontAtlas + nkentseu::NkFont) : une "page" (atlas
4  rasterisé + texture GPU) par taille de caractère demandée.

```

Une page par taille demandée : chaque nouvelle `characterSize` construit un nouvel atlas. Pratique, mais rappelez-vous l'invariant du `Build()` statique — c'est un point à surveiller si vous mélangez ce wrapper avec un atlas à vous.

7.11 Récapitulatif des trois chemins

Chemin	Quand	Ce que vous écrivez
<code>NkGuiFont</code>	interface <code>NKGui</code>	3 lignes, rien de plus
<code>renderer::NkFont + NkText</code>	scène <code>NkCanvas</code>	chargement + dessin
Atlas manuel	rendu maison	atlas, conversion, quads

Le premier est celui de ce cours. Le troisième est celui qui *explique*, et c'est pourquoi nous l'avons détaillé : le jour où votre texte est flou, décalé ou invisible, c'est dans ces quatre étapes que le problème se trouve.

7.12 Exercices

1 — Le plus court chemin vers un texte à l'écran

Dans une application NKGui minimale, remplacez le chargement de police par une boucle sur les dix polices embarquées : pour chaque `NkEmbeddedFontId` de 0 à 9, testez `IsAvailable`, récupérez son nom avec `GetName`, et affichez la liste dans un panneau. Puis ajoutez un bouton qui passe à la police suivante. Vous devrez recharger l'atlas et le ré-uploader dans le backend — l'exercice est là. Vérifiez ce qui se passe si vous oubliez le ré-upload.

2 — Voir l'atlas

Récupérez le bitmap alpha8 avec `GetTexDataAsAlpha8`, enveloppez-le dans une `NkImage` — le chapitre 5 vous a donné tout ce qu'il faut — et sauvegardez-le en PNG. Ouvrez le fichier. Vous verrez les glyphes empilés en étagères, avec deux pixels de marge (`texGlyphPadding`). Comptez-les et comparez avec `face->glyphCount`. Puis recommencez avec une plage de glyphes plus large et observez l'atlas grandir. Enfin, mettez `sdfMode = true` et regardez à quoi ressemble un champ de distance.

3 — Écrire son propre rendu de texte

Sans utiliser NKGui, écrivez une fonction qui prend une `NkFont*`, une chaîne UTF-8, une position et une couleur, et remplit deux tableaux de sommets et d'indices — exactement comme `NkGuiDrawList::AddText`. Testez-la sans GPU : affichez pour chaque caractère son *codepoint*, ses quatre coins et ses UV. Vérifiez ensuite que la somme des `advanceX` est bien égale à `face->CalcTextSizeX(texte)`. Puis arrondissez volontairement `advanceX` à l'entier et observez l'écart se creuser avec la longueur de la chaîne : vous venez de reproduire le bug contre lequel le commentaire de `NkGuiDrawList.cpp:257-266` met en garde.

4 — Toucher les plafonds

Écrivez un programme console qui ajoute des polices à un atlas dans une boucle, en incrémentant la taille d'un pixel à chaque tour, et qui s'arrête quand `AddToAtlas` renvoie `nullptr`. Combien d'itérations ? Ensuite, tentez d'ajouter une police avec `GetGlyphRangesChineseFull()` et regardez ce que vaut `face->glyphCount` après `Build()`. Comparez avec `NK_FONT_MAX_GLYPHS`. Que se passe-t-il pour les caractères au-delà ? Cherchez la réponse dans le comportement observé, puis dans `NkFontAtlas.cpp` — et notez si le module vous a prévenu ou non.

NKAudio : faire du bruit

Un bouton qui ne fait aucun bruit quand on clique dessus paraît cassé. C’est irrationnel et c’est ainsi. Ce chapitre vous donne les moyens de corriger cela en une dizaine de lignes, puis explique tout ce qu’il y a derrière — parce que le module va beaucoup plus loin qu’un simple «jouer un fichier».

NKAudio est, des cinq modules d’intégration, celui dont la façade est la plus aboutie. Un seul en-tête suffit :

L’unique inclusion nécessaire

```
1 #include "NKAudio/NkAudio.h"
```

et tout vit dans le *namespace* `nkentseu::audio`.

8.1 Le module, et sa dépendance surprenante

Kernel/Runtime/NKAudio/NKAudio.jenga:30-35

```
1 nkentseudependson(
2     # NKMedia : decodeur Opus from-scratch (codec .opus via NkOpusCodec).
3     ["NKPlatform", "NKCore", "NKMemory", "NKContainers", "NKLogger", "NKThreading",
4     "NKFileSystem", "NKStream", "NKMedia"],
5     selfexport="NKAudio",
6     extra_includes=["src"],
7 )
```

NKAudio dépend de NKMedia, qui dépend lui-même de NKImage. La raison est mince — le décodeur Opus vit dans NKMedia — mais elle est réelle : dans l’ordre de construction, NKImage vient d’abord, puis NKMedia, puis NKAudio. Nous enseignons l’audio avant la vidéo parce que la façade audio est plus simple, mais retenez la direction de la flèche : c’est l’audio qui appelle la vidéo, jamais l’inverse.

Kernel/Runtime/NKAudio/ — arborescence (abrégée)

```
1 NKAudio/
2   src/NKAudio/
3     NkAudio.h                (1427 l.) <- EN-TETE PUBLIC UNIQUE : tout est la
4     NkAudioEngineCore.cpp    <- AudioEngine (PIMPL)
5     NkAudioLoader.cpp        <- AudioLoader (WAV natif + dispatch codecs)
6     NkAudioMixer.cpp · NkAudioGenerator.cpp · NkAudioAnalyzer.cpp
7     NkAudioEffects.{h,cpp}   <- effets DSP concrets
8     NkAudioBackends.{h,cpp}  <- WASAPI / CoreAudio / ALSA / AAudio / WebAudio / Null
9     NkAudioBus.{h,cpp}       <- bus hiérarchiques Master -> SFX/Music/Voice/UI
10    NkAudioCapture.{h,cpp}    <- MICRO (entree)
11    Codecs/ MP3/ · OGG/ · FLAC/ · Opus/
12    Streaming/
13      NkAudioStream.{h,cpp}    <- IAudioStream (pull) + WavStream/MemoryStream
```

```

14 NkAudioStreamPlayer.{h,cpp} <- thread decodeur + ring buffer
15 NkContainerAudioStream.{h,cpp}<- piste audio d'un conteneur video

```

8.2 L'invariant fondateur : un seul format interne

Avant toute chose, comprenez ce que le module fait de vos fichiers :

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:258-260

```

1 @note Format interne normalisé : Float32 interleaved stéréo.
2 La conversion est faite au chargement.

```

Peu importe que votre fichier soit un WAV 8 bits mono à 22 050 Hz ou un FLAC 24 bits à 96 000 Hz : après `AudioLoader::Load`, vous avez des flottants 32 bits entrelacés. Cela simplifie énormément la suite — le mixeur n'a qu'un seul cas à traiter — et cela signifie qu'un fichier chargé occupe souvent plus de mémoire que sur le disque.

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:262-284

```

1 struct AudioSample {
2     float32 *data = nullptr; ///< Samples Float32 (interleaved)
3     usize frameCount = 0;    ///< Nombre de frames (samples/channels)
4     int32 sampleRate = 48000;
5     int32 channels = 2;
6     AudioFormat format = AudioFormat::UNKNOWN;
7     float32 GetDuration() const;
8     usize GetSampleCount() const;
9     bool IsValid() const;
10    memory::NkAllocator *mAllocator = nullptr; ///< Allocateur responsable
11 };

```

Notez le vocabulaire, qu'il faut fixer une fois pour toutes : une *frame* est un instant, un *sample* est une valeur. En stéréo, une *frame* contient deux *samples*. `frameCount` compte les instants, pas les nombres — d'où `GetSampleCount() = frameCount * channels`.

Notez aussi `mAllocator` : chaque `AudioSample` se souvient de qui l'a alloué. Nous verrons pourquoi c'est vital.

8.3 Le patron canonique en cinq temps

L'application `NkAudioPlayer` ouvre son fichier principal sur la recette, et il n'y a rien à y ajouter :

Applications/NkAudioPlayer/src/main.cpp:2-9

```

1 // Le patron CORRECT pour Lire un fichier audio vers Les haut-parleurs, ET L'afficher :
2 // 1. AudioEngine::Instance().Initialize() -> ouvre Le device + thread de mixage
3 // 2. AudioLoader::Load(path) -> décode WAV/MP3/OGG/FLAC/Opus en AudioSample
4 // 3. engine.Play(sample, params) -> Lance une voix (bus "Music")
5 // 4. boucle : GetPlaybackPosition -> tête de Lecture sur La FORME D'ONDE dessinée
6 // 5. engine.Shutdown()

```

8.3.1 Temps 1 — initialiser le moteur

Applications/NkAudioPlayer/src/main.cpp:113-121

```
1 // — 1) Moteur audio —————
2 audio::AudioEngine &engine = audio::AudioEngine::Instance();
3 audio::AudioEngineConfig cfg;
4 if (!engine.Initialize(cfg)) {
5     logger.Error("[NkAudioPlayer] AudioEngine::Initialize a echoue (device indisponible
6     ?)");
7     return -2;
8 }
9 logger.Info("[NkAudioPlayer] device reel : {0} Hz, {1} canaux", engine.GetSampleRate(),
10 engine.GetChannels());
```

AudioEngine est un **singleton** : Instance() rend toujours la même. Initialize ouvre le périphérique de sortie et lance le fil de mixage.

La ligne de journalisation n'est pas décorative. Le périphérique impose son taux d'échantillonnage et son nombre de canaux; votre cfg n'exprime qu'un souhait. Lisez GetSampleRate() et GetChannels() après Initialize(), jamais avant.

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:396-411

```
1 struct AudioEngineConfig {
2     AudioBackendType backend = AudioBackendType::AUTO;
3     int32 sampleRate = AUDIO_DEFAULT_SAMPLE_RATE; // 48000
4     int32 channels = AUDIO_DEFAULT_CHANNELS; // 2
5     int32 bufferSize = AUDIO_DEFAULT_BUFFER_SIZE;
6     int32 maxVoices = AUDIO_MAX_VOICES;
7     bool enableHrtf = false; bool enableDoppler = true;
8     float32 masterVolume = 1.0f;
9     memory::NkAllocator *allocator = nullptr; ///< nullptr = allocateur global
10    bool enableMasterLimiter = true;
11    float32 masterLimiterThresholdDb = -0.5f;
12    ResamplingQuality resamplingQuality = ResamplingQuality::LINEAR;
13};
```

Les valeurs par défaut conviennent. **Ne touchez pas au champ backend** — nous verrons dans un instant pourquoi.

Un seul Initialize, un seul Shutdown

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1071-1073

```
1 @note Initialize() et Shutdown() sont appelés une seule fois
2     depuis le thread principal. Toutes les autres méthodes
3     sont thread-safe via des atomics et queues lock-free.
```

Les contrats sont formels : Initialize exige !IsInitialized(), Shutdown exige IsInitialized() (NkAudio.h:1100-1111). Toutes les autres méthodes, elles, sont appelables de n'importe où.

8.3.2 Temps 2 — décoder le fichier

Applications/NkAudioPlayer/src/main.cpp:115-121

```
1 // — 2) Charge/décoder Le fichier —————
2 audio::AudioSample sample = audio::AudioLoader::Load(path.CStr());
3 if (!sample.IsValid()) {
4     logger.Error("[NkAudioPlayer] impossible de charger/décoder : {0}", path.CStr());
5     engine.Shutdown();
6     return -3;
7 }
```

AudioLoader est entièrement statique :

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:583-648

```
1 static AudioFormat DetectFormat(const uint8* data, usize size);
2 static AudioFormat DetectFormatFromPath(const char* path);
3 static AudioSample Load(const char* path, memory::NkAllocator* allocator = nullptr);
4 static AudioSample LoadFromMemory(const uint8* data, usize size,
5                                   AudioFormat format = AudioFormat::UNKNOWN,
6                                   memory::NkAllocator* allocator = nullptr);
7 static bool SaveWAV(const char* path, const AudioSample& sample);
8 static void Free(AudioSample& sample);
9 static void ConvertSampleFormat(AudioSample& sample, SampleFormat target);
10 static void Resample(AudioSample& sample, int32 targetSampleRate, bool highQuality = true);
11 static void ConvertChannels(AudioSample& sample, int32 targetChannels);
```

Load détecte le format par le contenu, pas par l'extension : renommer un MP3 en .wav ne le trompe pas.

8.3.3 Temps 3 — lancer une voix

Applications/NkAudioPlayer/src/main.cpp:158-162

```
1 // — 4) Joue + prepare La forme d'onde —————
2 audio::VoiceParams vp;
3 vp.bus = "Music";
4 vp.volume = 1.0f;
5 audio::AudioHandle handle = engine.Play(sample, vp);
```

Une voix est une instance de lecture. Jouer trois fois le même AudioSample crée trois voix indépendantes, chacune avec son volume, sa position de lecture et son *handle*.

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:346-357

```
1 struct VoiceParams {
2     float32 volume = 1.0f;    float32 pitch = 1.0f;    float32 pan = 0.0f;
3     bool looping = false;    float32 loopStart = 0.0f; float32 loopEnd = -1.0f;
4     float32 fadeInTime = 0.0f; float32 startOffset = 0.0f;
5     int32 priority = 128;
6     AudioSource3D source3d;
7     const char *bus = "SFX";    ///< Bus de routage (SFX/Music/Voice/UI)
8 };
```

Le `AudioHandle` est un simple entier enveloppé (`NkAudio.h:227-249`), avec un `IsValid()` et une conversion implicite en `bool`. Il sert à piloter la voix après coup :

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1145-1166

```
1 void Stop(AudioHandle handle, float32 fadeOutTime = 0.0f);
2 void Pause(AudioHandle handle); void Resume(AudioHandle handle);
3 bool IsPlaying(AudioHandle) const; bool IsPaused(AudioHandle) const; bool
  IsLooping(AudioHandle) const;
4 void SetVolume/SetPitch/SetPan/SetLooping(AudioHandle, ...);
5 float32 GetVolume/GetPitch/GetPan(AudioHandle) const;
6 float32 GetPlaybackPosition(AudioHandle) const;
7 void SetPlaybackPosition(AudioHandle, float32 seconds);
```

Play peut échouer sans rien dire

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1125

```
1 @return Handle de contrôle, invalide si pool plein
```

Le nombre de voix simultanées est plafonné par `maxVoices`. Passé ce seuil, `Play` renvoie un *handle* invalide et rien ne se joue. Si vous comptez réutiliser le *handle*, testez `h.IsValid()`. Si vous n'en avez pas besoin — un son de clic, par exemple — ignorer le retour est parfaitement légitime.

8.3.4 Temps 4 — piloter pendant la lecture

Applications/NkAudioPlayer/src/main.cpp:185-191

```
1         else if (k->GetKey() == NkKey::NK_SPACE) {
2             paused = !paused;
3             if (paused)
4                 engine.Pause(handle);
5             else
6                 engine.Resume(handle);
7         }
```

et la détection de fin naturelle :

Applications/NkAudioPlayer/src/main.cpp:254-255

```
1         if (!engine.IsPlaying(handle) && !paused)
2             running = false; // fin naturelle
```

Notez la condition composée : une voix en pause n'est pas « en train de jouer ». Sans le `&& !paused`, appuyer sur la barre d'espace fermerait l'application.

8.3.5 Temps 5 — l'ordre de fermeture, qui n'est pas négociable

Applications/NkAudioPlayer/src/main.cpp:258-259

```
1 engine.Shutdown();
2 audio::AudioLoader::Free(sample);
```

Shutdown() d'abord, Free() ensuite. Toujours.

La documentation de Play pose le contrat :

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1123

```
1 @param sample Sample source (doit rester valide pendant la lecture)
```

Play ne copie pas vos échantillons : la voix lit *directement* dans `sample.data`, depuis le fil audio. Libérer le `AudioSample` avant d'avoir arrêté le moteur, c'est laisser un fil temps réel lire de la mémoire rendue au système.

Le symptôme est particulièrement déplaisant : cela ne plante pas toujours. Selon ce qui a été réalloué à cet endroit, vous obtiendrez un grésillement, un bruit blanc, un silence — ou un plantage, mais dix secondes plus tard, ailleurs.

Retenez la séquence : arrêter le moteur, puis libérer les données. Toujours dans cet ordre, dans les cinq chemins de sortie de votre programme.

Et la libération elle-même a sa règle :

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:284, 620

```
1 memory::NkAllocator *mAllocator = nullptr; ///< Allocateur responsable (jamais nullptr si
    IsValid())
2 @note Doit être appelé avec le même allocateur utilisé pour Load()
```

C'est la même règle que pour `NKImage` : jamais `delete`, jamais `free`. Un `IAudioStream` qu'on n'a confié à personne se détruit avec `memory::NkGetDefaultAllocator().Delete(stream)` — le dépôt le note explicitement, avec la mention du plantage `c000374` en cas de mélange ([Applications/NkAudioDemo/src/main.cpp:44](#)).

8.4 Jouer un son au clic d'un bouton

Voici l'objectif annoncé. Tout est en place; il ne reste qu'à assembler.

8.4.1 Au démarrage — une seule fois

Exemple écrit pour le cours (API : `NkAudio.h:1091, 1104, 595`)

```
1 #include "NKAudio/NkAudio.h"
2 using namespace nkentseu;
3
4 audio::AudioEngine &engine = audio::AudioEngine::Instance();
5 if (!engine.Initialize(audio::AudioEngineConfig{})) {
6     logger.Error("[UI] audio indisponible : les sons d'interface seront muets");
7     // ... mais l'application continue : le son n'est pas vital.
8 }
```

```

9  audio::AudioSample clic = audio::AudioLoader::Load("Resources/Audio/clic.wav");
10 if (!clic.IsValid())
11     logger.Warn("[UI] son de clic introuvable");

```

Remarquez la posture : l'échec de l'audio ne doit pas empêcher l'application de démarrer. Une carte son occupée, un périphérique débranché, une machine sans sortie : ce sont des situations normales.

8.4.2 Dans la frame — quand le widget renvoie true

Exemple écrit pour le cours (API : NkGuiWidgets.h:44, NkAudio.h:1127)

```

1  if (nkgui::Button(ctx, "Valider")) {
2      audio::VoiceParams vp;
3      vp.bus = "UI"; // bus dedie : volume UI réglable a part
4      engine.Play(clic, vp); // handle ignore : son court, « fire and forget »
5      // ... l'action du bouton ...
6  }

```

Trois lignes. C'est tout — et c'est possible parce que Play est *non bloquant* : il inscrit la voix dans le mixeur et rend la main immédiatement. Votre frame ne ralentit pas.

8.4.3 À la fermeture

Exemple écrit pour le cours (API : NkAudio.h:1114, 622)

```

1  engine.Shutdown();
2  audio::AudioLoader::Free(clic);

```

Pourquoi le bus « UI »

Ranger les sons d'interface sur leur propre bus n'est pas de la coquetterie : cela permet à l'utilisateur de les couper sans toucher à la musique, par un simple `engine.GetBus("UI")->SetMute(true)`. Si vous les mettez sur le bus « SFX » par défaut, vous ne pourrez plus les distinguer des bruitages du jeu. Le coût de ce choix est nul ; le coût de sa correction plus tard ne l'est pas.

8.5 Les bus : régler des groupes de sons ensemble

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1246-1251

```

1  Le bus Master est cree automatiquement a Initialize() avec ses
2  4 sous-buses standard : SFX (default), Music, Voice, UI.
3
4  Volume effectif d'une voix = voix.volume * bus.volume *
5                          bus.parent.volume * ... * Master.volume.

```

Les quatre bus existent dès `Initialize()` : vous n'avez rien à créer. Écrire `vp.bus = "Music"` suffit.

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1255-1276

```
1 NkAudioBus* GetMasterBus();
2 NkAudioBus* GetBus(const char* name);
3 NkAudioBus* GetOrCreateBus(const char* name, NkAudioBus* parent = nullptr);
```

Kernel/Runtime/NKAudio/src/NKAudio/NkAudioBus.h:53-137 (abrégé)

```
1 static constexpr int32 MAX_EFFECTS = 8;
2 static constexpr int32 MAX_CHILDREN = 16;
3 static constexpr int32 MAX_NAME_LEN = 32;
4
5 NkAudioBus* GetParent() const noexcept;
6 NkAudioBus* GetChild(int32 idx) const noexcept;
7 NkAudioBus* FindDescendant(const char* name) noexcept;
8 void SetVolume(float32) noexcept; float32 GetVolume() const noexcept;
9 float32 GetEffectiveVolume() const noexcept; // produit récursif
10 void SetMute(bool) / IsMuted() / SetSolo(bool) / IsSoloed()
11 bool AddEffect(IAudioEffect*) / RemoveEffect / ClearEffects
12 void SetSidechainFromBus(NkAudioBus* sourceBus, float32 amount = 0.7f,
13 float32 threshold = 0.05f) noexcept;
```

Un cas d'usage typique — baisser la musique sans toucher aux bruitages :

Exemple écrit pour le cours (API : NkAudio.h:1266, NkAudioBus.h:84)

```
1 if (audio::NkAudioBus *bus = engine.GetBus("Music"))
2     bus->SetVolume(0.35f);
```

GetBus renvoie nullptr si le moteur n'est pas initialisé : le if n'est pas facultatif.

Le *sidechain* mérite un mot, parce qu'il résout un problème courant : SetSidechainFromBus(voix, 0.7f) sur le bus Music fait automatiquement baisser la musique quand un dialogue joue. C'est ce que font tous les jeux, et cela tient ici en un appel.

Enfin, le fondu croisé entre deux musiques :

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1297

```
1 AudioHandle PlayMusicCrossfade(const AudioSample& newMusic, float32 fadeTime = 2.0f,
2 const VoiceParams& params = VoiceParams{});
```

Le limiteur master, et pourquoi le son ne sature jamais

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:409-411

```
1 bool enableMasterLimiter = true;
2 float32 masterLimiterThresholdDb = -0.5f;
```

Un limiteur est actif par défaut sur la sortie. C'est pourquoi mettre un volume supérieur à 1,0 ne produit pas de distorsion : le signal est compressé au lieu d'être écrêté. C'est un filet de sécurité utile — mais cela veut aussi dire que si vous poussez tous vos volumes, vous n'obtiendrez pas « plus fort », vous obtiendrez « plus compressé ».

8.6 Les fichiers longs : le streaming

Charger une heure de podcast en Float32 stéréo à 48 kHz représente environ 1,4 Go en mémoire. Pour tout ce qui dépasse quelques dizaines de secondes, il faut décoder au fil de l'eau.

Kernel/Runtime/NKAudio/src/NKAudio/Streaming/NkAudioStream.h:42-68

```
1  class IAudioStream {
2      virtual ~IAudioStream() = default;
3      virtual int32 ReadFrames(float32* outBuf, int32 maxFrames) noexcept = 0;
4      virtual bool Seek(nk_int64 frameIdx) noexcept = 0;
5      virtual nk_int64 GetFrameCount() const noexcept = 0;
6      virtual int32 GetSampleRate() const noexcept = 0;
7      virtual int32 GetChannels() const noexcept = 0;
8      virtual bool IsEOF() const noexcept = 0;
9  };
10 IAudioStream* OpenAudioStream(const char* path) noexcept;
```

C'est une interface *pull* : on demande des *frames*, on en reçoit. Il faut donc quelqu'un pour tirer régulièrement, et à l'avance — c'est le rôle du lecteur de flux, qui lance un fil décodeur et remplit un tampon circulaire :

Kernel/Runtime/NKAudio/src/NKAudio/Streaming/NkAudioStreamPlayer.h:41-135 (abrégé)

```
1  class AudioStreamPlayer {
2      bool Init(int32 sampleRate, int32 channels, int32 ringBufferFrames = 88200)
3      noexcept;
4      void Shutdown() noexcept;
5      bool Play(IAudioStream* stream, bool loop = false) noexcept;
6      void Stop() noexcept; void Pause() noexcept; void Resume() noexcept;
7      int32 ReadFrames(float32* outBuf, int32 maxFrames) noexcept;
8      bool IsPlaying() const noexcept;
9      void SetVolume(float32) / GetVolume()
10     void SetSpeed(float32) / GetSpeed()
11     void SeekContent(float32 seconds) noexcept;
12     void FlushRing() noexcept;
13     bool IsActive() const noexcept; bool IsFinished() const noexcept;
14 };
```

L'ouverture, telle qu'elle est écrite dans la démo :

Applications/NkAudioDemo/src/main.cpp:27-50 (abrégé)

```
1  static int RunStreamingTest(const char *path) {
2      IAudioStream *stream = OpenAudioStream(path);
3      if (!stream) {
4          logger.Error("[NkAudioDemo] OpenAudioStream echec");
5          return 1;
6      }
7      int32 sampleRate = stream->GetSampleRate();
8      int32 channels = stream->GetChannels();
9
10     AudioStreamPlayer player;
11     if (!player.Init(sampleRate, channels, 88200)) {
12         memory::NkGetDefaultAllocator().Delete(stream); // voir note c0000374
13         return 2;
14     }
```

```

15     if (!player.Play(stream, /*Loop=*/false)) {
16         player.Shutdown();
17         return 3;
18     }

```

Après **Play**, le lecteur possède le flux : c'est lui qui le détruira. Avant **Play** — donc dans les chemins d'erreur — c'est à vous de le faire, avec l'allocateur du moteur.

Reste à brancher ce lecteur sur le moteur. C'est le rôle du *callback* procédural.

8.7 Le *callback* procédural, et le fil audio

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1140-1141

```

1 using ProceduralCallback = NkFunction<void(float32* buffer, int32 frames, int32 channels)>;
2 AudioHandle PlayProcedural(ProceduralCallback callback, const VoiceParams& params =
    VoiceParams{});

```

Au lieu de lire un `AudioSample` figé, la voix appelle votre fonction chaque fois qu'elle a besoin d'échantillons. C'est ainsi qu'on branche un flux :

Applications/NkVideoPlayer/src/main.cpp:396-402

```

1         audio::VoiceParams vp;
2         vp.bus = "Music";
3         audioHandle = engine.PlayProcedural(
4             [&streamPlayer](float32 *buf, int32 frames, int32 /*channels*/) {
5                 streamPlayer.ReadFrames(buf, frames);
6             },
7             vp);

```

C'est aussi ainsi qu'on écrit un synthétiseur : rien ne vous oblige à lire un fichier, vous pouvez calculer les échantillons.

Ce *callback* tourne sur le fil audio

Kernel/Runtime/NKAudio/src/NKAudio/NkAudio.h:1134-1139

```

1 Le callback est appelé depuis le thread audio pour remplir
2 les buffers à la demande. Idéal pour synthétiseur logiciel.
3 @note TEMPS RÉEL : callback ne doit pas allouer/locker

```

« Temps réel » veut dire ici : votre fonction dispose de quelques millisecondes, et si elle les dépasse, l'utilisateur entend un craquement.

Sont donc interdits dans ce *callback* : allouer de la mémoire, prendre un verrou, ouvrir un fichier, journaliser, redimensionner un conteneur. Tout ce qui peut faire attendre un fil peut faire craquer le son.

Ce que vous pouvez faire : lire des tableaux préalloués, calculer, écrire dans le tampon fourni, manipuler des atomiques.

8.7.1 L'ordre de destruction avec un lecteur de flux

Ce commentaire du lecteur vidéo mérite d'être lu deux fois :

Applications/NkVideoPlayer/src/main.cpp:672-676

```
1  streamPlayer.Shutdown() joint le thread décodeur et libère le IAudioStream
2  qu'il possède (ContainerAudioStream/MemoryStream/WavStream) ; l'ordre importe :
3  arrêter l'engine (qui appelle le callback procédural depuis son thread audio)
4  AVANT de détruire streamPlayer, pour ne jamais mixer un stream en cours de
5  destruction.
```

D'où la séquence complète, dans le bon ordre :

Exemple écrit pour le cours (invariant : NkVideoPlayer/src/main.cpp:672-676)

```
1  engine.Shutdown();      // 1) arrête le fil audio -> plus aucun appel au callback
2  player.Shutdown();      // 2) joint le fil décodeur et libère le stream
```

La règle générale des trois fermetures

engine.Shutdown() vient **toujours en premier**, parce que c'est lui qui fait tourner le fil qui touche à tout le reste. Ensuite seulement on détruit ce qu'il lisait : les lecteurs de flux, puis les AudioSample.

8.8 Le micro

Kernel/Runtime/NKAudio/src/NKAudio/NkAudioCapture.h:27-75 (abrégé)

```
1  struct NkCaptureDeviceInfo { /* ... */ bool isDefault = false; };
2  struct NkCaptureConfig {
3      int32 sampleRate = 48000; int32 channels = 1; int32 ringSeconds = 4;
4  };
5  using NkAudioInCallback = NkFunction<void(const float32* interleaved, int32 frames, int32
channels)>;
6
7  class NkAudioCapture {
8      static NkVector<NkCaptureDeviceInfo> EnumerateDevices();
9      bool Open(const NkCaptureConfig& config); void Close();
10     bool Start(); void Stop(); bool IsCapturing() const;
11     int32 Read(float32* out, int32 maxFrames);
12     int32 Available() const;
13     void SetCallback(NkAudioInCallback cb);
14     int32 SampleRate() const; int32 Channels() const;
15     static bool SelfTest();
16 };
```

Deux modes coexistent : le *pull* (Read depuis votre boucle) et le *push* (SetCallback). Le premier est plus simple et suffit largement :

Exemple écrit pour le cours (API : NkAudioCapture.h:53-63)

```
1  #include "NKAudio/NkAudioCapture.h"
2
3  audio::NkAudioCapture cap;
4  audio::NkCaptureConfig ccfg;      // 48000 Hz, mono, ring de 4 s
5  if (!cap.Open(ccfg) || !cap.Start())
6      return 1;
7
```

```

8  NkVector<float32> bloc; bloc.Resize(1024);
9  while (enregistre) {
10     const int32 n = cap.Read(bloc.Data(), 1024);    // 0 si rien de dispo
11     /* ... accumuler les n frames ... */
12 }
13 cap.Stop();
14 cap.Close();

```

Read qui renvoie 0 n'est pas une erreur : c'est « rien de neuf ». Le tampon circulaire fait quatre secondes par défaut ; si vous ne lisez pas assez souvent, les plus anciennes données sont perdues.

NkAudioCapture est l'une des deux seules classes du module à posséder un auto-test ([NkAudioCapture.cpp:930](#)) ; l'autre est NkDenoiser ([NkDenoiser.cpp:304](#)). Il n'y a d'auto-test ni pour AudioEngine, ni pour AudioLoader, ni pour les codecs.

L'application NkMicRecord donne au passage le test de codecs le plus court du dépôt :

Applications/NkMicRecord/src/main.cpp:76-89 (abrégé)

```

1  // Mode décodage : NkMicRecord --decode <in.(wav|mp3|ogg|flac|opus)> <out.wav>
2  // Charge via AudioLoader (auto-détection du format, dont Ogg-Opus) et
3  // réécrit en WAV 16-bit – test end-to-end des codecs NkAudio.
4  if (argc >= 4 && NkString(argv[1]) == NkString("--decode")) {
5      audio::AudioSample smp = audio::AudioLoader::Load(argv[2]);
6      if (!smp.IsValid()) {
7          printf("[DECODE] ERREUR : chargement impossible : %s\n", argv[2]);
8          return 1;
9      }
10     const bool ok = audio::AudioLoader::SaveWAV(argv[3], smp);

```

8.9 Tester sans carte son

Voici la fonctionnalité la plus utile pour apprendre — et la moins connue.

Kernel/Runtime/NkAudio/src/NkAudio/NkAudio.h:1321

```

1  void RenderToBuffer(float32* outputBuffer, int32 frameCount, int32 channels = 2);

```

Couplée au backend NULL_OUTPUT, elle permet de faire tourner tout le moteur — voix, bus, effets, limiteur — *sans aucun périphérique*, et de récupérer le mixage dans un tableau que vous pouvez inspecter :

Exemple écrit pour le cours (API : NkAudio.h:122-136, 1321)

```

1  audio::AudioEngineConfig cfg;
2  cfg.backend = audio::AudioBackendType::NULL_OUTPUT;    // aucune sortie physique
3  engine.Initialize(cfg);
4  engine.Play(smp);
5
6  NkVector<float32> mix; mix.Resize(48000 * 2);           // 1 s de stereo
7  engine.RenderToBuffer(mix.Data(), 48000, 2);           // appel SYNCHRONE
8  // mix contient maintenant exactement ce qui serait sorti des haut-parleurs.

```

Ce qu'il faut retenir

NULL_OUTPUT + RenderToBuffer transforme l'audio en calcul pur, donc en quelque chose de **vérifiable**. Un test qui vérifie que le mixage n'est pas silencieux, qu'il ne dépasse pas 1,0, ou qu'un bus muet produit bien du zéro, tourne sans matériel et sans attendre. C'est ainsi qu'il faut aborder les exercices de ce chapitre.

8.10 État réel du module

8.10.1 Ce qui fonctionne

Décodage	WAV natif, MP3, OGG Vorbis, FLAC, Opus — tous écrits dans le dépôt
Sortie	WASAPI, CoreAudio, ALSA, AAudio, WebAudio, Null
Capture	NkAudioCapture complet, avec auto-test
Streaming	IAudioStream, AudioStreamPlayer, piste audio de conteneur vidéo
Mixage	bus hiérarchiques, sidechain, fondu croisé musique, limiteur master
3D	atténuation, occlusion, absorption de l'air, HRTF synthétique
Hors ligne	RenderToBuffer

Le décodage mérite une précision, parce que le dépôt se calomnie lui-même : le commentaire de tête de `NkAudioLoader.cpp:3` parle encore de « stubs MP3/OGG/FLAC ». **C'est périmé**. Les quatre décodeurs sont branchés sur de vrais codecs (`NkAudioLoader.cpp:411-429`). Encore un cas où le commentaire ment et le code dit vrai.

8.10.2 Ce qui ne fonctionne pas

Ne jamais demander DIRECTSOUND

Kernel/Runtime/NKAudio/src/NKAudio/NkAudioBackends.cpp:445-451

```
1      bool DirectSoundAudioBackend::Initialize(int32 sr, int32 ch, int32 buf) {
2          mSampleRate = sr;
3          mChannels = ch;
4          mBufferSize = buf;
5          mImpl = memory::NkGetDefaultAllocator().New<DsImpl>();
6          // DirectSoundCreate8, CreateSoundBuffer, etc.
7          return true;
8      }
```

Lisez bien la dernière ligne : `return true`. Le backend annonce le succès sans avoir ouvert quoi que ce soit. Vous obtiendrez un moteur « initialisé », des voix « en lecture », des positions qui avancent — et un silence absolu, sans le moindre message d'erreur.

Laissez `cfg.backend` sur `AUTO`, qui choisit WASAPI sous Windows, CoreAudio sous macOS, ALSA sous Linux. Trois autres valeurs de l'énumération — `PULSE_AUDIO`, `OPEN_SL_ES`, `CUSTOM` — n'ont pas de classe de backend correspondante dans `NkAudioBackends.h` : elles existent dans l'énumération et nulle part ailleurs.

Le rééchantillonnage « haute qualité » est du linéaire

Kernel/Runtime/NKAudio/src/NKAudio/NkAudioLoader.cpp:588-592

```
1 void AudioLoader::ResampleSinc(AudioSample &sample, int32 targetSampleRate) {  
2     // Pour l'instant, fallback linéaire  
3     // TODO: Implémenter Kaiser-windowed Sinc pour qualité studio  
4     ResampleLinear(sample, targetSampleRate);  
5 }
```

Donc `AudioLoader::Resample(sample, taux, /*highQuality=*/true)` fait exactement la même chose que `false`. C'est le jumeau exact du piège `NK_LANCZOS3` du chapitre 5 : ça ne plante pas, ça ment.

Par symétrie, les valeurs `SINC_4` et `SINC_8` de `ResamplingQuality` dans `AudioEngineConfig` sont à vérifier avant d'être présentées comme différenciantes.

La bonne nouvelle, c'est que vous n'avez normalement pas à rééchantillonner vous-même :

Applications/NkAudioPlayer/src/main.cpp:15-16

```
1 NOTE (agent NKCode) : le mixage NKAudio convertit le TAUX du fichier vers celui du  
2 device (fix mixBuffer/format-device). Un lecteur n'a RIEN à faire côté  
   rééchantillonnage.
```

8.10.3 Le reste de la surface, en survol

Le module contient bien davantage que ce chapitre ne couvre. Pour que vous sachiez au moins que cela existe :

- `IAudioEffect` ([NkAudio.h:438](#)) et les effets concrets de `NkAudioEffects.h` — réverbération, écho, chorus, compresseur, filtres, *pitch shift*... Ils s'attachent à une voix (`AddEffect`), à un bus, ou à la sortie (`AddMasterEffect`). Leur propriété reste à l'appelant ([NkAudio.h:1230](#)) : l'effet doit survivre à ce à quoi il est attaché ;
- l'audio 3D : `SetSourcePosition`, `SetListenerOrientation`, `SetSourceOcclusion`. Notez que c'est l'application qui calcule l'occlusion — « L'application fait le raycast pour calculer cette valeur » ([NkAudio.h:1177-1178](#)). Pour le HRTF, si vous n'avez pas de fichier de données, `GenerateSyntheticHrtf()` ([NkAudio.h:1211](#)) n'en demande aucun ;
- `AudioGenerator` ([NkAudio.h:695](#)) — synthèse de formes d'onde, enveloppes ADSR ;
- `AudioAnalyzer` ([NkAudio.h:947](#)) — FFT, avec un `FftResult` qui porte les magnitudes et une conversion `BinToFrequency`. De quoi dessiner un spectre en temps réel.

8.11 Relier le son à l'image

NKAudio ne dépend d'aucun module graphique : le raccordement est entièrement à votre charge, et c'est très bien ainsi. Trois motifs reviennent :

1. **Le son au clic** — le widget renvoie `true`, vous appelez `Play` avec `vp.bus = "UI"`. C'est ce que nous avons fait ;
2. **La forme d'onde** — `AudioSample::data` et `frameCount` vous donnent tout pour pré-calculer une enveloppe min/max par colonne de pixels, et `GetPlaybackPosition(handle)` vous

donne la tête de lecture. C'est exactement ce que fait `ComputePeaks` dans `Applications/NkAudioPlayer/src/main.cpp:56-87`, avant d'envoyer un quad par colonne au renderer;

3. Le spectre — `AudioAnalyzer` rend un `FftResult`, une barre par *bin*.

Le calcul de la fraction lue, pour dessiner un curseur de lecture :

`Applications/NkAudioPlayer/src/main.cpp:212-215`

```
1 // Position de Lecture -> fraction [0..1].
2 const float32 pos = engine.GetPlaybackPosition(handle);
3 const float32 frac = (durSec > 0.0) ? math::NkClamp((float32)(pos / durSec), 0.0f,
4 1.0f) : 0.0f;
5 const int32 headCol = (int32)(frac * (float32)cols);
```

Le `NkClamp` n'est pas superflu : la position peut légèrement dépasser la durée en fin de lecture, à cause de la granularité du tampon.

8.12 Exercices

1 — Le son au clic, complet

Ajoutez à votre application `NKGui` trois boutons et trois sons différents, tous sur le bus «UI». Puis ajoutez un `Checkbox` «sons d'interface» qui appelle `engine.GetBus("UI")->SetMute(...)`.

Vérifiez ensuite trois choses : que couper le bus UI n'affecte pas un son joué sur «SFX»; que l'application démarre normalement si vous renommez les fichiers audio pour les rendre introuvables; et que la fermeture n'émet aucun bruit parasite — c'est-à-dire que vous appelez bien `Shutdown()` avant `Free()`.

2 — Mesurer sans écouter

Écrivez un programme console qui initialise le moteur en `NULL_OUTPUT`, joue un son, et rend une seconde de mixage dans un tableau avec `RenderToBuffer`. Calculez et affichez le maximum absolu et la moyenne quadratique du tableau.

Recommencez en réglant le bus sur `SetVolume(0.5f)` : le maximum doit être divisé par deux. Puis avec `SetMute(true)` : le tableau doit être rigoureusement nul. Vous venez d'écrire un test audio automatisable, sans matériel.

3 — Prouver l'ordre de fermeture

Écrivez volontairement la mauvaise séquence : `AudioLoader::Free(sample); puis engine.Shutdown();` sur un son de plusieurs secondes en cours de lecture. Lancez plusieurs fois et notez ce que vous entendez et ce qui se passe.

Puis remettez le bon ordre. Cet exercice n'a l'air de rien : il vous vaccine contre une classe entière de bugs qu'on ne diagnostique pas au débogueur, parce que le plantage arrive ailleurs et plus tard.

4 — Un synthétiseur en dix lignes

Utilisez `PlayProcedural` avec un *callback* qui remplit le tampon d'une sinusoïde, en gardant la phase dans une variable capturée par référence. Ajoutez un `SliderFloat` `NKGui` qui règle la fréquence.

Attention : le *slider* s'écrit depuis le fil principal et le *callback* lit depuis le fil audio. Réfléchissez à ce que vous partagez — et relisez l'interdiction d'allouer et de verrouiller. Une variable atomique de type `f1oat` est la bonne réponse ; un `NkVector` redimensionné dans le *callback* est la mauvaise.

NKMedia : lire et écrire de la vidéo

Ce chapitre est le plus court des cinq, et c'est délibéré.

NKMedia est de très loin le plus gros module du moteur. Il contient des décodeurs H.264, HEVC, VP8, VP9, AV1, MPEG-2, Theora, AAC, Opus (avec ses deux moitiés CELT et SILK), AMR — tous écrits à la main, sans ffmpeg. Le seul décodeur AV1 pèse 286 Ko de source. Un chapitre pour débutants qui essaierait d'en faire le tour serait illisible et faux.

Nous nous limiterons donc à **quatre classes de façade**, plus une cinquième en bonus. Tout le reste est de la plomberie interne, et vous n'avez aucune raison d'y toucher.

Le périmètre de ce chapitre

- NkMediaProbe — qu'y a-t-il dans ce fichier ?
- NkVideoWriter — écrire une vidéo, en MJPEG ;
- NkVideoReader — la lire image par image ;
- NkVideoRecorder — enregistrer ce que l'application affiche ;
- NkImageSequenceWriter — écrire une séquence d'images.

Cinq classes, aucune ligne de codec. Si vous maîtrisez ces cinq-là, vous savez faire tout ce dont une application a besoin.

9.1 Le module, et une inversion inhabituelle

Kernel/Runtime/NKMedia/NKMedia.jenga:17-22

```
1 nkentseudependson(
2     ["NKCore", "NKPlatform", "NKMemory", "NKContainers", "NKMath", "NKLogger",
3     "NKStream", "NKFileSystem", "NKImage", "NKThreading", "NKTime"],
4     selfexport="NKMedia",
5     extra_includes=["src"],
6 )
```

NKImage est dans la liste, et pour une raison élégante : **le codec MJPEG n'est rien d'autre que le codec JPEG de NKImage appliqué image par image**. Les séquences d'images passent, elles aussi, par les codecs PNG, BMP, TGA et QOI de NKImage. Le chapitre 5 n'était donc pas un préalable arbitraire.

Il n'y a pas d'en-tête parapluie : on inclut le sous-en-tête précis dont on a besoin. Tout vit dans le *namespace* `nkentseu::media`.

Kernel/Runtime/NKMedia/ — arborescence (abrégée)

```
1 NKMedia/
2   NKMedia.jenga · ROADMAP.md (1529 l. — la source de verite sur l'etat reel)
3   src/NKMedia/
4     NkMediaProbe.{h,cpp}      <- detection conteneur + pistes
```

```

5  NkMediaDemux.{h,cpp}      <- extraction des paquets audio
6  Video/
7  NkVideoTypes.h           <- NkVideoInputFormat
8  NkVideoReader.{h,cpp}    <- LECTURE : conteneur -> RGBA8
9  NkVideoWriter.{h,cpp}    <- ECRITURE simple
10 NkVideoRecorder.{h,cpp}   <- CAPTURE A/V threadée
11 NkVideoConverter.{h,cpp}  <- sequence/AVI -> video
12 NkImageSequenceWriter.{h,cpp}
13 Containers/ NkAviWriter · NkMovWriter · NkMp4H264Writer · NkWebmWriter
14 Audio/Containers/NkWavWriter.{h,cpp}
15 Codecs/ Video/{H264,HEVC,VP8,VP9,AV1,Mpeg1,Mpeg2,Theora} · AAC · Opus · AMR

```

9.1.1 Ici, les commentaires *sous-estiment* le module

Nous avons pris l'habitude, depuis le chapitre 5, de nous méfier des commentaires qui promettent trop. NKMedia présente le problème inverse, et il faut le savoir pour ne pas renoncer à des fonctionnalités qui existent.

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoReader.h:8-11

```

1 // - Conteneur MOV/MP4 (ISOBMFF) : piste vidéo -> MJPEG (mjpa/jpeg). H264 = à venir.
2 // H264/AVC (MP4 courant) exige un décodeur complet (gros chantier séparé) : la
3 // Lecture d'une piste avc1 renvoie une erreur claire tant que le décodeur n'existe pas.

```

C'est faux. Le fichier `Video/NkVideoReader.cpp` affecte réellement `info.codec` à "h264" (lignes 1592, 1828, 2002), "hevc" (955, 1563), "vp8" (1572, 1650), "vp9" (1578, 1635), "av1" (1586, 1644), "mpeg2" (1869), "theora" (1944), en plus de "mjpeg", "rawrgb" et "image". Les décodeurs sont branchés.

De même, `Video/NkVideoWriter.h:5-6` annonce «AVI pour l'instant; MP4/WebM à venir» alors que les écrivains MOV/MP4 et WebM sont livrés.

La source de vérité de ce module

Pour NKMedia, la vérité n'est ni dans les en-têtes ni dans les `.jenga` : elle est dans [Kernel/Runtime/NKMedia/ROADMAP.md](#), un document de 1 529 lignes tenu à jour. Prenez le réflexe de l'ouvrir avant de conclure qu'une fonctionnalité manque.

Voici l'essentiel de cette feuille de route, au moment de la rédaction :

Brique	Statut
Probe / démux conteneurs	fini — MP4, WebM, WAV, OGG, MP3, FLAC
Extraction de paquets	fini — MP4 (stb1 et fMP4), WebM
Opus CELT / SILK	fini (SILK exact au bit face à libopus)
Dispatcher Opus	<i>partiel</i> — mode hybride non fait
Décodeur AAC-LC	fini, exact au bit
Muxers (écriture)	fini — AVI, MOV/MP4, WAV, WebM
Vidéo (décodage)	fini — H.264 Main+High, VP8, VP9, HEVC
Vidéo (encodage)	en cours — RAW/MJPEG/MPEG-1 + H.264 baseline
AV1 / MPEG-2 / Theora	restent à faire

Ce sur quoi ce cours ne s'engage pas

- **L'encodage H.264** est marqué « en cours » et limité au profil *baseline*. Il fonctionne — le NkVideoRecorder l'utilise par défaut — mais ce n'est pas un chemin sur lequel bâtir un exercice de débutant. **Restez en MJPEG**;
- **AV1, MPEG-2 et Theora** : le code de décodage existe, la feuille de route ne les déclare pas terminés;
- **Opus en mode hybride** et en stéréo peut être « refusé proprement » : un fichier .opus courant n'est pas garanti;
- **HEVC** refuse proprement les tuiles, le PCM, les échantillonnages 4 :2 :2 et 4 :4 :4;
- **VP8** refuse la segmentation par macrobloc et les partitions multiples.

« Refusé proprement » est une bonne nouvelle : cela signifie une erreur claire plutôt qu'une image corrompue.

9.2 NkMediaProbe : qu'y a-t-il dans ce fichier ?

C'est la classe par laquelle commencer, parce qu'elle ne décode rien : elle lit les en-têtes et vous dit ce que contient le fichier.

Kernel/Runtime/NkMedia/src/NkMedia/NkMediaProbe.h:21-70 (abrégé)

```
1 enum class NkMediaContainer { NK_UNKNOWN, NK_MP4, NK_WEBM, NK_WAV, NK_OGG, NK_MP3, NK_FLAC };
2 enum class NkMediaTrackType { NK_UNKNOWN, NK_AUDIO, NK_VIDEO };
3
4 struct NkMediaTrack {
5     NkMediaTrackType type = NkMediaTrackType::NK_UNKNOWN;
6     NkString codec;           // "aac", "opus", "vorbis", "vp8", "vp9", "h264", "pcm", "mp3", ...
7     int32 sampleRate = 0;     int32 channels = 0;       // audio
8     int32 width = 0;          int32 height = 0;         // video
9     int32 bitsPerSample = 0;  bool pcmBigEndian = false;
10    NkVector<nk_uint8> codecPrivate;                       // OpusHead pour Opus, etc.
11 };
12 struct NkMediaInfo {
13     NkMediaContainer container = NkMediaContainer::NK_UNKNOWN;
14     NkVector<NkMediaTrack> tracks;
15     const char* ContainerName() const;
16     const NkMediaTrack* FirstAudio() const;
17     const NkMediaTrack* FirstVideo() const;
18 };
19 struct NkMediaProbe {
20     static bool Probe(const uint8* data, usize size, NkMediaInfo& out);
21     static bool ProbeFile(const char* path, NkMediaInfo& out);
22     static NkMediaContainer DetectContainer(const uint8* data, usize size);
23     static bool SelfTest();
24 };
```

L'usage réel, tiré de l'application de test :

Applications/NkMediaTest/src/main.cpp:331-353 (abrégé)

```
1 if (argc >= 2) {
2     media::NkMediaInfo info;
3     if (!media::NkMediaProbe::ProbeFile(argv[1], info)) {
4         printf("[ERREUR] probe echoue : %s\n", argv[1]);
```

```

5         return 1;
6     }
7     printf("Conteneur : %s\n", info.ContainerName());
8     printf("Pistes : %d\n", (int)info.tracks.Size());
9     for (uint64 i = 0; i < info.tracks.Size(); ++i) {
10         const media::NkMediaTrack &t = info.tracks[i];
11         printf(" [%d] %-5s codec=%-6s", (int)i, TrackTypeName(t.type), t.codec.CStr());
12         if (t.type == media::NkMediaTrackType::NK_AUDIO)
13             printf(" %d Hz, %d canal(aux)", t.sampleRate, t.channels);
14         if (t.type == media::NkMediaTrackType::NK_VIDEO)
15             printf(" %dx%d", t.width, t.height);
16         printf("\n");
17     }

```

FirstAudio() et FirstVideo() évitent de parcourir soi-même les pistes quand on ne s'intéresse qu'à la principale — ils rendent nullptr s'il n'y en a pas.

Ce qu'il faut retenir

Avant d'ouvrir un fichier avec NkVideoReader, sondez-le. Vous saurez alors si le codec est l'un de ceux que vous savez traiter, et vous pourrez afficher un message utile au lieu d'un échec muet. ProbeFile est instantané : il ne décode aucune image.

9.3 Écrire une vidéo

9.3.1 L'API

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoWriter.h:26-81

```

1  enum class NkVideoCodec { RAW_BGR, MJPEG, MPEG1 };
2  enum class NkVideoContainer { AVI, MOV, ELEMENTARY };
3  // (NkVideoInputFormat { RGB24, RGBA32, BGR24 } vit dans Video/NkVideoTypes.h:16-20)
4
5  struct NkVideoConfig {
6      int32 width = 0; int32 height = 0;
7      int32 fpsNum = 30; int32 fpsDen = 1;
8      NkVideoCodec codec = NkVideoCodec::MJPEG;
9      NkVideoContainer container = NkVideoContainer::AVI;
10     int32 quality = 90; // MJPEG 1..100
11     int32 audioSampleRate = 0; // 0 = video muette
12     int32 audioChannels = 0;
13 };
14 class NkVideoWriter {
15     bool Open(const char* path, const NkVideoConfig& cfg);
16     bool WriteFrame(const uint8* pixels, NkVideoInputFormat fmt);
17     bool AddAudioSamples(const int16* interleaved, usize frameCount);
18     bool Close();
19     bool IsOpen() const; int32 FrameCount() const;
20 };

```

Notez la cadence exprimée en fraction : fpsNum / fpsDen. Pour du 30 images/s, 30/1; pour du 29,97 (NTSC), 30000/1001. Les formats vidéo n'aiment pas les flottants.

9.3.2 Le squelette de référence

Applications/NKVideoTest/src/main.cpp:83-112

```
1  bool MakeVideo(const char *path, media::NkVideoCodec codec, media::NkVideoContainer
   container, int32 w, int32 h,
2      int32 fps, int32 nframes) {
3      media::NkVideoConfig cfg;
4      cfg.width = w;
5      cfg.height = h;
6      cfg.fpsNum = fps;
7      cfg.fpsDen = 1;
8      cfg.codec = codec;
9      cfg.container = container;
10     cfg.quality = 88;
11
12     media::NkVideoWriter vw;
13     if (!vw.Open(path, cfg)) {
14         ::printf(" [ERREUR] ouverture %s\n", path);
15         return false;
16     }
17     uint8 *rgb = (uint8 *)memory::NkAlloc((usize)w * h * 3);
18     for (int32 fr = 0; fr < nframes; ++fr) {
19         RenderFrame(rgb, w, h, fr, nframes);
20         if (!vw.WriteFrame(rgb, media::NkVideoInputFormat::RGB24)) {
21             ::printf(" [ERREUR] trame %d\n", fr);
22             memory::NkFree(rgb);
23             vw.Close();
24             return false;
25         }
26     }
27     memory::NkFree(rgb);
28     vw.Close();
29     return true;
30 }
```

Quatre choses à retenir de ce bloc.

1. **Le tampon est alloué une fois**, hors de la boucle, et réutilisé. Allouer une image par trame est le meilleur moyen de rendre l'encodage plus lent que le décodage;
2. **la mémoire vient de memory::NkAlloc** et repart par memory::NkFree — la règle du chapitre 5 ne change pas;
3. **le chemin d'erreur appelle quand même Close()**;
4. **le format d'entrée est explicite** : NkVideoInputFormat::RGB24. Le writer accepte aussi RGBA32 et BGR24 et fait la conversion.

Close() n'est pas optionnel

Un fichier AVI se termine par un index; un MP4 par un atome moov. Aucun des deux n'est écrit tant que Close() n'a pas été appelé. Sans lui, vous obtenez un fichier de la bonne taille, contenant toutes vos images, et qu'**aucun lecteur au monde n'ouvrira**. Le message typique est «moov atom not found».

Le dépôt prend explicitement cette précaution dans son code de capture :

Applications/NKViewportDemo/src/NKViewportDemo/main.cpp:358-362

```

1 // Robustesse : si la fenêtre est fermée AVANT la fin de l'enregistrement, on
  finalise quand même
2 // (écrit le moov) → le MP4 reste lisible (sinon "moov atom not found").
3 if (rec.IsRecording()) {
4     rec.End();
5 }

```

La même règle vaut pour `NkVideoRecorder::End()` et `NkWebmWriter::Finalize()`. **Prévoyez une finalisation sur tous les chemins de sortie, y compris la fermeture brutale de la fenêtre.**

9.3.3 Deux limites de l'écriture

- **L'audio n'est pas supportée partout** : `Video/NkVideoWriter.h:50-52` précise que la piste audio est « supportée par AVI ... et MOV/MP4 ... Non supportée par ELEMENTARY/MPEG1 (Open échoue si demandée) ». Au moins l'échec est franc ;
- **le sens des pixels** : les encodeurs veulent du *haut en bas* (`Video/NkVideoTypes.h:16-20`). Or OpenGL rend de bas en haut. Nous verrons le paramètre qui corrige cela.

9.3.4 La séquence d'images, à la manière de Blender

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkImageSequenceWriter.h:21-42

```

1 enum class NkImageSeqFormat { PNG, JPEG, BMP, TGA, QOI };
2 struct NkImageSequenceWriter {
3     bool Open(const char* dir, const char* basename, int32 width, int32 height,
4               NkImageSeqFormat fmt, int32 padding = 4, int32 quality = 90);
5     bool WriteFrame(const uint8* pixels, NkVideoInputFormat fmt);
6     bool Close(); int32 FrameCount() const;
7 };

```

Applications/NKVideoTest/src/main.cpp:185-201

```

1 media::NkImageSequenceWriter seq;
2 bool ok = seq.Open(outDir, "nkframe", W, H, media::NkImageSeqFormat::PNG, 4, 90);
3 if (ok) {
4     uint8 *rgb = (uint8 *)memory::NkAlloc((usize)W * H * 3);
5     for (int32 fr = 0; fr < 5 && ok; ++fr) { // 5 images d'exemple
6         RenderFrame(rgb, W, H, fr, N);
7         ok = seq.WriteFrame(rgb, media::NkVideoInputFormat::RGB24);
8     }
9     memory::NkFree(rgb);
10    seq.Close();
11 }

```

Cela produit `nkframe_0000.png`, `nkframe_0001.png`... Cette classe délègue entièrement aux codecs de `NkImage` : c'est le chemin le plus fiable du module, et le plus facile à déboguer — vous pouvez ouvrir chaque image.

Et pour transformer ensuite la séquence en vidéo, il existe un raccourci en une ligne :

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoConverter.h:24-30

```
1 static int32 ImageSequenceToVideo(const char* prefix, int32 digits, const char* suffix,
2                                   int32 first, int32 count, const char* outPath,
3                                   int32 fpsNum, int32 fpsDen, int32 quality = 90);
4 static int32 MjpegAviToVideo(const char* aviPath, const char* outPath, int32 quality = 90);
```

Ces deux fonctions renvoient le *nombre de trames* traitées, pas un booléen : zéro signifie l'échec.

9.4 Lire une vidéo

9.4.1 L'API

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoReader.h:26-70

```
1 struct NkVideoFrame {
2     NkVector<nk_uint8> rgba;           // width*height*4, row-major, haut->bas
3     int32 width = 0;   int32 height = 0;
4     int64 timestampMs = 0;
5     int32 index = -1;
6 };
7 struct NkVideoReaderInfo {
8     int32 width = 0;   int32 height = 0;
9     int32 frameCount = -1;           // -1 = inconnu
10    double fps = 0.0;
11    NkString codec;                  // "mjpeg"|"rawrgb"|"h264"|"hevc"|"vp8"|"vp9"|...
12    NkString container;              // "avi"|"mov"|"mp4"|"sequence"
13 };
14 class NkVideoReader {
15     NkVideoReader(const NkVideoReader&) = delete;           // NON COPIABLE
16     bool Open(const char* path);
17     bool IsOpen() const;
18     const NkVideoReaderInfo& Info() const;
19     bool ReadFrame(NkVideoFrame& out);                       // false = fini
20     bool SeekFrame(int32 index);
21     int32 CurrentIndex() const;
22     void Close();
23     static bool SelfTest();
24 };
```

Le point capital est dans le commentaire de `NkVideoFrame::rgba` : **RGBA8, ligne par ligne, de haut en bas**. C'est exactement le format qu'attendent `NkTexture::Update` et `UploadImageRGBA`. Il n'y a aucune conversion à écrire.

9.4.2 La boucle de lecture

Applications/NkVideoReadTest/src/main.cpp:3041-3070 (abrégé)

```
1 const char *path = argv[1];
2 NkVideoReader rd;
3 if (!rd.Open(path)) {
4     printf(" [K0] impossible d'ouvrir/lire : %s\n", path);
5     return 1;
6 }
7 const NkVideoReaderInfo &in = rd.Info();
8 printf(" conteneur=%s codec=%s %dx%d %.2f fps frames=%d\n", in.container.CStr(),
```



```

    in.codec.CStr(), in.width,
9      in.height, in.fps, in.frameCount);
10
11     int32 count = 0;
12     NkVideoFrame fr;
13     while (rd.ReadFrame(fr)) {
14         /* ... traiter fr.rgba ... */
15         ++count;
16     }

```

Remarquez que `fr` est déclarée *hors* de la boucle et réutilisée : son `NkVector` conserve sa capacité d'une trame à l'autre. La déclarer dans la boucle provoquerait une allocation par image.

`Open` accepte aussi un **dossier d'images** : le champ `container` vaut alors "sequence" et le codec "image". C'est le moyen le plus simple de tester votre code de lecture sans avoir de vidéo sous la main.

9.4.3 Le seek n'est pas ce que vous croyez

Applications/NkVideoReadTest/src/main.cpp:2762-2765

```

1  SeekFrame se contente de repositionner sur l'IDR précédente (voir implémentation) : il
2  faut ensuite redécoder EN AVANT jusqu'à la cible, exactement comme le fait la boucle de
3  rattrapage du lecteur (Applications/NkVideoPlayer).

```

Dans un format à trames inter-dépendantes comme H.264, l'image numéro 1 000 ne peut pas être décodée seule : elle décrit des différences par rapport aux précédentes. `SeekFrame(1000)` vous replace donc sur la dernière image autonome avant la cible, et c'est à vous d'appeler `ReadFrame` jusqu'à ce que `CurrentIndex() >= 1000`.

Sur une séquence d'images ou un AVI MJPEG — où chaque image est indépendante — le seek est exact.

9.4.4 Décoder coûte cher

Une phrase à graver : **ne décidez jamais plus d'une image par image affichée**. Le lecteur du dépôt borne explicitement sa boucle de rattrapage :

Applications/NkVideoPlayer/src/main.cpp:562-620 (extrait)

```

1      if (!paused && !ended) {
2          if (audioOn && !streamPlayer.IsFinished())
3              mediaClock = streamPlayer.GetPositionSeconds();
4          else
5              mediaClock += (float64)(dt * speed);
6          const int32 targetIdx = (int32)(mediaClock * (float64)fps);
7          /* resynchronisation dure si on a plus de 1,5 s de retard */
8          const int32 kHardResyncLagFrames = (int32)(fps * 1.5f);
9          if (!firstFrame && (targetIdx - reader.CurrentIndex()) > kHardResyncLagFrames) {
10             if (reader.SeekFrame(targetIdx) && reader.ReadFrame(fr))
11                 havePendingFrame = true;
12         }
13         NkChrono burstTimer;
14         while ((firstFrame || reader.CurrentIndex() < targetIdx) &&
15             burstTimer.Elapsed().ToMilliseconds() < 150.0) {
16             if (reader.ReadFrame(fr)) { havePendingFrame = true; firstFrame = false; }
17             else if (loop) { /* SeekFrame(0) + ReadFrame */ }
18             else { paused = true; break; }

```

```

19         }
20         if (havePendingFrame)
21             pushFrame(); // upload + dessin UNE SEULE FOIS, avec la dernière image de la
    rafale
22     }

```

Trois garde-fous dans ce seul bloc : la rafale de rattrapage est limitée à 150 ms de temps mur; au-delà d'une seconde et demie de retard on saute carrément; et surtout, **l'upload GPU n'a lieu qu'une fois**, avec la dernière image de la rafale. Décoder dix images pour n'en afficher qu'une est acceptable; en uploader dix ne l'est pas.

L'horloge maîtresse

Notez la première ligne : quand l'audio joue, c'est *lui* qui donne l'heure (`mediaClock = streamPlayer.GetPositionSeconds()`). Ce n'est qu'en son absence qu'on accumule le *delta time*. La raison est physiologique : l'oreille détecte un hoquet audio de 20 ms, l'œil ne voit pas une image manquante. On synchronise donc toujours l'image sur le son, jamais l'inverse.

9.5 Afficher la vidéo

9.5.1 Dans une fenêtre NkCanvas

Le principe : créer **une** texture à la taille de la vidéo, puis la mettre à jour à chaque nouvelle image.

Applications/NkVideoPlayer/src/main.cpp:355-362

```

1 // — 4) Texture RGBA de la taille vidéo + sprite d'affichage —————
2 NkTexture frameTex;
3 if (!frameTex.Create(*target.GetRenderer(), (uint32)vidW, (uint32)vidH, NkColor2D{0, 0,
    0, 255})) {
4     logger.Error("[NkVideoPlayer] echec creation texture {0}x{1}", vidW, vidH);
5     window.Close();
6     return -5;
7 }
8 NkSprite sprite(frameTex);

```

puis, à chaque image décodée :

Applications/NkVideoPlayer/src/main.cpp:424-431

```

1 auto pushFrame = [&]() {
2     rawW = fr.width;
3     rawH = fr.height;
4     rawFrame = fr.rgba;
5     applyLook(fr.rgba.Data(), (uint64)fr.width * fr.height);
6     frameTex.Update(fr.rgba.Data(), (uint32)fr.width, (uint32)fr.height, 0, 0);
7     haveFrame = true;
8 };

```

et le dessin, qui n'a rien de particulier :

Applications/NkVideoPlayer/src/main.cpp:664-668

```
1 target.Clear(NkColor2D{0, 0, 0, 255});
2 if (haveFrame)
3     target.Draw(static_cast<const NkDrawable &>(sprite));
4 target.Display();
```

Ne recréez jamais la texture par image

Create alloue une texture GPU ; Update y recopie des pixels. Appeler Create à chaque frame, c'est allouer et libérer une texture soixante fois par seconde : la mémoire GPU se fragmente et le pilote finit par protester. Créez une fois, mettez à jour ensuite.

9.5.2 Dans une interface

Le motif est le même, avec l'API du backend d'interface. L'IDE du dépôt en donne la version complète, y compris le cas du changement de dimensions :

Applications/NKCode/src/NKCode/Shell/NkVideoViewer.h:70-82

```
1 inline void NkVideoUpload(editorkit::NkEditorShell *shell, NkVideoClip *c) {
2     if (!shell || c->frame.rgba.Empty() || c->frame.width <= 0 || c->frame.height <=
3         0)
4         return;
5     const uint8 *px = c->frame.rgba.Data();
6     if (c->texId == 0 || c->texW != c->frame.width || c->texH != c->frame.height) {
7         c->texId = shell->UploadRGBA(px, c->frame.width, c->frame.height);
8         c->texW = c->frame.width;
9         c->texH = c->frame.height;
10    } else {
11        shell->UpdateRGBA(c->texId, px, c->frame.width, c->frame.height);
12    }
13    c->haveFrame = c->texId != 0;
14 }
```

Premier appel ou changement de taille : UploadRGBA, qui crée. Sinon : UpdateRGBA, qui recopie. C'est exactement la même logique que pour NKImage au chapitre 5, avec une texture qui vit plus longtemps qu'une frame.

Et le cadencement, côté interface :

Applications/NKCode/src/NKCode/Shell/NkVideoViewer.h:153-170

```
1 float32 dt = ctx.input.dt;
2 if (dt <= 0.f || dt > 0.25f)
3     dt = 1.f / 60.f; // garde-fou (onglet revenu au premier plan / hoquet)
4 const float32 frameDur = 1.f / (c->fps * (c->speed > 0.01f ? c->speed : 0.01f));
5 if (c->playing && !c->ended) {
6     c->acc += dt;
7     if (c->acc > frameDur * 4.f)
8         c->acc = 0.f; // evite le rattrapage explosif
9     while (c->acc >= frameDur) {
10        c->acc -= frameDur;
11        if (c->reader->ReadFrame(c->frame)) {
12            c->index = c->frame.index;
13            NkVideoUpload(shell, c);
14        }
15    }
16    /* ... Loop / fin ... */
```

```

16         }
17     }

```

Les deux garde-fous méritent d'être notés, parce qu'ils correspondent à deux situations réelles : un *delta time* aberrant quand l'onglet revient au premier plan après plusieurs secondes, et un accumulateur qui a débordé et voudrait décoder quarante images d'un coup.

9.6 Enregistrer ce que l'application affiche

9.6.1 L'API

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoRecorder.h:39-77

```

1  enum class NkRecorderCodec { H264, MJPEG };
2
3  struct NkVideoRecorder {
4      bool Begin(const char* path, int32 width, int32 height, int32 fpsNum = 60, int32
        fpsDen = 1,
5          int32 qp = 20, int32 maxQueuedFrames = 32,
6          NkRecorderCodec codec = NkRecorderCodec::H264, int32 mjpegQuality = 90);
7      int32 AddAudio(int32 sampleRate, int32 channels, const char* lang3 = nullptr);
8      int32 AddSubtitleTrack(const char* lang3 = nullptr);
9      bool PushVideo(const uint8* pixels, NkVideoInputFormat fmt, bool flipVertical =
        false);
10     void PushAudio(int32 trackIdx, const int16* interleaved, uint32 frames);
11     void AddSubtitle(int32 trackIdx, const char* utf8, uint32 startMs, uint32 durMs);
12     bool End();
13     bool IsRecording() const; int32 FrameCount() const;
14     int32 QueueDepth(); uint64 DroppedFrames(); double EncodeFps();
15 };

```

La différence avec NkVideoWriter est architecturale :

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoRecorder.h:5-8

```

1  **Encodage sur un THREAD de fond** : la boucle de rendu pousse les trames capturées
2  dans une file (rapide) et un worker encode en arrière-plan → l'application reste
3  FLUIDE pendant la capture (l'encodeur H.264 est lourd).

```

PushVideo rend la main immédiatement; l'encodage a lieu ailleurs. End() joint ce fil : ne détruisez jamais un recorder sans avoir appelé End().

9.6.2 La capture, de bout en bout

Le point de jonction avec le reste du moteur est délicieusement simple : la capture d'écran est une NkImage.

Applications/NKViewportDemo/src/NKViewportDemo/main.cpp:329-362 (abrégé)

```

1      if (record && target->CaptureToImage(capImg) && capImg.IsValid()) {
2          if (!rec.IsRecording()) {
3              if (rec.Begin("viewport_capture.mp4", capImg.Width(), capImg.Height(), 30, 1,
4                  24)) {
5                  recAudio = rec.AddAudio(kRate, 1, "fre");
6                  const int32 st = rec.AddSubtitleTrack("fre");

```

```

6         rec.AddSubtitle(st, "Capture live NKCanvas (DX11) - Nkentseu", 0, 4000);
7     }
8 }
9     if (rec.IsRecording()) {
10         rec.PushVideo(capImg.Pixels(), media::NkVideoInputFormat::RGBA32);
11         rec.PushAudio(recAudio, recTone.Data(), (uint32)kAudioPerFrame);
12         if (recFrames >= kRecTotal) { rec.End(); running = false; }
13     }
14 }

```

`NkRenderWindow::CaptureToImage(NkImage&)` remplit une `NkImage`, et `capImg.Pixels()` part directement dans `PushVideo`. `NKImage`, `NKCanvas` et `NKMedia` se rejoignent en une ligne.

9.6.3 Trois pièges du recorder

L'ordre des appels

`Video/NkVideoRecorder.h:51-54` : `AddAudio` et `AddSubtitleTrack` sont à appeler « juste après `Begin` (avant les trames) ». Ajouter une piste en cours d'enregistrement n'a pas de sens dans un conteneur MP4, dont la table des pistes est écrite en tête.

En MJPEG, l'audio est ignorée — silencieusement

`Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoRecorder.h:36-38`

```
1   conteneur MOV/MP4, **VIDEO SEULE (audio/sous-titres ignorees dans ce mode)**
```

C'est le piège le plus vicieux du module : `AddAudio` vous rend un index parfaitement valide, `PushAudio` accepte vos échantillons sans broncher, et le fichier produit est muet. Rien ne vous prévient.

Vous êtes donc devant un choix : MJPEG (léger, sûr, muet) ou H.264 (avec audio, mais l'encodeur est marqué « en cours »). Pour apprendre, prenez MJPEG et enregistrez l'audio à part.

Le recorder abandonne des trames

`Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoRecorder.h:45-46`

```
1   `maxQueuedFrames` BORNE la file video : si l'encodeur (H.264 lourd) prend du retard,
2   les nouvelles trames sont ABANDONNEES (drop-newest) au lieu de gonfler la memoire
3   (temps reel : perdre une trame vaut mieux que geler). Abandons comptes
   (DroppedFrames()).
```

C'est le bon comportement — mais il faut le savoir, sinon on s'étonne qu'une capture de dix secondes à 60 images/s en contienne 400. Surveillez `DroppedFrames()` et `QueueDepth()`, et réglez si nécessaire :

`Applications/Sandbox/src/Demo/main.cpp:1025-1050 (extrait)`

```
1         const bool encoderBusy = recorder && recorder->QueueDepth() >= 24;
2         if (!encoderBusy)
3             (void)recordCapture.EnqueueCopy(...);
```

9.6.4 Le sens des pixels

Kernel/Runtime/NKMedia/src/NKMedia/Video/NkVideoRecorder.h:56-57

```
1  `flipVertical` pour framebuffer bottom-up (OpenGL)
```

OpenGL considère l'origine en bas à gauche; les formats vidéo la placent en haut à gauche. Si votre capture provient d'un *framebuffer* OpenGL lu directement, passez `flipVertical = true`. Le symptôme d'un oubli est sans ambiguïté : la vidéo est à l'envers.

9.7 Le démux audio, en deux mots

Nous n'en aurons pas besoin dans ce cours, mais vous croiserez la classe :

Kernel/Runtime/NKMedia/src/NKMedia/NkMediaDemux.h:22-48

```
1  struct NkMediaPacket {
2      usize offset = 0;    // position dans le buffer SOURCE (ne copie pas !)
3      usize size = 0;
4      int64 timestampMs = 0;
5      int64 granule = -1;    // OGG
6      int64 discardPaddingNs = 0;    // WebM
7  };
8  struct NkMediaDemux {
9      static bool ExtractAudioPackets(const uint8* data, usize size, const NkMediaInfo&
10         info,
11                                     NkVector<NkMediaPacket>& out);
12      static bool ExtractAudioPacketsFile(const char* path, NkVector<nk_uint8>& outBytes,
13         NkMediaInfo& outInfo, NkVector<NkMediaPacket>&
14         out);
15      static bool SelfTest();
16  };
```

Les paquets pointent dans le tampon source

`NkMediaPacket` ne contient pas de données : il contient un *décalage*. Le commentaire est explicite — « Un paquet encodé (pointe dans le buffer d'origine; ne copie pas) » (`NkMediaDemux.h:22`), et pour la variante fichier : « `outBytes` reçoit le buffer (à garder vivant car les paquets pointent dedans) » (`NkMediaDemux.h:47`).

Si votre `NkVector<nk_uint8>` bytes sort de portée avant que vous ayez fini de lire les paquets, vous lisez de la mémoire libérée. Gardez les deux ensemble, dans la même portée.

9.8 Vérifier que tout marche sur votre machine

NKMedia est le module le mieux doté du moteur en auto-tests — il en compte 52. L'application `NkVideoReadTest`, lancée *sans argument*, les enchaîne :

Applications/NkVideoReadTest/src/main.cpp:252-276

```
1  if (argc < 2) {
2      printf(" [self-test] ecrire AVI MJPEG -> relire -> verifier...\n");
3      bool ok = NkVideoReader::SelfTest();
4      printf(" [ %s ] NkVideoReader::SelfTest (AVI MJPEG round-trip)\n", ok ? "OK " :
5      "KO");
6      bool okH264 = NkH264Decoder::SelfTest();
```

```

6      bool okCavlc = NkH264Cavlc::SelfTest();
7      bool okVp9 = NkVp9Decoder::SelfTest();
8      bool okAv1 = NkAv1Decoder::SelfTest();
9      bool okHevc = NkHevcDecoder::SelfTest();
10     bool okHevcCabac = NkHevcCabacState::SelfTest();
11     bool okWav = NkWavWriter::SelfTest();
12     bool okWebm = NkWebmWriter::SelfTest();
13     bool all = ok && okH264 && okCavlc && okVp9 && okAv1 && okHevc && okHevcCabac &&
        okWav && okWebm;
14     printf("=== %s ===\n", all ? "LECTURE VIDEO OPERATIONNELLE" : "ECHEC");
15     return all ? 0 : 1;
16 }

```

Le premier de la liste est le plus parlant : il écrit un AVI MJPEG, le relit, et vérifie que ce qui sort correspond à ce qui est entré. C'est ce *round-trip* qui valide indirectement NkVideoWriter, qui n'a pas d'auto-test propre — pas plus que NkVideoRecorder, NkVideoConverter, NkImageSequenceWriter, ni les écrivains de conteneurs.

Deux détails pratiques :

- ces auto-tests écrivent dans le répertoire courant : `nkvideoreader_selftest.avi`, `nkwavwriter_selftest.wav`, `nkwebmwriter_selftest.webm`. Lancez-les depuis un dossier de travail, pas depuis la racine du dépôt;
- le décodeur HEVC lit une variable d'environnement, `NK_HEVC_THREADS` : absente ou à 0, il choisit tout seul; à 1, il travaille séquentiellement (`NkVideoReadTest/src/main.cpp:249-250`). Utile pour départager un bug de décodage d'un bug de parallélisme.

9.9 Une leçon de performance qui dépasse la vidéo

La feuille de route documente un bug de performance instructif :

Kernel/Runtime/NKMedia/ROADMAP.md:1447-1458 (abrégé)

```

1  le vrai coupable etait `NkVideoReader::ReadFrame` [...] : la gestion du DPB (liste de
2  references) COPIAIT EN PROFONDEUR (au lieu de déplacer) jusqu'a 16 `NkH264Frame`
3  (plans Y/Cb/Cr + grilles de mouvement, ~380 Ko/image) DEUX FOIS par frame decodee
4  [...] cout mesure ~80 ms/frame et CROISSANT [...] >90% du temps total. Fix : traits::NkMove

```

Quatre-vingts millisecondes par image, dont plus de 90 % en copies inutiles. La leçon est transférable bien au-delà de la vidéo : **dans une boucle chaude, une copie profonde qui aurait dû être un déplacement peut représenter l'essentiel du temps d'exécution**. Le champ `NkVideoFrame::rgba` est un conteneur : le copier par valeur à chaque image coûte cher, et le lecteur du dépôt ne le fait que parce qu'il a explicitement besoin d'une copie non retouchée (`Applications/NkVideoPlayer/src/main.cpp:427`).

Le même document ajoute un avertissement de méthode, qui vaut pour toute mesure :

Kernel/Runtime/NKMedia/ROADMAP.md:1449-1452

```

1  (mesure fiable : chrono mur autour d'un run borne, PAS le champ `t=` de `NkVideoReadTest`
2  qui affiche le PTS video, pas le temps de decodage reel – piège rencontre une fois)

```

Le *PTS* est l'instant auquel une image doit être affichée dans la vidéo. Il n'a rien à voir avec le temps qu'il a fallu pour la décoder. Confondre les deux, c'est mesurer la durée du film au lieu de la vitesse du lecteur.

9.10 Exercices

1 — Sonder avant d'ouvrir

Écrivez un petit outil en ligne de commande qui prend un chemin, appelle `NkMediaProbe::ProbeFile`, et affiche le conteneur et toutes les pistes. Passez-lui successivement un WAV, un MP3, un MP4 et un fichier texte renommé en `.mp4`. Ajoutez ensuite une règle : si la première piste vidéo n'a pas un codec parmi `"mjpeg"`, `"rawrgb"` ou `"image"`, affichez un avertissement disant que la lecture n'est pas garantie par ce cours. Vous venez d'écrire le garde-fou qui manque à la plupart des applications.

2 — Le *round-trip* complet

Générez une vidéo de 100 images en MJPEG/AVI, avec un motif dont vous pouvez prédire le contenu — par exemple un rectangle qui se déplace d'un pixel par image. Relisez-la ensuite avec `NkVideoReader` et vérifiez que le nombre d'images correspond, et que le rectangle est bien là où vous l'attendez à l'image 50. Recommencez avec le conteneur MOV. Puis essayez `RAW_BGR` et comparez la taille des fichiers : vous aurez une idée concrète de ce que compresser veut dire.

3 — Prouver que `Close()` est obligatoire

Reprenez l'exercice précédent et commentez l'appel à `vw.Close()`. Regardez la taille du fichier produit — elle est presque identique — puis essayez de l'ouvrir avec votre lecteur vidéo habituel, et avec `NkVideoReader`. Cet exercice prend deux minutes et vous fera gagner une soirée le jour où votre application se fermera brutalement en cours d'enregistrement.

4 — Un lecteur cadencé, dans votre interface

Ajoutez à votre application `NKGui` un panneau qui lit une séquence d'images depuis un dossier (c'est le chemin le plus simple : `Open` accepte un dossier) et l'affiche avec le widget `Image` du chapitre 5. Cadencez sur `Info().fps` avec un accumulateur, en reprenant les deux garde-fous de `NkVideoViewer` : *delta time* aberrant et accumulateur débordé. Ajoutez un bouton pause et un `SliderFloat` de vitesse. Vérifiez que vous n'appellez `UploadRGBA` qu'une seule fois — au premier affichage — et `UpdateRGBA` ensuite.

NKNetwork : faire parler deux machines

Le réseau est le dernier des cinq modules, et le plus abstrait : rien ne s'affiche, rien ne s'entend, et quand ça ne marche pas, il n'y a souvent aucun message. C'est aussi celui où le périmètre honnête est le plus important à poser, parce que le module contient beaucoup de code que *rien* dans le dépôt n'utilise.

10.1 Où il vit, et ce dont il a besoin

Première singularité : contrairement aux quatre autres, NKNetwork n'est pas dans `Kernel/Runtime` mais dans `Kernel/System`. C'est un service système, au même titre que le système de fichiers ou les fils d'exécution.

Kernel/System/NKNetwork/ — arborescence

```

1 NKNetwork/
2   NKNetwork.jenga · ROADMAP.md
3   tests/.jenga/          <- DOSSIER VIDE (voir plus bas)
4   src/NKNetwork/
5     NKNetwork.h           <- en-tete PARAPLUIE + alias de commodite
6     Core/NkNetDefines.h/.cpp <- types, enums, constantes
7     Transport/
8       NkSocket.h/.cpp      <- socket UDP/TCP brut
9       NkReliableUDP.h/.cpp <- ACK, retransmit, fenetre (usage INTERNE)
10    Protocol/
11      NkBitStream.h/.cpp    <- NkBitWriter / NkBitReader
12      NkConnection.h/.cpp   <- NkConnection + NkConnectionManager
13      Replication/NkNetWorld.h/.cpp
14      RPC/NkRPC.h/.cpp
15      Lobby/NkLobby.h/.cpp
16      HTTP/NkHTTPClient.h/.cpp

```

Tout vit dans le `namespace nkentseu::net`.

Kernel/System/NKNetwork/NKNetwork.jenga:27-42 (abrégé)

```

1 _NET_DEPS = ["NKCore", "NKPlatform", "NKMemory", "NKContainers", "NKLogger", "NKThreading",
2             "NKMath", "NKTime", "NKFileSystem"]
3 _NET_INCLUDES = ["src", "pch"]
4 _NET_DEFINES = []
5 if _WANT_MBEDTLS:
6     _NET_DEPS.append("NKMbedTLS")
7     _NET_INCLUDES.append("%{wks.location}/Externals/Libs/NKMbedTLS/include")
8     _NET_DEFINES.append("NKENTSEU_HTTP_USE_MBEDTLS")

```

Il faut lier ws2_32 sous Windows

Le module utilise les sockets Berkeley, qui s'appellent Winsock sous Windows. Votre *application* doit donc ajouter la bibliothèque à son édition de liens — le module ne le fait

pas pour vous. Les deux consommateurs du dépôt le font : [Applications/NkNetWorldDemo/NkNetWorldDemo.jenga:35-40](#) et [Sandbox/System/NKNetwork/NKNetworkSandbox.jenga:74-78](#) ajoutent tous deux "ws2_32" à leur `links()`.

L'oubli se manifeste par des erreurs d'édition de liens sur des symboles `__imp_socket`, `__imp_WSASStartup...` — obscures si on ne sait pas d'où elles viennent, évidentes ensuite.

10.1.1 Deux avertissements sur la documentation du module

La description du `.jenga` parle d'un autre module

[Kernel/System/NKNetwork/NKNetwork.jenga:3-15](#) décrit « un système de réflexion runtime » avec les fichiers `NkType`, `NkClass`, `NkProperty`, `NkMethod`, `NkRegistry`. C'est la description de **NKReflection**, copiée par erreur. Aucun de ces fichiers n'existe dans `NKNetwork`.

Le dossier `tests/` est vide

[Kernel/System/NKNetwork/tests/](#) ne contient qu'un sous-dossier vide. Le glob `testfiles(["tests/**/*.cpp"])` de [NKNetwork.jenga:92-95](#) ne correspond à aucun fichier : le bloc `with test()` : est un squelette mort.

De plus, il n'existe **aucune** fonction `SelfTest()` dans tout le module — là où `NKMedia` en compte 52 et `NKAudio` 2.

Cela ne veut pas dire que le module n'est pas testé : le test existe, mais c'est une application séparée, et la feuille de route le dit :

[Kernel/System/NKNetwork/ROADMAP.md:131-138](#)

```
1  ### Tests (2026-07-12)
2  - `Sandbox/System/NKNetwork` (cible `SandboxNKNetwork`) : **67 checks verts** –
3    NkBitStream round-trip + overflow, NkNetId Pack/Unpack, NkNetInterpolator,
4    réplication client/serveur sur messages forgés, **bout-en-bout loopback réel**
5    (StartServer + Connect 127.0.0.1, handshake, snapshot → spawn → delta →
6    despawn). Mode manuel --https <url> pour valider TLS.
```

Attention au chemin : ce sandbox est dans [Sandbox/System/NKNetwork/](#) — à la racine du dépôt — et **pas** dans [Applications/Sandbox/](#). Ses binaires sont bâtis en Debug et en Release.

10.2 Le périmètre de ce chapitre

Ce que nous enseignons, et pourquoi

Enseigné — parce que du code du dépôt l'exerce réellement :

- `NkSocket` (initialisation de plateforme, et UDP brut pour la découverte réseau);
- `NkAddress`;
- `NkConnectionManager` — la classe centrale;
- `NkBitWriter` / `NkBitReader`;
- `NkNetWorld` — la réplication d'entités.

Non enseigné — parce qu'*aucune* application du dépôt ne s'en sert :

- `NkLobby`, `NkSession`, `NkMatchmaker`, `NkDiscovery` — 67 Ko d'en-tête sans un seul

consommateur;

- NkRPC / NkRPCRouter — la feuille de route le classe « livré (déclaratif) », ce qui n'est pas la même chose que « prouvé »;
- NkReliableUDP en usage direct : il n'est employé qu'en interne par NkConnection.

Ce n'est pas un jugement sur la qualité de ce code. C'est une règle de prudence : un cours ne doit enseigner que ce qu'il peut montrer en train de fonctionner.

Sont également absents, d'après [ROADMAP.md:32-40](#) : la prédiction client et la réconciliation serveur, le relais des entités à autorité client, les transports WebSocket et WebRTC, la compression des instantanés, le chiffrement DTLS, et la traversée de NAT (STUN/TURN/ICE).

10.3 Le socle : la plateforme

Kernel/System/NKNetwork/src/NKNetwork/Transport/NkSocket.h:746-753

```
1 static NkNetResult PlatformInit() noexcept;
2 static void PlatformShutdown() noexcept;
```

Sous Windows, PlatformInit appelle WSStartup. Sous les autres systèmes, il ne fait presque rien — mais il faut l'appeler quand même, sinon votre code ne sera pas portable. La feuille de route est catégorique : « PlatformInit() requis avant usage » ([ROADMAP.md:52-54](#)).

Applications/Pong/src/Pong/Net/NetworkSession.cpp:28-40

```
1 void NetworkSession::PlatformInit() {
2     const auto r = net::NkSocket::PlatformInit();
3     if (r != net::NkNetResult::NK_NET_OK) {
4         logger.Error("[Net] PlatformInit failed: {0}", (int)r);
5     } else {
6         logger.Info("[Net] PlatformInit OK");
7     }
8 }
9
10 void NetworkSession::PlatformShutdown() {
11     net::NkSocket::PlatformShutdown();
12     logger.Info("[Net] PlatformShutdown");
13 }
```

10.3.1 Les codes de retour

Kernel/System/NKNetwork/src/NKNetwork/Core/NkNetDefines.h:268-291

```
1 enum class NkNetResult : uint8 {
2     NK_NET_OK = 0, NK_NET_INVALID_ARG, NK_NET_NOT_CONNECTED, NK_NET_ALREADY_CONNECTED,
3     NK_NET_CONNECTION_REFUSED, NK_NET_TIMEOUT, NK_NET_PACKET_TOO_LARGE, NK_NET_SOCKET_ERROR,
4     NK_NET_BUFFER_FULL, NK_NET_NOT_INITIALIZED, NK_NET_PLATFORM_UNSUPPORTED,
5     NK_NET_AUTH_FAILED, NK_NET_BANNED, NK_NET_UNKNOWN
6 };
7 inline const char* NkNetResultStr(NkNetResult r) noexcept; // -> texte lisible
```

NkNetResultStr existe : utilisez-le dans vos journaux plutôt que d'imprimer un entier. Un « erreur réseau 6 » ne vous dira rien dans six mois.

10.3.2 Les adresses

Kernel/System/NKNetwork/src/NKNetwork/Transport/NkSocket.h:172-353

```
1 class NkAddress {
2     NkAddress(const char* ip, uint16 port);
3     static NkAddress Loopback(uint16 port, Family f = Family::NK_IP_V4) noexcept;
4     static NkAddress Any(uint16 port, Family f = Family::NK_IP_V4) noexcept;
5     static NkAddress Broadcast(uint16 port) noexcept;
6     bool IsValid() const noexcept;
7     NkString ToString() const noexcept;
8 };
```

Trois fabriques nommées, à comprendre une fois pour toutes :

- Loopback(port) — 127.0.0.1, la machine locale. Pour tester;
- Any(port) — « toutes mes interfaces réseau ». C'est ce à quoi un serveur se lie. Any(0) laisse même le système choisir le port;
- Broadcast(port) — 255.255.255.255, tout le réseau local.

Toujours valider une adresse saisie

NkAddress("192.168.1.20", 7777) *analyse* la chaîne. Si l'utilisateur a tapé n'importe quoi, l'adresse est invalide et Connect échouera de façon obscure. Pong teste systématiquement avant ([Applications/Pong/src/Pong/Net/NetworkSession.cpp:122-129](#)) et affiche un message utile.

10.4 NkConnectionManager : la classe du chapitre

C'est par elle que tout passe. Elle fait tenir, derrière une API courte, un protocole UDP fiable complet : poignée de main, accusés de réception, retransmissions, canaux, déconnexion gracieuse.

Kernel/System/NKNetwork/src/NKNetwork/Protocol/NkConnection.h:733-790 (abrégé)

```
1 class NkConnectionManager {
2     NkConnectionManager() noexcept = default;
3     ~NkConnectionManager() noexcept; // (Shutdown auto)
4     NkConnectionManager(const NkConnectionManager&) = delete; // NON COPIABLE
5
6     NkNetResult StartServer(uint16 port, uint32 maxClients = 64) noexcept;
7     NkNetResult Connect(const NkAddress& serverAddr, uint16 localPort = 0) noexcept;
8     void Shutdown() noexcept;
9
10    NkNetResult SendTo(NkPeerId peer, const uint8* data, uint32 size, NkNetChannel ch)
11    noexcept;
12    NkNetResult Broadcast(const uint8* data, uint32 size, NkNetChannel ch) noexcept;
13    void DrainAll(NkVector<NkReceiveMsg>& out) noexcept;
14    void Disconnect(NkPeerId peer, const char* reason = nullptr) noexcept;
15    void DisconnectAll(const char* reason = nullptr) noexcept;
16    bool IsServer() const noexcept; bool IsRunning() const noexcept;
17    uint32 ConnectedPeerCount() const noexcept;
18    bool GetConnectionStats(NkPeerId peer, NkConnectionStats& outStats) const noexcept;
19
20    /* NkFunction */ onPeerConnected; ///< (NkPeerId)
21    /* NkFunction */ onPeerDisconnected; ///< (NkPeerId, const char* reason)
```

```

21     uint32 maxConnections = kNkMaxConnections;
22 };

```

10.4.1 Trois invariants de fil d'exécution

Tout le reste du chapitre découle de ces trois faits.

1. StartServer et Connect lancent un fil.

Kernel/System/NKNetwork/src/NKNetwork/Protocol/NkConnection.h:762-788

```

1  @note Crée un socket UDP et le lie à l'adresse Any(port).
2  @note Démarre le thread réseau interne pour polling automatique.
3  [...]
4  @note Envoie une déconnexion gracieuse à tous les pairs connectés.
5  @note Attend la fin du thread réseau avant retour (join).

```

Vous n'avez donc pas à interroger le réseau vous-même : un fil le fait en permanence, et Shutdown() l'attend proprement.

2. Les *callbacks* s'exécutent sur ce fil. onPeerConnected et onPeerDisconnected ne sont **pas** appelés depuis votre boucle principale. N'y faites rien de lourd : pas de création d'objets de jeu, pas de manipulation de l'interface, pas d'allocation. Pong s'y limite à un incrément atomique (Applications/Pong/src/Pong/Net/NetworkSession.cpp:60-63).

3. La poignée de main est asynchrone. Connect() renvoie NK_NET_OK *avant* que la connexion soit établie. Ce code de retour signifie « la demande est partie », pas « je suis connecté ». Le seul test valable est de surveiller ConnectedPeerCount() dans une boucle bornée.

10.4.2 Le bout-en-bout, tel qu'il est testé

Sandbox/System/NKNetwork/src/main.cpp:313-334

```

1  void TestLoopback() {
2      NK_CHECK(NkSocket::PlatformInit() == NkNetResult::NK_NET_OK);
3
4      constexpr uint16 kPort = 48213;
5
6      NkConnectionManager server;
7      NK_CHECK(server.StartServer(kPort, 8) == NkNetResult::NK_NET_OK);
8
9      NkConnectionManager client;
10     NK_CHECK(client.Connect(NkAddress("127.0.0.1", kPort)) == NkNetResult::NK_NET_OK);
11
12     // Attente de l'établissement de la connexion (handshake 3-way).
13     bool connected = false;
14     for (int i = 0; i < 500; ++i) {
15         if (server.ConnectedPeerCount() >= 1 && client.ConnectedPeerCount() >= 1) {
16             connected = true;
17             break;
18         }
19         NkChrono::SleepMilliseconds(static_cast<int64>(10));
20     }
21     NK_CHECK(connected);

```

Cinq cents itérations de dix millisecondes : cinq secondes au maximum. La boucle est **bornée** — un test qui attend indéfiniment n'est pas un test. Et on vérifie les *deux* côtés : le serveur voit un pair, le client aussi.

10.4.3 On ne reçoit pas, on draine

Kernel/System/NKNetwork/src/NKNetwork/Protocol/NkConnection.h:229-235

```
1 struct NkReceiveMsg {
2     uint8 data[kNkMaxPayloadSize] = {};    ///< buffer INLINE (1380 octets), pas un
        pointeur
3     uint32 size = 0;
4     NkPeerId from;
5     NkNetChannel channel = NkNetChannel::NK_NET_CHANNEL_UNRELIABLE;
6     NkTimestampMs receivedAt = 0;
7 };
```

Le fil réseau accumule les messages; votre boucle les récupère d'un coup :

Sandbox/System/NKNetwork/src/main.cpp:397-410

```
1     bool applied = false;
2     NkVector<NkReceiveMsg> msgs;
3     for (int i = 0; i < 500 && !applied; ++i) {
4         serverWorld.Update(0.02f);
5         msgs.Clear();
6         client.DrainAll(msgs);
7         for (usize m = 0; m < msgs.Size(); ++m) {
8             (void)clientWorld.HandleMessage(msgs[m]);
9         }
10        applied = spawned && clientPawn.x == 12.5f && clientPawn.y == -7.25f;
11        NkChrono::SleepMilliseconds(static_cast<int64>(10));
12    }
```

msgs.Clear() avant chaque DrainAll

DrainAll *ajoute* à votre conteneur. Sans le Clear(), vous retraiterez les anciens messages à chaque tour, indéfiniment.

Et il y a une seconde raison de vider ce conteneur : NkReceiveMsg contient un tampon **en ligne** de 1 380 octets, pas un pointeur. Un NkVector<NkReceiveMsg> de cent éléments pèse 138 Ko. Ne le copiez jamais inutilement, et videz-le entre deux images.

Corollaire de ce tampon fixe : **un message ne peut pas dépasser 1380 octets**. Au-delà, SendTo renvoie NK_NET_PACKET_TOO_LARGE.

Kernel/System/NKNetwork/src/NKNetwork/Core/NkNetDefines.h:145-175

```
1 static constexpr uint32 kNkMaxConnections    = 256u;
2 static constexpr uint32 kNkMaxPacketSize     = 1400u;    // taille MTU-safe
3 static constexpr uint32 kNkMaxPayloadSize    = 1380u;
4 static constexpr uint32 kNkMaxChannels       = 8u;
5 static constexpr uint32 kNkMaxRetransmits    = 5u;
6 static constexpr uint32 kNkMaxFragments     = 16u;
```

Ces 1 400 octets ne sont pas arbitraires : c'est la taille sous laquelle un datagramme traverse Internet sans être fragmenté par les routeurs. La fragmentation IP est la première cause de pertes

silencieuses.

Tester `msg.data` avant de le déréférencer

Le dépôt contient deux copies presque identiques du même fichier, et leur **seule** différence est instructive : [Applications/Pong/src/Pong/Net/NetworkSession.cpp:253](#) déréférence `msg.data[0]` sans garde; la copie corrigée ajoute `&& msg.data != nullptr`. Enseignons la version corrigée.

10.5 Choisir son canal

C'est la décision de conception la plus importante du module, et le fichier de définitions la documente mieux que n'importe quel manuel :

Kernel/System/NKNetwork/src/NKNetwork/Core/NkNetDefines.h:505-525

```
1 enum class NkNetChannel : uint8 {
2     NK_NET_CHANNEL_UNRELIABLE = 0,          ///< UDP pur – positions, inputs, animations
3     NK_NET_CHANNEL_RELIABLE_ORDERED,        ///< TCP-like – chat, événements gameplay
4     NK_NET_CHANNEL_RELIABLE_UNORDERED,      ///< Livraison garantie, ordre libre
5     NK_NET_CHANNEL_SEQUENCED,               ///< Seul le plus récent est livré
6     NK_NET_CHANNEL_SYSTEM                   ///< RÉSERVÉ AU MOTEUR – ne pas utiliser
7 };
```

Le raisonnement à tenir est le suivant : **une donnée qui sera de toute façon remplacée dans 16 millisecondes ne mérite pas d'être retransmise**. La position d'un joueur perdue en route est sans importance — la suivante arrive. En revanche, un message de chat perdu est perdu pour toujours.

Vous envoyez	Canal
Position, entrées, animation	UNRELIABLE
Chat, événements de jeu, commandes critiques	RELIABLE_ORDERED
Fichiers, gros blocs où l'ordre importe peu	RELIABLE_UNORDERED
États continus (points de vie, énergie)	SEQUENCED
Rien du tout	SYSTEM

Le canal SYSTEM est réservé au moteur — «usage interne uniquement, comportement non défini si utilisé». Ce n'est pas une suggestion.

Pong choisit son canal par un simple booléen, ce qui est le bon niveau d'abstraction pour une application :

Applications/Pong/src/Pong/Net/NetworkSession.cpp:234-243

```
1 bool NetworkSession::Broadcast(const uint8 *data, uint32 size, uint8 reliable) {
2     if (mConnMgr == nullptr)
3         return false;
4     if (mState.load() != NetworkState::Connected)
5         return false;
6     const auto ch = reliable ? net::NkNetChannel::NK_NET_CHANNEL_RELIABLE_ORDERED
7                             : net::NkNetChannel::NK_NET_CHANNEL_UNRELIABLE;
8     const auto r = mConnMgr->Broadcast(data, size, ch);
9     return r == net::NkNetResult::NK_NET_OK;
10 }
```

Dans le jeu, l'entrée du joueur et l'instantané d'état partent en non fiable (`GameplayScene.cpp:705` et `:849`), tandis que la pause, le but marqué et la demande de revanche partent en fiable (`:227`, `:293`, `:309`).

10.6 NkBitStream : ne pas gaspiller le réseau

Avec 1380 octets par message, chaque bit compte. C'est là qu'intervient la partie la plus élégante du module.

Kernel/System/NKNetwork/src/NKNetwork/Protocol/NkBitStream.h:165-352 (abrégé)

```
1 void WriteBool(bool) ; WriteU8/U16/U32/U64 ; WriteI8/I16/I32 ; WriteF32
2 void WriteF32Q(float32 v, float32 minV, float32 maxV, float32 prec) noexcept; // quantifie
3 void WriteInt(int32 v, int32 minV, int32 maxV) noexcept; // borne
4 void WriteVec3f(const NkVec3f&) ; WriteVec3fQ(v, minV, maxV, prec) ; WriteQuatf(const
    NkQuatf&);
5 void WriteString(const char* s, uint32 maxLen = 256) noexcept;
6 void WriteBytes(const uint8* data, uint32 size) noexcept;
7 void WriteBits(uint32 v, uint32 numBits) noexcept;
8 void AlignToByte() noexcept;
9 bool IsOverflowed() const noexcept;
10 uint32 BytesWritten() const;
```

Les méthodes de lecture sont exactement symétriques (`ReadBool`, `ReadU8`, `ReadF32Q`, `ReadInt...`).

10.6.1 L'argument : la quantification

Un `float` coûte 32 bits. Mais si vous savez qu'un score est entre 0 et 999, il tient sur 10 bits. Et si une position est entre -100 et $+100$ au centimètre près, elle tient sur environ 15 bits au lieu de 32.

Exemple écrit pour le cours (API : `NkBitStream.h:237, 252`)

```
1 #include "NKNetwork/Protocol/NkBitStream.h"
2
3 uint8 buf[256];
4 net::NkBitWriter w(buf, sizeof(buf));
5 w.WriteU8(kMonTypeDeMessage);
6 w.WriteInt(scoreJoueur, 0, 999); // 10 bits au lieu de 32
7 w.WriteF32Q(posX, -100.f, 100.f, 0.01f); // quantifie, ~15 bits
8 w.AlignToByte();
9 if (!w.IsOverflowed())
10     client.SendTo(peer, buf, (uint32)w.BytesWritten(),
11         net::NkNetChannel::NK_NET_CHANNEL_RELIABLE_ORDERED);
```


Écriture et lecture doivent être des miroirs exacts

Exemple écrit pour le cours (API : NkBitStream.h:455-635)

```
1 net::NkBitReader r(m.data, m.size);
2 const uint8 type = r.ReadU8();
3 const int32 score = r.ReadInt(0, 999); // MEMES bornes
4 const float32 x = r.ReadF32Q(-100.f, 100.f, 0.01f); // MEMES bornes, MEME
    precision
5 if (r.IsOverflowed()) { /* message tronque : jeter */ }
```

Un flux de bits n'est pas auto-descriptif : il n'y a ni noms de champs, ni balises. Si le lecteur écrit `ReadInt(0, 9999)` là où l'écrivain a écrit `WriteInt(0, 999)`, il ne lira pas le bon nombre de bits, et **tout ce qui suit sera décalé**. Les symptômes sont absurdes : une position correcte suivie d'un identifiant aberrant.

Deux disciplines s'imposent. D'abord, écrire les deux fonctions *côte à côte*, dans le même fichier, immédiatement l'une après l'autre. Ensuite, tester systématiquement `IsOverflowed()` des deux côtés : c'est la seule détection d'erreur dont vous disposez.

Le *round-trip* de test est le meilleur exercice du module, parce qu'il ne demande aucun réseau :

Sandbox/System/NKNetwork/src/main.cpp:47-89 (abrégé)

```
1 void TestBitStream() {
2     uint8 buffer[256];
3     NkBitWriter writer(buffer, sizeof(buffer));
4
5     writer.WriteBool(true);
6     writer.WriteU8(0xAB);
7     writer.WriteU16(0x1234);
8     writer.WriteU32(0xDEADBEEF);
9     writer.WriteU64(0x0123456789ABCDEFu11);
10    writer.WriteI32(-42);
11    writer.WriteF32(3.5f);
12    writer.WriteBits(5u, 3);
13    writer.AlignToByte();
14    const uint8 raw[4] = {1, 2, 3, 4};
15    writer.WriteBytes(raw, 4);
16    NK_CHECK(!writer.IsOverflowed());
17
18    NkBitReader reader(buffer, writer.BytesWritten());
19    NK_CHECK(reader.ReadBool() == true);
20    NK_CHECK(reader.ReadU8() == 0xAB);
21    /* ... symetrique ... */
22    NK_CHECK(!reader.IsOverflowed());
23
24    // Overflow en ecriture comme en lecture.
25    uint8 tiny[2];
26    NkBitWriter tinyWriter(tiny, sizeof(tiny));
27    tinyWriter.WriteU32(0xFFFFFFFFFu);
28    NK_CHECK(tinyWriter.IsOverflowed());
29 }
```

10.7 NkNetWorld : répliquer des entités

Envoyer des octets, c'est bien. Synchroniser l'état d'un monde, c'est le vrai sujet. NkNetWorld fournit la mécanique.

Kernel/System/NKNetwork/src/NKNetwork/Replication/NkNetWorld.h:128-515 (abrégé)

```
1 struct NkNetEntity {
2     NkNetId netId; uint32 prefabId; void* user;
3     WriteStateFn writeState; /// serialise l'etat - appele cote AUTORITE
4     ReadStateFn readState;  /// applique un etat reçu - cote replique
5 };
6
7 class NkNetWorld {
8     struct Config { float32 tickRate; uint32 keyframeInterval; /* ... */ };
9     void Init(NkConnectionManager* mgr, bool isServer, Config cfg = {}) noexcept;
10    NkNetId AllocateNetId() noexcept;
11    bool RegisterEntity(const NkNetEntity& desc) noexcept;
12    bool UnregisterEntity(NkNetId netId, bool notifyPeers = true) noexcept;
13    NkNetEntity* FindEntity(NkNetId netId) noexcept;
14    uint32 EntityCount() const noexcept;
15    void Update(float32 dt) noexcept;
16    bool HandleMessage(const NkReceiveMsg& msg) noexcept;
17    uint32 CurrentTick() const noexcept;
18    void DrainInputs(NkPeerId peer, NkVector<NkNetInput>& out) noexcept;
19    SpawnCb onEntitySpawn;    /// (NkNetId, uint32 prefabId, NkPeerId owner)
20    DespawnCb onEntityDespawn; /// (NkNetId)
21 };
```

Le modèle tient en trois phrases :

Kernel/System/NKNetwork/src/NKNetwork/Replication/NkNetWorld.h:20-22

```
1 HandleMessage(). Entité inconnue → callback onEntitySpawn (l'application
2 crée son objet et l'enregistre) ; état reçu → readState ; destruction →
3 onEntityDespawn. Les inputs locaux remontent via SendInput().
```

Côté serveur — l'autorité — on décrit comment sérialiser :

Sandbox/System/NKNetwork/src/main.cpp:349-361

```
1 NkNetEntity desc;
2 desc.netId = serverWorld.AllocateNetId();
3 desc.prefabId = 99;
4 desc.user = &serverPawn;
5 desc.writeState = [](void *user, NkBitWriter &w) {
6     TestPawn *p = static_cast<TestPawn *>(user);
7     w.WriteF32(p->x);
8     w.WriteF32(p->y);
9 };
10 NK_CHECK(serverWorld.RegisterEntity(desc));
```

Côté client — la réplique — on réagit à l'apparition d'une entité inconnue :

Sandbox/System/NKNetwork/src/main.cpp:171-203 (abrégé)

```
1 world.onEntitySpawn = [&](NkNetId netId, uint32 prefabId, NkPeerId /*owner*/) {
2     spawnedPrefab = prefabId;
```

```

3      NkNetEntity desc;
4      desc.netId = netId;
5      desc.prefabId = prefabId;
6      desc.user = &pawn;
7      desc.readState = [](void *user, NkBitReader &r) {
8          TestPawn *p = static_cast<TestPawn *>(user);
9          p->x = r.ReadF32();
10         p->y = r.ReadF32();
11     };
12     world.RegisterEntity(desc);
13 };

```

Le prefabId est votre code de type : « ceci est un joueur », « ceci est un projectile ». C'est à vous de décider ce que chaque numéro signifie et de créer l'objet correspondant.

Les deux règles des fonctions d'état

writeState doit être déterministe : « deux appels sur le même état doivent produire les mêmes octets » ([Replication/NkNetWorld.h:124-126](#)). Sinon le calcul de différences envoie des mises à jour perpétuelles pour un état qui n'a pas bougé.

Écrivez des lambdas, pas des pointeurs de fonction nus. Le pont ECS du moteur le documente ([Engine/Noge/src/Noge/ECS/Replication/NkNetWorld.cpp:132-138](#)) : un pointeur de fonction brut résout mal la surcharge du type de rappel.

10.8 Le socket brut : la découverte sur le réseau local

Il reste un cas où l'on descend sous NkConnectionManager : annoncer un serveur pour que les autres machines le trouvent sans qu'on ait à taper une adresse IP.

Kernel/System/NKNetwork/src/NKNetwork/Transport/NkSocket.h:504-715 (abrégé)

```

1 class NkSocket {
2     enum class Type : uint8 { NK_UDP, NK_TCP /* ... */ };
3     NkNetResult Create(const NkAddress& localAddr, Type type = Type::NK_UDP) noexcept;
4     void Close() noexcept;
5     NkNetResult SetNonBlocking(bool v) noexcept;
6     NkNetResult SetBroadcast(bool v) noexcept;
7     NkNetResult SendTo(const void* data, uint32 size, const NkAddress& to) noexcept;
8     NkNetResult RecvFrom(void* buf, uint32 bufSize, uint32& outSize, NkAddress& outFrom)
9         noexcept;
10    bool IsValid() const noexcept;
11 };

```

Le socket qui émet la balise :

Applications/Pong/src/Pong/Net/NetworkDiscovery.cpp:44-63 (abrégé)

```

1     mBeaconSock = new net::NkSocket();
2     // Bind sur une socket UDP IPv4 quelconque (port 0 = Laisse l'OS choisir),
3     // on n'a besoin que d'envoyer. SetBroadcast active SO_BROADCAST
4     // nécessaire pour pouvoir SendTo vers 255.255.255.255.
5     const auto rc = mBeaconSock->Create(net::NkAddress::Any(0),
6     net::NkSocket::Type::NK_UDP);
7     if (rc != net::NkNetResult::NK_NET_OK) { /* ... */ return false; }
8     (void)mBeaconSock->SetNonBlocking(true);
9     const auto rb = mBeaconSock->SetBroadcast(true);

```

```
9         if (rb != net::NkNetResult::NK_NET_OK) { /* ... */ return false; }
```

et celui qui écoute, lié au port de balise sur toutes les interfaces :

Applications/Pong/src/Pong/Net/NetworkDiscovery.cpp:84-96 (abrégé)

```
1         mScanSock = new net::NkSocket();
2         // Bind sur kBeaconPort, INADDR_ANY : on reçoit les broadcasts
3         // émises vers 255.255.255.255:kBeaconPort par d'autres machines du LAN.
4         const auto rc = mScanSock->Create(net::NkAddress::Any(kBeaconPort),
        net::NkSocket::Type::NK_UDP);
5         /* ... */
6         (void)mScanSock->SetNonBlocking(true);
7         (void)mScanSock->SetBroadcast(true);
```

Sans SetBroadcast(true), l'émission échoue

Le système d'exploitation refuse par défaut d'envoyer vers une adresse de diffusion — c'est une protection. Il faut l'autoriser explicitement, des deux côtés. C'est l'oubli numéro un de la découverte réseau.

Le *tick*, avec ses deux garde-fous :

Applications/Pong/src/Pong/Net/NetworkDiscovery.cpp:115-160 (abrégé)

```
1         if (mBeaconSock != nullptr) {
2             mBeaconTimer -= dt;
3             if (mBeaconTimer <= 0.0f) {
4                 const auto dst = net::NkAddress::Broadcast(kBeaconPort);
5                 const auto rs = mBeaconSock->SendTo(&mBeaconPkt, sizeof(mBeaconPkt), dst);
6                 mBeaconTimer = kBeaconIntervalSec;
7             }
8         }
9         if (mScanSock != nullptr) {
10            // Boucle : on lit tant qu'il y a des datagrammes pendants.
11            // En non-bloquant, RecvFrom OK avec outSize=0 = rien à lire.
12            constexpr int kMaxPerTick = 32;
13            for (int i = 0; i < kMaxPerTick; ++i) {
14                uint8 buf[256];
15                uint32 received = 0;
16                net::NkAddress from;
17                const auto rr = mScanSock->RecvFrom(buf, sizeof(buf), received, from);
18                if (rr != net::NkNetResult::NK_NET_OK) break;
19                if (received == 0) break;
20                /* filtre magic + version, puis UpdateHost(pkt, from) */
21            }
22        }
```

Deux points à retenir :

1. **en mode non bloquant, un RecvFrom qui réussit avec outSize == 0 signifie «rien à lire», pas «erreur».** C'est la condition d'arrêt normale de la boucle;
2. **la boucle est bornée à 32 datagrammes par image.** Sur un réseau chargé — ou face à un émetteur mal réglé — une boucle non bornée monopoliserait la frame.

Notez enfin le filtrage «magic + version» : un port UDP reçoit tout ce que n'importe qui envoie. Toujours vérifier une signature avant d'interpréter.

10.9 HTTP, et pourquoi HTTPS échoue

Kernel/System/NKNetwork/src/NKNetwork/HTTP/NkHTTPClient.h:383-717 (abrégé)

```
1 struct NkHTTPResponse {
2     uint32 statusCode = 0;
3     NkString body;
4     NkString error;
5     bool IsOK() const noexcept;
6 };
7 class NkHTTPClient {
8     NkHTTPResponse Get(const char* url) noexcept;
9     NkHTTPResponse Post(const char* url, const char* json) noexcept;
10 };
```

HTTPS est désactivé par défaut

Le support TLS est optionnel, activé par une variable d'environnement au moment de la construction :

Kernel/System/NKNetwork/NKNetwork.jenga:21-25

```
1 # TLS/HTTPS opt-in : NK_ENABLE_TLS=1 (ou mbedtls) active le backend mbed-TLS
2 # (bibliotheque NKmbedTLS, C pur, cross-compilee pour toutes les plateformes y
3 # compris iOS). Desactive par défaut -> HTTP simple uniquement, aucune dependance.
4 _TLS_OPT = os.getenv("NK_ENABLE_TLS", "").strip().lower()
5 _WANT_MBEDTLS = _TLS_OPT in ("1", "true", "on", "yes", "mbedtls")
```

Sans cette variable, toute URL en https:// échoue — proprement, au moins :

Kernel/System/NKNetwork/src/NKNetwork/HTTP/NkHTTPClient.cpp:1050

```
1         response.error = "HTTPS not compiled in (rebuild with NK_ENABLE_TLS=1)";
```

Et si vous activez TLS, votre application doit lier NKmbedTLS en plus, comme le fait le sandbox. Pour ce cours, tenons-nous-en à http://.

Le mode de test manuel du sandbox montre l'usage complet :

Sandbox/System/NKNetwork/src/main.cpp:441-451 (abrégé)

```
1 int main(int argc, char **argv) {
2     // Mode HTTPS manuel : SandboxNKNetwork --https <url>
3     // Nécessite un build avec NK_ENABLE_TLS=1 (backend mbedTLS), sinon le
4     // client renvoie proprement "HTTPS not compiled in".
5     if (argc >= 3 && NkString(argv[1]) == NkString("--https")) {
6         NkHTTPClient http;
7         const NkHTTPResponse resp = http.Get(argv[2]);
8         /* ... affichage de resp.statusCode, resp.body.Length(), resp.error.CStr() ... */
9         return (resp.statusCode >= 200 && resp.statusCode < 400) ? 0 : 1;
10    }
```

10.10 Comment structurer une application en réseau

Vous avez maintenant toutes les briques. Reste la question d'architecture, et Pong y répond de la façon qu'il faut copier.

1. **Une classe de session** possède le `NkConnectionManager` et n'expose que des verbes simples : `StartHost`, `StartJoin`, `Broadcast`, `DrainReceived`, `Shutdown`, plus un état atomique (`Idle` / `Hosting` / `Connected`) (`Applications/Pong/src/Pong/Net/NetworkSession.h`);
2. un `Tick(dt)` appelé une fois par image draine le fil réseau et met à jour cet état :

`Applications/Pong/src/Pong/Game/PongApp.cpp:162-169`

```
1 // Tick reseau (drain interne du thread reseau). Aucun cout si
2 // la session est Idle.
3 mNetwork.Tick(dt);
4 // Tick decouverte LAN : emet beacon (host) + drain scan (client).
5 mDiscovery.Tick(dt);
```

3. **les écrans d'interface lisent l'état, jamais le manager.** Aucune scène de Pong ne touche au `NkConnectionManager` : elles appellent `DrainReceived` et consultent `NetworkState`.

Le routage des messages se fait dans la session, par type, avec la garde qui manquait :

`Applications/Pong/src/Pong/Net/NetworkSession.cpp:245-260 (abrégé)`

```
1 void NetworkSession::DrainInternal() {
2     if (mConnMgr == nullptr)
3         return;
4     // Drain brut depuis NkConnectionManager.
5     NkVector<net::NkReceiveMsg> all;
6     mConnMgr->DrainAll(all);
7     for (uint32 i = 0; i < all.Size(); ++i) {
8         const auto &msg = all[i];
9         if (msg.size >= sizeof(netproto::PktHello) && msg.data[0] ==
netproto::kMsgHello) {
10             netproto::PktHello pkt;
11             /* ... recopie des sizeof(pkt) octets de msg.data dans pkt ... */
12             continue;
13         }
14         // Message non-interne : reserve pour le caller via DrainReceived.
15         mPendingForUser.PushBack(msg);
16     }
17 }
```

Notez le test `msg.size >= sizeof(...)` avant de lire le contenu : un pair malveillant, ou simplement une version différente de votre programme, peut envoyer n'importe quoi.

Le découpage à retenir

Le réseau vit sur son fil; la session fait tampon; l'interface ne voit qu'un état et une file de messages. Aucune scène, aucun widget ne doit connaître `NkConnectionManager`. C'est ce qui rend le jeu testable sans réseau et le réseau modifiable sans toucher au jeu.

10.11 L'ordre de fermeture

Sandbox/System/NKNetwork/src/main.cpp:430-433

```
1      client.Shutdown();
2      server.Shutdown();
3      NkSocket::PlatformShutdown();
```

et, avec une couche de réplication au-dessus :

Applications/NkNetWorldDemo/src/main.cpp:205-211

```
1      serverNet.Shutdown();
2      clientNet.Shutdown();
3      client.Shutdown();
4      server.Shutdown();
5      net::NkSocket::PlatformShutdown();
```

La règle est celle des poupées russes : **on éteint du plus haut vers le plus bas**. Les systèmes de réplication d'abord, puis les gestionnaires de connexion — qui joignent leur fil réseau —, puis la plateforme. Inverser reviendrait à fermer Winsock pendant qu'un fil s'en sert encore.

N'incluez pas l'en-tête parapluie dans une application ECS

Voici le piège le plus coûteux du module, documenté deux fois dans le dépôt :

Applications/NkNetWorldDemo/src/main.cpp:20-37

```
1  // PAS L'umbrella NKNetwork/NKNetwork.h : il injecte des alias de commodité
2  // dans `nkentseu` (dont `using NkNetSystem = net::NkNetSystem;` et
3  // `using NkNetEntity = net::NkNetEntity;`) qui entrent en collision directe
4  // avec l'adaptateur ECS de Noge (`nkentseu::NkNetSystem`, composant
5  // `nkentseu::ecs::NkNetEntity`). On inclut donc uniquement les en-têtes
6  // précis nécessaires.
7  #include "Noge/ECS/Replication/NkNetWorld.h"
8  #include "NKECS/World/NkWorld.h"
9  #include "NKNetwork/Transport/NkSocket.h"
10 #include "NKTime/NkChrono.h"
11 #include "NKLogger/NkLog.h"
12
13 using namespace nkentseu;
14 using namespace nkentseu::ecs;
15 // PAS de `using namespace nkentseu::net;` : net::NkNetEntity (descripteur de
16 // réplication NKNetwork) et ecs::NkNetEntity (composant ECS) partagent le
17 // même nom -- même piège que Noge/ECS/Replication/NkNetWorld.cpp.
```

C'est l'exception à la règle générale du moteur — « incluez toujours l'en-tête parapluie » du chapitre 1. Ici, l'en-tête parapluie injecte des alias dans le *namespace* global du moteur, et deux types différents portent le même nom. Incluez les en-têtes précis, et n'ouvrez pas le *namespace* `nkentseu::net` en grand.

10.12 Exercices

1 — Le flux de bits, sans réseau

Écrivez un programme console qui sérialise une petite structure de jeu — un identifiant, un score entre 0 et 999, deux coordonnées entre -100 et $+100$ au centimètre, un booléen — puis la relit et vérifie l'égalité champ par champ. Affichez `BytesWritten()`. Comparez avec ce que coûterait la même structure en écriture brute (`WriteU32`, `WriteF32`). Puis introduisez volontairement une erreur : lisez le score avec les bornes (0, 9999). Observez que ce n'est pas seulement le score qui devient faux, mais tout ce qui suit.

2 — Serveur et client dans le même programme

Reproduisez `TestLoopback` : dans un seul `main`, démarrez un serveur sur 127.0.0.1, connectez un client, attendez la poignée de main dans une boucle bornée, échangez trois messages sur trois canaux différents, puis fermez dans le bon ordre. Ajoutez de la journalisation dans `onPeerConnected` — et regardez sur quel fil elle s'exécute. Puis, volontairement, supprimez la boucle d'attente et envoyez immédiatement après `Connect()`. Que se passe-t-il, et pourquoi ?

3 — Un chat dans votre interface

Ajoutez à votre application `NKGui` deux boutons « Héberger » et « Rejoindre », un `InputText` pour l'adresse, un second pour le message, et une liste des messages reçus. Respectez l'architecture de `Pong` : une petite classe de session qui possède le manager, un `Tick(dt)` appelé une fois par image, et une interface qui ne voit qu'un état et une file. Utilisez `RELIABLE_ORDERED` — un message de chat perdu ne se rattrape pas. Testez avec deux instances de votre programme sur la même machine, puis, si vous le pouvez, sur deux machines du même réseau.

4 — Mesurer ce que coûte la fiabilité

Reprenez l'exercice 2 et envoyez mille messages, une fois en `UNRELIABLE`, une fois en `RELIABLE_ORDERED`, en comptant ceux qui arrivent et en chronométrant. Sur une boucle locale, vous ne perdrez probablement rien : les deux chiffres seront identiques. C'est un résultat en soi — **tester le réseau sur 127.0.0.1 ne teste pas le réseau**. Consultez ensuite `GetConnectionStats` et voyez ce que le module sait vous dire du temps d'aller-retour et des pertes.

Projet final : NkAtelier

Vous avez maintenant traversé toute la pile : la fenêtre, les événements, le rendu 2D, les widgets, les images, les polices, le son, la vidéo, le réseau. Ce dernier chapitre ne présente aucune API nouvelle. Il fait une seule chose, et c'est la plus utile : **il assemble**.

Nous allons construire, du fichier de build jusqu'à l'exécutable qui s'affiche, une petite application appelée NkAtelier. Elle tient en un seul `main.cpp`, elle ne dépend d'**aucun fichier externe** — pas une police, pas une image, pas un son — et elle prouve, à l'écran, que cinq modules coopèrent.

11.1 Ce que nous allons construire

NkAtelier est une fenêtre avec trois panneaux.

1. **Panneau « Image »** — une image *fabriquée à l'exécution* par les primitives de dessin de NkImage, redimensionnée en vignette, envoyée au backend et affichée par le widget Image. Un curseur change une couleur ; l'image est refabriquée et ré-uploadée.
2. **Panneau « Son »** — trois boutons qui jouent trois sons *synthétisés* par AudioGenerator, sur le bus « UI », avec une case à cocher pour couper ce bus.
3. **Panneau « Vidéo »** — au démarrage, l'application *écrit* une petite vidéo MJPEG dans un fichier AVI, puis la *relit* image par image et l'affiche, cadencée, dans l'interface. Boutons lecture, pause et retour au début.

Le tout avec du texte rendu par une police embarquée.

Pourquoi aucun actif externe

C'est un choix pédagogique délibéré, et il vaut d'être expliqué.

Une application d'apprentissage qui exige un `logo.png` et un `clic.wav` échoue à la première exécution, parce que le répertoire courant n'est pas celui qu'on croit. Le lecteur passe alors une heure sur un problème de chemin relatif au lieu d'apprendre le moteur — c'est exactement ce que la démo du dépôt contourne avec sa liste de chemins candidats ([Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:294-299](#)).

Ici, chaque ressource est **produite** :

- la police vient de NkFontEmbedded (chapitre 6) ;
- l'image est dessinée par NkImage::FillRect et consorts (chapitre 5) ;
- le son est synthétisé par AudioGenerator::GenerateTone (chapitre 7) ;
- la vidéo est écrite par NkVideoWriter avant d'être relue (chapitre 8).

L'application ne peut pas échouer faute de fichier. C'est également ce qui la rend facile à donner à quelqu'un d'autre.

11.2 Étape 0 — l'arborescence

Un module ou une application du dépôt, c'est un dossier avec un `.jenga`. Créez :

Arborescence à créer

```
1 Applications/NkAtelier/  
2   NkAtelier.jenga  
3   src/  
4     main.cpp
```

C'est tout. Un seul fichier source; c'est suffisant pour ce que nous faisons, et cela évite de mélanger l'apprentissage du moteur avec celui d'une organisation de projet.

11.3 Étape 1 — le fichier de build

11.3.1 Le contenu

Le fichier ci-dessous est modelé sur `Applications/NKGuiDemo/NKGuiDemo.jenga`, allégé de ce dont nous n'avons pas besoin (la réflexion, la détection de Vulkan) et augmenté de ce qu'il nous faut (`NKAudio`, `NKMedia`).

Applications/NkAtelier/NkAtelier.jenga — écrit pour le cours, modelé sur NKGuiDemo.jenga:41-95

```
1 #!/usr/bin/env python3  
2 # -*- coding: utf-8 -*-  
3 """NkAtelier – projet final du cours : image + texte + son + video dans une UI NKGui."""  
4  
5 from Jenga import *  
6 from jengaconfig import *  
7  
8  
9 with project("NkAtelier"):  
10     windowedapp()  
11     language("C++")  
12     cppdialect("C++17")  
13     location(".")  
14  
15     files(["src/**/*.cpp"])  
16  
17     nkentseudependson(  
18         ["NKGui", "NKCanvas", "NKWindow", "NKEvent", "NKGlad",  
19          "NKFont", "NKImage", "NKAudio", "NKMedia",  
20          "NKStream", "NKFileSystem", "NKLogger", "NKMath", "NKTime",  
21          "NKContainers", "NKMemory", "NKCore", "NKPlatform", "NKThreading"],  
22         extra_includes=["src", "%{NKGlad.location}/include"],  
23     )  
24  
25     objdir("%{wks.location}/Build/Obj/%{cfg.buildcfg}-%{cfg.system}%{prj.name}")  
26     targetdir("%{wks.location}/Build/Bin/%{cfg.buildcfg}-%{cfg.system}%{prj.name}")  
27  
28     # ===== Windows =====  
29     with filter("system:Windows && !options:windows-runtime=uwp && !system:XboxSeries &&  
!system:XboxOne"):  
30         windowedapp()  
31         usetoolchain(TC_WINDOWS)  
32         defines(["WIN32_LEAN_AND_MEAN", "_UNICODE", "UNICODE"])  
33         links([  
34             "user32", "gdi32", "opengl32", "dwmapi", "shell32",
```

```

35         "uuid", "ole32",
36         "d3d11", "d3d12", "dxgi", "d3dcompiler",
37     ])
38
39     # ===== Linux (XLib) =====
40     with filter("system:Linux && options:linux-backend=xlib || system:Linux &&
!options:linux-backend && !options:headless"):
41         windowedapp()
42         usetoolchain("clang-native")
43         defines(["NKENTSEU_FORCE_WINDOWING_XLIB_ONLY"])
44         links(["pthread", "X11", "Xext", "GL"])
45
46     # ===== macOS =====
47     with filter("system:macOS"):
48         windowedapp()
49         usetoolchain("clang-native")
50         frameworks(["Cocoa", "QuartzCore", "OpenGL"])
51
52     with filter("config:Debug"):
53         defines(["_DEBUG", "DEBUG"])
54         optimize("Off")
55         symbols(True)
56     with filter("config:Release"):
57         defines(["NDEBUG"])
58         optimize("Speed")
59         symbols(False)

```

11.3.2 Ce qu'il faut comprendre dans ce fichier

- `windowedapp()` — application fenêtrée, pas console. Sous Windows, cela évite qu'une console noire s'ouvre derrière votre fenêtre;
- `nkentseudependson([...])` — la liste des modules. Elle *doit* contenir tout ce que vous incluez, transitivement ou non;
- `extra_includes` — NKGLad est nécessaire parce que le backend OpenGL de NkCanvas expose ses en-têtes dans les vôtres;
- les blocs `with filter(...)` — la configuration par plateforme. Ne les inventez pas : recopiez-les d'une application existante qui fonctionne.

La liste des dépendances n'est pas décorative

Un module absent de `nkentseudependson` ne provoque pas une erreur claire « module manquant ». Il produit soit une erreur de compilation sur un en-tête introuvable — encore lisible —, soit une erreur d'édition de liens sur des symboles non résolus, qui ne dit pas quel module ajouter.

Le réflexe : quand un symbole `nkentseu::quelquechose` manque à l'édition de liens, cherchez dans quel module il est déclaré, et ajoutez ce module.

11.3.3 Enregistrer le projet dans l'espace de travail

Un `.jenga` isolé n'est pas construit. Il faut l'inclure depuis le descripteur racine, `Nkentseu.jenga`, à côté des autres applications :

Nkentseu.jenga — ajout, forme identique à Nkentseu.jenga:1394-1396

```
1 with include("Applications/NkAtelier/NkAtelier.jenga"):
2
3     pass
```

11.4 Étape 2 — le squelette : fenêtre, contexte, backend, police

11.4.1 Les inclusions

Applications/NkAtelier/src/main.cpp — écrit pour le cours (modèle : NKGuiDemo/main.cpp:7-31)

```
1 #include "NKWindow/NKWindow.h"
2 #include "NKWindow/NKMain.h"
3 #include "NKEvent/NkWindowEvent.h"
4 #include "NKEvent/NkMouseEvent.h"
5 #include "NKEvent/NkKeyboardEvent.h"
6
7 #include "NKCanvas/Core/NkContextDesc.h"
8 #include "NKCanvas/Core/NkGraphicsApi.h"
9 #include "NKCanvas/Renderer/Targets/NkRenderWindow.h"
10 #include "NKCanvas/UI/NKGuiCanvasBackend.h" // pont NKGui -> NKCanvas (header-only)
11
12 #include "NKGui/NKGui.h" // TOUJOURS L'en-tête parapluie de NKGui
13 #include "NKImage/Core/NkImage.h"
14 #include "NKAudio/NkAudio.h"
15 #include "NKMedia/Video/NkVideoWriter.h"
16 #include "NKMedia/Video/NkVideoReader.h"
17
18 #include "NKTime/NkClock.h"
19 #include "NKMemory/NkUniquePtr.h"
20 #include "NKMemory/NKMemory.h"
21 #include "NKLogger/NkLog.h"
22 #include "NKMath/NkColor.h"
23
24 using namespace nkentseu;
25 using namespace nkentseu::nkgui;
26 using namespace nkentseu::renderer;
```

Deux remarques d'ordre général.

D'abord, on inclut l'en-tête parapluie de NKGui (NKGui/NKGui.h) et jamais un en-tête interne — le chapitre 1 en a donné la raison : c'est lui qui neutralise les macros de X11 sous Linux.

Ensuite, on n'inclut pas d'en-tête parapluie pour NKMedia : il n'en existe pas. On prend les deux sous-en-têtes précis dont on a besoin.

11.4.2 Fenêtre et cible de rendu

main.cpp — écrit pour le cours (modèle : NKGuiDemo/main.cpp:240-263)

```
1 int nkmain(const NkEntryState &state) {
2     (void)state;
3
4     NKWindow window;
5     NKWindowConfig cfg;
6     cfg.title = "NkAtelier - projet final du cours Nkentseu";
7     cfg.width = 1180;
```

```

8     cfg.height = 760;
9     cfg.centered = true;
10    cfg.resizable = true;
11    if (!window.Create(cfg))
12        return -1;
13
14    NkContextDesc desc;
15    desc.api = NkGraphicsApi::NK_GFX_API_AUTO;
16    if (desc.api == NkGraphicsApi::NK_GFX_API_AUTO) {
17    #if defined(NKENTSEU_PLATFORM_WINDOWS)
18        desc.api = NkGraphicsApi::NK_GFX_API_DX11;
19    #else
20        desc.api = NkGraphicsApi::NK_GFX_API_OPENGL;
21    #endif
22    }
23    auto target = memory::NkMakeUnique<NkRenderWindow>(window, desc);
24    if (!target || !target->IsValid())
25        return -1;

```

Le point d'entrée s'appelle `nkmain` et vient de `NKWindow/NKMain.h` : c'est lui qui gère les différences de démarrage entre plateformes.

Si rien ne s'affiche, changez de backend

Le tableau du chapitre 1 rappelait que, à la date de sa rédaction, seul le backend OpenGL était validé à l'exécution; DX11 et le rasteriseur logiciel avaient un écran noir signalé, Vulkan et DX12 des plantages ([Kernel/Runtime/NKCanvas/ROADMAP.md](#), lignes 185-200 et 601-607).

Ces états sont datés et le dépôt bouge — mais si votre fenêtre s'ouvre noire, **la première chose à essayer n'est pas de relire votre code** : c'est de remplacer `NK_GFX_API_DX11` par `NK_GFX_API_OPENGL` et de relancer. Cinq secondes, et cela élimine la moitié des causes possibles.

11.4.3 Contexte NKGui et backend

main.cpp — écrit pour le cours (modèle : NKGuiDemo/main.cpp:264-279)

```

1     auto ctxPtr = memory::NkMakeUnique<NkGuiContext>();
2     if (!ctxPtr)
3         return -1;
4     NkGuiContext &ctx = *ctxPtr;
5     ctx.Init(static_cast<int32>(cfg.width), static_cast<int32>(cfg.height));
6     SetCurrentContext(&ctx);
7
8     NkGuiCanvasBackend backend;
9     if (!backend.Init(target->GetRenderer()))
10        return -1;

```

L'ordre compte : le backend reçoit le render, donc le render doit exister. Et, souvenez-vous du chapitre 5, toutes les textures doivent être créées *après* l'initialisation du render — ce qui est le cas ici, puisque nos uploads viendront plus bas.

11.4.4 La police

main.cpp — écrit pour le cours (modèle : NKGuiDemo/main.cpp:281-288)

```
1  auto fontPtr = memory::NkMakeUnique<NkGuiFont>();
2  if (!fontPtr->LoadEmbedded(NkEmbeddedFontId::DroidSans, 17.f)) {
3      fontPtr->LoadEmbedded(NkEmbeddedFontId::ProggyClean, 16.f);
4  }
5  ctx.font = fontPtr.Get();
6  if (fontPtr->Valid()) {
7      backend.UploadFontGray8(fontPtr->TexId(), fontPtr->pixels, fontPtr->atlasW,
8      fontPtr->atlasH);
9  } else {
10     logger.Error("[NkAtelier] aucune police embarquee disponible : textes invisibles");
11 }
```

Trois lignes utiles et un repli, exactement comme au chapitre 6. Le `else` n'est pas superflu : sans lui, une police manquante donnerait une interface entièrement muette, sans le moindre indice.

11.5 Étape 3 — les événements et la boucle

11.5.1 Le branchement des entrées

main.cpp — écrit pour le cours (modèle : NKGuiDemo/main.cpp:326-347)

```
1  auto &events = NkEvents();
2  bool running = true;
3  events.AddEventCallback<NkWindowCloseEvent>([&](NkWindowCloseEvent *) { running = false;
4  });
5  events.AddEventCallback<NkMouseMoveEvent>([&](NkMouseMoveEvent *e) {
6      ctx.input.mousePos = {static_cast<float32>(e->GetX()),
7      static_cast<float32>(e->GetY())};
8  });
9  events.AddEventCallback<NkMouseButtonPressEvent>([&](NkMouseButtonPressEvent *e) {
10     if (e->GetButton() == NkMouseButton::NK_MB_LEFT)
11         ctx.input.mouseDown[0] = true;
12 });
13 events.AddEventCallback<NkMouseButtonReleaseEvent>([&](NkMouseButtonReleaseEvent *e) {
14     if (e->GetButton() == NkMouseButton::NK_MB_LEFT)
15         ctx.input.mouseDown[0] = false;
16 });
17 events.AddEventCallback<NkMouseWheelVerticalEvent>([&](NkMouseWheelVerticalEvent *e) {
18     ctx.input.wheel += static_cast<float32>(e->GetDeltaY());
19 });
```

C'est le strict minimum pour des boutons, des curseurs et du défilement. Si vous ajoutez un champ de saisie, il vous faudra en plus `NkTextInputEvent` et la traduction des touches d'édition — la démo en donne le modèle complet ([Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:367-405](#)).

11.5.2 La boucle, et l'invariant d'ordre

main.cpp — écrit pour le cours (modèle : NKGuiDemo/main.cpp:429-470)

```
1  NkClock clock;
2  uint32 lastW = 0, lastH = 0;
3
4  while (running && window.IsOpen()) {
5      float32 dt = clock.Tick().delta;
6      if (dt <= 0.f) dt = 1.f / 60.f;
7      if (dt > 0.1f) dt = 0.1f;          // garde-fou : fenetre deplacee, machine chargee
8
9      while (NkEvent *ev = NkEvents().PollEvent()) {
10         (void)ev;                      // Le travail est fait par les callbacks
11     }
12     if (!running)
13         break;
14
15     // Synchroniser la swapchain sur la taille de la FENETRE.
16     const math::NkVec2u wsz = target->GetWindow().GetSize();
17     if (wsz.x > 0 && wsz.y > 0 && (wsz.x != lastW || wsz.y != lastH)) {
18         target->OnResize(wsz.x, wsz.y);
19         lastW = wsz.x;
20         lastH = wsz.y;
21     }
22     const math::NkVec2u sz = target->GetSize();
23     if (sz.x > 0 && sz.y > 0) {
24         ctx.viewW = static_cast<int32>(sz.x);
25         ctx.viewH = static_cast<int32>(sz.y);
26     }
27
28     ctx.BeginFrame(dt);
29     // ... Les trois panneaux, sections suivantes ...
30     ctx.EndFrame();
31
32     target->Clear();
33     backend.Submit(ctx.dl, sz.x, sz.y);
34     backend.Submit(ctx.dlOverlay, sz.x, sz.y);
35     target->Display();
36 }
```

L'invariant d'ordre, une dernière fois

événements → BeginFrame(dt) → widgets → EndFrame() → Clear → Submit ×2 → Display.

Les deux Submit ne sont pas une coquille : le second envoie la couche des popups et menus, qui doit se dessiner par-dessus tout le reste. L'oublier donne une application où les menus déroulants passent derrière les panneaux — un bug qu'on cherche longtemps parce qu'il ressemble à un problème de profondeur.

Le bornage de dt mérite aussi un mot. Quand l'utilisateur déplace la fenêtre, le système bloque la boucle : le *delta time* suivant peut valoir plusieurs secondes. Toutes vos animations feraient alors un bond. Un plafond à 100 ms coûte une ligne et supprime une catégorie entière de bizarreries.

11.6 Étape 4 — l'image (NKImage)

11.6.1 Fabriquer l'image

Nous n'avons pas de fichier; nous dessinons. Voici la fonction, écrite pour ce cours, qui produit une image et l'envoie au backend :

main.cpp — écrit pour le cours (API : NkImage.h:232, 587-765, 404, 775)

```
1 // Fabrique une image 512x512, en tire une vignette, et l'uploade sous `texId`.
2 // Renvoie false si quoi que ce soit echoue.
3 static bool ConstruireVignette(NkGuiCanvasBackend &backend, uint32 texId, float32 teinte,
4                               int32 &outW, int32 &outH) {
5     // 1) Une image sur la PILE : rien a liberer.
6     NkImage source;
7     if (!source.Create(512, 512, math::NkColor(18, 20, 26, 255), 4))
8         return false;
9
10    // 2) Dessin CPU (chapitre 5). Toutes ces primitives sont inline dans l'en-tete.
11    const uint8 r = static_cast<uint8>(60.f + 195.f * teinte);
12    const uint8 g = static_cast<uint8>(200.f - 120.f * teinte);
13    source.FillRect(40, 40, 432, 120, math::NkColor(r, g, 90, 255));
14    source.DrawRect(40, 40, 432, 120, math::NkColor(255, 255, 255, 255));
15    for (int32 i = 0; i < 12; ++i)
16        source.DrawCircle(256, 300, 20 + i * 12, math::NkColor(0, 220, 255, 255 - i * 18));
17    source.DrawLine(0, 0, 511, 511, math::NkColor(255, 255, 255, 90));
18    source.DrawLine(0, 511, 511, 0, math::NkColor(255, 255, 255, 90));
19
20    // 3) Vignette : Resize renvoie un NkImage* -> il FAUT le liberer.
21    NkImage *vignette = source.Resize(192, 192, NkResizeFilter::NK_BILINEAR);
22    if (!vignette)
23        return false;
24
25    // 4) Aplattir en RGBA CONTIGU (Le stride ne vaut pas forcément w*4 : chapitre 5).
26    static NkVector<uint8> rgba; // statique : on evite de realloquer a chaque appel
27    const int32 w = vignette->Width();
28    const int32 h = vignette->Height();
29    rgba.Resize(static_cast<usize>(w) * h * 4u);
30    for (int32 y = 0; y < h; ++y) {
31        const uint8 *src = vignette->RowPtr(y);
32        uint8 *dst = rgba.Data() + static_cast<usize>(y) * w * 4u;
33        for (int32 x = 0; x < w * 4; ++x)
34            dst[x] = src[x];
35    }
36
37    const bool ok = backend.UploadImageRGBA(texId, rgba.Data(), w, h);
38    outW = w;
39    outH = h;
40
41    vignette->Free(); // OBLIGATOIRE : issu d'une fabrique
42    return ok;       // ~NkImage() libere `source`
43 }
```

Quatre points de vigilance sont concentrés dans cette fonction, et ce sont exactement les quatre du chapitre 5 :

1. source est sur la pile, donc rien à libérer;
2. Resize renvoie un pointeur, donc Free() obligatoire — y compris si vous ajoutez un return anticipé après cette ligne;
3. la copie passe par RowPtr(y), pas par un memcpy global;
4. le tampon rgba est static : on ne réalloue pas 147 Ko à chaque changement de curseur.

11.6.2 L'appeler, et l'afficher

Avant la boucle :

main.cpp — écrit pour le cours

```
1   constexpr uint32 kTexVignette = 0x56494731u;    // 'VIG1'
2   int32 vigW = 0, vigH = 0;
3   float32 teinte = 0.35f;
4   bool vignetteOk = ConstruireVignette(backend, kTexVignette, teinte, vigW, vigH);
5   float32 teinteAppliquee = teinte;
```

Dans la frame :

main.cpp — écrit pour le cours (API : NkGuiWidgets.h:39, 65, 99, 140)

```
1   SetNextWindowSize(ctx, 340.f, 420.f);
2   if (Begin(ctx, "Image (NKImage)")) {
3       Text(ctx, "Image dessinée à l'exécution, sans aucun fichier.");
4       Separator(ctx);
5
6       if (vignetteOk)
7           Image(ctx, kTexVignette, static_cast<float32>(vigW),
8 static_cast<float32>(vigH));
9       else
10          Text(ctx, "(construction de l'image échouée)");
11
12      Separator(ctx);
13      SliderFloat(ctx, "Teinte", teinte, 0.f, 1.f);
14      // On ne refabrique QUE si la valeur a changé : sinon on redessinerait
15      // et re-uploaderait 512x512 pixels à chaque frame pour rien.
16      if (teinte != teinteAppliquee) {
17          vignetteOk = ConstruireVignette(backend, kTexVignette, teinte, vigW, vigH);
18          teinteAppliquee = teinte;
19      }
20      EndWindow(ctx);
21  }
```

Ne refaites pas le travail à chaque image

Le test `teinte != teinteAppliquee` est la seule chose qui sépare une application fluide d'une application qui redessine et ré-uploade une texture soixante fois par seconde sans raison.

C'est un réflexe à prendre : dans une interface en mode immédiat, *déclarer* un widget est bon marché, mais *réagir* à un widget doit être conditionné au changement. Les widgets qui renvoient `true` en cas de modification — `SliderFloat`, `Checkbox`, `DragFloat` — sont faits pour cela.

11.7 Étape 5 — le son (NKAudio)

11.7.1 Synthétiser trois sons

`AudioGenerator` produit des `AudioSample` sans aucun fichier. C'est ce que font quatre applications du dépôt, dont `Pong` :

Applications/Pong/src/Pong/Audio/AudioManager.cpp:31-45

```

1      static AudioSample MakePaddleHit() {
2          AudioSample s = AudioGenerator::GenerateTone(
3              /*frequency*/ 880.0f,
4              /*duration */ 0.06f,
5              /*waveform */ WaveformType::SINE,
6              /*sampleRate*/ 48000,
7              /*amplitude*/ 0.85f);
8          AdsrEnvelope env;
9          env.attackTime = 0.002f; // attack immediate
10         env.decayTime = 0.02f;
11         env.sustainLevel = 0.5f;
12         env.releaseTime = 0.03f;
13         AudioGenerator::ApplyEnvelope(s, env);
14         return s;
15     }

```

L'enveloppe ADSR n'est pas un raffinement optionnel : sans elle, un son qui commence et s'arrête brutalement produit un clic audible — l'oreille entend la discontinuité. Deux millisecondes d'attaque suffisent à le supprimer.

Adaptons pour notre atelier :

main.cpp — écrit pour le cours (API : NkAudio.h:707, 758 ; modèle : Pong/AudioManager.cpp:31-45)

```

1  static audio::AudioSample FaireSon(float32 freq, float32 duree, audio::WaveformType forme) {
2      audio::AudioSample s = audio::AudioGenerator::GenerateTone(freq, duree, forme, 48000,
3          0.75f);
4      audio::AdsrEnvelope env;
5      env.attackTime = 0.003f;
6      env.decayTime = 0.02f;
7      env.sustainLevel = 0.5f;
8      env.releaseTime = 0.04f;
9      audio::AudioGenerator::ApplyEnvelope(s, env);
10     return s;
11 }

```

11.7.2 Initialiser, jouer, couper

Avant la boucle :

main.cpp — écrit pour le cours (API : NkAudio.h:1091, 1104)

```

1      audio::AudioEngine &engine = audio::AudioEngine::Instance();
2      const bool audioOk = engine.Initialize(audio::AudioEngineConfig{}); // backend AUTO
3      if (!audioOk)
4          logger.Error("[NkAtelier] audio indisponible : l'application continue en silence");
5
6      audio::AudioSample sonGrave, sonMedium, sonAigu;
7      if (audioOk) {
8          logger.Info("[NkAtelier] device audio : {0} Hz, {1} canaux", engine.GetSampleRate(),
9              engine.GetChannels());
10         sonGrave = FaireSon(220.f, 0.08f, audio::WaveformType::SQUARE);
11         sonMedium = FaireSon(440.f, 0.07f, audio::WaveformType::SINE);
12         sonAigu = FaireSon(880.f, 0.06f, audio::WaveformType::SINE);
13     }
14     bool sonsUiActifs = true;

```

Dans la frame :

main.cpp — écrit pour le cours (API : NkGuiWidgets.h:44-45 ; NkAudio.h:1127, 1266)

```
1      SetNextWindowSize(ctx, 340.f, 260.f);
2      if (Begin(ctx, "Son (NkAudio)")) {
3          if (!audioOk) {
4              Text(ctx, "Moteur audio indisponible sur cette machine.");
5          } else {
6              Text(ctx, "Trois sons synthetises, joues sur le bus UI.");
7              Separator(ctx);
8
9              audio::VoiceParams vp;
10             vp.bus = "UI";           // bus dedie : coupable sans toucher a la musique
11
12             if (Button(ctx, "Grave (220 Hz)"))
13                 engine.Play(sonGrave, vp);
14             if (Button(ctx, "Medium (440 Hz)"))
15                 engine.Play(sonMedium, vp);
16             if (Button(ctx, "Aigu (880 Hz)"))
17                 engine.Play(sonAigu, vp);
18
19             Separator(ctx);
20             if (Checkbox(ctx, "Sons d'interface", sonsUiActifs)) {
21                 if (audio::NkAudioBus *bus = engine.GetBus("UI"))
22                     bus->SetMute(!sonsUiActifs);
23             }
24         }
25         EndWindow(ctx);
26     }
```

Remarquez que Play est appelé sans récupérer le *handle* : ce sont des sons courts, on ne les pilotera pas. Remarquez aussi le if autour de GetBus : il renvoie nullptr si le moteur n'est pas initialisé.

N'écrivez jamais `cfg.backend = DIRECTSOUND`

Rappel du chapitre 7 : le backend DirectSound est un stub qui renvoie true sans rien ouvrir (`Kernel/Runtime/NkAudio/src/NkAudio/NkAudioBackends.cpp:445-451`). Vous auriez un moteur « initialisé », des voix « en lecture », et le silence complet. Laissez AUTO.

11.8 Étape 6 — la vidéo (NKMedia)

C'est la partie la plus ambitieuse, et elle est faite d'un aller-retour : on écrit une vidéo, puis on la relit.

11.8.1 Écrire la vidéo

main.cpp — écrit pour le cours (modèle : NKVideoTest/src/main.cpp:83-112)

```
1 // Ecrit une petite video MJPEG/AVI de `nframes` images. Renvoie true si OK.
2 static bool EcrireVideoDemo(const char *chemin, int32 w, int32 h, int32 fps, int32 nframes) {
3     media::NkVideoConfig cfg;
4     cfg.width = w;
5     cfg.height = h;
6     cfg.fpsNum = fps;
7     cfg.fpsDen = 1;
```

```

8   cfg.codec = media::NkVideoCodec::MJPEG;           // Le chemin le plus sur (chapitre 8)
9   cfg.container = media::NkVideoContainer::AVI;
10  cfg.quality = 88;
11
12  media::NkVideoWriter vw;
13  if (!vw.Open(chemin, cfg)) {
14      logger.Error("[NkAtelier] impossible d'ouvrir {0} en ecriture", chemin);
15      return false;
16  }
17
18  // Un SEUL tampon, reutilise. RGB24 : 3 octets par pixel, contigu, haut->bas.
19  uint8 *rgb = static_cast<uint8 *>(memory::NkAlloc(static_cast<usize>(w) * h * 3));
20  if (!rgb) {
21      vw.Close();
22      return false;
23  }
24
25  bool ok = true;
26  for (int32 f = 0; f < nframes && ok; ++f) {
27      // Motif : un carre qui traverse l'image + un fond qui defile.
28      const int32 cx = (w - 60) * f / (nframes > 1 ? nframes - 1 : 1);
29      for (int32 y = 0; y < h; ++y) {
30          uint8 *ligne = rgb + static_cast<usize>(y) * w * 3;
31          for (int32 x = 0; x < w; ++x) {
32              const bool dansCarre = (x >= cx && x < cx + 60 && y >= h / 2 - 30 && y < h /
33  2 + 30);
34              ligne[x * 3 + 0] = dansCarre ? 255 : static_cast<uint8>((x + f * 3) & 0xFF);
35              ligne[x * 3 + 1] = dansCarre ? 140 : static_cast<uint8>((y * 2) & 0xFF);
36              ligne[x * 3 + 2] = dansCarre ? 0 : 60;
37          }
38          ok = vw.WriteFrame(rgb, media::NkVideoInputFormat::RGB24);
39      }
40
41      memory::NkFree(rgb);
42      vw.Close();           // OBLIGATOIRE : ecrit l'index AVI. Sans lui : fichier illisible.
43      return ok;
44  }

```

Notez que `Close()` est appelé **sur tous les chemins**, y compris quand `ok` est devenu `false`. C'est la règle du chapitre 8 : sans finalisation, le fichier existe et n'est lisible par personne.

Notez aussi la construction de la ligne : `rgb + y * w * 3`. Ici, pas de *stride* aligné — nous fabriquons nous-mêmes un tampon contigu, c'est ce que `WriteFrame` attend. Le *stride* est une notion de `NkImage`, pas de `NkVideoWriter`.

11.8.2 Relire la vidéo

Avant la boucle :

main.cpp — écrit pour le cours (API : `NkVideoReader.h:54-66`)

```

1   constexpr uint32 kTexVideo = 0x56494431u;           // 'VID1'
2   const char *kCheminVideo = "nkatelier_demo.avi";
3
4   media::NkVideoReader lecteur;
5   bool videoOk = false;
6   double videoFps = 25.0;
7   int32 videoW = 0, videoH = 0, videoNb = -1;
8

```

```

9   if (EcrireVideoDemo(kCheminVideo, 320, 240, 25, 75)) { // 3 secondes
10      if (lecteur.Open(kCheminVideo)) {
11         const media::NkVideoReaderInfo &in = lecteur.Info();
12         videoW = in.width;
13         videoH = in.height;
14         videoNb = in.frameCount;
15         videoFps = (in.fps > 1.0) ? in.fps : 25.0;
16         videoOk = true;
17         logger.Info("[NkAtelier] video : {0} {1} {2}x{3} @ {4} fps",
18                     in.container.CStr(), in.codec.CStr(), in.width, in.height, in.fps);
19      } else {
20         logger.Error("[NkAtelier] video ecrite mais illisible : {0}", kCheminVideo);
21      }
22   }
23
24   media::NkVideoFrame trame; // DECLAREE HORS BOUCLE : son buffer est reutilise
25   float32 accumulateur = 0.f;
26   bool videoEnLecture = true;
27   bool videoFinie = false;
28   int32 videoIndex = -1;

```

Le fait que la lecture soit tentée juste après l'écriture est en soi un test : si Open échoue, c'est que l'écriture a mal tourné.

11.8.3 Cadencer et afficher

main.cpp — écrit pour le cours (modèle : NKCode/Shell/NkVideoViewer.h:153-170)

```

1   SetNextWindowSize(ctx, 400.f, 420.f);
2   if (Begin(ctx, "Video (NKMedia)")) {
3       if (!videoOk) {
4           Text(ctx, "Video indisponible.");
5       } else {
6           // — Cadencement —
7           const float32 dureeImage = 1.f / static_cast<float32>(videoFps);
8           if (videoEnLecture && !videoFinie) {
9               accumulateur += dt;
10              if (accumulateur > dureeImage * 4.f)
11                  accumulateur = 0.f; // evite le rattrapage explosif
12              while (accumulateur >= dureeImage) {
13                  accumulateur -= dureeImage;
14                  if (lecteur.ReadFrame(trame)) {
15                      videoIndex = trame.index;
16                      // Une seule image decodee -> un seul upload.
17                      backend.UploadImageRGBA(kTexVideo, trame.rgba.Data(),
trame.width, trame.height);
18                  } else {
19                      videoFinie = true; // fin du fichier
20                      break;
21                  }
22              }
23          }
24
25          // — Affichage —
26          if (videoIndex >= 0)
27              Image(ctx, kTexVideo, static_cast<float32>(videoW),
static_cast<float32>(videoH));
28          else
29              Text(ctx, "(premiere image en attente)");
30      }

```

```

31         Separator(ctx);
32         Text(ctx, videoFinie ? "Etat : terminee" : (videoEnLecture ? "Etat : lecture"
: "Etat : pause"));
33
34         BeginHBox(ctx);
35         if (Button(ctx, videoEnLecture ? "Pause" : "Lecture"))
36             videoEnLecture = !videoEnLecture;
37         if (Button(ctx, "Debut")) {
38             if (lecteur.SeekFrame(0)) {
39                 videoFinie = false;
40                 accumulateur = 0.f;
41                 videoIndex = -1;
42             }
43         }
44         EndHBox(ctx);
45     }
46     EndWindow(ctx);
47 }

```

Les trois règles de la lecture vidéo, réunies

1. **Un seul upload par image affichée.** La boucle while peut décoder plusieurs trames si l'application a pris du retard, mais on n'uploade que celle qu'on va montrer;
2. **l'accumulateur est borné ($> \text{dureeImage} * 4$) :** sans ce garde-fou, un décrochage de deux secondes déclencherait le décodage de cinquante images d'un coup, ce qui aggraverait le décrochage;
3. **NkVideoFrame est déclarée hors boucle.** Son NkVector garde sa capacité; la déclarer à l'intérieur provoquerait une allocation par image.

Le rebouclage n'est exact que sur les formats non compressés en temps

SeekFrame(0) fonctionne bien ici parce que notre vidéo est en MJPEG : chaque image est indépendante. Sur du H.264, SeekFrame(n) ne fait que revenir à l'image autonome précédente, et il faut redécoder vers l'avant jusqu'à la cible (chapitre 8). Notre choix du MJPEG n'était donc pas seulement un choix de prudence : il simplifie aussi la navigation.

L'upload répété dépend de NkTexture::Update

Regardez l'implémentation du pont : UploadImageRGBA crée la texture au premier appel, puis appelle tex->Update(...) aux suivants ([Kernel/Runtime/NKCanvas/src/NKCanvas/UI/NkGuiCanvasBackend.h:94-117](#)). C'est exactement le bon comportement — on ne recrée pas la texture par image.

Mais souvenez-vous de l'avertissement de Pong (chapitre 6) : «Update exige gTextureBackend.Update qui n'est pas câblé sur tous les backends → échec silencieux» ([Applications/Pong/src/Pong/Render/FontAtlas.cpp:107-109](#)).

Si votre vignette statique s'affiche mais que la vidéo reste figée sur sa première image, la cause est probablement là — et la solution est de changer de backend graphique, pas de réécrire votre code de lecture.

11.9 Étape 7 — la fermeture

C'est le moment où l'on récolte ce qu'on a semé dans les cinq chapitres précédents. Chaque module a son ordre, et ils s'imbriquent.

main.cpp — écrit pour le cours (invariants : NkAudio.h:1114 ; NkAudioPlayer/main.cpp:258-259)

```
1 // — Fermeture, dans cet ordre exact —————
2
3 // 1) Video : fermer Le lecteur (il possede son etat interne).
4 lecteur.Close();
5
6 // 2) Audio : ARRETER LE MOTEUR AVANT de Liberer Les samples.
7 if (audioOk) {
8     engine.Shutdown();
9     audio::AudioLoader::Free(sonGrave);
10    audio::AudioLoader::Free(sonMedium);
11    audio::AudioLoader::Free(sonAigu);
12 }
13
14 // 3) Interface.
15 SetCurrentContext(nullptr);
16 ctx.Shutdown();
17 return 0;
18 // Les NkUniquePtr (target, ctxPtr, fontPtr) se detruisent ici, dans L'ordre
19 // inverse de Leur declaration. Le backend et l'atlas de police suivent.
20 }
```

Module	Ce qu'on ferme	Piège si l'ordre est inversé
NKMedia	lecteur.Close()	fichier verrouillé
NKAudio	Shutdown() puis Free()	lecture après libération
NKImage	rien (pile) ou Free()	fuite ou double libération
NKFont	rien (l'atlas possède tout)	double libération si fusion
NKGui	ctx.Shutdown()	—

Le principe unique derrière tous ces ordres

On éteint d'abord ce qui **tourne**, ensuite ce qui est **lu**.

Le moteur audio a un fil qui lit vos échantillons; le gestionnaire de connexion a un fil qui lit vos tampons; l'enregistreur vidéo a un fil qui lit vos images. Dans les trois cas, la fonction d'arrêt joint le fil — et ce n'est qu'après qu'on a le droit de libérer ce qu'il lisait. Si vous ne deviez retenir qu'une phrase de ces six chapitres, ce serait celle-là : elle explique l'ordre de fermeture de NKAudio, de NKNetwork et de NKMedia d'un seul coup.

11.10 Compiler et lancer

Ligne de commande (voir README.md:147-163)

```
1 jenga build --target NkAtelier --config Debug
```

L'exécutable atterrit dans `Build/Bin/Debug-Windows/NkAtelier/` — ou `Debug-Linux`, `Debug-macOS` selon la plateforme, suivant le `targetdir` que nous avons écrit dans le `.jenga`.

Pour la version optimisée :

Ligne de commande

```
1 jenga build --target NkAtelier --config Release
```

Le répertoire courant au lancement

Notre application écrit `nkatelier_demo.avi` dans le **répertoire courant**, pas à côté de l'exécutable. Selon que vous la lancez depuis un terminal, depuis un explorateur de fichiers ou depuis un environnement de développement, ce répertoire diffère.

Ce n'est pas grave ici — nous écrivons *et* lisons au même endroit dans la même exécution — mais c'est exactement le mécanisme qui fait qu'un `assets/logo.png` « introuvable » l'est seulement dans certaines conditions de lancement. Si vous voulez voir le fichier produit, lancez l'application depuis un terminal, dans un dossier que vous avez choisi.

Souvenez-vous aussi que les auto-tests de NKMedia écrivent, eux aussi, dans le répertoire courant (chapitre 8).

11.11 Le catalogue des silences

Voici la partie la plus utile de ce chapitre. Toutes les pannes ci-dessous ont un point commun : **aucune ne produit de message d'erreur**. Gardez cette liste sous la main.

Symptôme	Cause à vérifier en premier
Fenêtre noire, rien du tout	Backend graphique. Essayez <code>NK_GFX_API_OPENGL</code> .
Tout s'affiche sauf les textes	Atlas de police non uploadé, ou <code>ctx.font</code> non affecté.
Les menus passent <i>derrière</i>	<code>Second Submit(ctx.dlOverlay, ...)</code> oublié.
L'image apparaît « penchée »	<code>memcpy</code> global au lieu de <code>RowPtr(y)</code> .
L'image est vide	Texture créée avant <code>render.Initialize()</code> .
Aucun son, mais tout indique que ça joue	Backend <code>DIRECTSOUND</code> demandé explicitement.
Grésillement ou plantage à la fermeture	<code>Free(sample)</code> appelé avant <code>engine.Shutdown()</code> .
Le fichier vidéo est illisible	<code>Close()</code> non appelé sur le <code>NkVideoWriter</code> .
La vidéo reste sur la première image	<code>NkTexture::Update</code> non câblé dans ce backend.
La vidéo est à l'envers	<code>flipVertical</code> avec un framebuffer OpenGL.
Vignette de mauvaise qualité malgré Lanczos	<code>NK_LANCZOS3</code> retombe sur du bilinéaire.
Aucun pair ne se connecte	Poignée de main asynchrone : attendre <code>ConnectedPeerCount()</code> .
Erreurs d'édition de liens <code>_imp_socket</code>	<code>ws2_32</code> absent du <code>links()</code> .

La méthode générale

Ces silences ont tous la même parade, et c'est une habitude à prendre plutôt qu'une astuce : **journalisez ce que vous chargez, et journalisez aussi ce que vous ne chargez**

pas.

Toutes les fonctions d'initialisation de ce chapitre ont un `else` qui écrit une ligne. Cela paraît excessif quand tout fonctionne. Le jour où quelque chose manque, cette ligne vous fait gagner une heure — et c'est précisément ce que fait la démo du dépôt : « Image demo introuvable (cwd) -> section image vide » (`Applications/NKGuiDemo/src/NKGuiDemo/main.cpp:422`).

11.12 Le programme, vu de haut

Récapitulons la structure complète, dans l'ordre du fichier :

Applications/NkAtelier/src/main.cpp — plan d'ensemble

```
1 // --- inclusions (etape 2) ---
2
3 // --- fonctions utilitaires ---
4 static bool ConstruireVignette(backend, texId, teinte, &w, &h); // etape 4
5 static audio::AudioSample FaireSon(freq, duree, forme); // etape 5
6 static bool EcrireVideoDemo(chemin, w, h, fps, nframes); // etape 6
7
8 int nkmain(const NkEntryState &state) {
9     // 1. fenetre + cible de rendu // etape 2
10    // 2. contexte NKGui + backend // etape 2
11    // 3. police embarquee + upload atlas // etape 2
12    // 4. callbacks d'evenements // etape 3
13    // 5. image : premiere construction + upload // etape 4
14    // 6. audio : Initialize + synthese des trois sons // etape 5
15    // 7. video : ecriture du fichier puis ouverture en lecture // etape 6
16
17    while (running && window.IsOpen()) {
18        // dt + pompage des evenements + synchro swapchain // etape 3
19        ctx.BeginFrame(dt);
20        // panneau Image // etape 4
21        // panneau Son // etape 5
22        // panneau Video (cadencement + upload + affichage) // etape 6
23        ctx.EndFrame();
24        // Clear + Submit x2 + Display // etape 3
25    }
26
27    // fermeture : video, puis audio (Shutdown avant Free), puis UI // etape 7
28 }
```

Environ 300 lignes en tout. C'est peu pour cinq modules — et c'est le point. La difficulté du moteur n'est pas dans la quantité de code à écrire, elle est dans les ordres à respecter : ordre d'initialisation, ordre de la frame, ordre de fermeture.

11.13 Pour aller plus loin

Trois extensions, par difficulté croissante. Chacune réutilise un chapitre précédent sans rien introduire de nouveau.

11.13.1 Enregistrer ce que l'atelier affiche

Ajoutez un bouton « Enregistrer » qui, à chaque frame, capture la fenêtre et pousse l'image dans un `NkVideoRecorder` en mode MJPEG :

```

1  if (enregistrement && target->CaptureToImage(capture) && capture.IsValid()) {
2      if (!rec.IsRecording())
3          rec.Begin("nkatelier_capture.mp4", capture.Width(), capture.Height(),
4                  30, 1, 22, 32, media::NkRecorderCodec::MJPEG, 92);
5      if (rec.IsRecording())
6          rec.PushVideo(capture.Pixels(), media::NkVideoInputFormat::RGBA32);
7  }
8  // ... et, sur TOUS les chemins de sortie :
9  if (rec.IsRecording())
10     rec.End();

```

Attention aux deux pièges du chapitre 8 : en mode MJPEG l'audio est ignorée silencieusement, et `End()` doit être appelé même si la fenêtre est fermée brutalement.

11.13.2 Ajouter le réseau

Ajoutez à `NkAtelier` un quatrième panneau : deux boutons « Héberger » et « Rejoindre », et une liste de messages. Chaque clic sur un bouton de son envoie un message aux autres instances, qui jouent le même son.

Vous aurez besoin d'ajouter `NKNetwork` aux dépendances du `.jenga` et `ws2_32` au `links()` sous Windows. Le chapitre 9 donne la structure à suivre : une petite classe de session, un `Tick(dt)` par frame, et une interface qui ne voit qu'un état.

11.13.3 Faire du décodage un travail de fond

Notre vidéo fait 320 par 240 pixels : le décodage est instantané. Essayez avec une vidéo de 1920 par 1080, et vous verrez la boucle hoqueter.

La solution — décoder sur un fil, uploader sur le fil principal — est celle du chapitre 5, appliquée à la vidéo. C'est un excellent exercice, mais commencez par **mesurer** : entourez `ReadFrame` d'un chronomètre mural et regardez. Et souvenez-vous de l'avertissement du chapitre 8 : mesurez le temps mur, pas le *PTS* de la vidéo.

11.14 Exercices

1 — Faire tomber chaque panneau, un par un

Pour chacun des quatre modules, cassez volontairement une chose et notez ce que vous observez :

- n'appellez pas `UploadFontGray8`;
- supprimez le `vignette->Free()` et lancez l'application quelques minutes en bougeant le curseur de teinte (surveillez la mémoire);
- inversez `Shutdown()` et `Free()` à la fermeture;
- supprimez le `vw.Close()` dans `EcrireVideoDemo`.

Écrivez, pour chaque cas, la phrase que vous auriez aimé lire dans un journal. Puis ajoutez-la vraiment à votre code. Vous venez d'écrire votre propre catalogue des silences.

2 — Une vraie vignette de fichier

Ajoutez un cinquième panneau qui charge une image *depuis le disque* avec `NkImage::Load`, en essayant plusieurs chemins candidats comme le fait la démo du dépôt. Affichez le format d'origine (`SourceFormat()`), les dimensions, le nombre de canaux et le *stride*. Faites-en une vignette de 128 pixels de large en préservant le rapport d'aspect, et affichez côte à côte l'original réduit et la vignette. Puis ajoutez un bouton qui sauvegarde la vignette en PNG et un autre qui tente de la sauvegarder en WebP. Affichez les deux valeurs de retour côte à côte. Que constatez-vous ?

3 — Le mélangeur

Ajoutez trois `SliderFloat` — Master, UI, Music — reliés aux bus correspondants par `GetBus(...)->SetVolume(...)`, en n'appelant le *setter* que lorsque la valeur change. Ajoutez ensuite un son long en boucle sur le bus « Music » (`vp.looping = true`) et vérifiez, à l'oreille, la formule du chapitre 7 : le volume effectif est le produit du volume de la voix, de celui du bus, et de tous ses parents jusqu'au Master. Vérifiez enfin que couper « UI » ne touche pas à « Music ».

4 — Une chaîne complète, de l'image à la vidéo et retour

Modifiez `EcrireVideoDemo` pour qu'elle ne dessine plus ses pixels à la main, mais qu'elle utilise une `NkImage` et ses primitives de dessin — comme `ConstruireVignette` — puis qu'elle envoie `image.Pixels()` à `WriteFrame`. Attention : `WriteFrame` attend un tampon **contigu**, et `NkImage::Pixels()` a un *stride* aligné sur 4 octets. Choisissez donc des dimensions telles que `stride == width * bpp`, ou bien recopiez ligne par ligne. Vérifiez votre raisonnement en affichant les deux valeurs avant d'écrire la première image. Relisez ensuite la vidéo produite et affichez-la : vous aurez fait circuler les mêmes pixels à travers `NKImage`, le codec JPEG, le conteneur AVI, le décodeur, et enfin le GPU. C'est toute la chaîne de ce cours en une seule fonction.

5 — Le vôtre

Reprenez `NkAtelier` et transformez-le en quelque chose qui vous est utile : une visionneuse de dossier d'images, un métronome, un petit lecteur de séquences, un banc d'essai de vos propres filtres d'image. Une seule contrainte, et c'est celle de tout ce cours : **pour chaque fonction du moteur que vous emploierez, vérifiez dans la source qu'elle fait bien ce que son nom annonce**. Vous savez maintenant où regarder — le `.cpp`, pas le commentaire ; le `ROADMAP.md`, pas le `.jenga` — et vous savez que la réponse est parfois « non ».